

バス通学(Bus)



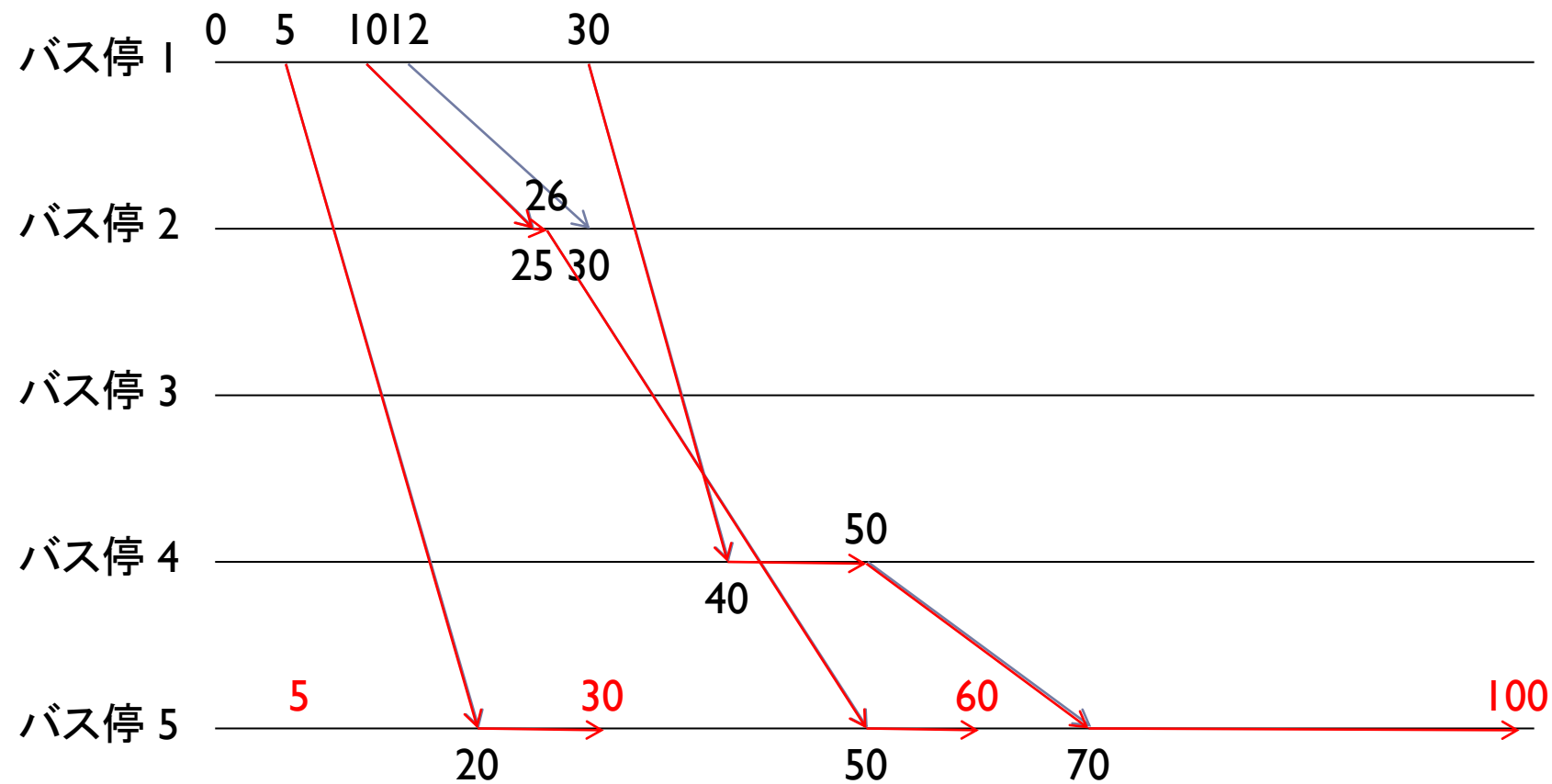
解説：カサウラカズミ

問題

- ▶ バスの時刻表が与えられる
- ▶ それぞれのバスはある時刻にあるバス停を出発しある時刻にあるバス停に到着する
- ▶ バスを乗り継いでバス停 1 からバス停 N に行く
- ▶ バス停 N に到着すべき時刻が Q 個与えられる
- ▶ それぞれについてバス停 1 の出発時刻を求めよ



例



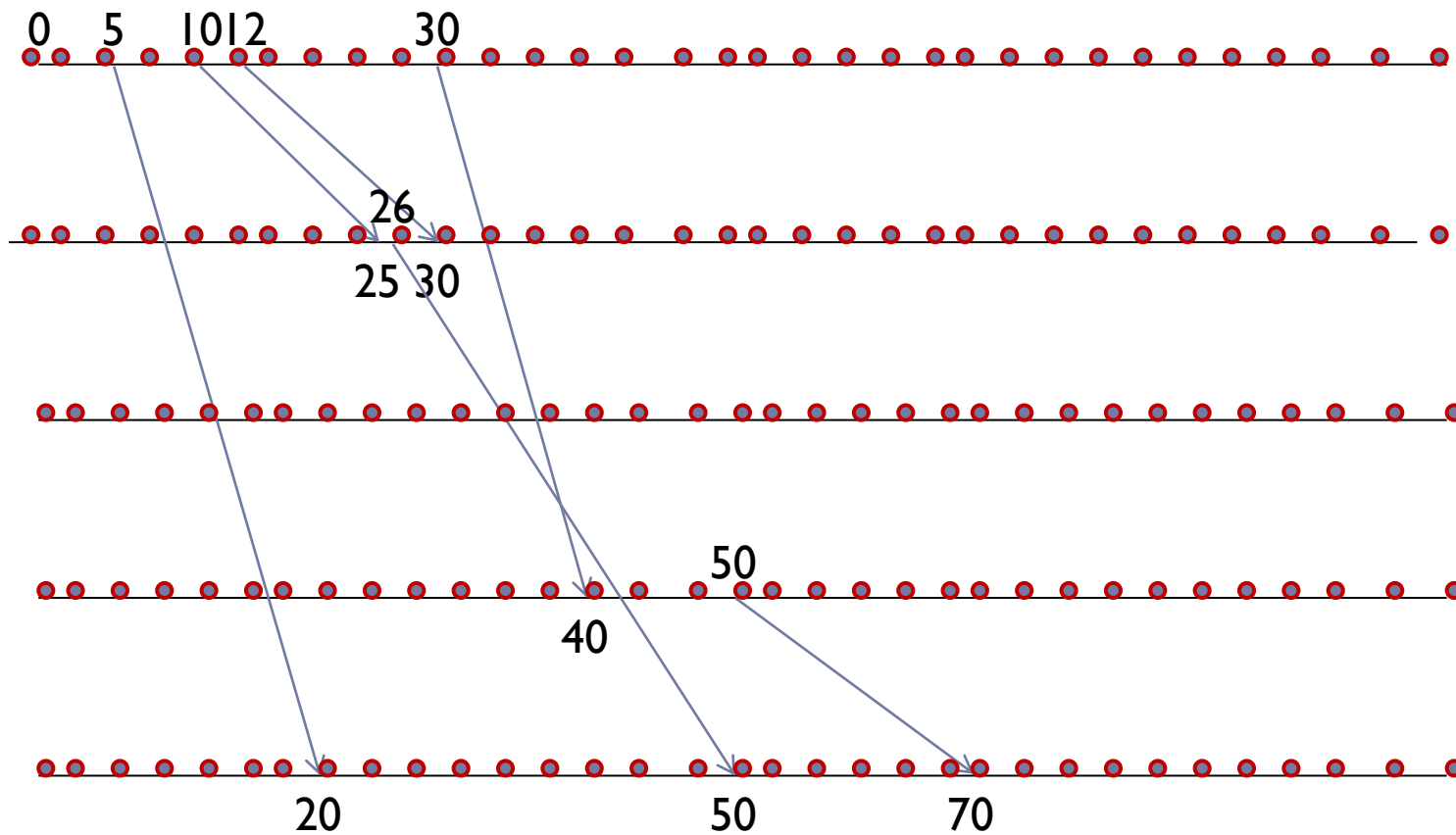
この間0.1秒

愚直な解法

- ▶ $Q=1$ のときを考える
- ▶ ダイアグラムをグラフ化
- ▶ バス停の番号と時間の組をグラフの頂点とみなす



愚直な解法



愚直な解法

- ▶ それぞれの頂点から一ミリ秒後の頂点に辺を張る
- ▶ $\langle A, t \rangle \rightarrow \langle A, t+1 \rangle$
- ▶ バス停 A を時刻 X に出発し バス停 B に時刻 Y に到着するバスに対し辺を張る
- ▶ $\langle A, X \rangle \rightarrow \langle B, Y \rangle$

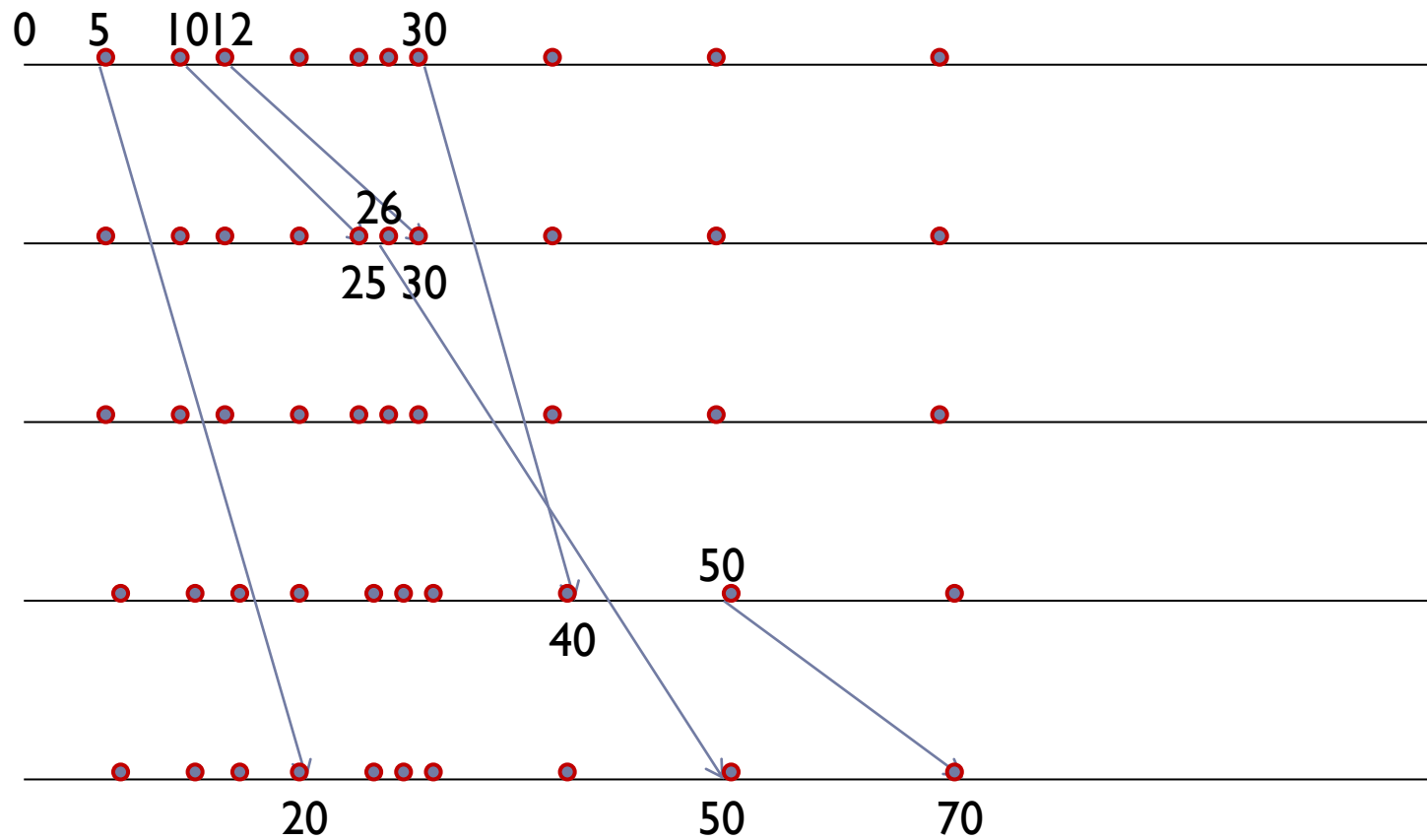


愚直な解法

- ▶ 時刻の取りうる値 86400000 通り ← 多すぎ
- ▶ 頂点数は $N * 86400000$ ← 多すぎ
- ▶ 頂点を減らそう
- ▶ 座標圧縮 $2 * M$ 通り
- ▶ 点は全部で $2 * N * M$
- ▶ Subtask 1,2 なら $N \leq 2000$ $M \leq 2000$ なのでなんとかメモリに収まる



愚直な解法

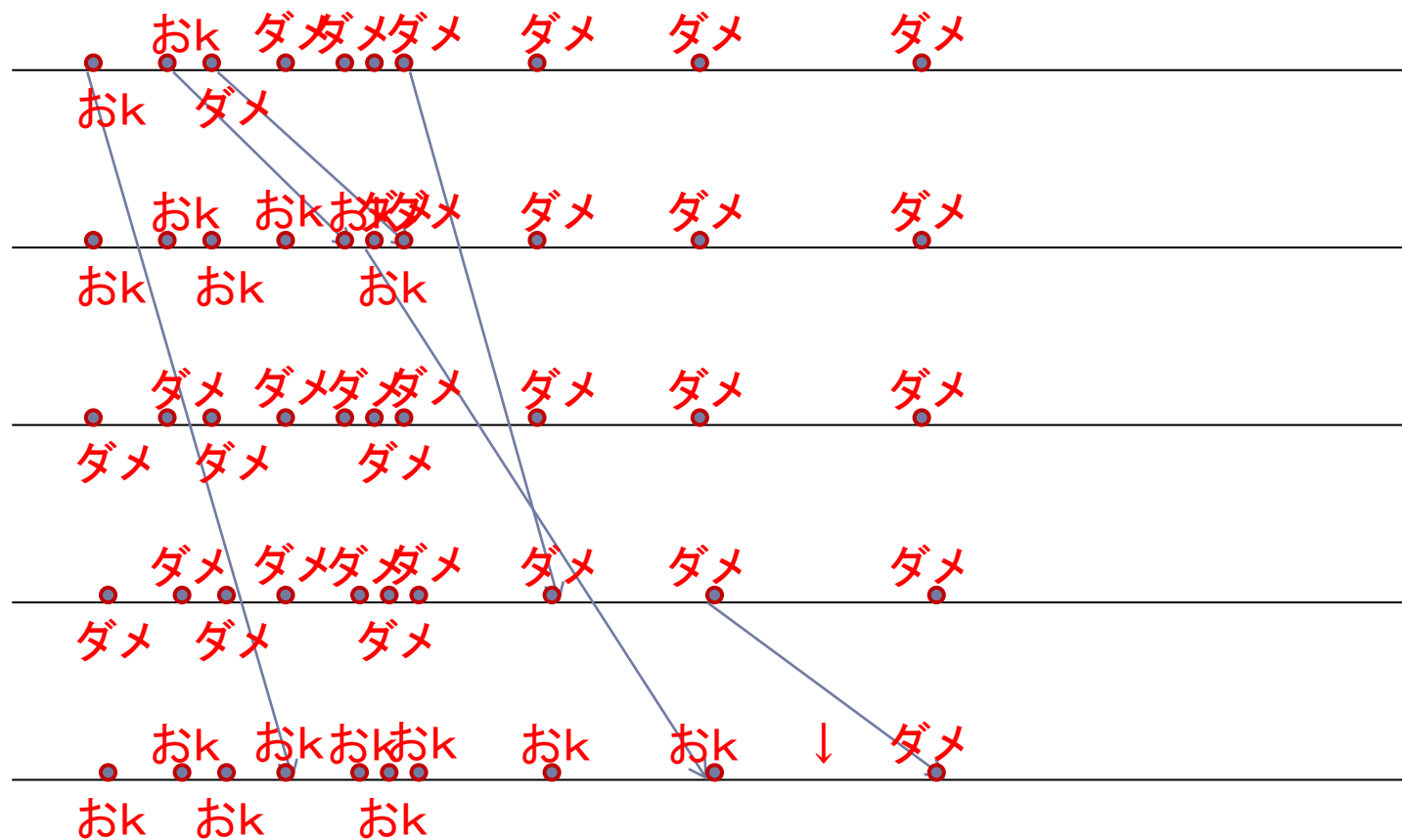


愚直な解法

- ▶ それぞれの頂点について所定時刻にバス停 N にたどりつけるか、すなわち辺をたどって $\langle N, L \rangle$ にたどりつけるかを調べる。
- ▶ すべての辺は時刻の早い点から遅い点に向かっている
- ▶ 時刻の遅い点から調べていけばいい



愚直な解法



愚直な解法

- ▶ 時間計算量 $O(NM)$ 空間計算量 $O(NM)$
- ▶ Subtask 1 $N \leq 2000, M \leq 2000$ ならで解けて 20 点。

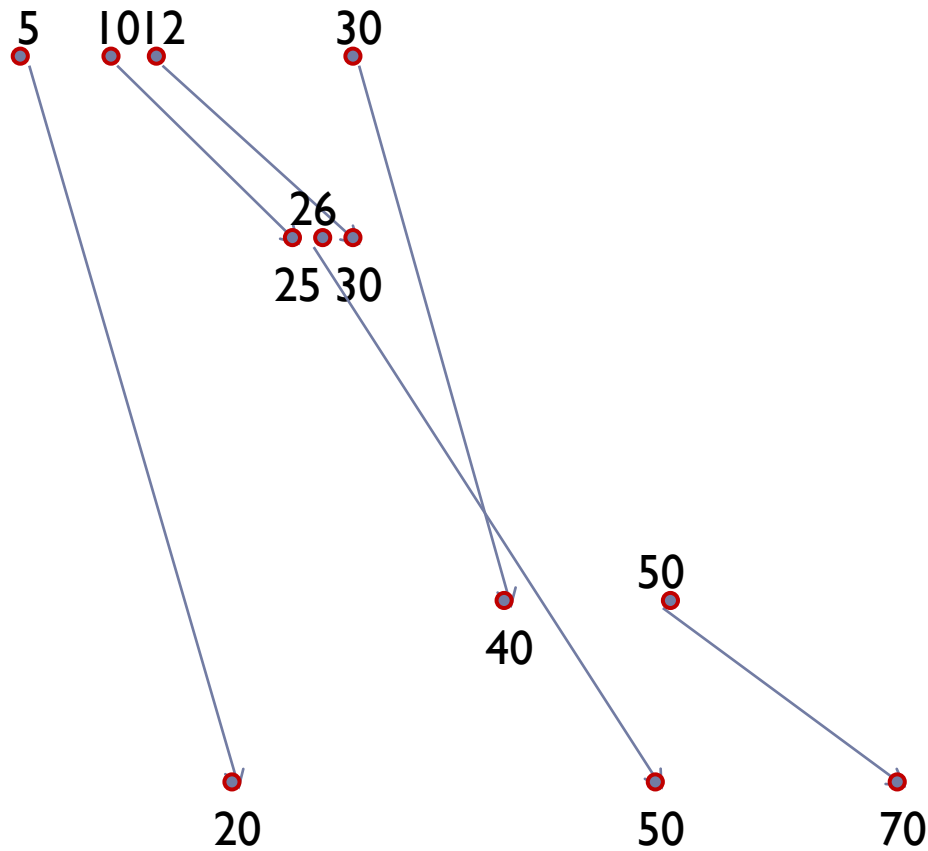


無駄をなくしスマートなグラフに

- ▶ 無駄な点が多い
- ▶ バス辺の出発点と到着点だけあればよい



無駄をなくしスマートなグラフに

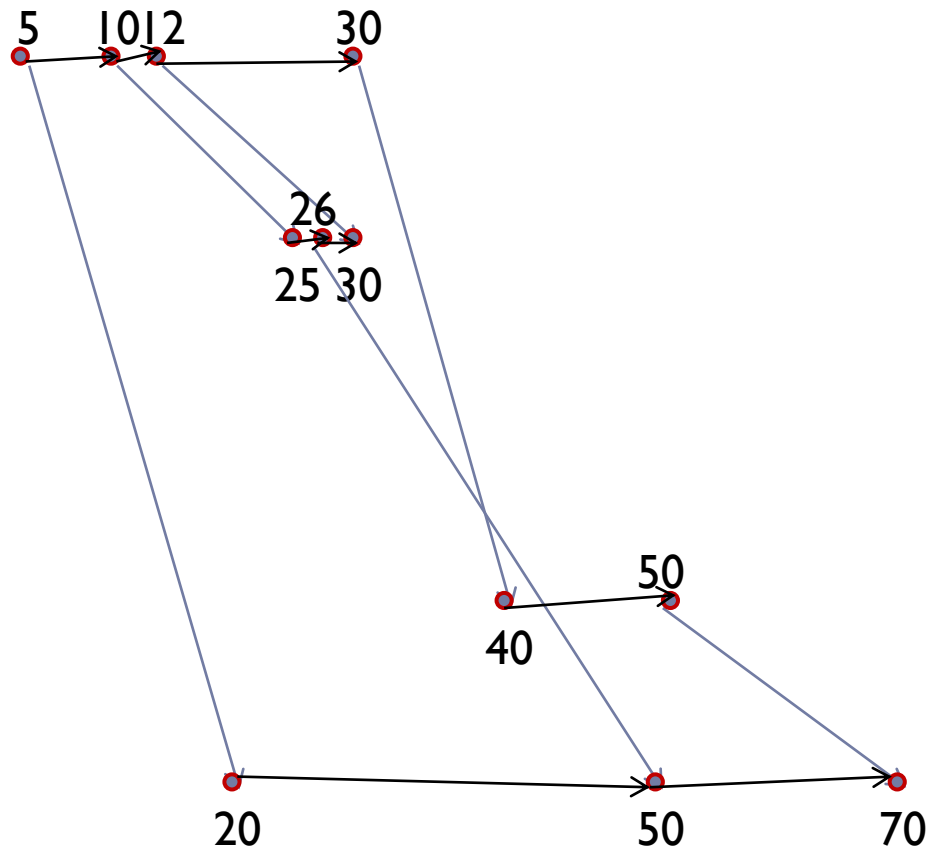


無駄をなくしスマートなグラフに

- ▶ 各バス停について、点を時刻でソートし順番に辺を張る



無駄をなくしスマートなグラフに

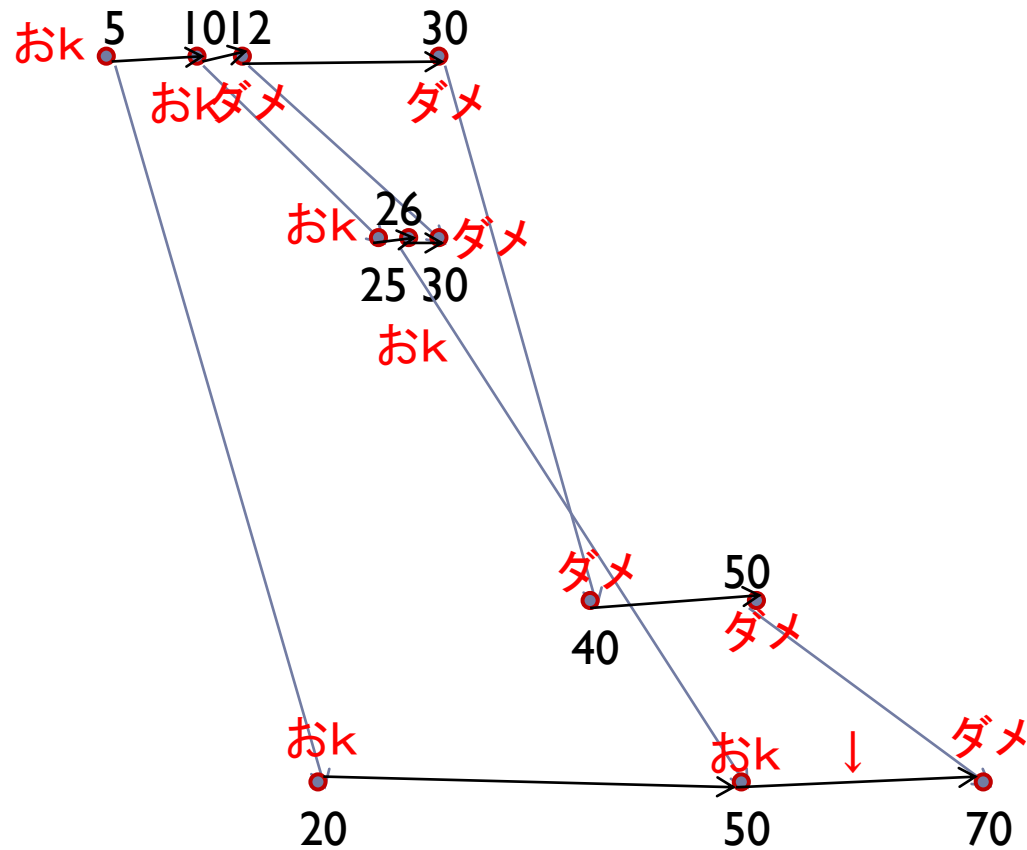


無駄をなくしスマートなグラフに

- ▶ あとは同様に、遅い時刻の点から目的の点に到達可能かどうか調べていけばよい



無駄をなくしスマートなグラフに



無駄をなくしスマートなグラフに

- ▶ これで Subtask 1,3 がとけて 35 点。



Q>1 に対応させよう

- ▶ グラフの構築はあらかじめできるので $O(M \log M + MQ)$ → Subtask 2 なら通るかな？
- ▶ よく考えたら バス停 N にバスが到着する時刻についてだけ調べればいい
- ▶ $O(M^2 + Q \log M)$ → Subtask 2 なら安全
- ▶ 残り 50 点
- ▶ なんとか前処理をしてそれぞれのクエリに高速で答えられるようにしたい
- ▶ 実はさっきの方法をちょっと変えるだけでいい

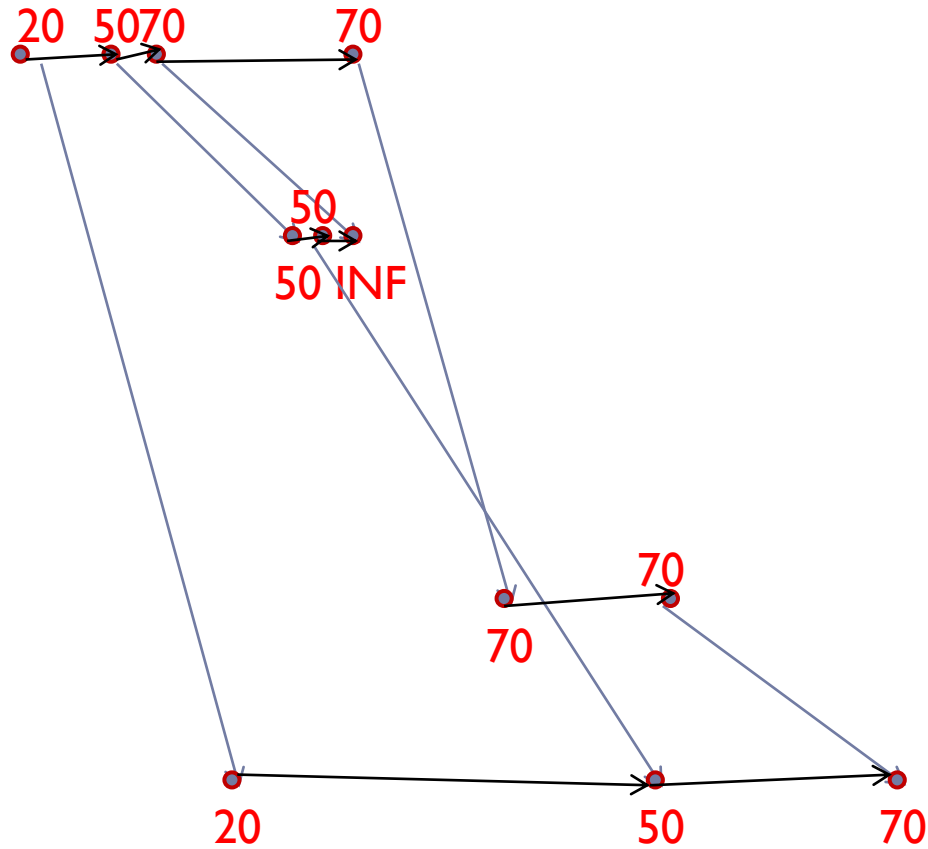


Q>1 に対応させよう

- ▶ $\langle N, 50 \rangle$ に到達できるなら $\langle N, 100 \rangle$ に到達できるのは当然
- ▶ 各頂点についてそこからバス停 N に到達できる最早の時刻を計算
- ▶ さっきのグラフ上で同様に遅い点から計算していけばいい



Q>1 に対応させよう



Q>1 に対応させよう

- ▶ あとはバス停 1 の頂点たちのデータに対して bound すればクエリに答えられる
 - ▶ 逆でもよい
 - ▶ 各頂点に対し、「そこに到達するためには遅くともいつバス停 1 を出発しなければならないか」を早い頂点から計算
 - ▶ バス停 N のデータたちを使えばクエリにこたえられる
 - ▶ 時間計算量 $O((M + Q)\log M)$
 - ▶ 空間計算量 $O(M)$ で 100 点！
 - ▶ ちなみにこれを Subtask 1 のグラフに使うと Subtask 2 がとけて 35 点
-



実装の工夫

- ▶ 実は実際にグラフを作らなくてもよい
- ▶ 後ろから見ていくことは同じ
- ▶ 各バス辺 e についてその到達点からバス停 N に到達できる最早時刻 $F[e]$ を計算し記録していく
- ▶ さらに各バス停 i に居ついて「いま考えている時刻」からバス停 N に到達できる時刻 $E[i]$ を計算し更新していく

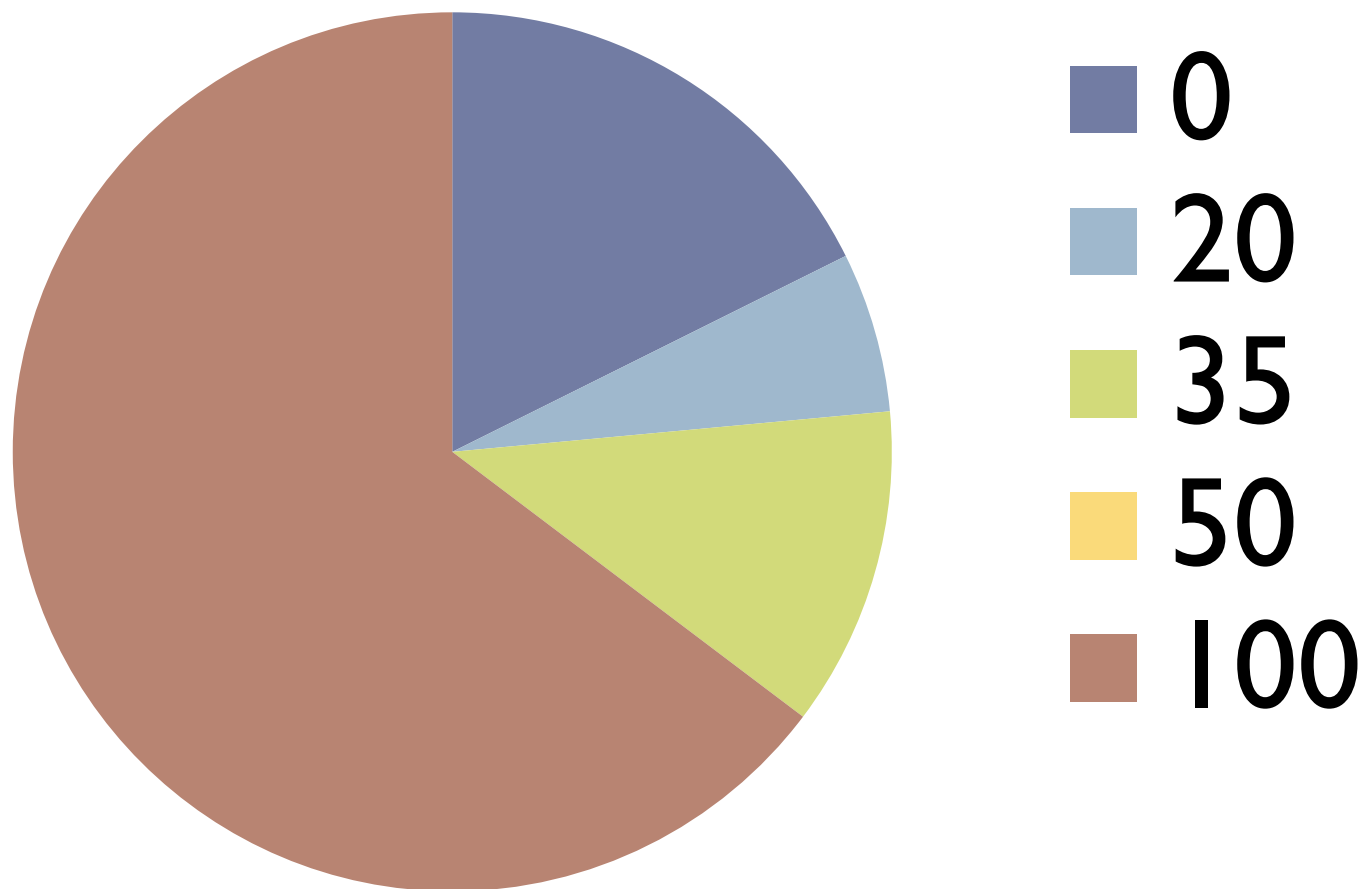


実装の工夫

- ▶ 出発時刻と到着時刻をソートする
- ▶ バスの出発と到着をイベントと考え、遅い時刻のイベントから順番に処理していく
- ▶ バス K の出発では出発バス停を A として
- ▶ $E[A] := \min(E[A], F[K])$
- ▶ バス K の到着では到着バス停を B として
- ▶ $F[K] := E[B]$
- ▶ 同時刻のイベントは出発を先にすることに注意
- ▶ これで本質的にさっきのグラフと同じことができる
- ▶ 逆向きに考えている場合は早いイベントから処理していく



得点分布



いつもと趣向を変えて円グラフにしてみました

The background is a dark gray gradient with numerous realistic water droplets of various sizes scattered across the surface. Some droplets are in sharp focus, showing highlights and reflections, while others are blurred in the background.

たのしい家庭菜園 (Growing Vegetables is Fun)

JOI Spring Camp 2014 Day-1

waf

問題概要

- IOI草が列になってたくさん (n 個) 生えている。
- IOI草は以下のどちらかの条件が満たされれば枯れずにすむ。
 - その草より左側に、その草より高い草が存在しない。
 - その草より右側に、その草より高い草が存在しない。
- うまくIOI草を並べ替えて、すべてのIOI草が枯れないようにする。
- 最小で何回の並べ替えをすれば良いか計算してください。

とりあえず全探索

- とりあえずよくわからないので**全探索**をする。
- 許される配置を列挙して、それぞれの配置の並べ替え回数を計算する。

全探索の実装を試みる

- 考えられる配置を `std::next_permutation(C++)` / 再帰関数 などを使って 適当に列挙したあと、条件を満たしているかどうか確認する。
- 端から順番に揃えていくと、無駄な並べ替えが起こらず並べ替え回数が最小となるので、並べ替えを実際にシミュレートして、並べ替え回数を計算する。
- 考えられる配置の列挙に $O(n!)$ がかかって、並べ替え回数の計算に $O(n^2)$ かかるので、全体で時間計算量 $O(n! \times n^2)$ となって小課題1までは解けそう。

許される配置を考える

- どういう配置が許されるのかもっと分かりやすい言い換えを考える。
- いちばん大きな草は最終的にどの位置にいても枯れることはない。
- 二番目に大きな草は、いちばん大きな草たちの間に挟まれなければ良い。
つまり、いちばん大きな草のうち、最も左側にある草のさらに左側か、最も右側にある草のさらに右側に配置すれば良い。
- どの草もそれより大きな草より**外側**に配置すれば良い。

わかりやすい図



問題の整理

- 整数列が与えられる。
- 隣り合った要素を入れ替えることを繰り返す。
- ある要素までは**広義単調増加**、そこから先は**広義単調減少**、となるように整数列をうまく並び替えたい。
- 最小で何回の入れ替えが必要か計算してください。

頭の良い解法？

- 初期状態の列を、ある一点で切り分けて、左側/右側のそれぞれの列でソートして、最後に連結する。
- 切り分ける点をすべて試して最小の並べ替え回数を計算する。
- 素朴に実装すると $O(n^3)$ かかるが、並べ替え回数を計算するところを高速化すれば、満点が取れそう。

というのは嘘で

- [6, 5, 1, 4, 3, 2] というテストケースに対して正しく答えることができない。
- 正しい答えは [1, 6, 5, 4, 3, 2] で 2回 となるが、先ほどの嘘解法では 3回 となってしまう。
- ちゃんと考えなおしましょう。

ちょっと頭の良い全列挙

- 最終的に許される配置のみをすべて列挙していく。
- まずいちばん高い草を中央に配置する。
- より高い草から順番に、すでに出来上がっている配置の左右どちら側に配置するか、で**全探索**していく。
- 並べ替え回数の計算のアルゴリズムは先ほどのものと一緒で良い。
- 列の列挙に $O(2^n)$ 、並べ替え回数の計算に $O(n^2)$ 、高い順に草をリストアップするのに $O(n \log n)$ にかけて、全体で $O(2^n \times n^2)$ となるので小課題 2までは解けそう。

わかりやすい図 その2



並べるときの注意点

- 同じ高さの草が**複数**ある場合を考慮する必要がある。
- 同じ高さの草たちを入れ替えるのは明らかに無駄であるから、同じ高さの草たちだけを取り出した場合、それらの中では入れ替えは必要ない。
- 同じ高さの草が M 個あるとき、初期状態で列の左端に近いものから順に L 個選んで、**順番を保ったまま**、すでに出来上がってる列の左側に挿入する。残った $M - L$ 個も、**順番を保ったまま**、すでに出来上がってる列の右側に挿入する。これをすべての L に対して試す。
- 時間計算量は変わらない。

まだまだ無駄がたくさんある

- 高い草から順番に配置しているので、ある草を配置しようとしているとき、それ以前に配置した草たちの順列は**無視**してしまっても良い？
- それ以前に配置した草たちの数が一緒ならば、そのなかで並べ替え回数が最小のもの以外の配置のしかたは、すべて切り捨ててしまっても良い？

かんたんなDP

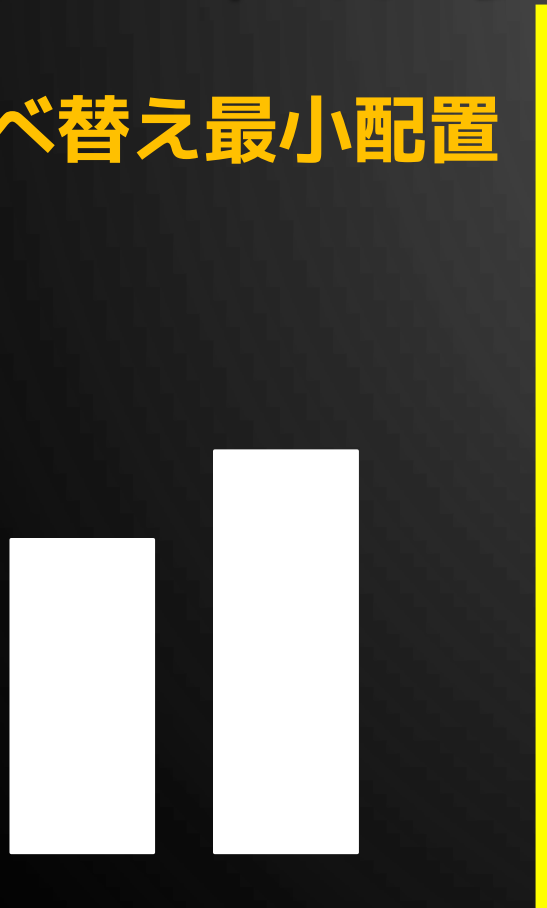
- $dp[i] := i$ 番目に大きな草まで並べたときの最小の並べ替え回数
- $a[i] := i$ 番目に大きな草を列に加えるときの最小の並べ替え回数
- $dp[i] = dp[i - 1] + a[i]$
- $a[i]$ を計算するには……

無理なら逆から並べてみる

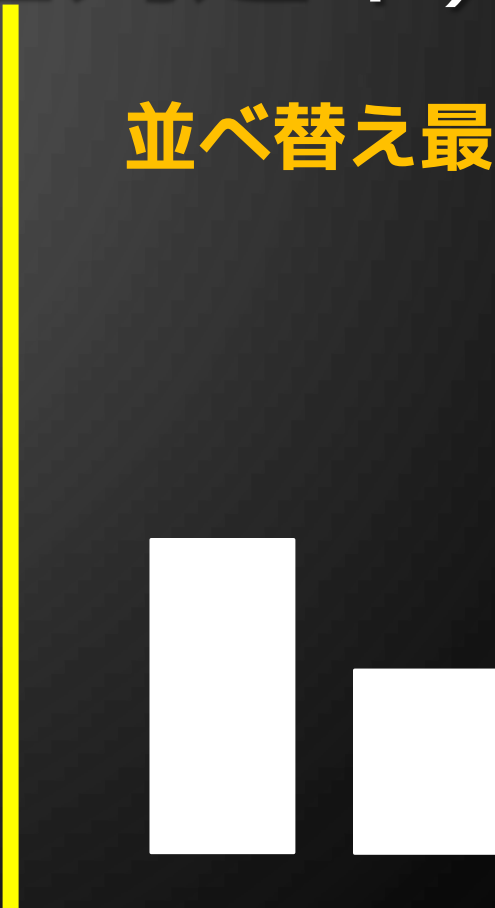
- 列が中央から出来上がっていくので並び替え回数を数えるのが難しい。
- それ以前の草の配置を無視したらダメそう。
- 高い草から順番に配置してダメなら、低い草から順番に配置してみる。
- 列が左端/右端から出来上がっていくのでうまくいきそう。
- ある草を配置しようとしているとき、それ以前に配置した低い草たちの順列は完全に無視してしまっても良い。
- 小さい草から動かして、端から列を構築していくイメージ。

わかりやすい図 その3 (整列途中)

並べ替え最小配置



並べ替え最小配置



未整列の部分



ふつうのDP

- $dp[i] := i$ 番目に小さな草まで並べたときの最小の並べ替え回数
- $a[i] := i$ 番目に小さな草を列に加えるときの最小の並べ替え回数
- $dp[i] = dp[i - 1] + a[i]$
- $a[i]$ は計算できそう。

ふつうのDPを実装する(1)

- i 番目に小さな M 個の草のなかで、初期状態で列の左端に近いものから順に L 個選んで、順番を保ったまま、左側に出来上がっている列の末尾に挿入する。
- 残った $M - L$ 個も、順番を保ったまま、右側に出来上がっている列の末尾に挿入する。
- すべての L について試して最小の入れ替え回数となるものを計算する。

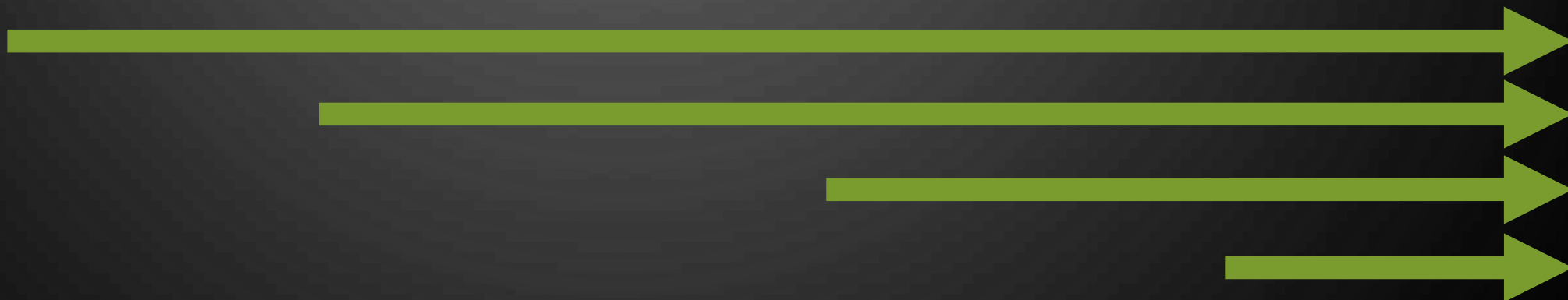
ふつうのDPを実装する (2)

- 高さが h で初期位置が x であるような草を左端の列に付け足すのに必要な入れ替え回数は、初期位置が $0, 1, 2, \dots, x - 1$ であるような草のなかで高さが h より高いものの個数と**一致**する。右端にもってくる場合も同様。
- 素朴に計算すると、各 L について $O(Mn)$ かかるので、ひとつの高さについて $O(M^2n)$ かかる。全体として $O(n^3)$ になる。
- 小課題3が解けない。つらい。

すこし高速化する

- 高さ h である M 個の草によって分割される $M + 1$ 個の区間について考える。
- M 個すべての草を右側に配置する場合は、右側から $i + 1$ 番目の区間を i 個の草が通過する。このときの入れ替え回数は、各区間に存在する高さ h 以上の草の個数を数え上げることで $O(n)$ で計算できる。
- $L + 1$ 個の草を左側に配置する場合の入れ替え回数は、(L 個の場合の入れ替え回数) - ($L + 1$ 個目の草を右に配置するための入れ替え回数) + (左に配置するための入れ替え回数) と等しく、これは累積和を用いて、前計算に $O(n)$ かけることで $O(1)$ で計算できる。

わかりやすい図 その4 (すべて右端)



わかりやすい図 その5 (1つ左に移す)



ふつうよりちょっと速いDP

- 高い順に草をリストアップするのに $O(n \log n)$ かかり、同じ高さの M 個の草についてまとめて $O(n)$ で計算できる。全体として $O(n^2)$ で計算できるので、小課題3までは解けるようになった。
- しかし小課題4を解くにはまだまだ遅い。

もっともっと高速化する

- 各区間に存在する高さ h 以上の草の個数を数えるところが遅い。
- この操作を高速に行えるデータ構造さえあれば……
- ん？ 区間に対する高速な操作？

Segment Tree

- JOI界では定番のデータ構造です。
- いろいろと応用が効く。
- 列に対する操作を高速化したいときに使えることがたまにある。
- ただし、慣れていないと実装が面倒である。

BITで実装する

- サイズ n のBITを用意する。まず、それぞれの草を初期位置に置いて、BITの各要素と対応付ける。BITの各要素を1で初期化する。
- さきほどのDP解と同じように小さな草から順番にすべてを見ていく。
- 草を順番に見ていくとき、ついでにいま注目している高さの草に対応する要素をすべて0にする操作を行っておく。こうすることで、いま注目している草より高い草に対応するBITの要素のみ1である、という状態が**常に保たれる**。
- ある区間のなかにある、いま注目している草より高い草の個数は、BITのsumを用いることで効率的に求められる。

高速化の評価

- それぞれの草に対応する要素を順番に0にしていくので、全体として更新操作は $O(n \log n)$ で行える。
- 同じ高さの M 個の草について注目しているとき、すべて右側に配置した場合の回数を計算するのに $M + 1$ 回、ひとつ左側に移し替えるのに 2回、sumが行われるので、全体として総和計算操作は $O(n \log n)$ で行える。
- 低い順に草をリストアップするのも $O(n \log n)$ で行える。
- 全体として $O(n \log n)$ で計算できるので、すべての小課題が解けそう。

歴史の研究 (Historical Research)

解説

秀 郁未(tozangezan)

概要

- 下図のように日付と出来事の対応が与えられる。

日付	1	2	3	4	5	6	7	8	
出来事	9	9	19	9	9	15	9	19	

- さらにクエリが与えられる。
- 各クエリは「(出来事の種類)*(範囲における個数)」という重要度の最大値を求める。
- $N \leq 100000$, $Q \leq 100000$, 種類の整数 $\leq 10^9$

例

- クエリの範囲が1日目～4日目

日付	1	2	3	4	5	6	7	8	
出来事	9	9	19	9	9	15	9	19	27

- $9 * 3 = 27$
- $15 * 0 = 0$
- $19 * 1 = 19$
- 27を出力

例

- クエリの範囲が3日目～6日目

日付	1	2	3	4	5	6	7	8		
出来事	9	9	19	9	9	15	9	19		19

- $9 * 2 = 18$
- $15 * 1 = 15$
- $19 * 1 = 19$
- 19を出力

例

- クエリの範囲が2日目～8日目

日付	1	2	3	4	5	6	7	8		
出来事	9	9	19	9	9	15	9	19	38	

- $9 * 4 = 36$
- $15 * 1 = 15$
- $19 * 2 = 38$
- 38を出力

例

- クエリの範囲が1日目～8日目

日付	1	2	3	4	5	6	7	8		
出来事	9	9	19	9	9	15	9	19	45	

- $9 * 5 = 45$
- $15 * 1 = 15$
- $19 * 2 = 38$
- 45を出力

ありふれた解法(小課題1)

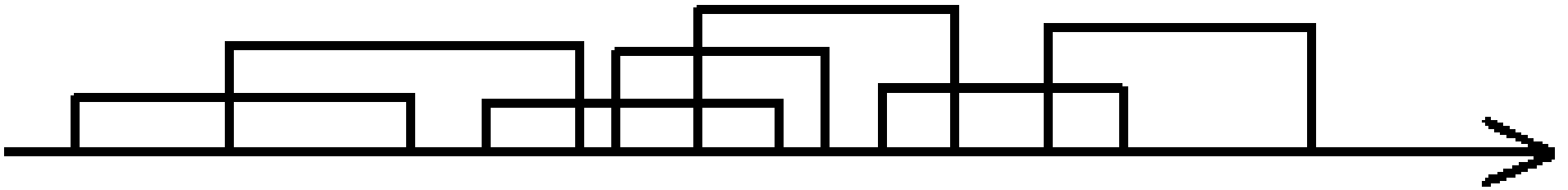
- 範囲の中にない種類の重要度は全部0
 - 範囲の中にある種類すべてに対して個数を数える
 - 最大値を求める
 - オーバーフローには注意
-
- $O(QN^2)$ $Q \leq 100, N \leq 100$ は通る。5点
 - おそらく今日の問題の中で最も簡単な部分点です
 - ちゃんと取りましたか?????

さっきの改善(小課題2)

- 結構さまざまな解法があると思います
- 累積和
 - 各数字について累積和を取る
 - クエリは各数字について範囲の和を求めてmaxをとる
 - $O(N(Q+N))$
- 二分探索
 - 各数字について出現場所をvectorとかに入れる
 - クエリごとにupper_boundとlower_boundで数える
 - $O(QN \log N)$
- 10点

特殊なケース(小課題3)

- クエリが互いに他を含まない。
- つまりこんな感じ



特殊なケース(小課題3)

- 矢印が書いてありますね…
- 左端から処理していこう。
- 尺取的に左端(開始日)と右端(終了日)を動かすとうまくいくだろう。
- 各段階で最大値を高速に取得できれば良い
- →ん？

特殊なケース(小課題3)

- Segment Tree
 - 詳しくは <http://www.slideshare.net/iwiwi/ss-3578491>
 - 今回は更新と最大値を求めるだけの単純なやつ
- 「出来事の種類」は高々N種類しかない
- 座標圧縮をしよう
- まずクエリを開始or終了が早い順にソート
- 座標圧縮→尺取(ここでSegment Treeを使う)でそれぞれのクエリに対する最大値を求める+更新
- $O((N+Q) \log (N+Q))$ **25点**

満点解法

- このままSegment Treeだけ使っていけるだろうか
... → いけなさそう...
- 何か変な制約があるだろうか ... → なさそう...
- じゃあ、データ構造で(半ば強引に)改善しよう
- 何を使おうかな??
 - なんとたら木 → 絶対やばい
 - 複雑なSegment Tree → 無理そうだとさっき言った

満点解法

- このままSegment Treeだけ使っていけるだろうか
… → いけなさそう…
- 何か変な制約があるだろうか … → なさそう…
- じゃあ、データ構造で(半ば強引に)改善しよう
- 何を使おうかな??
 - なんとら木 → 絶対やばい
 - 複雑なSegment Tree → 無理そうだとさっき言った
- **バ ケ ッ ト 法**

満点解法

- とりあえずバケットのサイズは B として後で決めることにしよう。
- バケット法による解法はいくつかあると思います。
- そのうち1つを紹介します。Segtree使いません。
- 一応、時間だけでなく空間計算量も考えることにしましょう。
- (バケット法の問題だとメモリ量ちゃんと考える必要があることも)

前処理①

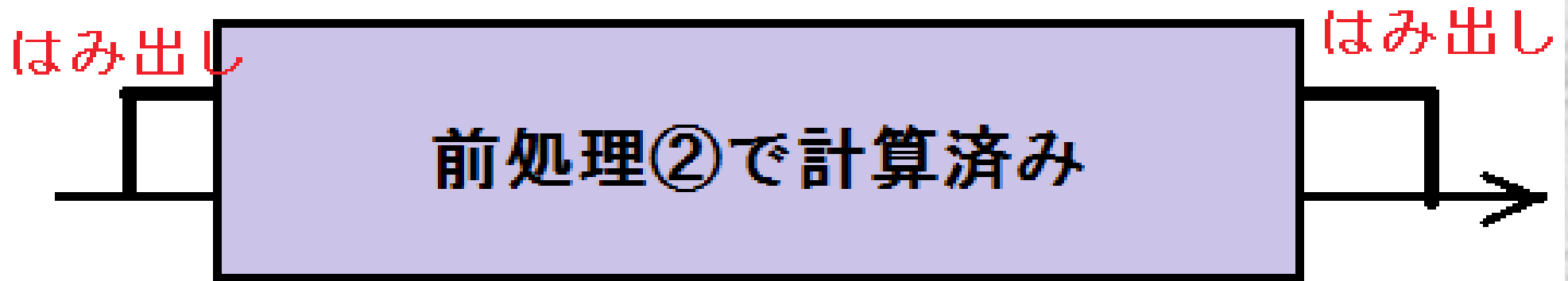
- 各数字について出現位置を昇順でvectorとかにつめておく。
- (座標圧縮が必要)
- ここの処理は 時間 $O(N \log N)$, 空間 $O(N)$

前処理②

- $(i*B+1)$ 日目から $j*B$ 日目 ($0 \leq i < j \leq N/B$)における「重要度の最大値」を求める
- この処理は、まずそれぞれの i に対してここを最初の日としたときの種類ごとの和を配列にもって、さらに最大値も持っておけばそれぞれの i に対し合計で時間 $O(N)$ ($j = i+1, \dots, N/B$ を一遍に計算)
- i の選び方は $O(N/B)$
- 時間 $O(N^2/B)$ 空間 $O(N^2/B^2)$

本処理

- クエリの範囲は、下図のように分けられる。



- 前処理②で計算済みの最大値を仮の答えとする
- それを越える答えがあるとしたら、その値に関する「種類」のものがはみ出した部分にある

本処理

- はみ出した部分のそれぞれの種類に対して、前処理①で作った出現位置のものを利用して upper_bound と lower_bound で個数を数える
- 前ページのものともあわせて最大値が答え。
- 時間 $O(QB \log N)$

計算量

- 前処理①

時間 $O(N \log N)$ 空間 $O(N)$

- 前処理②

時間 $O(N^2/B)$ 空間 $O(N^2/B^2)$

- 本処理

時間 $O(QB \log N)$

- 合計

時間 $O((N+QB) \log N + N^2/B)$ 空間 $O(N + N^2/B^2)$

- $B=100$ 程度で上手く間に合うものができる

ほかの解法

- 平方分割+Segment Tree(1箇所updateする,区間のmaxを求める)でも解けます
- $O(N * (N \log N)^{1.5})$ 程度？
- 何にせよこういう問題でTLEするときはバケットサイズをいろいろ試して送ることが大事

omake

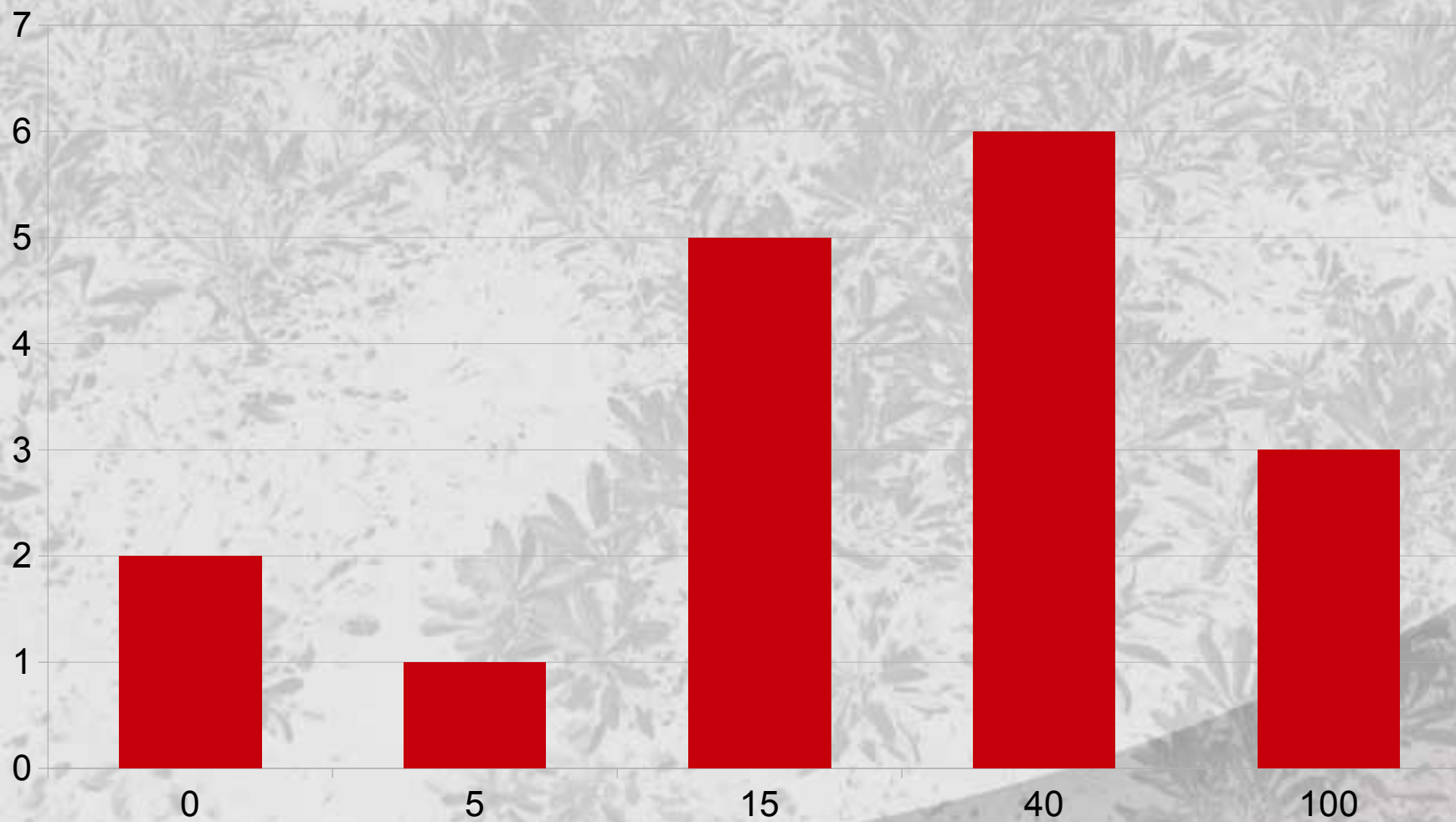
- なんか定数倍がきついらしい？
- バケット法とかの問題は基本的に定数倍が厳しく見えがちです。
- 定数倍改善力や謎エスパー力や嘘解法力がほしいあなたのために最適の練習環境をお教えします。(JOIで使える保障は無いです)

omake

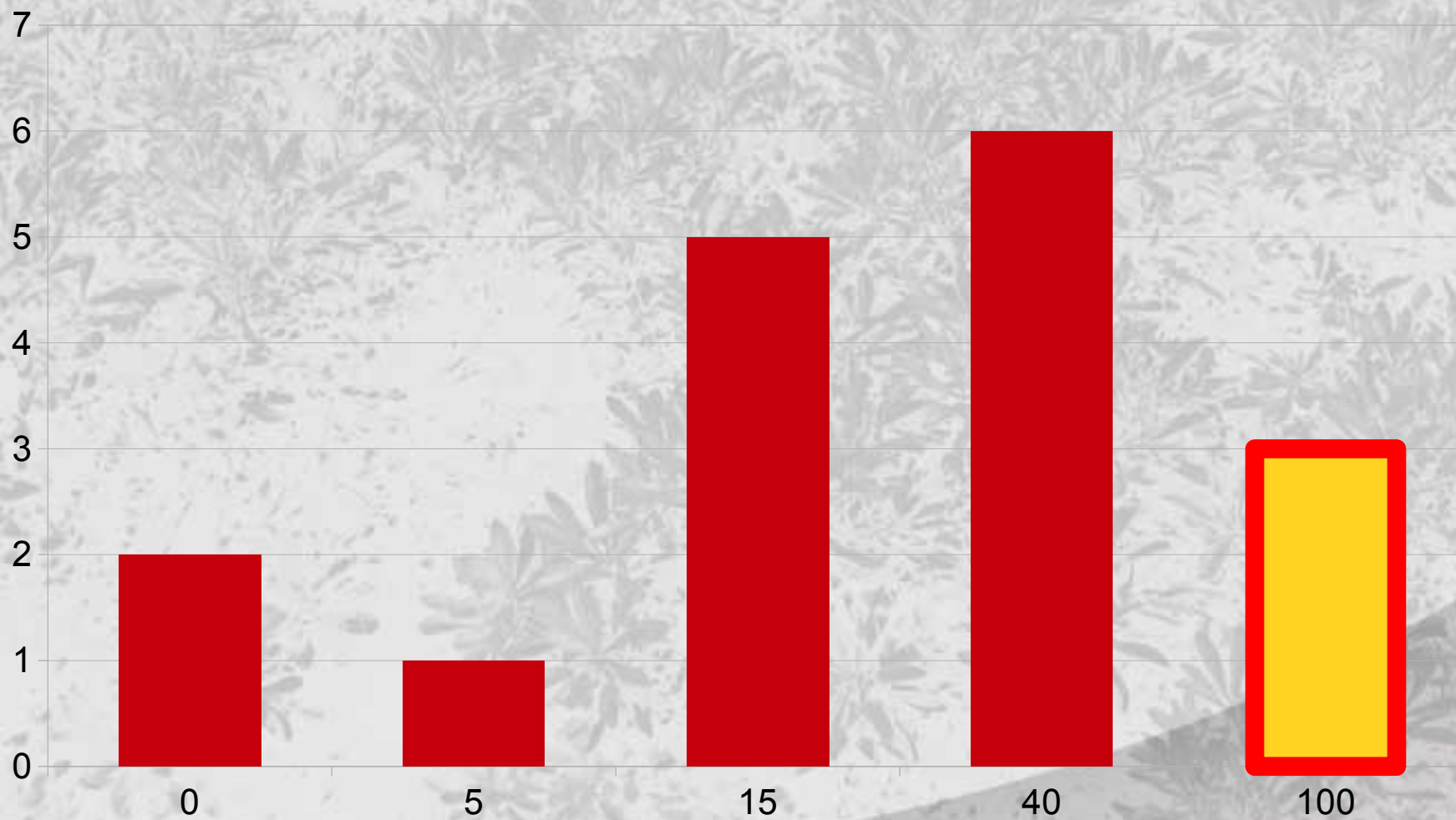
- なんか定数倍がきついらしい？
- バケット法とかの問題は基本的に定数倍が厳しく見えがちです。
- 定数倍改善力や謎エスパー力や嘘解法力がほしいあなたのために最適の練習環境をお教えします。(JOIで使える保障は無いです)

poj.org

得点分布



得点分布



Ramen 解説

2014/03/20 情報オリンピック春合宿にて

MASAKI HARA



問題概要

問題概要

ラーメン屋が N 軒ある



問題概要

ラーメン屋が N 軒ある

「こってり度」が定まっている

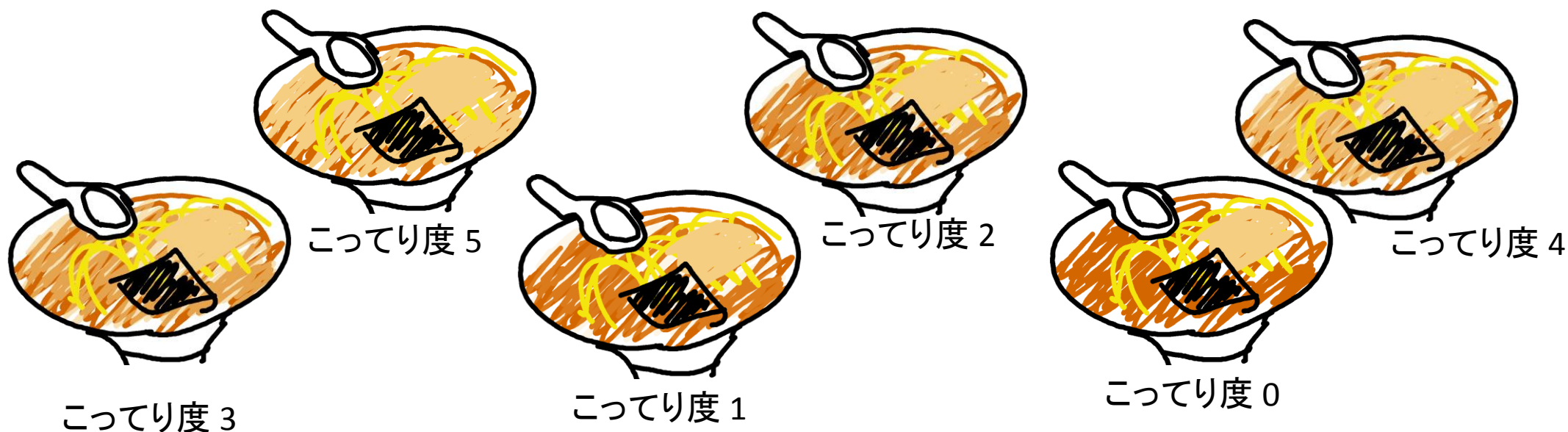


問題概要

ラーメン屋が N 軒ある

「こってり度」が定まっている

こってり度は互いに異なる



問題概要

JOI君はあっさりしたラーメンが好き



問題概要

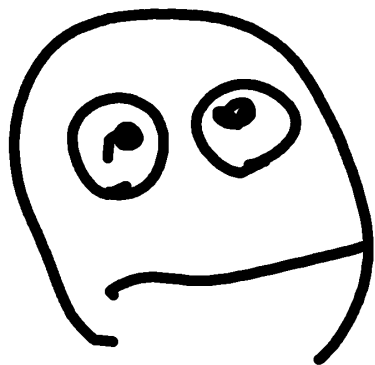
JOI君はあっさりしたラーメンが好き

IOIちゃんはこってりしたラーメンが好き



問題概要

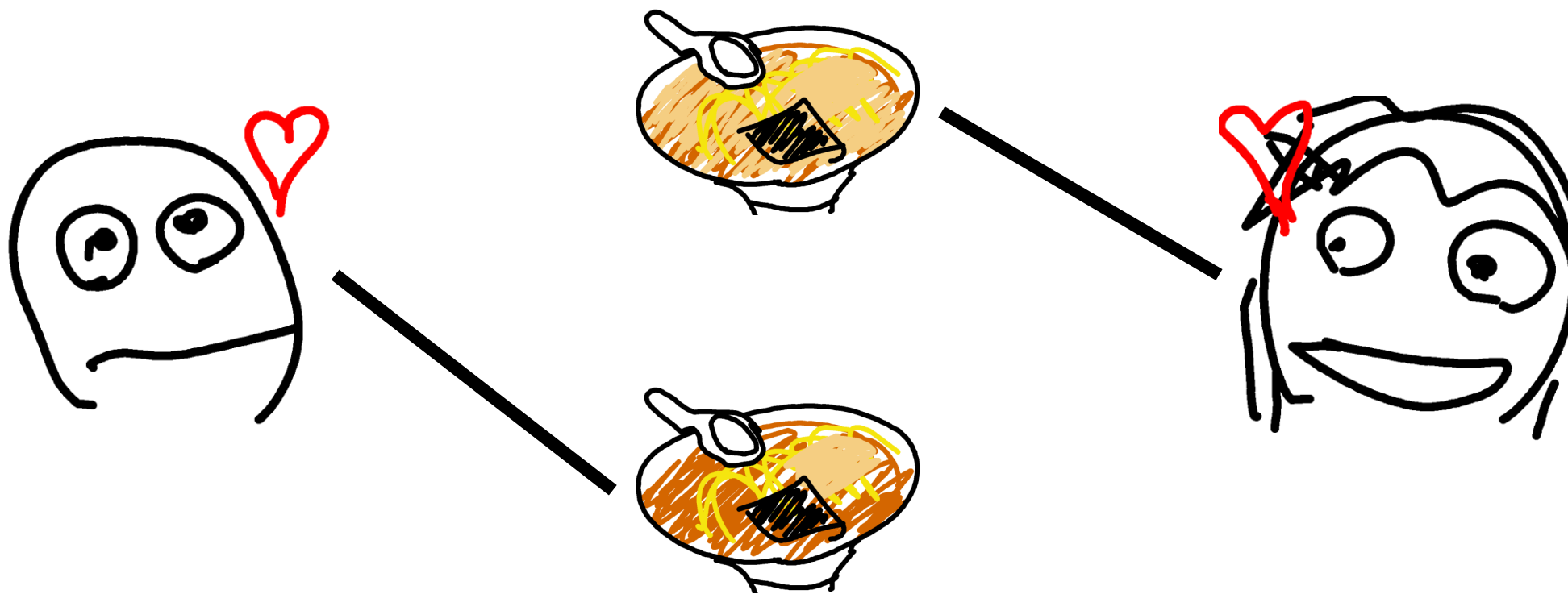
1日一回、食べ比べを行う



問題概要

1日一回、食べ比べを行う

二つのラーメン屋のこってり度の大小関係がわかる



問題概要

健康のため、食べ比べは**600日**より多くは行えない

問題概要

健康のため、食べ比べは**600日**より多くは行えない

問題概要

健康のため、食べ比べは**600日**より多くは行えない

一番あっさりしたラーメンと一番こってりしたラーメンを決めたい

小課題1 (20点)

$N \leq 30$

小課題1 (20点)

$$N \leq 30$$

$$\text{ラーメン屋の組み合わせは} \frac{N \times (N - 1)}{2} \leq 435$$

小課題1 (20点)

$$N \leq 30$$

ラーメン屋の組み合わせは $\frac{N \times (N - 1)}{2} \leq 435$

すべての組み合わせについてCompareを呼び出しても間に合う

小課題2 (累計50点)

$N \leq 300$

小課題2 (累計50点)

$N \leq 300$

こってり度最大を300手で決定できればよい

小課題2 (累計50点)

$N \leq 300$

こってり度最大を300手で決定できればよい (最小も同様にできるので)

小課題2 (累計50点)

$N \leq 300$

こってり度最大を300手で決定できればよい (最小も同様にできるので)

普通の最大値決定アルゴリズムでOK

小課題2 (累計50点)

普通の最大値決定アルゴリズム

小課題2 (累計50点)

普通の最大値決定アルゴリズム



小課題2 (累計50点)

普通の最大値決定アルゴリズム

暫定的な最大を決める

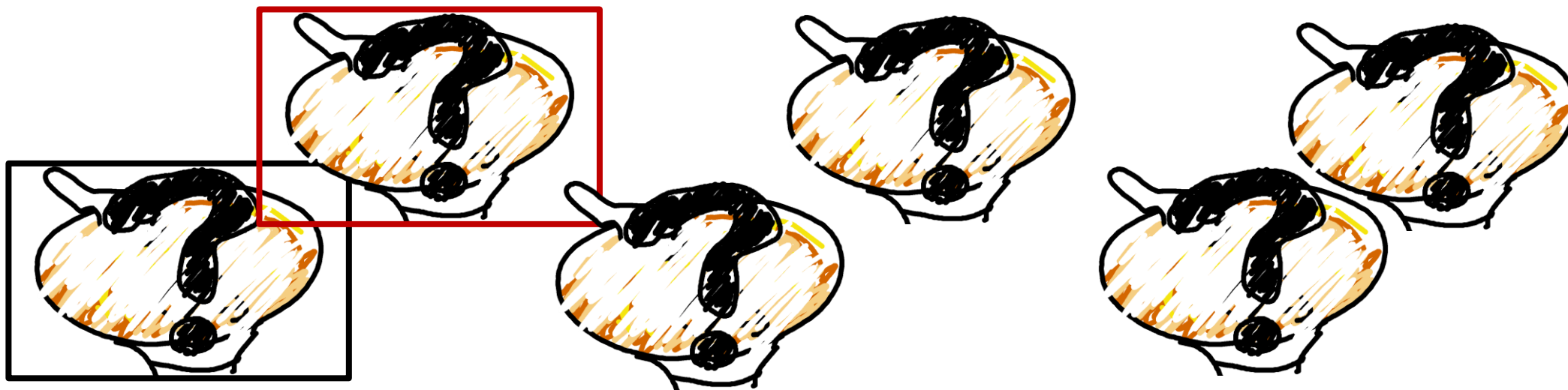


暫定的な最大

小課題2 (累計50点)

普通の最大値決定アルゴリズム

比較する

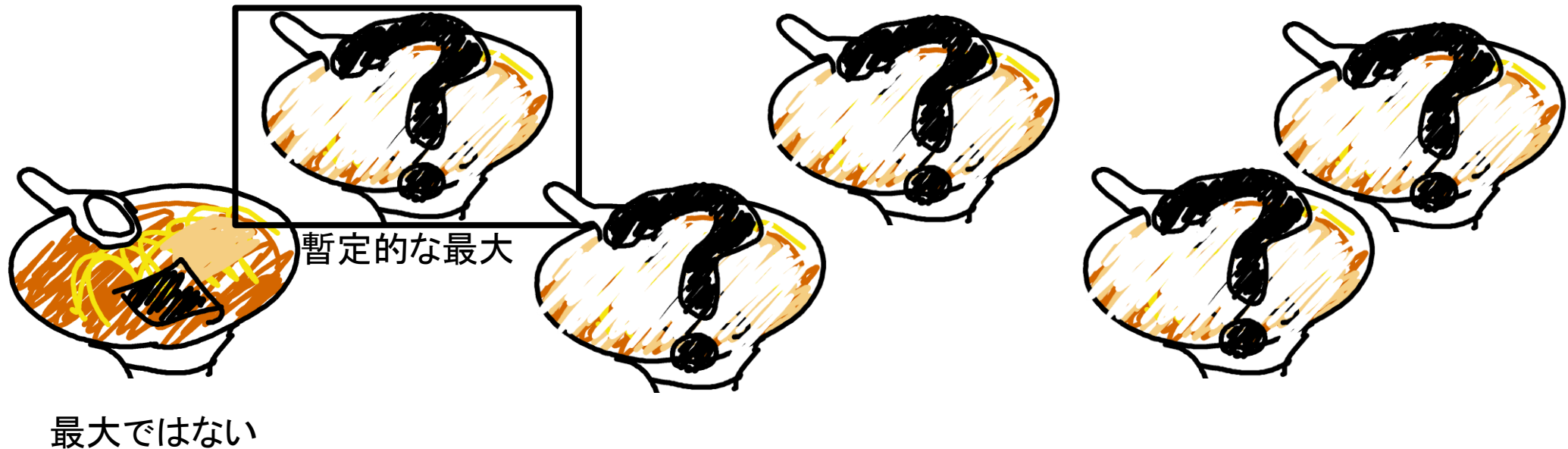


暫定的な最大

小課題2 (累計50点)

普通の最大値決定アルゴリズム

比較する→低いほうはリタイア



小課題2 (累計50点)

普通の最大値決定アルゴリズム

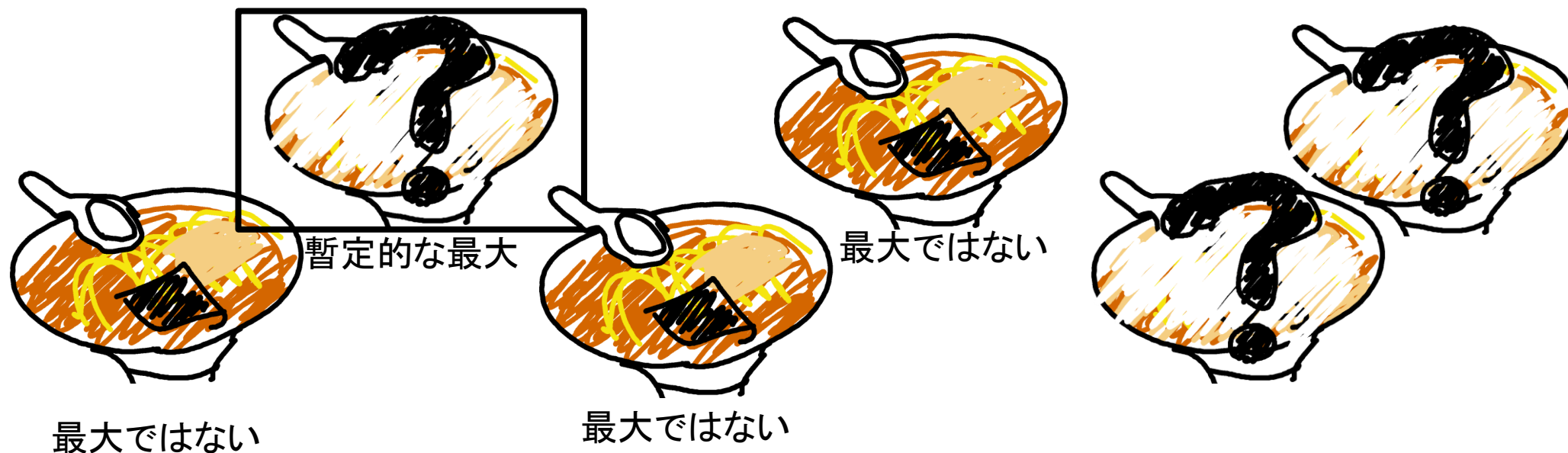
比較する→低いほうはリタイア



小課題2 (累計50点)

普通の最大値決定アルゴリズム

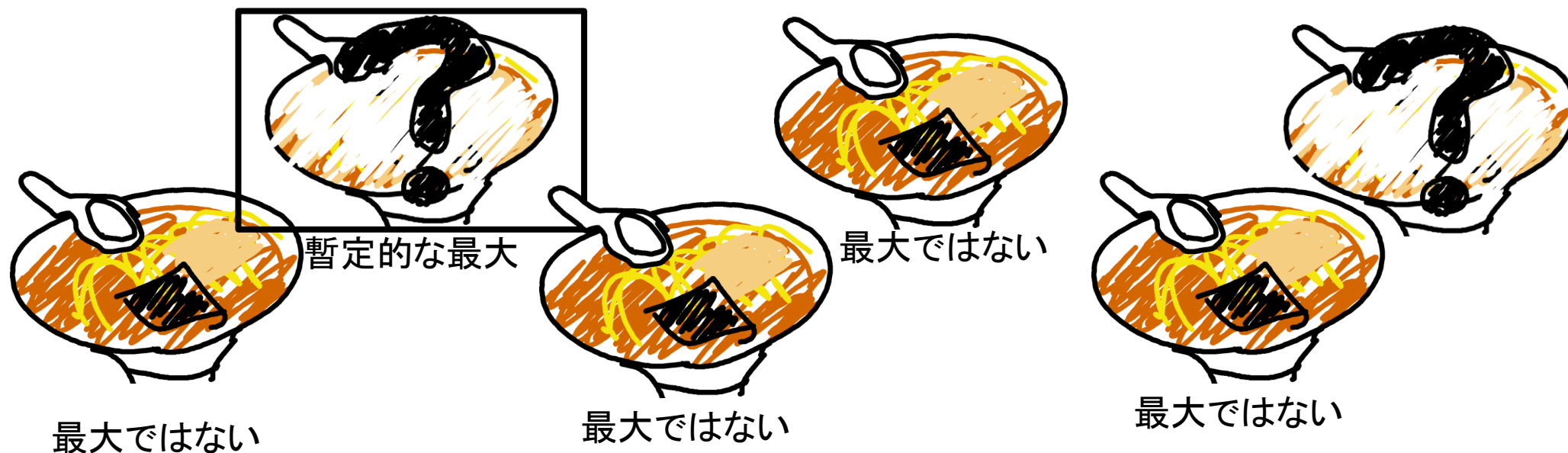
比較する→低いほうはリタイヤ



小課題2 (累計50点)

普通の最大値決定アルゴリズム

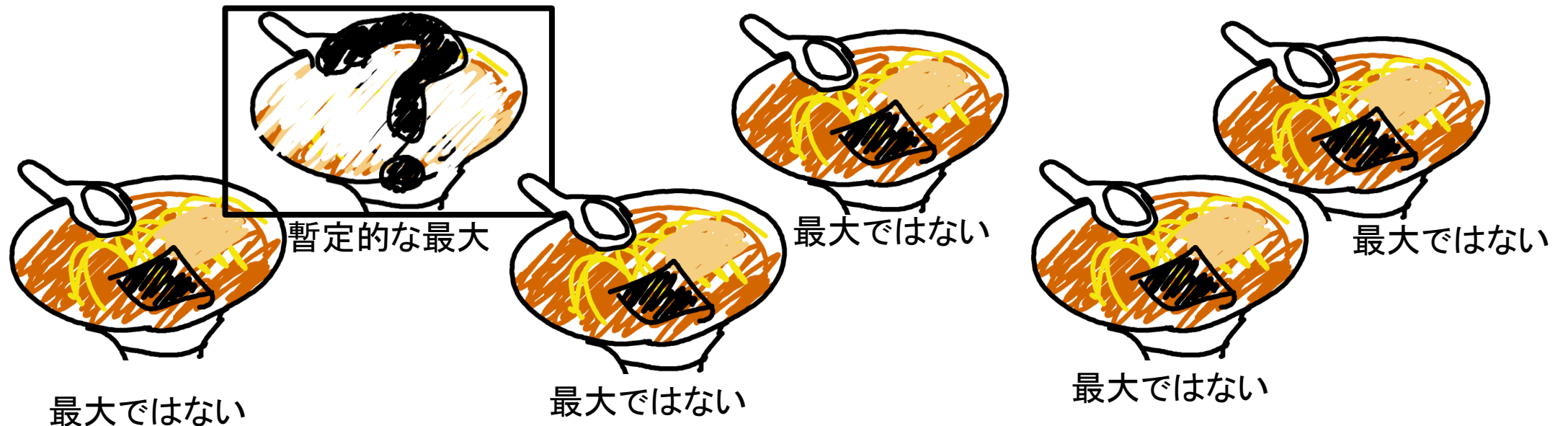
比較する→低いほうはリタイヤ



小課題2 (累計50点)

普通の最大値決定アルゴリズム

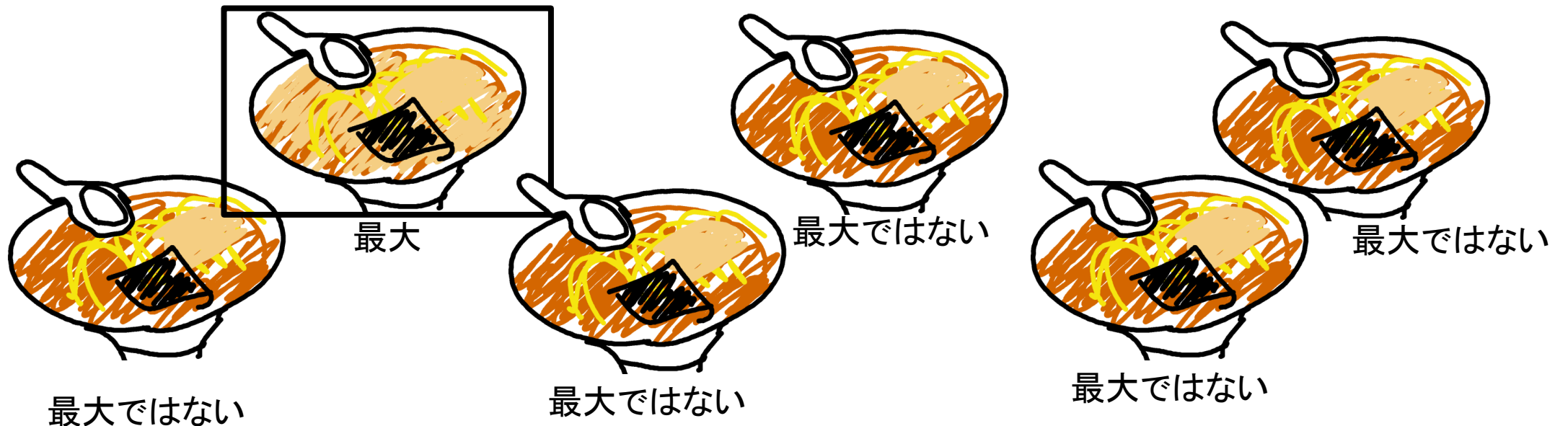
比較する→低いほうはリタイヤ



小課題2 (累計50点)

普通の最大値決定アルゴリズム

比較する→低いほうはリタイヤ



小課題2 (累計50点)

最大値を決めるのに $N - 1$ 回の比較でできる

小課題2 (累計50点)

最大値を決めるのに $N - 1$ 回の比較でできる

最大値と最小値を決めるのに $2(N - 1) \leq 598$ 回でできる

小課題3 (累計100点)

$N \leq 400$

小課題3 (累計100点)

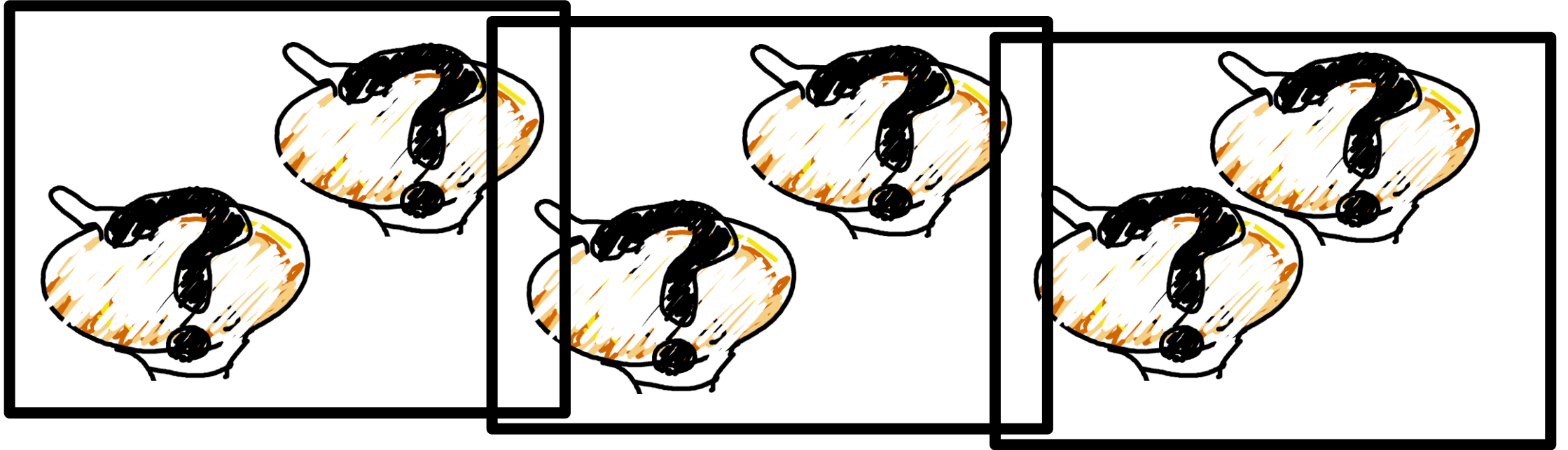
$N \leq 400$



小課題3 (累計100点)

$N \leq 400$

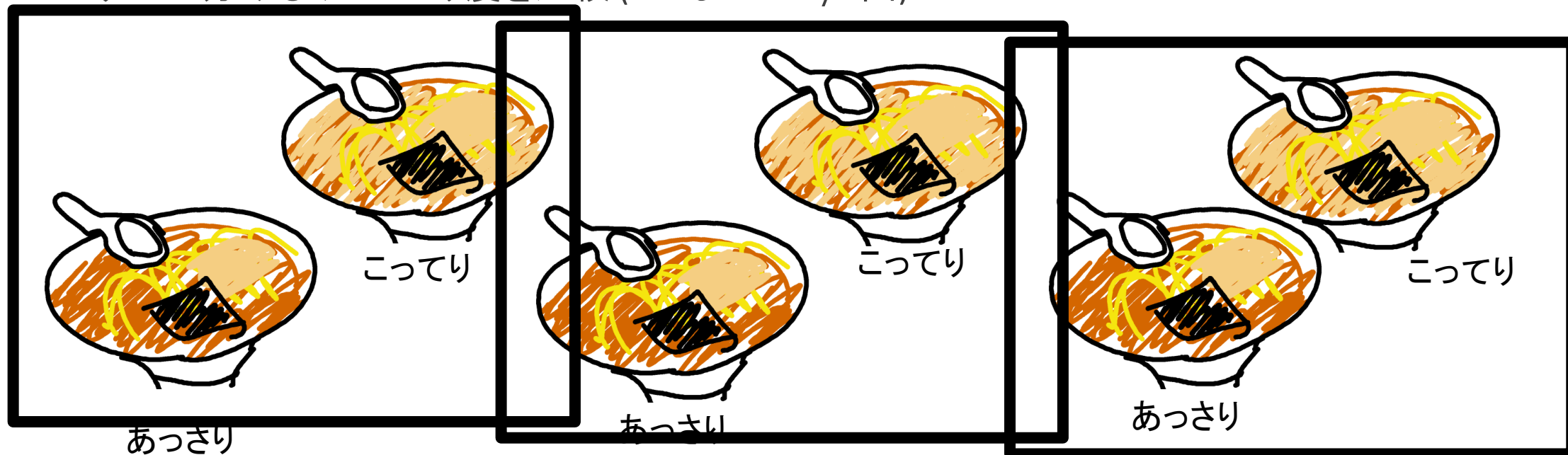
2つずつに分ける



小課題3 (累計100点)

$N \leq 400$

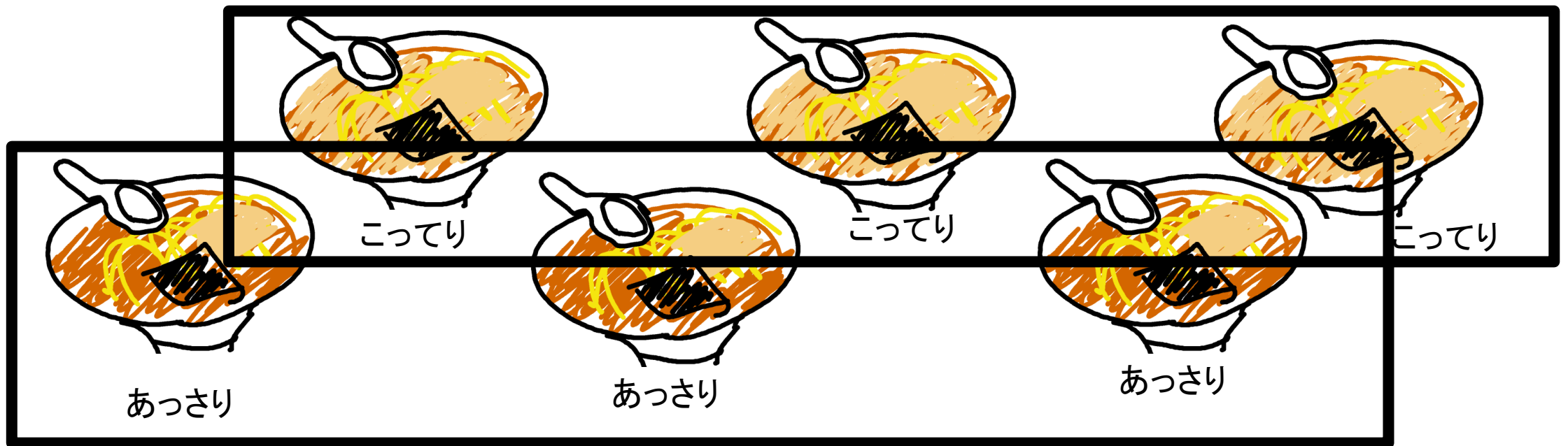
2つずつに分ける→こってり度を比較 (ここまでで $N/2$ 回)



小課題3 (累計100点)

$N \leq 400$

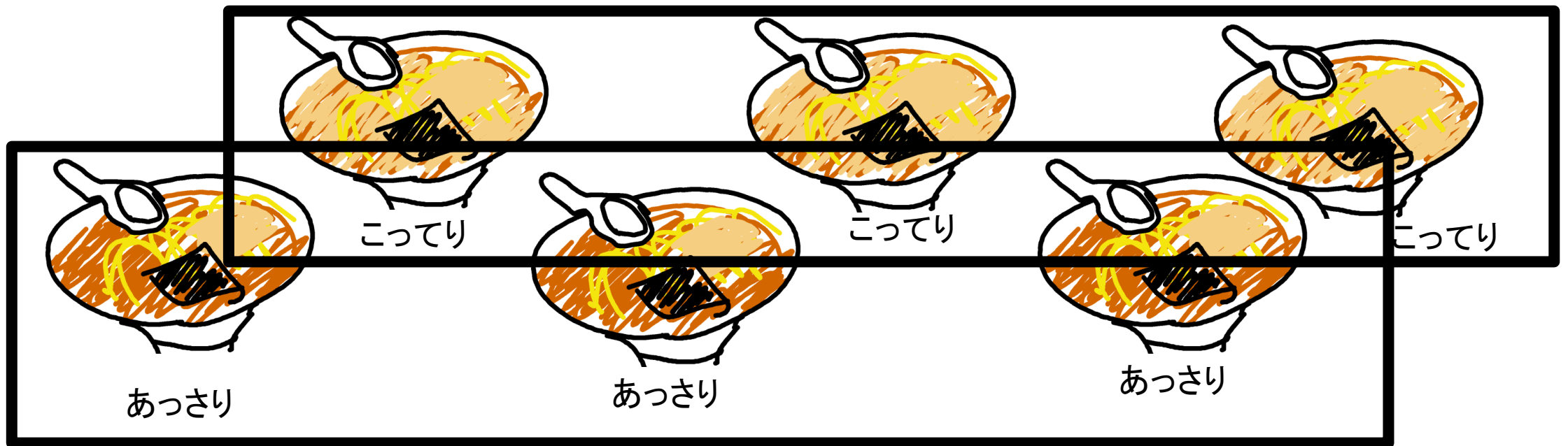
こってりグループとあっさりグループに分ける



小課題3 (累計100点)

$N \leq 400$

こってりグループとあっさりグループに分ける



小課題3 (累計100点)

$N \leq 400$

こってりグループ→こってり度最大を見つける



小課題3 (累計100点)

$N \leq 400$

こってりグループ→こってり度最大を見つける (ここで $N/2$ 回)



こってり度最大



小課題3 (累計100点)

$N \leq 400$

あっさりグループ→



小課題3 (累計100点)

$N \leq 400$

あっさりグループ→こってり度最小を見つける(ここで $N/2$ 回)



小課題3 (累計100点)

$N \leq 400$

小課題3 (累計100点)

$N \leq 400$

最初の分類に $\left\lfloor \frac{N}{2} \right\rfloor$ 回

小課題3 (累計100点)

$N \leq 400$

最初の分類に $\left\lfloor \frac{N}{2} \right\rfloor$ 回

こってり度最大を決めるのに $\left\lceil \frac{N}{2} \right\rceil - 1$ 回

小課題3 (累計100点)

$N \leq 400$

最初の分類に $\left\lfloor \frac{N}{2} \right\rfloor$ 回

こってり度最大を決めるのに $\left\lceil \frac{N}{2} \right\rceil - 1$ 回、こってり度最小を決めるのに $\left\lceil \frac{N}{2} \right\rceil - 1$ 回

小課題3 (累計100点)

$N \leq 400$

最初の分類に $\left\lfloor \frac{N}{2} \right\rfloor$ 回

こってり度最大を決めるのに $\left\lceil \frac{N}{2} \right\rceil - 1$ 回、こってり度最小を決めるのに $\left\lceil \frac{N}{2} \right\rceil - 1$ 回

合計 $\left\lceil \frac{3N}{2} \right\rceil - 2 \leq 598$ 回の比較でできる

コーナーケース

コーナーケース

小課題3を実装すると、はじめに `Compare(0, 1)` を呼び出すコードになることがある

コーナーケース

小課題3を実装すると、はじめに `Compare(0, 1)` を呼び出すコードになることがある

→ $N = 1$ に注意

下界の保証

比較回数は $\left\lceil \frac{3N}{2} \right\rceil - 2$ 回以上必要

下界の保証

比較回数は $\left\lceil \frac{3N}{2} \right\rceil - 2$ 回以上必要

「最大の可能性が残っているラーメン屋」の個数を K とおく

「最小の可能性が残っているラーメン屋」の個数を L とおく

下界の保証

比較回数は $\left\lceil \frac{3N}{2} \right\rceil - 2$ 回以上必要

「最大の可能性が残っているラーメン屋」の個数を K とおく

「最小の可能性が残っているラーメン屋」の個数を L とおく

「最大の可能性も最小の可能性も残っているラーメン屋」の個数を M とおく

下界の保証

比較回数は $\left\lceil \frac{3N}{2} \right\rceil - 2$ 回以上必要

「最大の可能性が残っているラーメン屋」の個数を K とおく

「最小の可能性が残っているラーメン屋」の個数を L とおく

「最大の可能性も最小の可能性も残っているラーメン屋」の個数を M とおく

$V = K + L - \frac{M}{2}$ とおく。

下界の保証

補題1: V の初期値は $\frac{3N}{2}$ である。

補題2: $N > 1$ のとき、最大と最小が決まった時点での V の値は 2 である。

補題2': $N = 1$ のとき、最大と最小が決まった時点での V の値は $\frac{3}{2}$ である。

下界の保証

補題1: V の初期値は $\frac{3N}{2}$ である。

補題2: $N > 1$ のとき、最大と最小が決まった時点での V の値は 2 である。

補題2': $N = 1$ のとき、最大と最小が決まった時点での V の値は $\frac{3}{2}$ である。

補題3: 初期状態から終了状態までの間に、 V の値は $\frac{3N}{2} - 2$ 以上減少する。

下界の保証

補題4: $\text{Compare}(X, Y)$ をどのように呼び出しても、 V の減少が 1 以下であるような結果が返ってくる可能性がある。

下界の保証

補題4は場合分けで証明できる(長いので適当に)

下界の保証

x, y の両方が、「最小かもしれないし、最大かもしれない」とき:

下界の保証

x, y の両方が、「最小かもしれないし、最大かもしれない」とき:

x と y の大小関係が決まると、 K と L は1ずつ減少し、 M は2減少する。

したがって、 V は1減少する。

下界の保証

Xは、「最小かもしれないし、最大かもしれない」が、Yは「最小ではないが、最大かもしれない」とき:

下界の保証

X は、「最小かもしれないし、最大かもしれない」が、 Y は「最小ではないが、最大かもしれない」とき:

$X < Y$ となる可能性がある。このとき、 K は 1 減少するが、 L は減少しない。

M は 1 減少する。したがって、 V は $\frac{1}{2}$ 減少する。

下界の保証

X は、「最小かもしれないし、最大かもしれない」が、 Y は「最小ではないが、最大かもしれない」とき:

$X < Y$ となる可能性がある。このとき、 K は 1 減少するが、 L は減少しない。

M は 1 減少する。したがって、 V は $\frac{1}{2}$ 減少する。

X は、「最小かもしれないし、最大かもしれない」が、 Y は「最小でも最大でもない」とき.....も同様。

下界の保証

X は、「最小かもしれないし、最大かもしれない」が、 Y は「最小ではないが、最大かもしれない」とき:

$X < Y$ となる可能性がある。このとき、 K は 1 減少するが、 L は減少しない。

M は 1 減少する。したがって、 V は $\frac{1}{2}$ 減少する。

X は、「最小かもしれないし、最大かもしれない」が、 Y は「最小でも最大でもない」とき.....も同様。

X と Y , 最大と最小を逆にしても同様。

下界の保証

XもYも「最大かもしれないが、最小ではない」とき:

下界の保証

x も y も「最大かもしれないが、最小ではない」とき:

x と y の大小関係が決まると、 K は 1 減少する。

したがって、 V は 1 減少する。

下界の保証

x も y も「最大かもしれないが、最小ではない」とき:

x と y の大小関係が決まると、 K は 1 減少する。

したがって、 V は 1 減少する。

最大と最小を逆にしても同様。

下界の保証

X は「最小かもしれないが、最大ではない」のに対し、 Y は「最大かもしれないが、最小ではない」とき:

下界の保証

X は「最小かもしれないが、最大ではない」のに対し、 Y は「最大かもしれないが、最小ではない」とき:

$X < Y$ となる可能性がある。このとき、 V は変化しない。

下界の保証

X は「最小かもしれないが、最大ではない」のに対し、 Y は「最大かもしれないが、最小ではない」とき:

$X < Y$ となる可能性がある。このとき、 V は変化しない。

最大と最小を逆にしても同様。

下界の保証

X は「最小かもしれないが、最大ではない」のに対し、 Y は「最大でも最小でもない」とき:

下界の保証

X は「最小かもしれないが、最大ではない」のに対し、 Y は「最大でも最小でもない」とき:

$X < Y$ となる可能性がある。このとき、 V は変化しない。

下界の保証

X は「最小かもしれないが、最大ではない」のに対し、 Y は「最大でも最小でもない」とき:

$X < Y$ となる可能性がある。このとき、 V は変化しない。

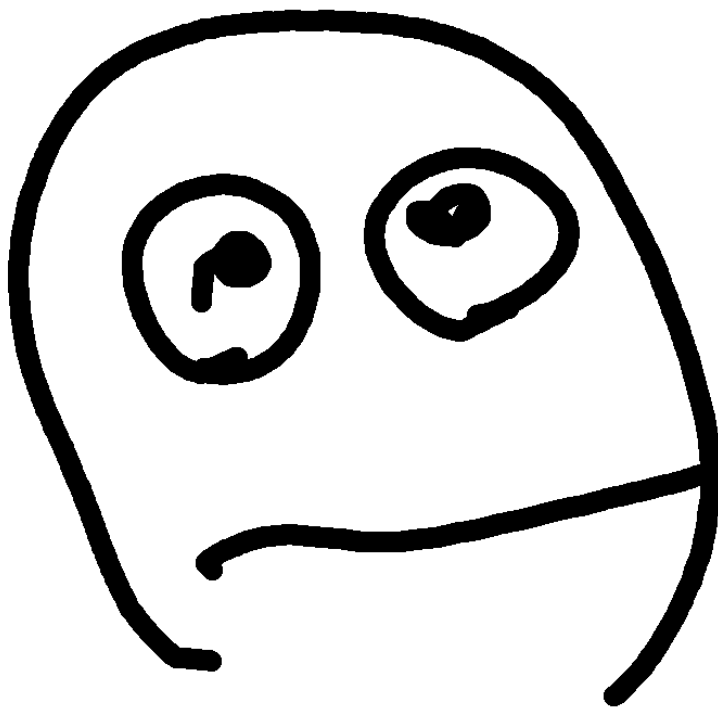
X と Y , 最大と最小を逆にしても同様。

下界の保証

XもYも「最大でも最小でもない」とき:

下界の保証

XもYも「最大でも最小でもない」とき:



下界の保証

以上の考察より、 V の減少量が 1 に満たないようなCompare呼び出しをたくさん行うプログラムはダメだということもわかる

別解

別解：小課題1

別解：小課題1

std::sortの比較関数としてCompareを用いる

別解: 小課題1

std::sortの比較関数としてCompareを用いる

```
bool cmp(int a, int b) {  
    return a != b && Compare(a, b) < 0;  
}
```

別解: 小課題1

std::sortの比較関数としてCompareを用いる

→std::sortの比較回数は $O(n \log n)$ なので普通はOK

別解: 小課題1

std::sortの比較関数としてCompareを用いる

→std::sortの比較回数は $O(n \log n)$ なので普通はOK

(オーダーの係数に関する保証はない / C++11以前は最悪計算量は保証されていない)

別解：小課題2

別解: 小課題2

`std::min_element` / `std::max_element` の比較関数としてCompareを用いる

別解: 小課題2

`std::min_element / std::max_element` の比較関数としてCompareを用いる

→比較回数はそれぞれ $N - 1$ であることが保証されているのでOK

別解：小課題3

別解: 小課題3

`std::minmax_element` の比較関数として `Compare` を用いる

別解: 小課題3

`std::minmax_element` の比較関数として `Compare` を用いる

→ 比較回数は $\min\left(\left\lfloor \frac{3(N-1)}{2} \right\rfloor, 0\right)$ であることが保証されているのでOK

- これはさっきの式と同じ

別解: 小課題3

`std::minmax_element` の比較関数として `Compare` を用いる

→ 比較回数は $\min\left(\left\lfloor \frac{3(N-1)}{2} \right\rfloor, 0\right)$ であることが保証されているのでOK

- これはさっきの式と同じ

C++11以降でしか使えない

別解: 小課題3

`std::minmax_element` の比較関数として `Compare` を用いる

→ 比較回数は $\min\left(\left\lfloor \frac{3(N-1)}{2} \right\rfloor, 0\right)$ であることが保証されているのでOK

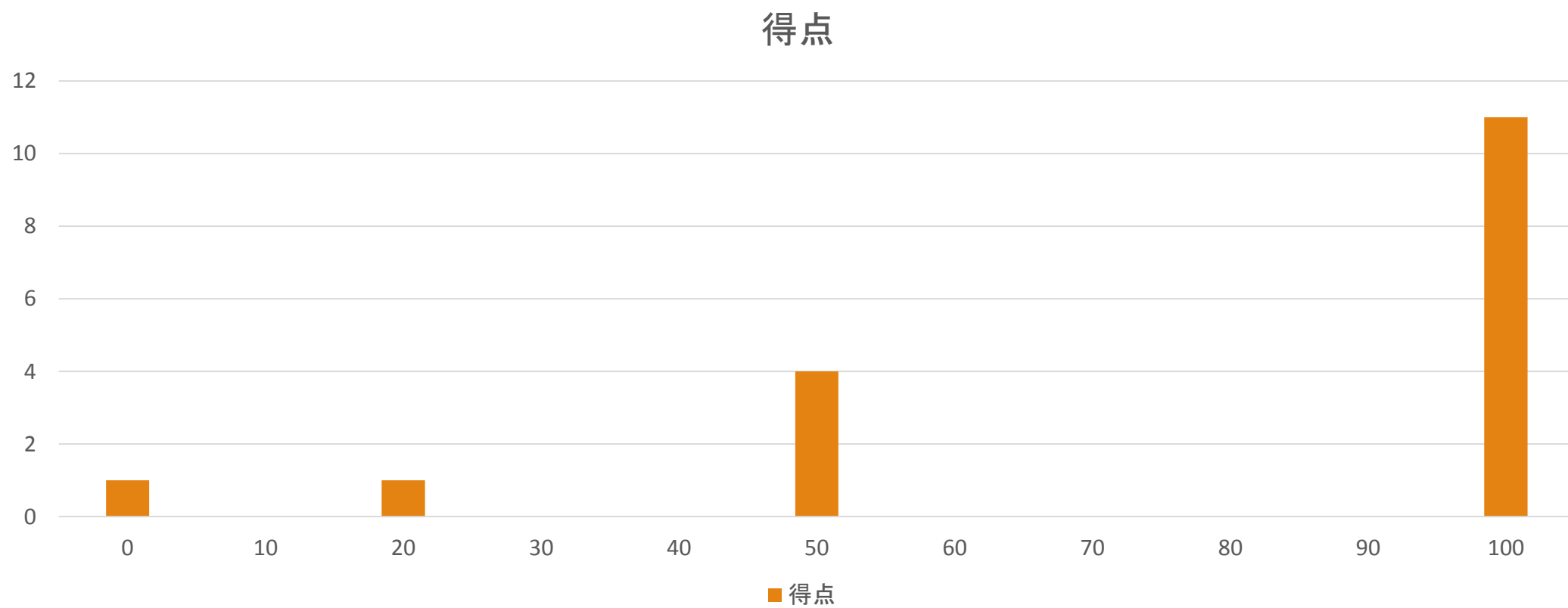
- これは $N \geq 1$ ではさっきの式と同じ

C++11以降でしか使えない

- 今回の合宿では使えなかったことになる

分布

分布



おわり

1. 問題概要
2. 小課題1 (20点)
3. 小課題2 (累計50点)
4. 小課題3 (累計100点)
5. コーナーケース
6. 下界の証明
7. 別解 (小課題1・小課題2・小課題3)
8. 分布

JOI 2013-2014 春合宿 Water Bottle 解説

問題概要

- グリッド状の街があり, 各マスは野原 or 建物 or 壁
 - 壁のマスには入れない
 - 野原を 1 マス通る時には水が 1 必要
 - 建物では水を(水筒の容量の範囲内で)無限に補給できる
 - 水は水筒に入れて持ち運ぶ
-
- 移動したい建物の組がいくつか与えられる
 - それぞれの組について, 必要な水筒の容量の最小値は？

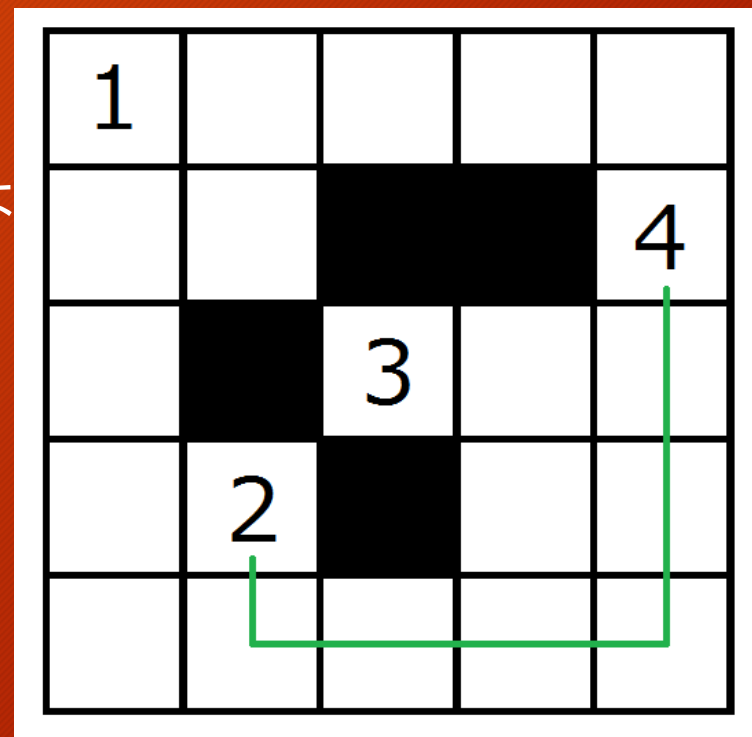
入力例 (1)

- Sample 1

1				
				4
		3		
	2			

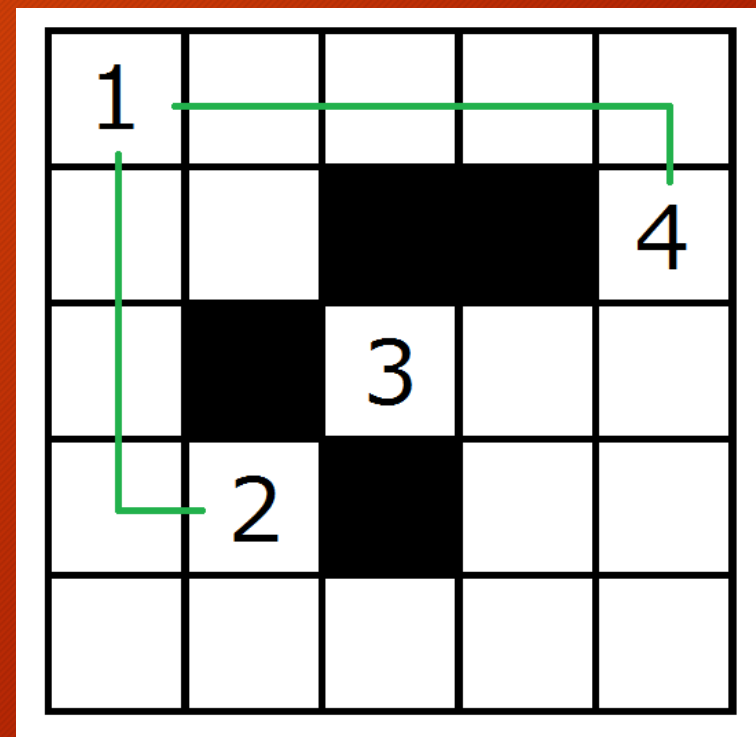
入力例 (2)

- 建物 2 から建物 4 まで移動する例
 - 直接移動してみた
 - この例では野原を連続して 6 マス通るので, 水筒の大きさは 6 必要



入力例 (3)

- 建物 2 から建物 4 まで移動する例
 - 遠回りだけど建物 1 を経由してみた
 - 2 -> 1 は野原 3 マス
 - 1 -> 4 は野原 4 マス
 - 水筒の大きさは 4 で十分
- 水筒の大きさを最小化するのが目的
- 本当の最短路には興味がない



小課題 1 (1)

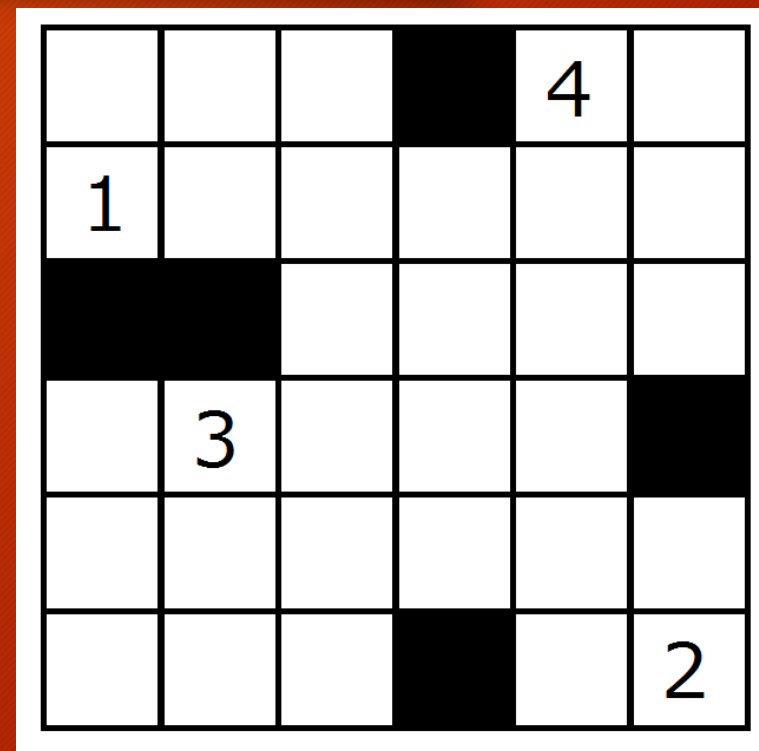
- 建物と建物の間を直接(途中で水を水筒に追加せずに)移動することをまず考える
- ある移動について, 必要な水の量は「グリッド上の最短距離 - 1」
- 各建物から BFS をすれば, すべての最短距離は $O(HWP)$ で求められる

小課題 1 (2)

- ある移動を考えたとき, 必要な水筒の大きさは?
 - 明らかに, その移動中の建物間の直接の移動の中での, 必要な水の量の最大値
- 最短路を求めるような気分で, 「パス上重みの最大値」の最小値も求められる
- P 個の頂点について, 全点間についてこの値は Warshall-Floyd で $O(P^3)$ で求められる(するとクエリはこの値を呼び出すだけ)
- 全部で $O(HWP + P^3 + Q)$
 - 小課題 1 が解けて, 10 点が得られる

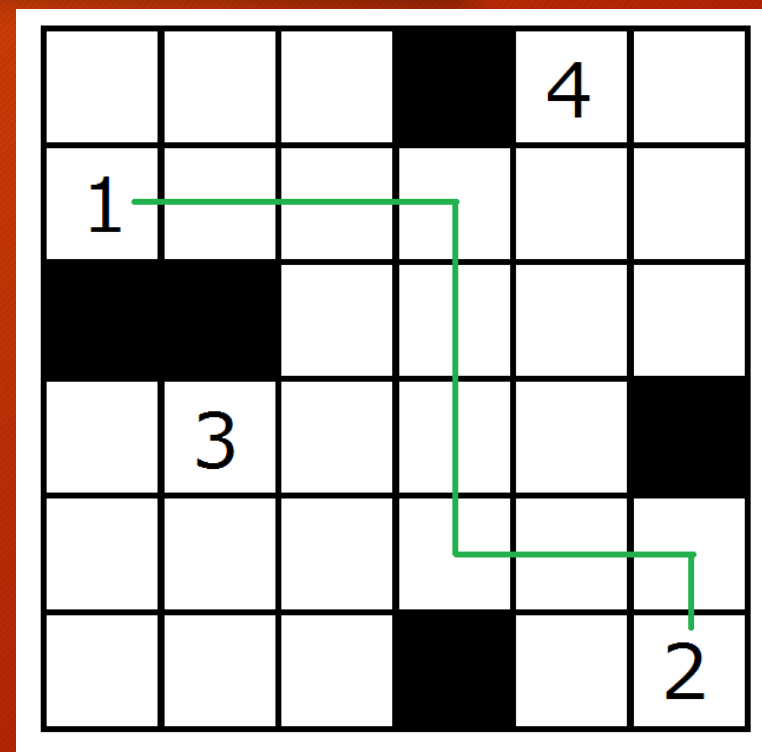
満点解法のために(1)

- 意味のある「直接の移動」とは何か？
- 右の図で 1 から 2 まで移動することを考えてみる



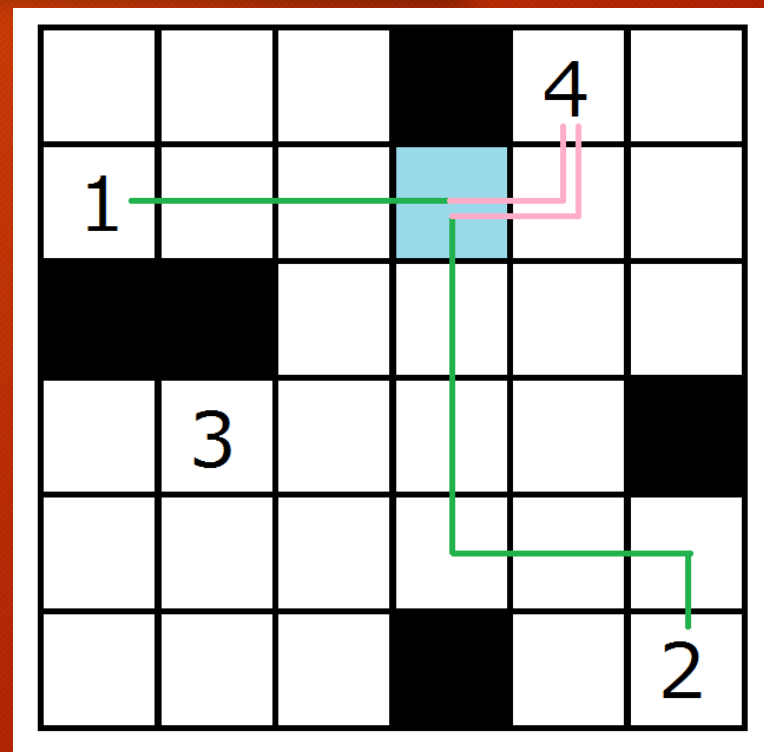
満点解法のために(2)

- とりあえず適当に直接の経路を書いてみた
- まだ無駄がありそう



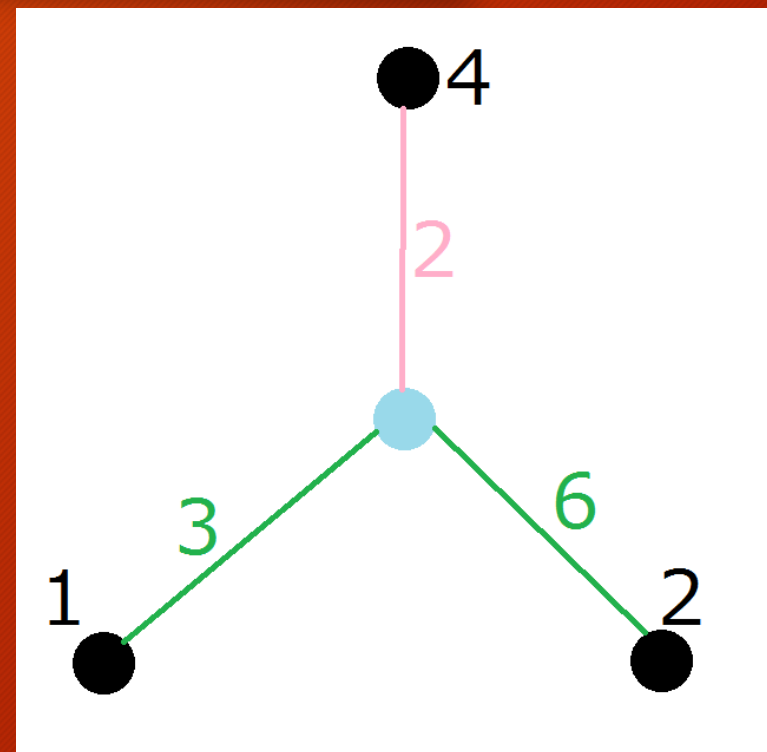
満点解法のために(3)

- 例えば右のように，途中で 4 に寄るともっと水筒の容量が少なくてよくなる(これでも明らかに無駄があるけど)
- 最初の移動で水色のマスを通っていたが，そこからあえて 4 に寄り道するようにした
- そうしたら，必要な水筒の大きさが小さくなった



満点解法のために(4)

- 関係するマスたちの距離についての構造のみ取り出してみた
- 1-2 を直接移動すると、最短距離は 9
- 1-4, 4-2 と移動すると、それぞれ最短距離は 5, 8
- 水色マスが 1, 2 よりも 4 に一番近いので、4 に寄り道したほうがうれしい

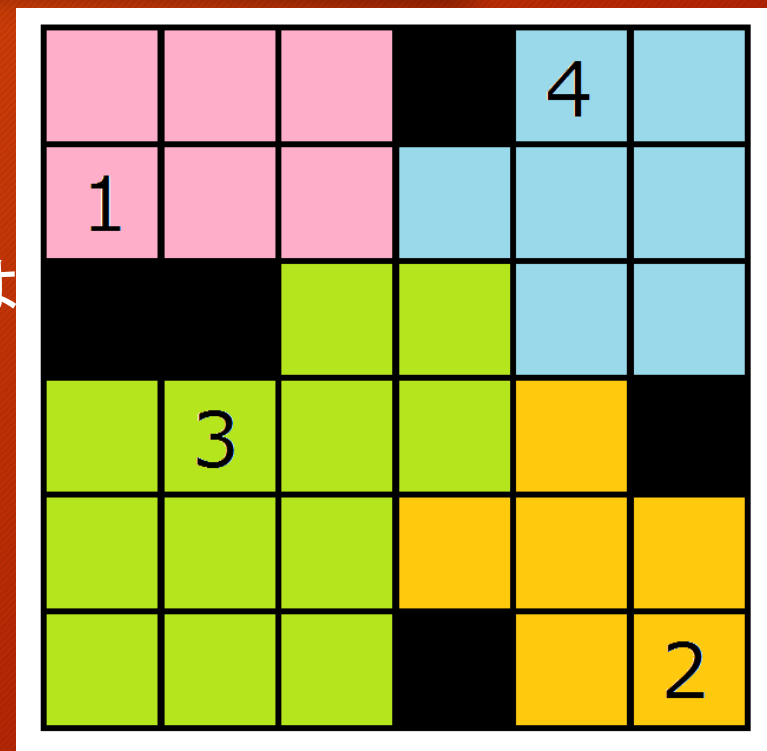


満点解法のために(5)

- 「a から b まで」直接移動しているときに、最寄りの建物が別の c になったら c を経由するとしてよい
- 逆に、直接移動するときは、常に最寄りの建物が a か b であるべき
- 実際には境界の問題もあるが、「他の建物までの距離も同じ」場合でも寄り道をして困ることはない

満点解法のために(6)

- 最寄りの建物で色分けすると右のようになる
- 建物間の移動では、色の境界を 1 回以上またぐ
- 逆に色の境界について考えると、その両端のマスからはそれぞれの最寄りのマスへ直行するのが最適
- 色の境界は $O(HW)$ 個なので、意味のある移動も実は $O(HW)$ 個しか存在しない！



小課題 2

- 各マスの最寄りの建物を求めるのは, 各建物から BFS をすれば $O(HW)$
- さらに $O(HW)$ で, 意味のある移動を全部列挙できる
- 重み最大値が最小のパスを求めるのは, Dijkstra が使える
- 密グラフについての Dijkstra を用いると, クエリあたり $O(P^2)$
- 全部で $O(HW + P^2 Q)$
 - 小課題 2 が解けて, 30 点が得られる
 - これだけでは小課題 1 は解けないことに注意
- 必要な辺の数が多いので, Dijkstra を $O(E \log P)$ でやっても小課題 3 を解くのは難しい

小課題 3 (1)

- クエリをもう少し高速化したい
- 水筒の大きさがどれだけあれば移動が可能か？と考える
 - 必要な大きさが小さい「直接の移動」から順にできることにしていって、いつ移動ができるようになるか調べる

小課題 3 (2)

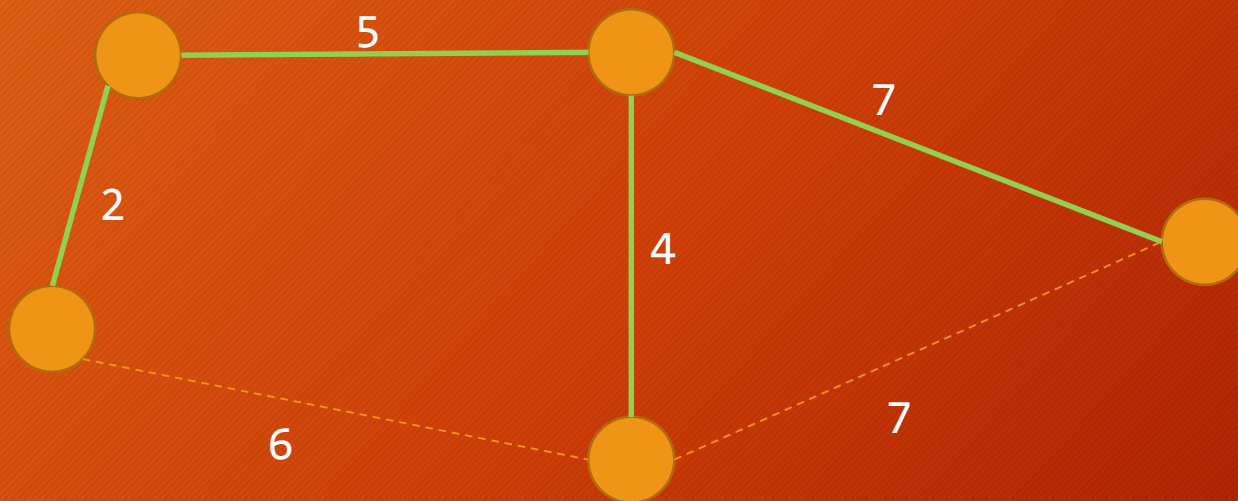
- 移動可能性は推移的
 - (x, y) の間が移動でき, (y, z) の間が移動できるなら (x, z) の間も移動できる
- 建物たちをグループに分けて, 同じグループの間は移動できる, とする
 - 新たに移動できるようになった場所は, グループを併合する
- このような処理は Union Find でできる

小課題 3 (3)

- 距離の小さい「直接の移動」から見ていって, 順に Union Find でつなげる
 - このとき, すでにつながっている 2 点間を結ぶ移動は使う必要がない!
 - 距離がそれ以下の「直接の移動」を用いて移動することができる
 - つながっていない 2 点間を結ぶのは, 高々 $P-1$ 回
 - 結局, 考えるべき「直接の移動」は高々 $P-1$ 個で済む
-
- 最後までつながらなかった点たちは, 十分大きい INF を距離としてつなげておくといいです

小課題 3 (4)

- 下のグラフにおいて、緑色の辺以外の辺はないと思っても問題ない



小課題 3 (5)

- 結局これは Kruskal 法と全く同じです
 - 最小全域木とは目標が少し違うが, 同じ結果になる
- 得られるものは木になる
- 最短路は, 普通に DFS とかで求めても $O(P)$ で求められる
- 結局, $O(HW + PQ)$
 - 小課題 1, 2, 3 が解けて, 70 点を得られる

小課題 4

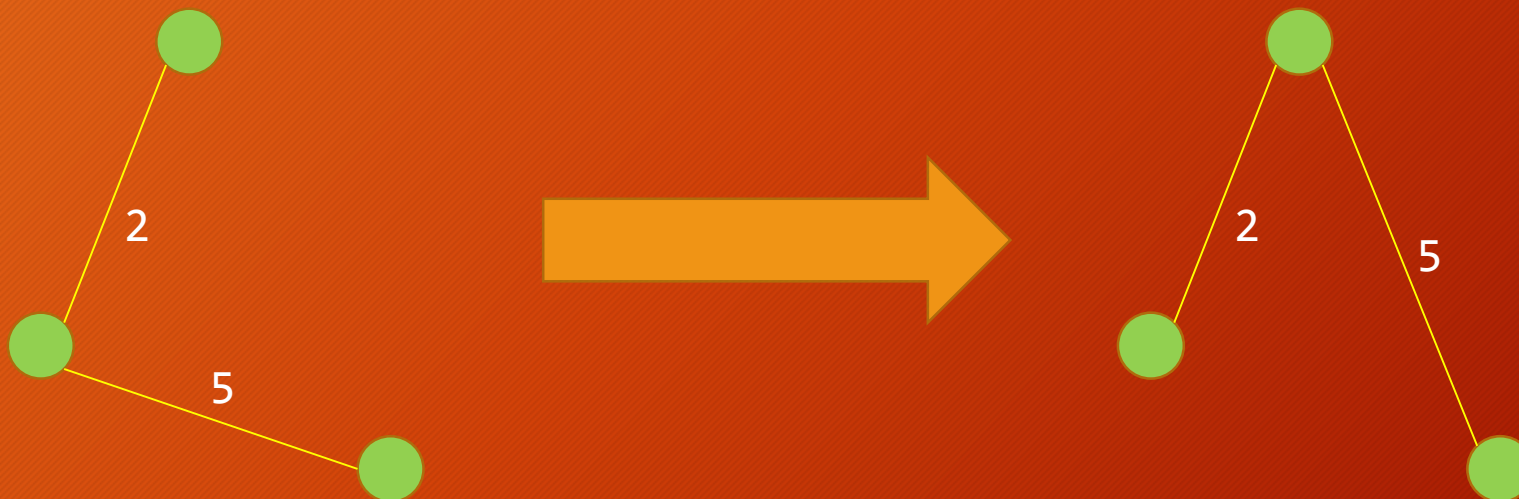
- 木の上で, パス上最大値を求めるクエリを高速で処理したい
- 木を勝手に根付き木にする
- Doubling の手法を使って LCA を行う
 - 最大値も同時に前計算させて同時に求める
 - 前処理 $O(P \log P)$, クエリ $O(\log P)$
 - 「蟻本」を見ましょう
- 全部で $O(HW + (P + Q) \log P)$
 - 満点が得られる

別解 (1)

- 作る木が, 必ずしも実際の構造に即している必要はない
- 「小さい方から見ていった時の連結性」さえ一致していれば何でもいい

別解 (2)

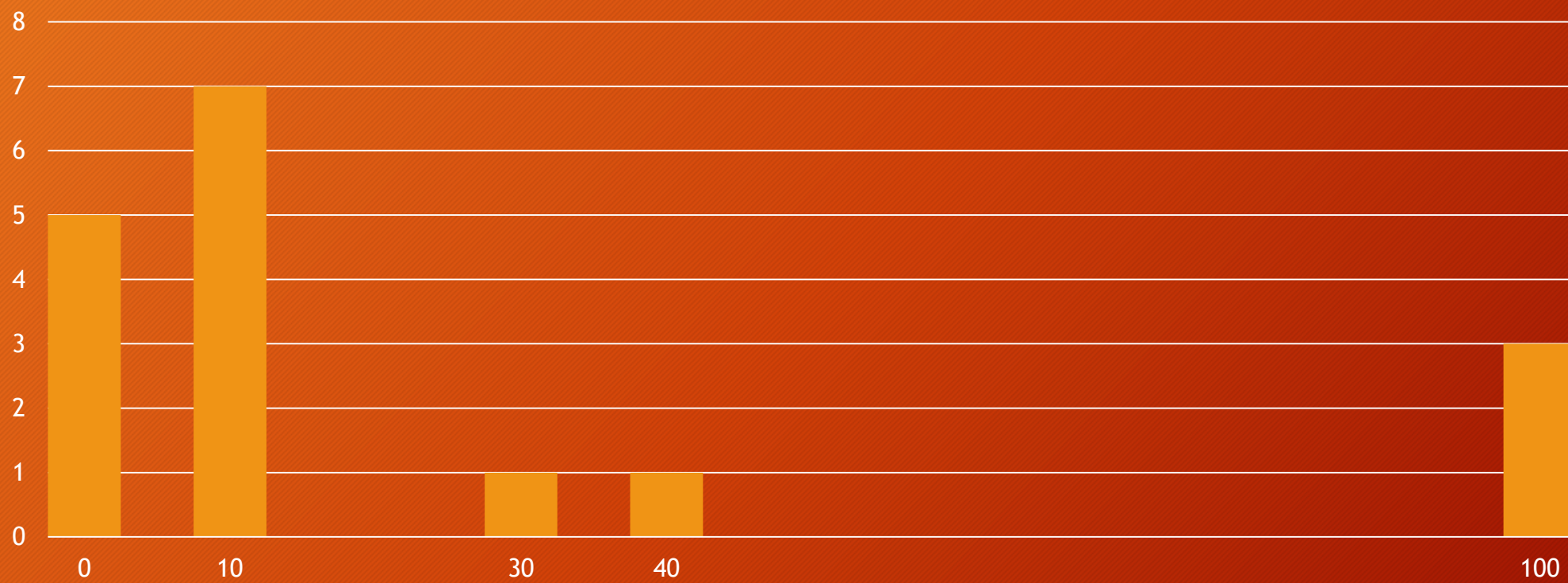
- 下のような変形をしても問題ない



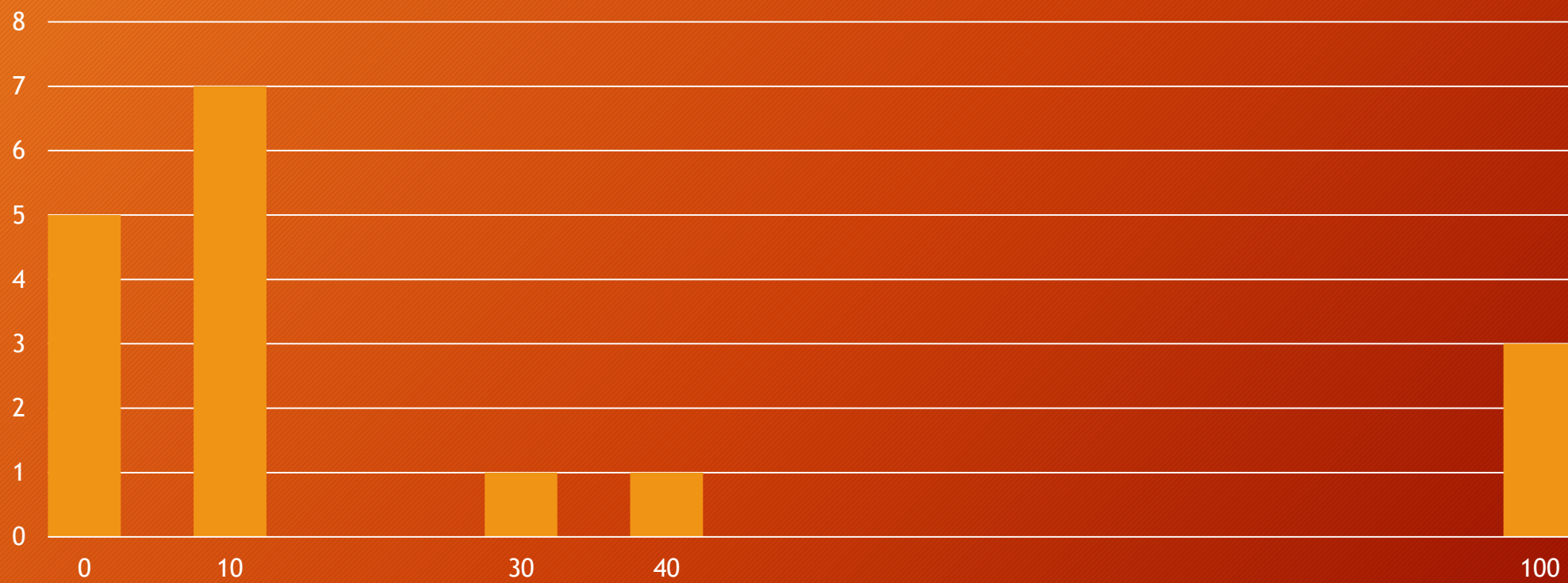
別解 (3)

- Union Find 木で, 経路圧縮をせずに, 辺に「どの距離の移動によってつながったか」を覚えさせる
- すると, 2 点間のパス上の最大値が答えになる
- 木は, 「小さい方を大きい方の下につける」方法により高さを $O(\log P)$ にできる
- すると, 根付きにして「共通の点になるまで 1 段ずつ根に近づく」方法をやるだけでクエリあたり $O(\log P)$
- 結局 $O(HW + (P + Q) \log P)$ なので, これも満点が得られる
 - さらに LCA で Doubling をしたりするとクエリあたり $O(\log \log P)$ にまでできる

得点分布



得点分布



Making Friends is Fun

今西 健介(@japlj)

問題概要

Making
Friends is Fun

あなたは歴史の裏舞台で活躍するエージェントであり世界の平和に向けて日々活動をしている。この世界には N 個の国がありそれぞれ 1 から N までの異なる番号がふられている。これらの N 個の国々の間にできる限り友好な関係を築いてもらうことがあなたの目的である。あなたはエージェントの仕事の計画を立てるため現在の国際関係を表す図を描いた。あなたは大きな画用紙を一枚用意し、そこにそれぞれの国を表す N 個の点を打った。次に現在の国際関係を表すために 2 つの国を結ぶ矢印を M 本描いた。国 a を表す点から別の国 b を表す点へと向かう矢印は、現在国 a が国 b に大使を派遣しているということを表す。以下では国 a を表す点から国 b を表す点へと向かう矢印を矢印 (a,b) と呼ぶ。こうして描いた N 個の点と M 本の矢印が現在の国際関係を表す図である。国同士の友好関係のきっかけとして 2 国間での友好条約締結会議（以下では単に「会議」という）を行うことを考えよう。ある 2 つの国 p, q が会議を行うためには両方の国に大使を派遣しているような国 x が仲介として必要である。そして会議を行った後にそれぞれの国は相手国に大使を派遣する。すなわち国 p と国 q が会議を行うためには矢印 (x,p) と矢印 (x,q) があるような国 x が存在していなければならない。会議を行った後では矢印 (p,q) と矢印 (q,p) を新たに描き加える。ただし矢印がすでに描かれている場合には新たに描き加えることはしない。あなたの仕事は会議を行うことができるような 2 つの国とその会議の仲介となる国を選び、会議を行わせることである。図を使ってこの仕事のシミュレーションをするにあたって、世界がどれほど平和に近づいているかについて画用紙の上の矢印の個数を基準に考えることにした。つまり 2 つの国を選んで会議を行わせるといったことを繰り返すことで画用紙

問題概要

Making
Friends is Fun

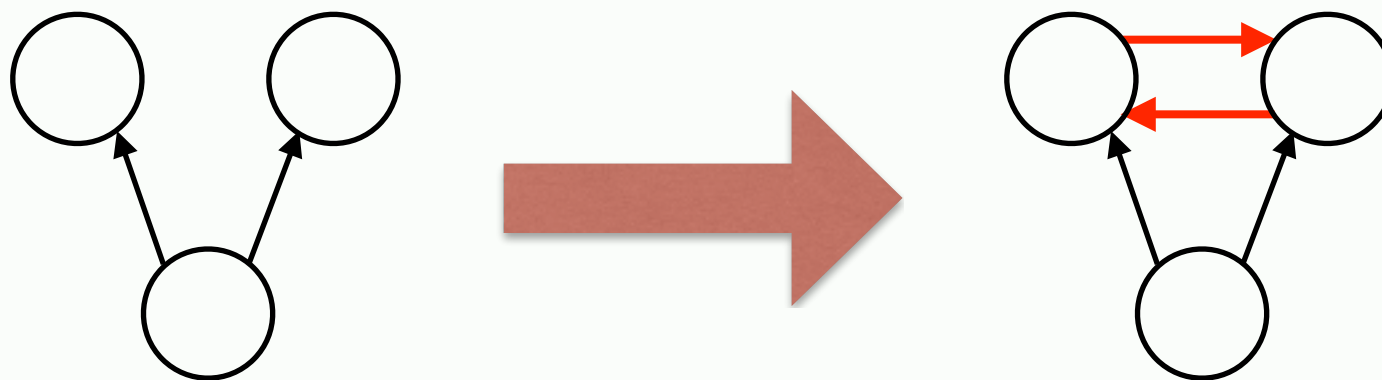
あなたは歴史の裏舞台で活躍するエージェントであり世界の平和に向けて日々活動をしている。この世界には N 個の国がありそれぞれ 1 から N までの異なる番号がふられている。これらの N 個の国々の間にできる限り友好な関係を築いてもらうことがあなたの目的である。あなたはエージェントの仕事の計画を立てるため現在の国際関係を表す図を描いた。あなたは大きな画用紙を一枚用意し、そこにそれぞれの国を表す N 個の点を打った。次に現在の国際関係を表すために 2 つの国を結ぶ矢印を M 本描いた。国 a を表す点から別の国 b を表す点へと向かう矢印は現在国 a が国 b に大使を派遣しているということを表す。以下では国 a を表す点から国 b を表す点へと向かう矢印を矢印 (a,b) と呼ぶことにする。描いた N 個の点と M 本の矢印が現在の国際関係を表す図である。同士の友好関係のさかん化として 2 国間の友好条約締結会議（以下で単に「会議」という）を行うことを考える。ある 2 つの国 p と q が会議を行うためには両方の国に大使を派遣しているような国 x が仲介として必要である。そして会議を行った後にそれぞれの国は相手国に大使を派遣する。すなわち国 p と国 q が会議を行うためには矢印 (x,p) と矢印 (x,q) があるような国 x が存在していなければならない。会議を行った後では矢印 (p,q) と矢印 (q,p) を新たに描き加える。ただし矢印がすでに描かれている場合には新たに描き加えることはしない。あなたの仕事は会議を行うことができるような 2 つの国とその会議の仲介となる国を選び、会議を行わせることである。図を使ってこの仕事のシミュレーションをするにあたって世界がどれほど平和に近づいているかについて画用紙の上の矢印の個数を基準に考えることにした。つまり 2 つの国を選んで会議を行わせるといったことを繰り返すことで画用紙

わかりやすい問題概要

Making
Friends is Fun

N 頂点 M 辺のグラフがある。

次のような**操作**を好きなだけ繰り返せる。



最終的に**辺を何本まで**増やすことができるか？

制約: $1 \leq N \leq 100\,000$, $1 \leq M \leq 200\,000$

典型……？

Making
Friends is Fun

よく知られた手法やよく出題される手法
は使えなさそう

動的計画法？ segment tree？ 強連結成分分解？



この問題特有の性質について考察する必要がある！

- ・ 有名なアルゴリズムを適用するような形では解けない
- ・ 「典型でない」問題

まずは簡単な性質から

性質1

どのような順番で操作を行っても
最終的な辺の本数は変わらない。

なぜか？

ある2点 (p, q) を結べる状態に一度なれば、
その後どうなっても (p, q) は結べるから。

部分点解法1

性質 1 だけを使って次のような解き方ができる:

- ・ 結べる 2 点が見つからなくなるまで操作を繰り返す

計算量は？

そもそも、辺は最大で $N(N-1)$ 本になりうる。

結べるペアを探すのに $\Omega(N)$ 時間ぐらいはかかる。

→ 書き方にもよるが、いずれにせよ $\Omega(N^3)$ 時間

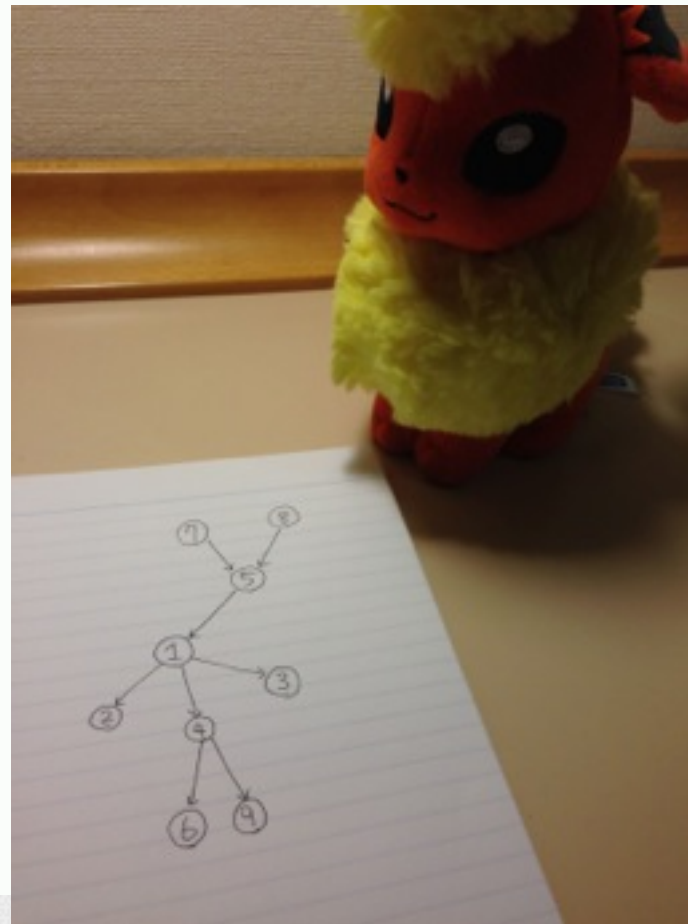
これだと小課題 1 だけが解ける(5点)

($N \leq 100$)

さらなる考察へ……

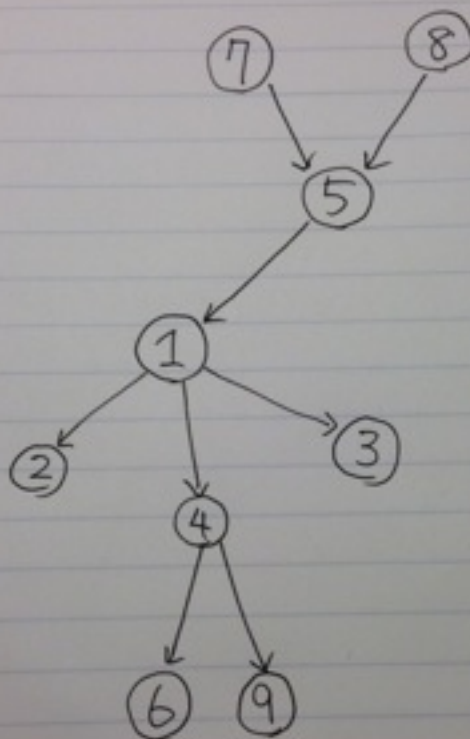
Making
Friends is Fun

こういう初期状態からだと、最終的に辺は何本ぐらいになるのでしょうか？これってトリビアになりませんか？



実際にやってみた(1/5)

Making
Friends is Fun



手で色々試してみるときは
極端な例や特殊な例
をやってみると役立つかも？

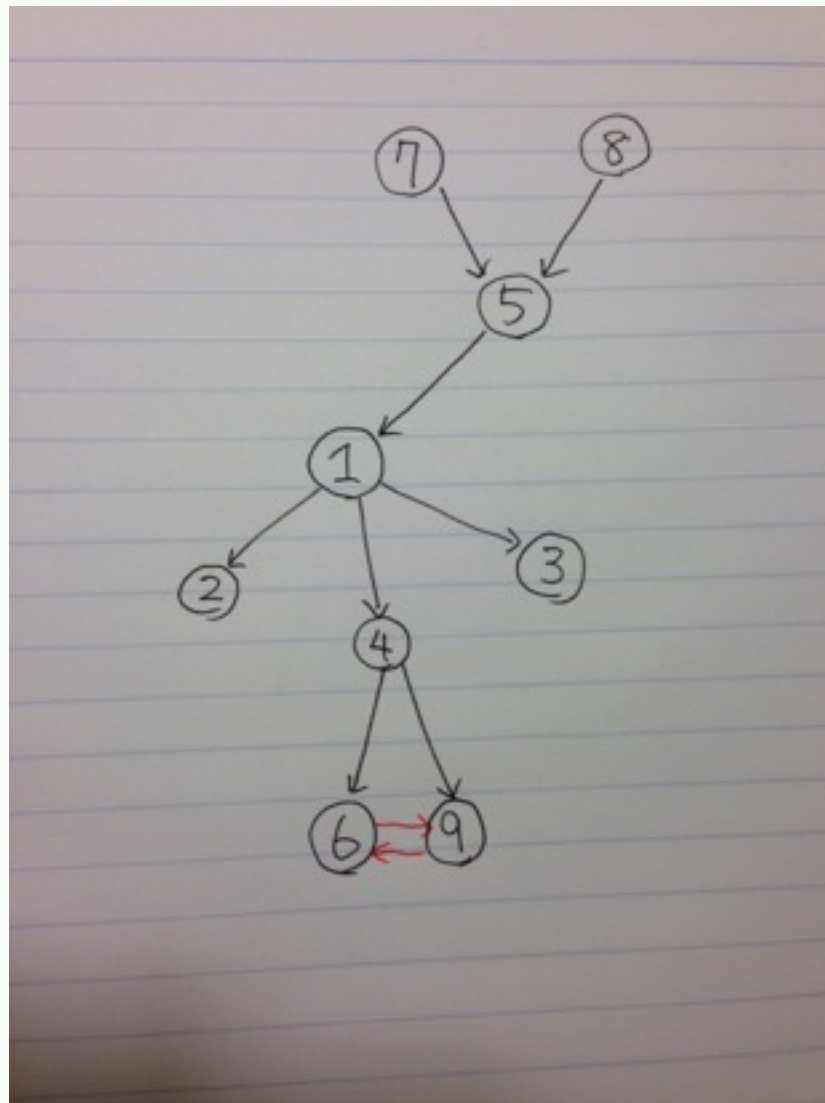
今回の場合は

- ・ 閉路がないグラフ

をやってみる

実際にやってみた(2/5)

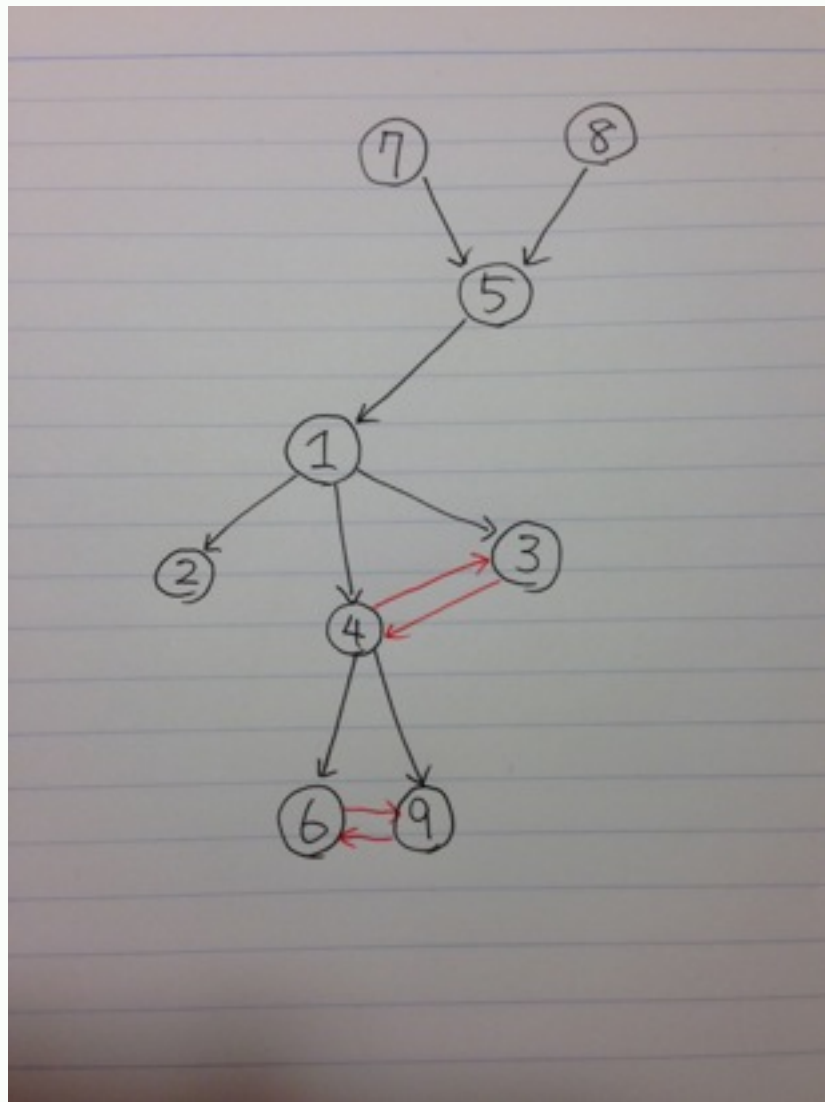
Making
Friends is Fun



4 を仲介に 6 と 9 が結べる

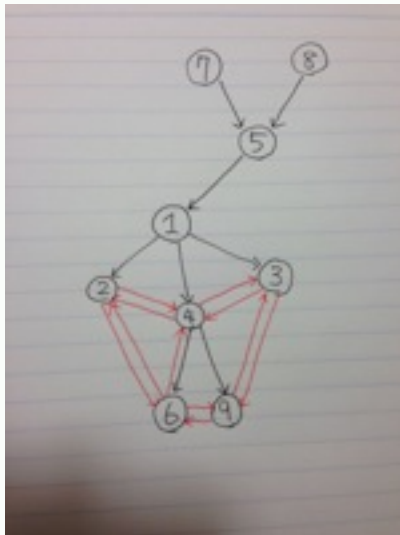
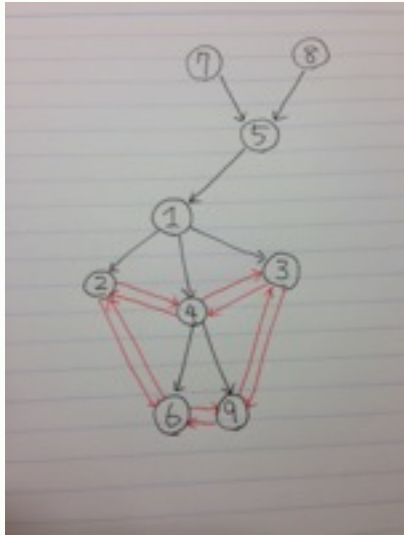
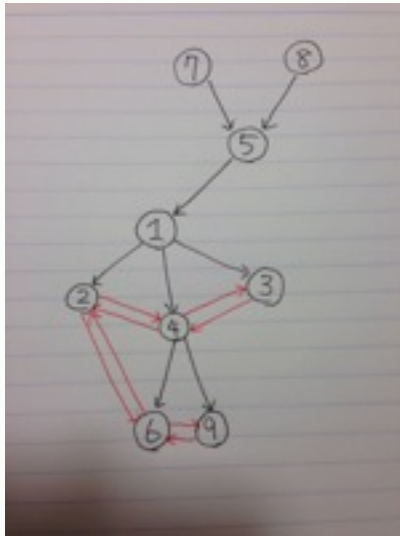
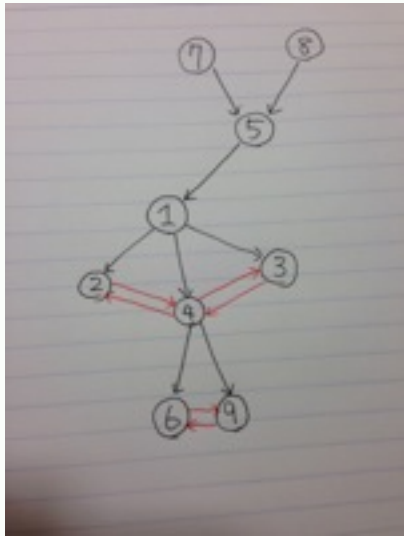
実際にやってみた(3/5)

Making
Friends is Fun



1 を仲介に 3 と 4 が結べる

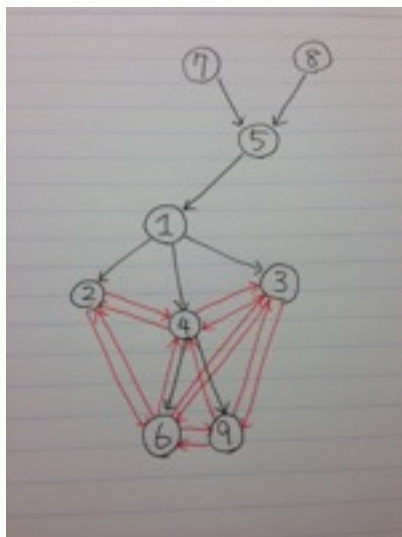
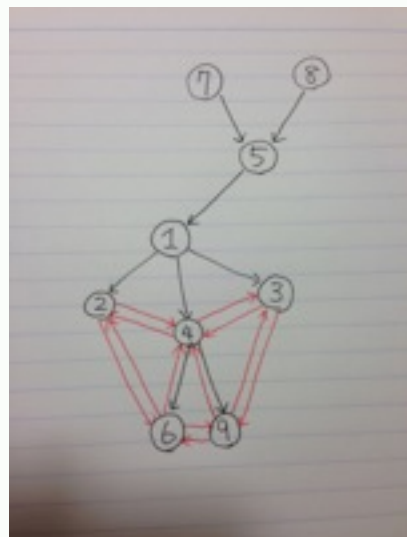
実際にやってみた(4/5)



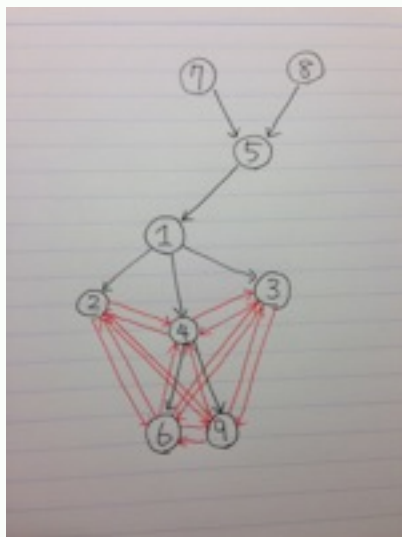
- 1 を仲介に 2 と 4 が結べる
- 4 を仲介に 2 と 6 が結べる
- 4 を仲介に 3 と 9 が結べる
- 2 を仲介に 4 と 6 が結べる

実際にやってみた(5/5)

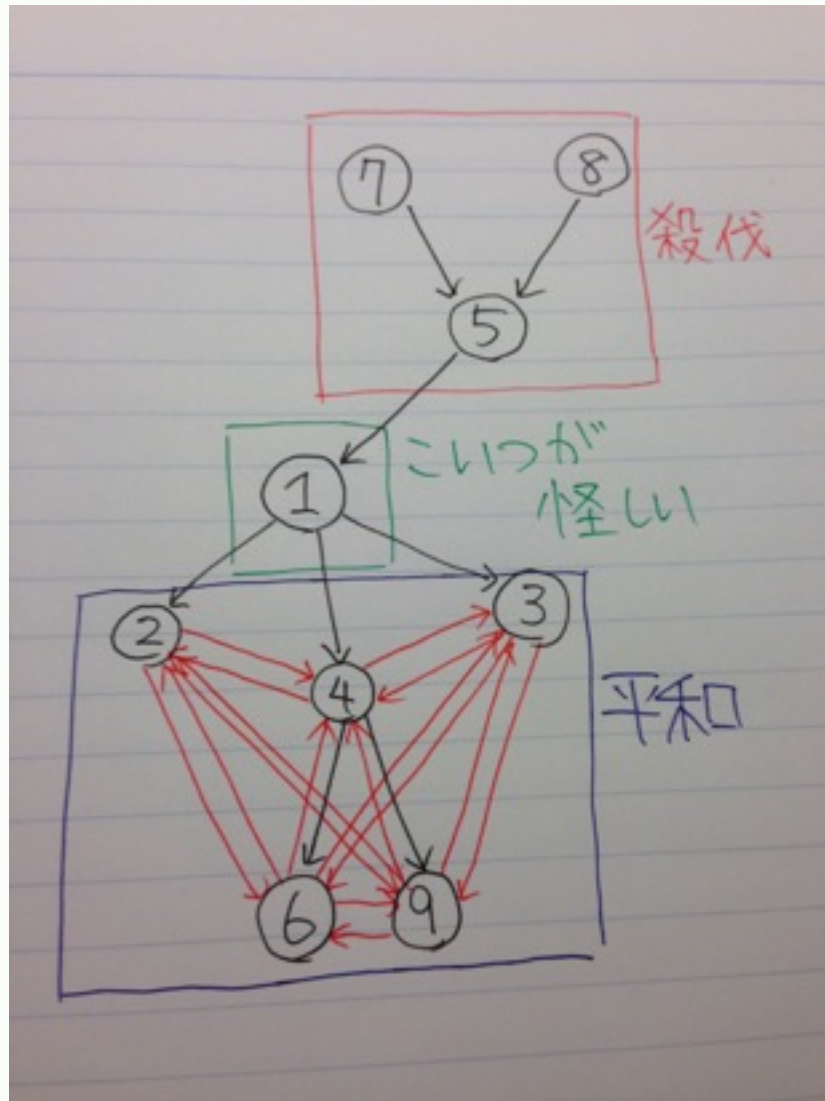
Making
Friends is Fun



3 を仲介に 4 と 9 が結べる
9 を仲介に 3 と 6 が結べる
6 を仲介に 2 と 9 が結べる



なにかがわかりそう



上のほうはめっちゃサツバツ

1 より下は平和そのもの

どうして差がついたのか……

慢心、環境の違い

わかること

性質 2

出て行く辺が 2 本以上あるような頂点 u があれば、
 u から到達できる頂点は全て互いに結ぶことができる。

なぜか？

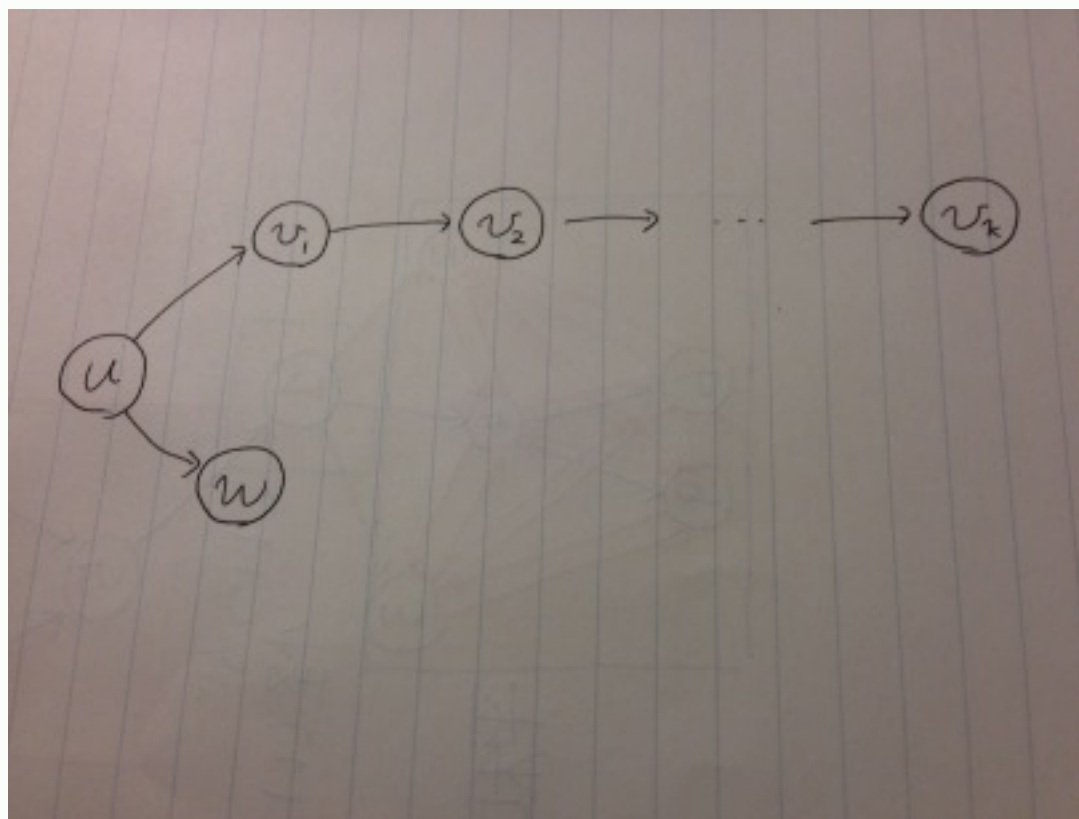
経路 $u \rightarrow v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k$ があったとする。

別の辺 $u \rightarrow w$ ($w \neq v_1$) がある (出て行く辺が 2 本以上ある) ので、

(v_1, w) を結ぶ、 (v_2, w) を結ぶ、 $\dots$ 、 (v_k, w) を結ぶ
と繋げていける。

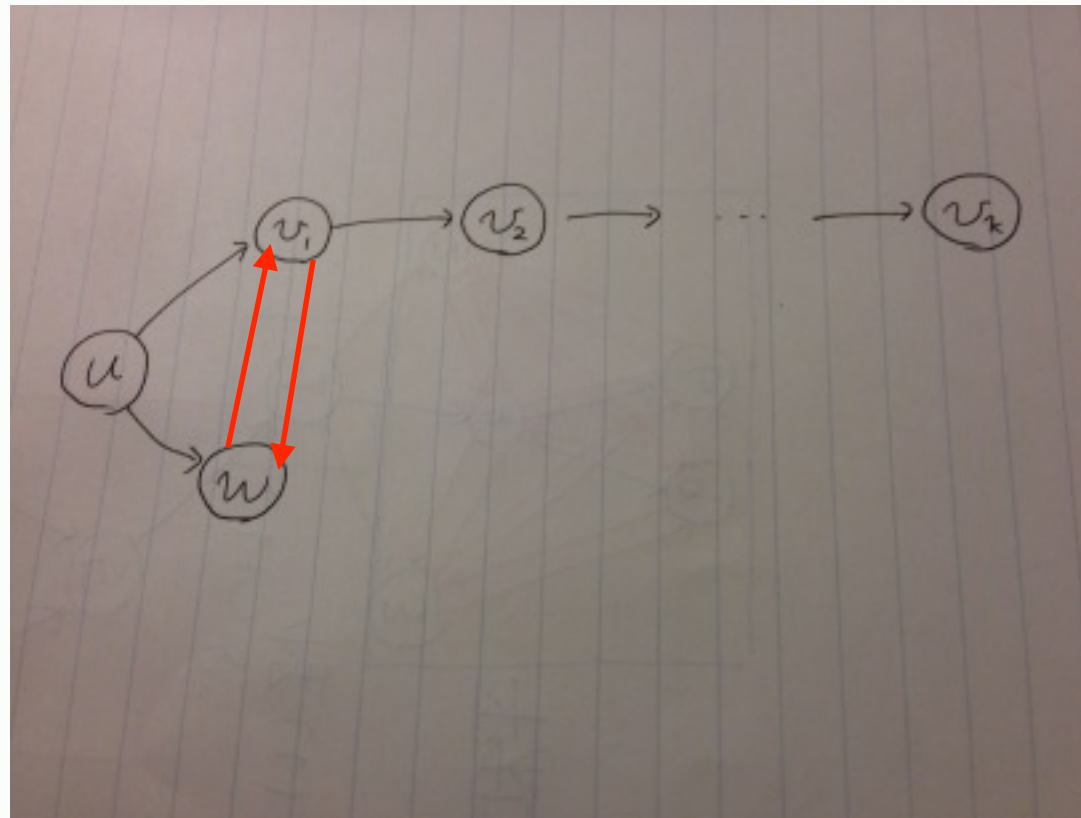
性質2 はやわかり

Making
Friends is Fun



性質2 はやわかり

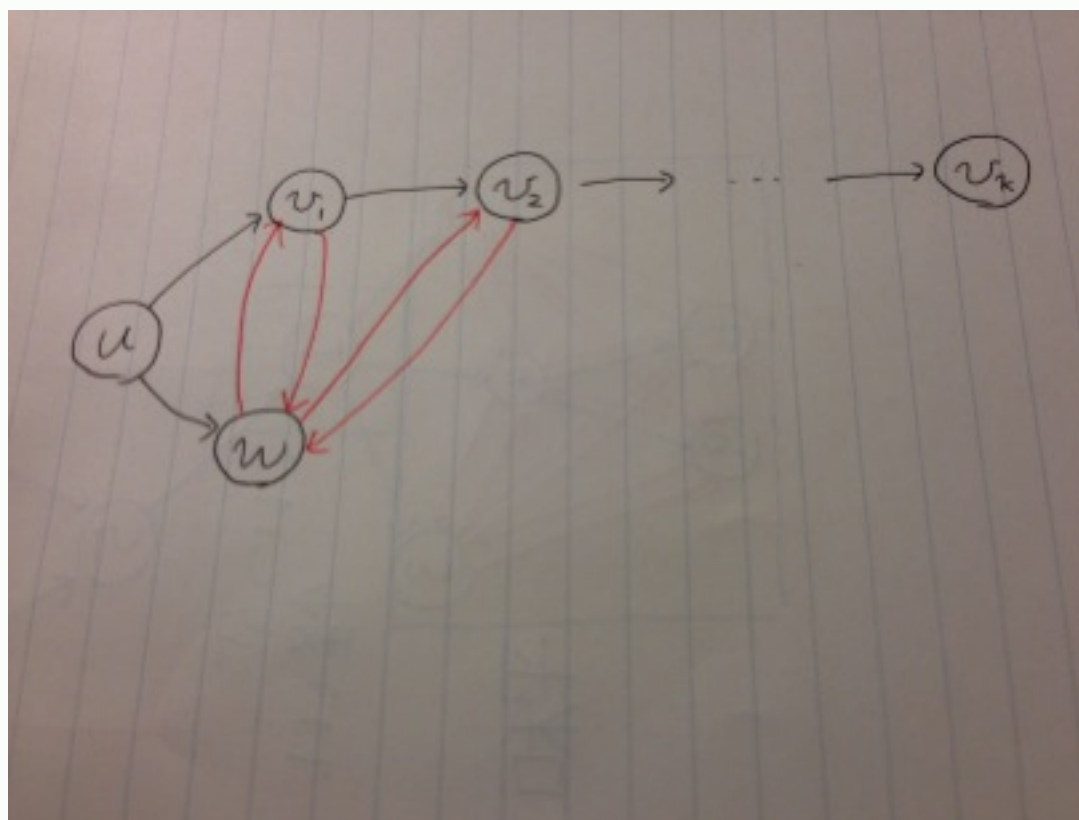
Making
Friends is Fun



この時点での写真ちゃんと撮ったのに
ファイル壊れてて取り込めなかった

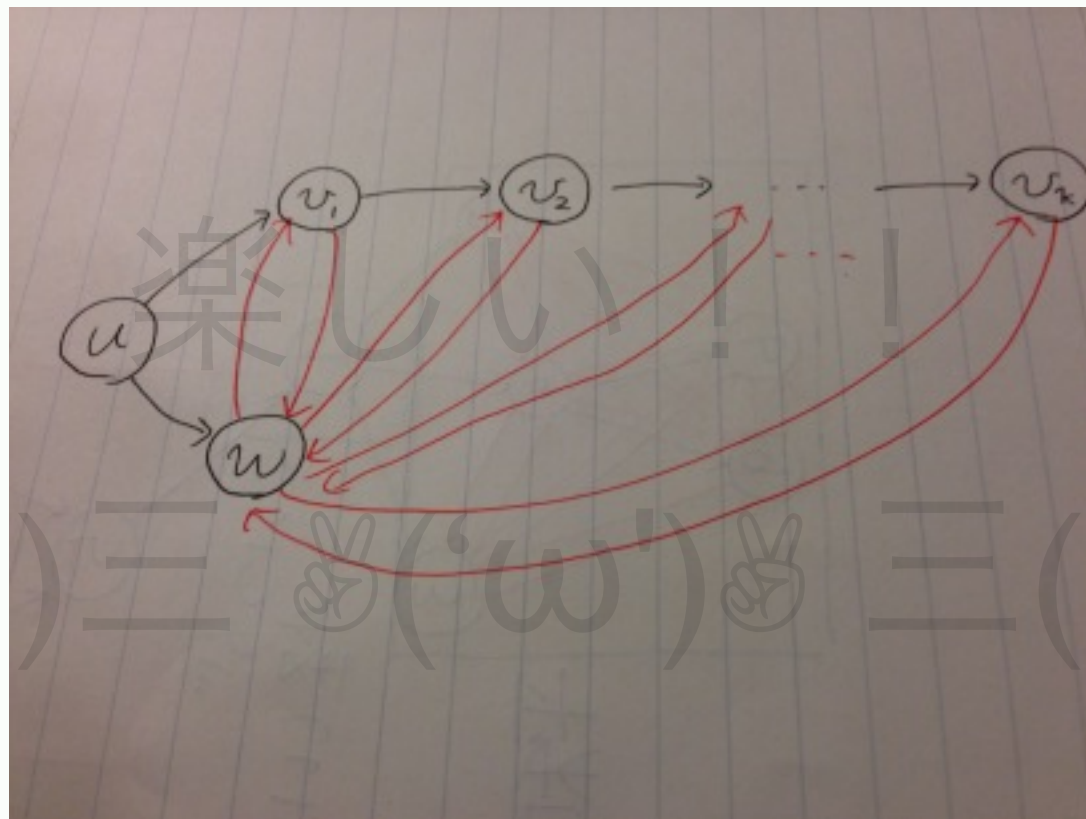
性質2 はやわかり

Making
Friends is Fun



性質2 はやわかり

Making
Friends is Fun



次のステップ

まずは考察

性質2 によれば、最終形には

「どの2点も双方向に結ばれているような部分」
が多く現れそう。

次に向けて

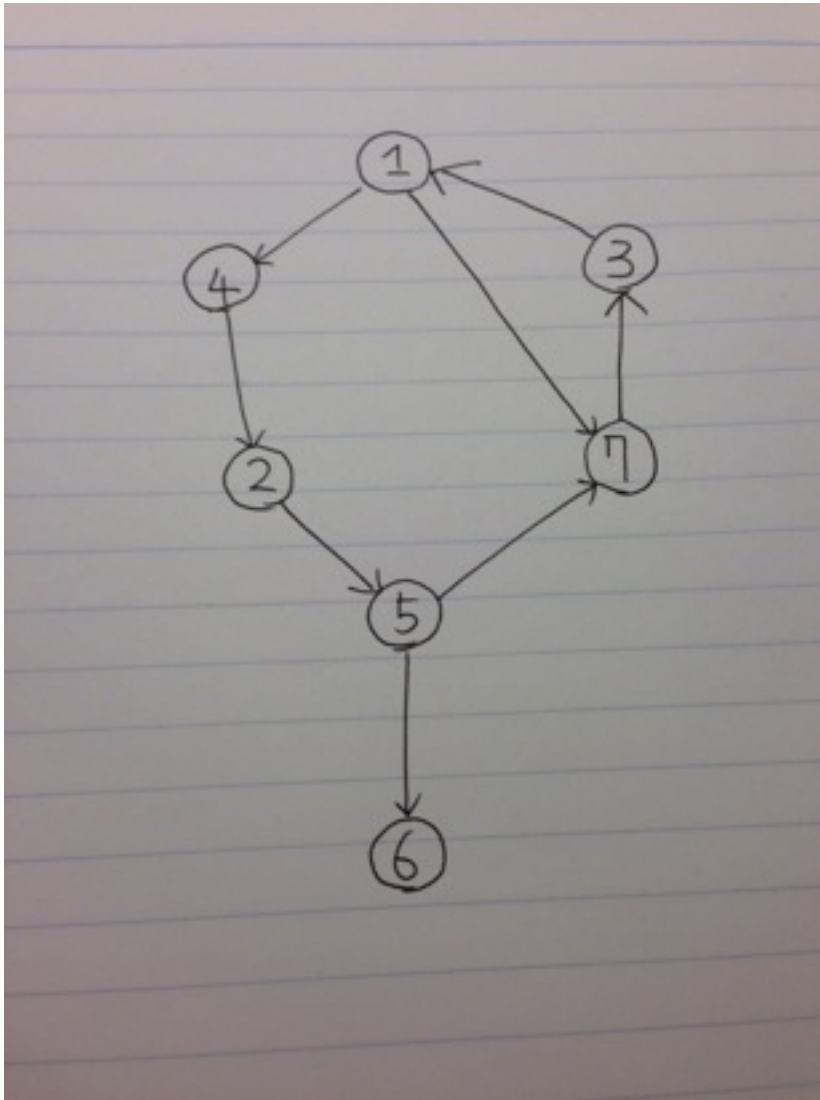
そういう部分のうち、頂点数が2以上のものを
「クリーク」と呼ぶことにする。

これはこの解説の中だけの用語

普通「クリーク」は無向グラフでの同様のものを指す

次はこいつや(1/4)

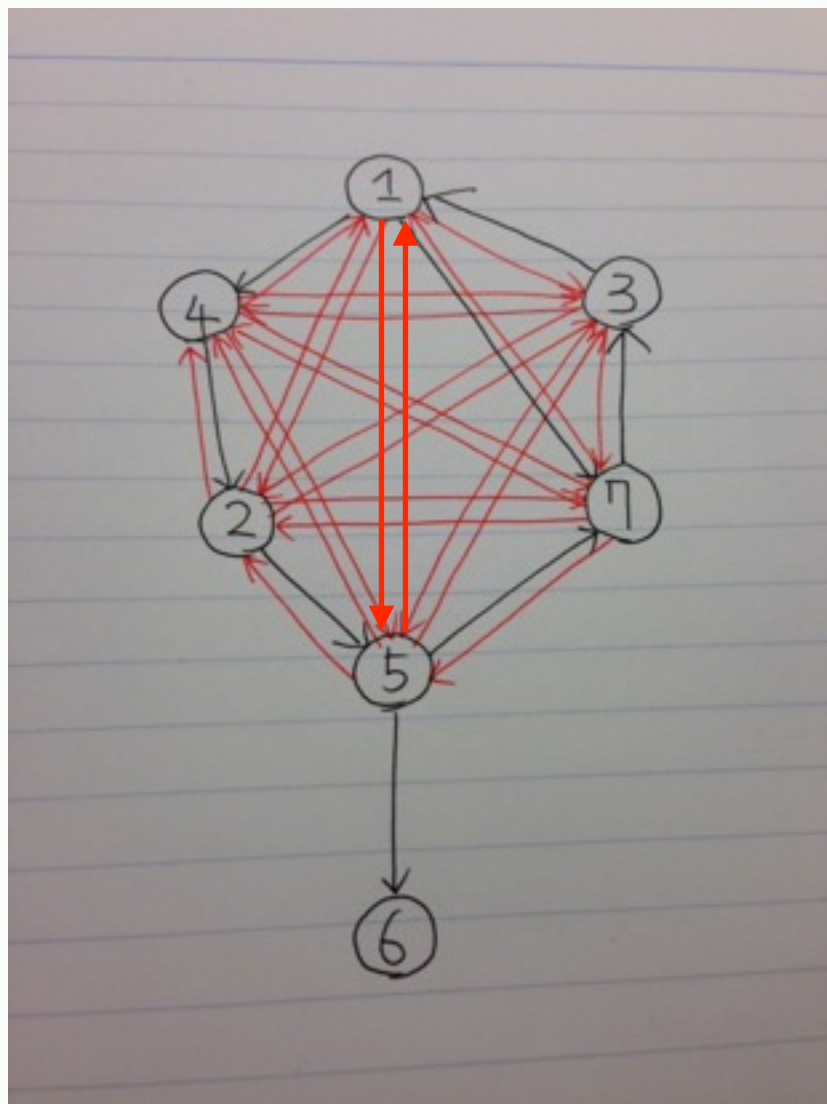
Making
Friends is Fun



← こういうのを

次はこいつや(2/4)

Making
Friends is Fun



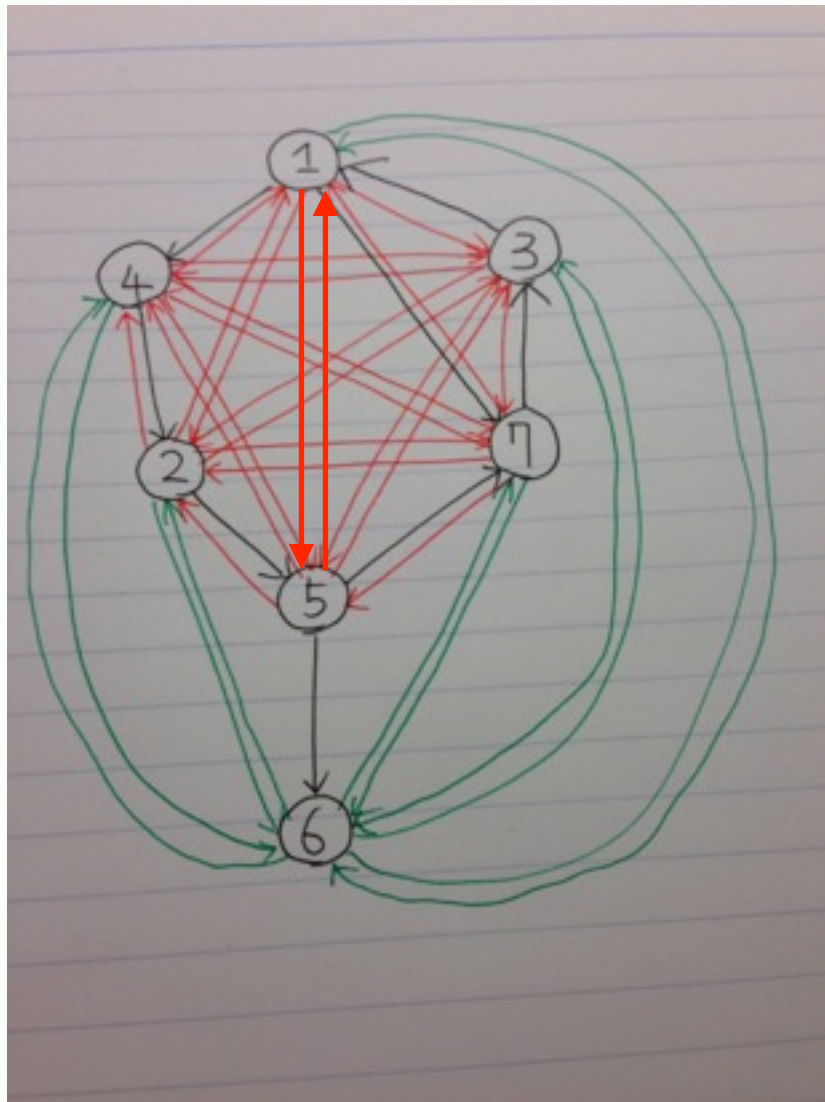
とりあえずここまでやる

← クリークから辺が出ている形

$1 \rightarrow 5, 5 \rightarrow 1$ の描き忘れに
さっき気づいた

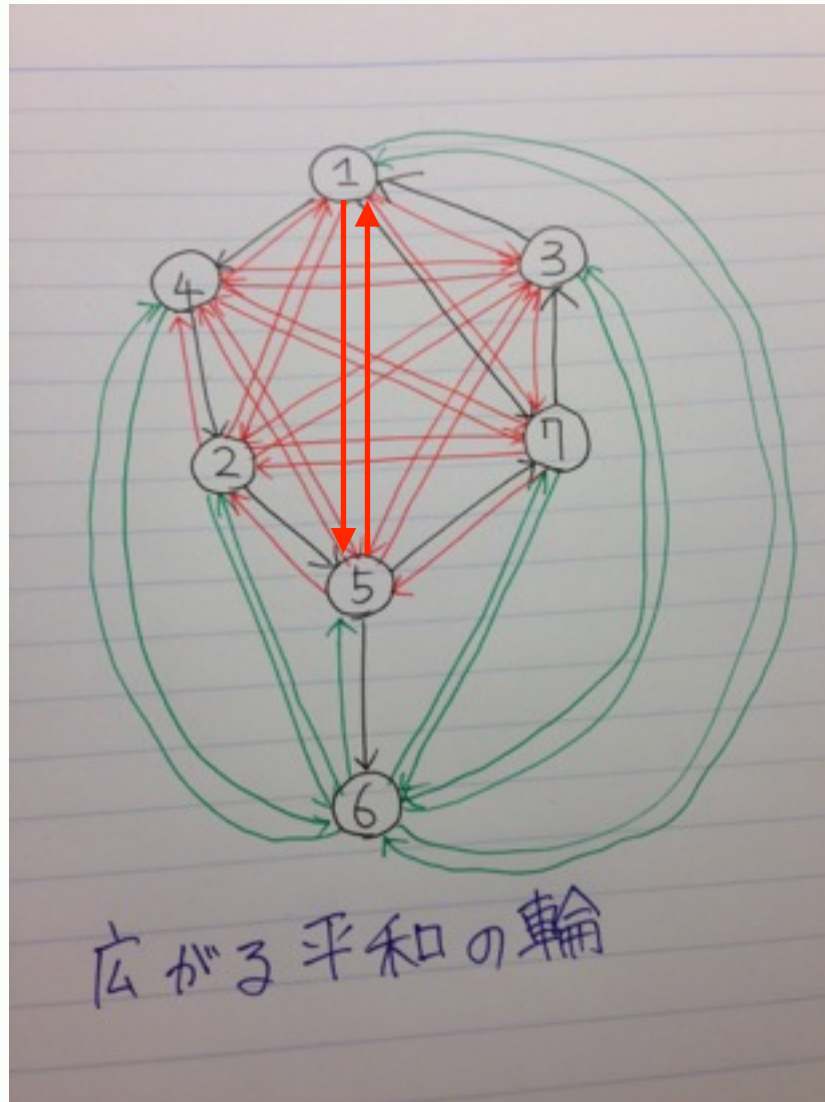
次はこいつや(3/4)

Making
Friends is Fun



5 は 1, 2, 3, 4, 7 全てに
辺を出しているので (クリークだからね)
それらと 6 を結べる

次はこいつや(4/4)



すると、
残った **5** と **6** も結べる！

広がる平和の輪

Making
Friends is Fun

性質3

クリークから辺が出ているとき、
その先の頂点もクリークに吸い込まれる。

なぜか？

さっきの具体例から明らかですね！

クリーク内の点 u から外の点 w に $u \rightarrow w$ と辺があるとする。

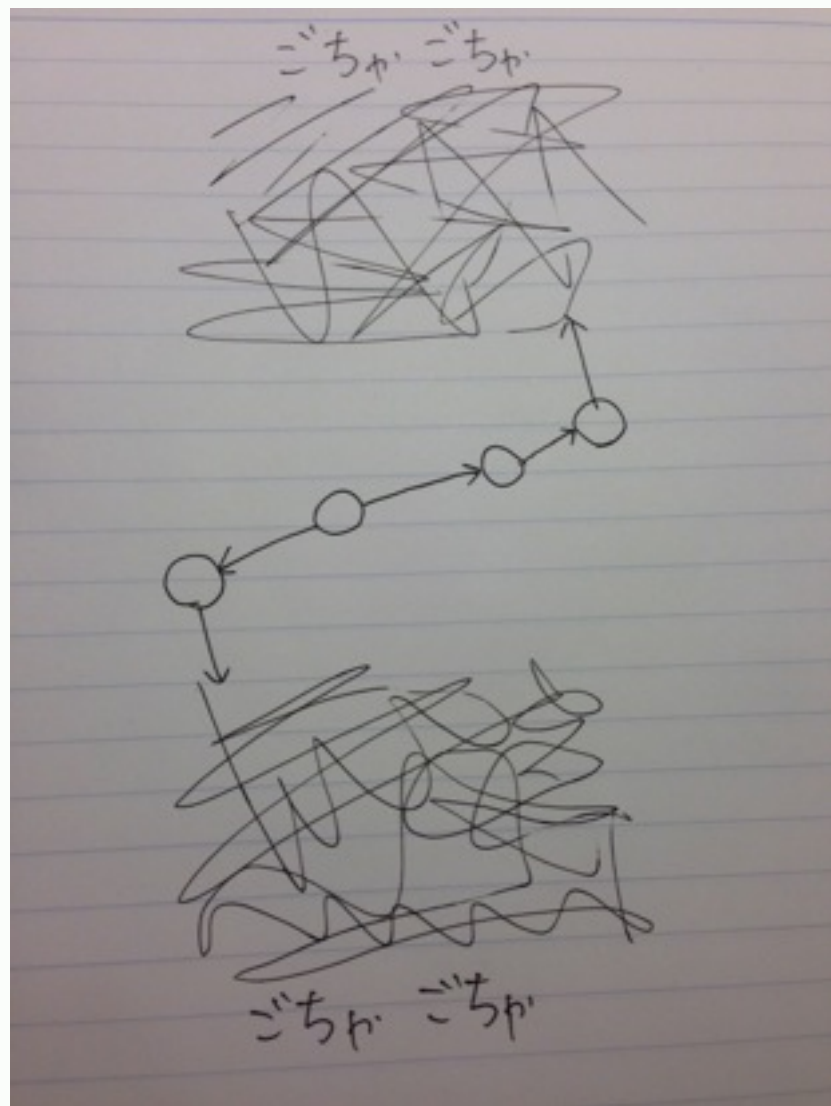
クリーク内に別の点 v ($u \neq v$) があるから(クリークは頂点数 2 以上と定義したネ)

$u \rightarrow v$ と $u \rightarrow w$ が両方存在するので (v, w) を結ぶことができる。

すると $v \rightarrow w$ と $v \rightarrow u$ も両方あることになるので (u, w) も結べる。

さいごのステップ(1/5)

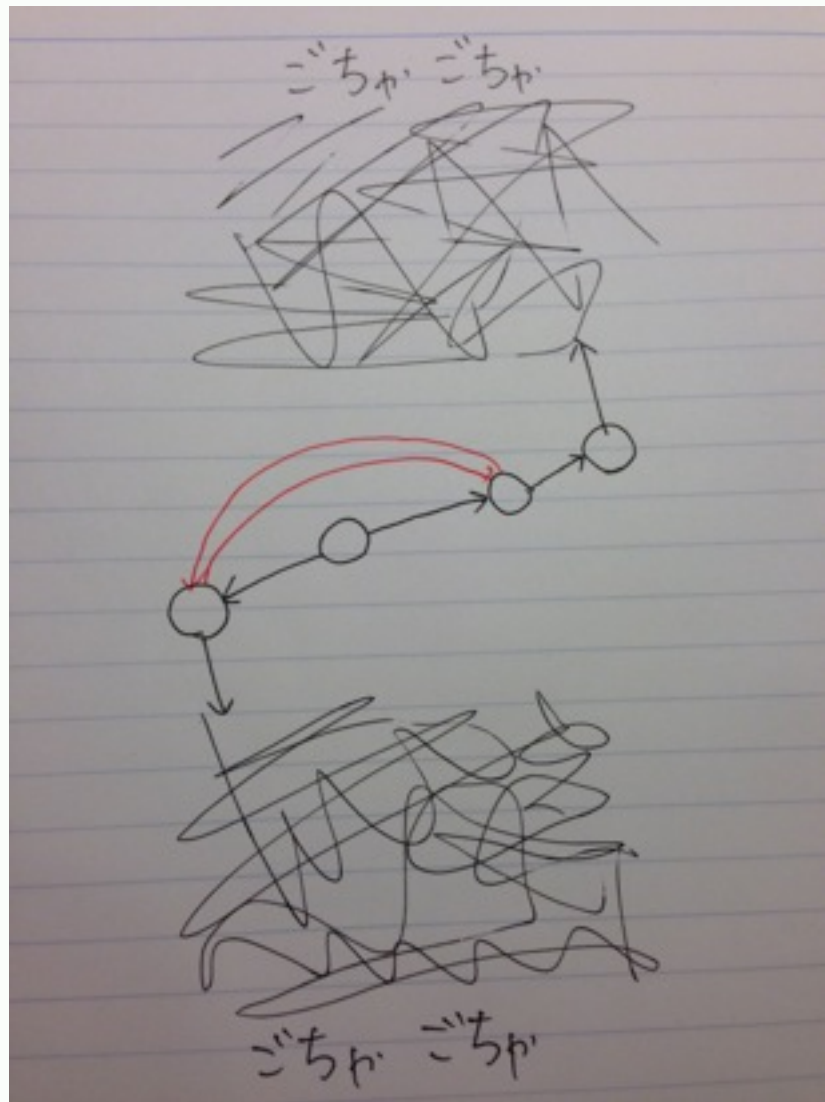
Making
Friends is Fun



クリークに入っていく辺は
普通はどうしようもないけど
2つのクリークが繋がっている
場合は？

さいごのステップ(2/5)

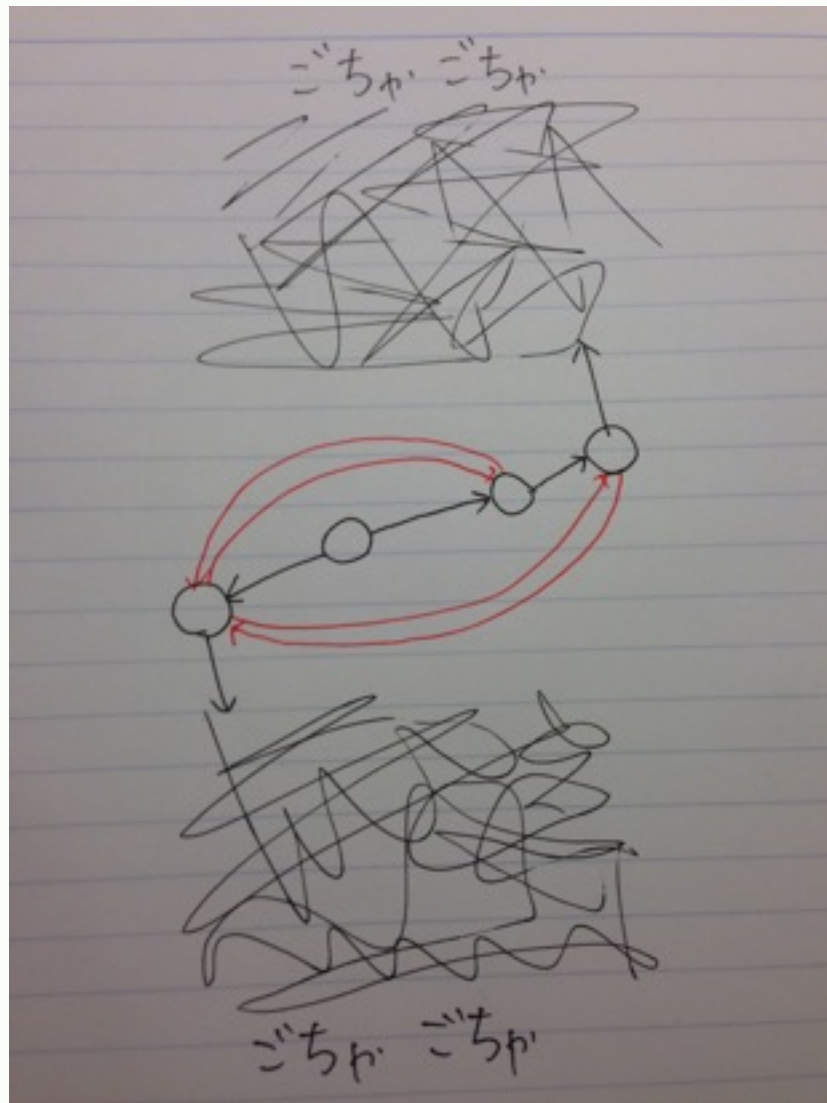
Making
Friends is Fun



平和のために
できることからやっ払いこう

さいごのステップ(3/5)

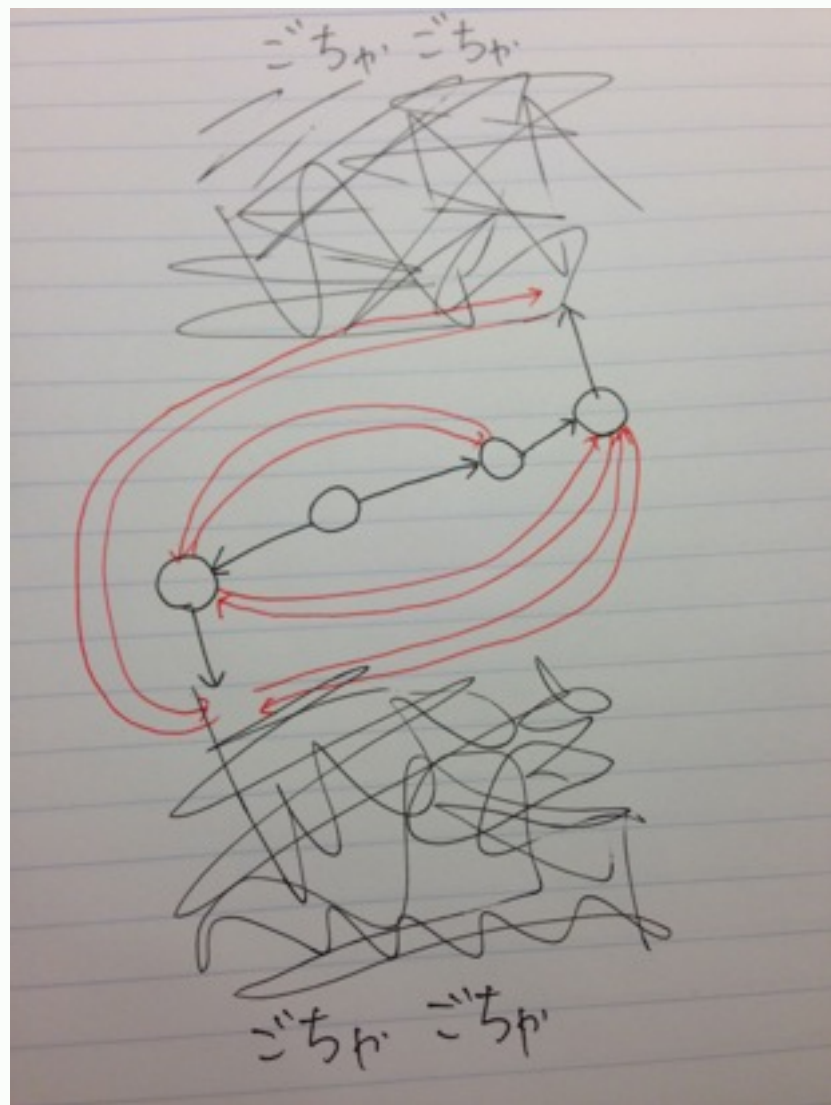
Making
Friends is Fun



まだつながらない

さいごのステップ(4/5)

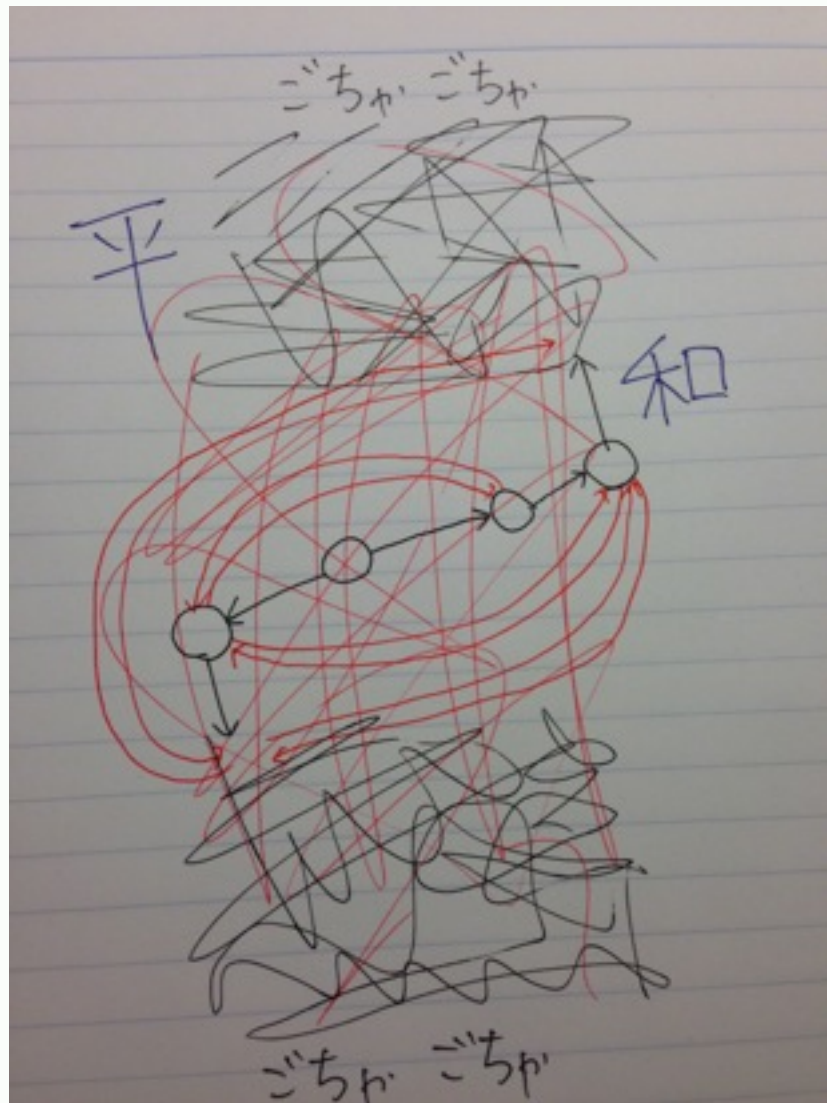
Making
Friends is Fun



つながりそう

さいごのステップ(5/5)

Making
Friends is Fun



ここまでくればもう
あとはなし崩しの繋がる！

【拡散希望】 平和

Making
Friends is Fun

性質4

最終形では、辺の方向を無視した連結成分において
クリークは高々 1 つしか存在しない。

なぜか？

具体例からもわかるように、
クリークが 2 個以上あればそれらは
繋がってさらに大きな 1 つのクリークになる。

ようやく解法へ

Making
Friends is Fun

ここまでの考察を積み重ねることで解法へ至る！

おさらいしておきましょう:

性質1: 操作の順番は不問

性質2: 2 辺以上出てる所から先はクリークになる

性質3: クリークから辺が出てる先は吸収できる

性質4: 連結成分ごとにクリークは高々 1 つ

ここまで来れば解法自体は単純です

辺が 2 本以上出ている頂点を見つけたら、
そこから到達できる頂点をクリークとしてまとめる。

以上。

実装

Making
Friends is Fun

この解法に至りさえすれば、
計算量は普通は $O(N^2+M)$ ぐらいには収まるはず
(前ページの解法を素直に実装するだけです)

ただしこれでは小課題 2 までしか解けません(35点)
($N \leq 5\,000$)

考察をもっと活用してより簡潔かつ高速な実装を！

満点解法

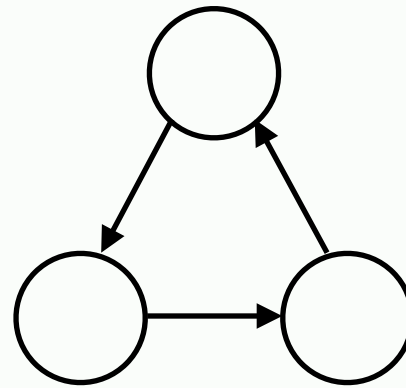
Making
Friends is Fun

- 入力が無向グラフだと思って**連結成分に分ける**
(以降は連結成分ごとに解けばOK → クリークは高々1個)
- **出て行く辺が 2 本以上**ある頂点たちから BFS なり DFS なりを行い、到達可能な点の個数を数える
(こうして数えた個数はこの連結成分にある唯一のクリークの頂点数に他ならない)
- 元々の辺のうち、**クリークに属さないもの**を数える
(BFS や DFS で、どの点がクリークに属すかは分かっている)
- 計算量は $O(N + M)$

強連結成分は意味ないです

Making
Friends is Fun

サイクルからはもう辺が増えない

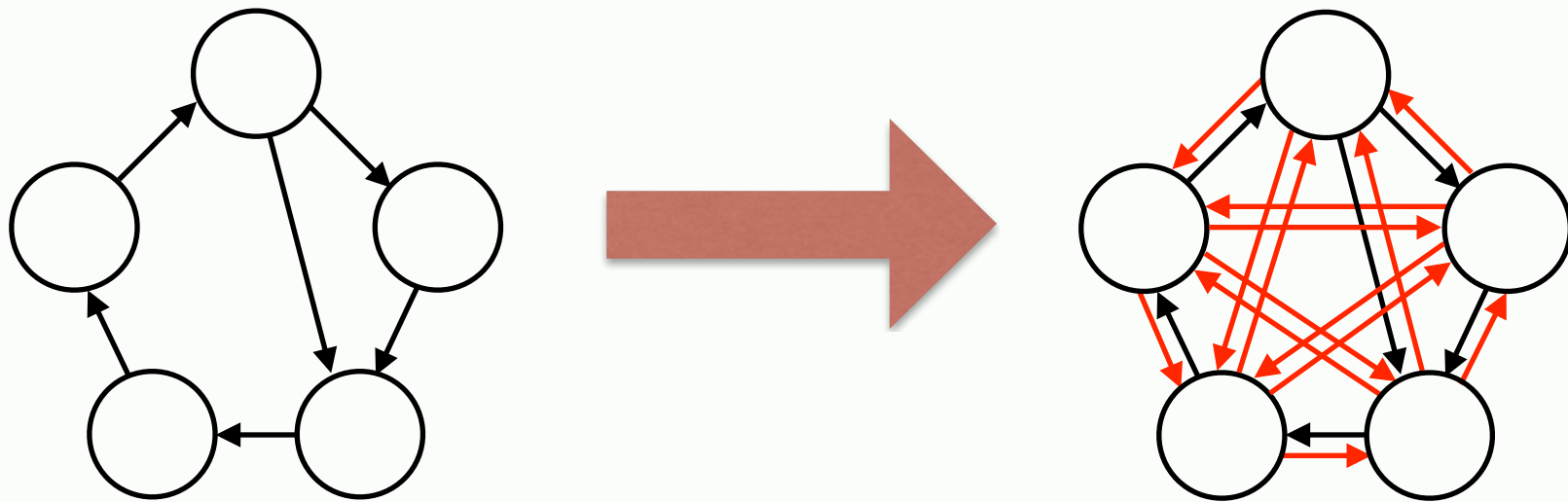


強連結成分分解すると、将来性のないサイクルと
将来性のあるつよい部分が同一視されてしまう
→やばい

long long int ans;

Making
Friends is Fun

世界全体が平和になることももちろんある(希望)



このとき辺の数は $N(N-1)$ 本

→最大 9 999 900 000 本は 32 ビットに収まらない！

おわりに

Making
Friends is Fun

問題について

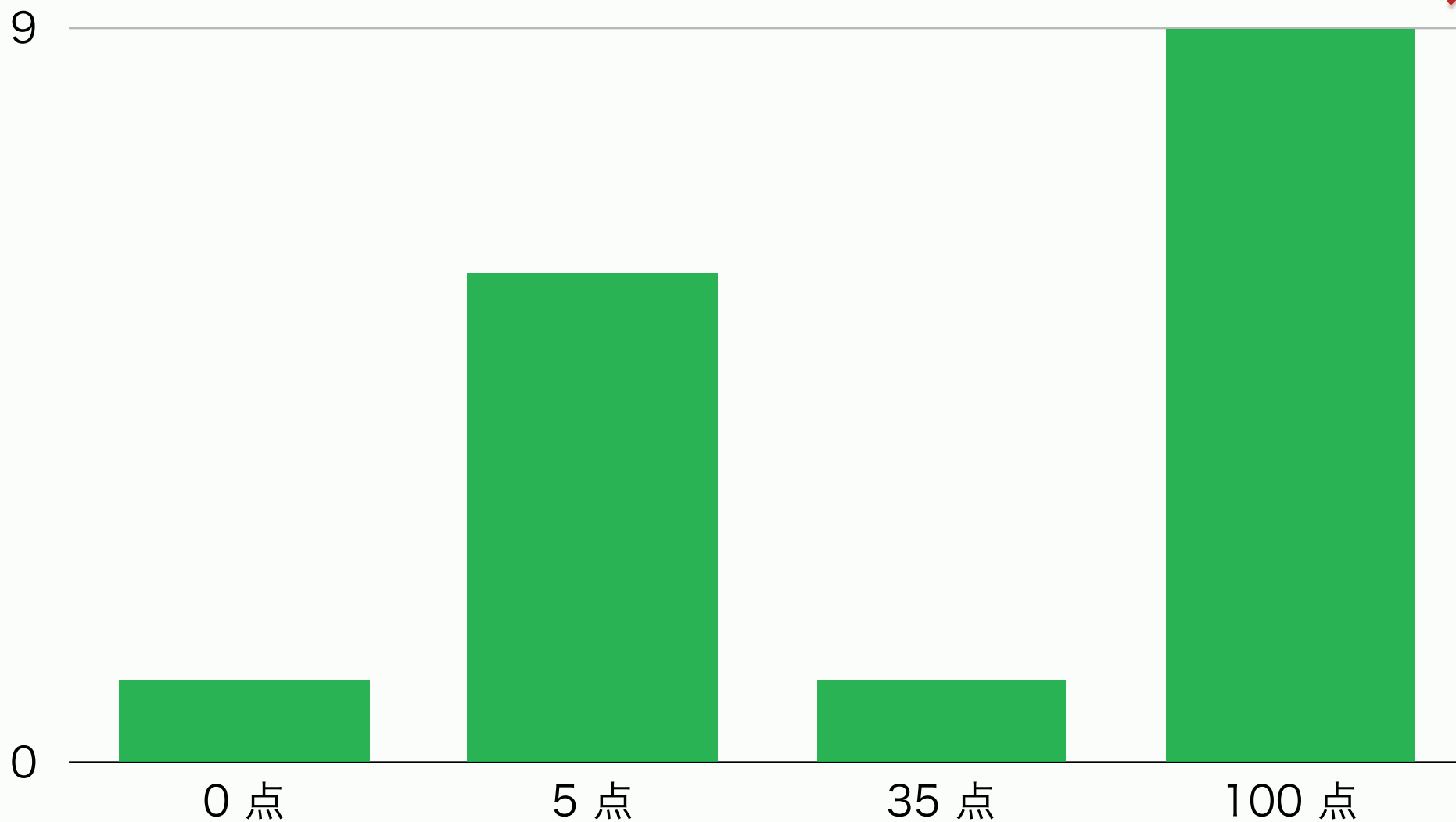
実装が簡潔で、**考察が本質的な**問題でした。
パッと見てよく分からないなあ、と思ったら
具体例を手で解いてみて性質を発見するのもアリです。

ちなみに

Union-Find を知っていれば、
クリークが高々 1 個、等の考察は使わなくても
けっこう簡単にコードが書けます。

得点分布

Making
Friends is Fun

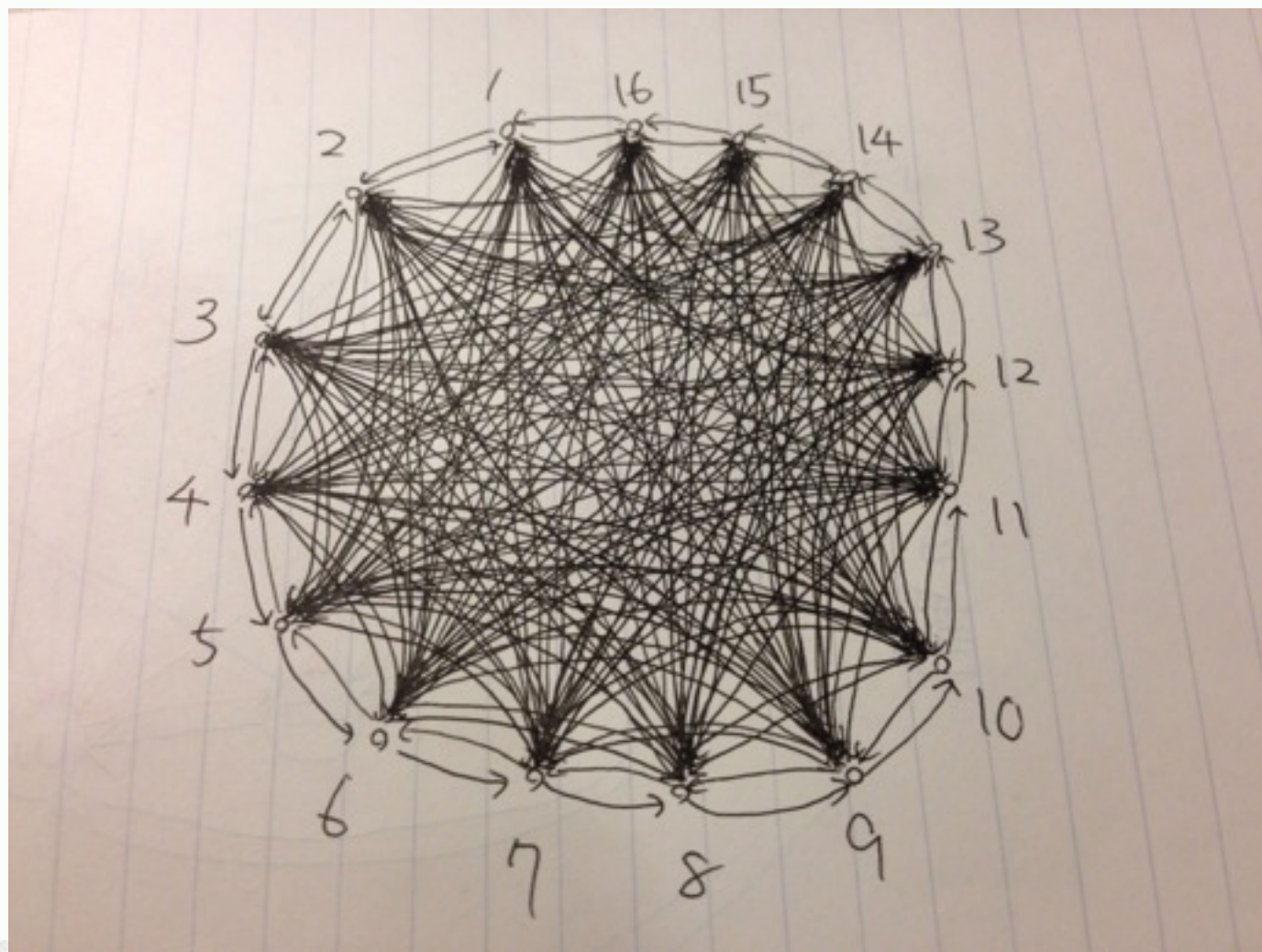


おまけ

Making
Friends is Fun

エージェントのお仕事体験！！！！

- $N = 16$
- 答えが最大
- 後悔してる
- ボールペン
ごめん



スタンプラリー解説



問題概要

問題文を読みましょう。

問題概要

上り線と下り線のある路線があり、各駅の

上り線のホーム → スタンプ台

下り線のホーム → スタンプ台

スタンプ台 → 上り線のホーム

スタンプ台 → 下り線のホーム

の移動時間と駅間の移動時間が定まっている。

各駅にひとつずつあるスタンプを集める最短時間を求めよ。

Subtask 1 (10点)

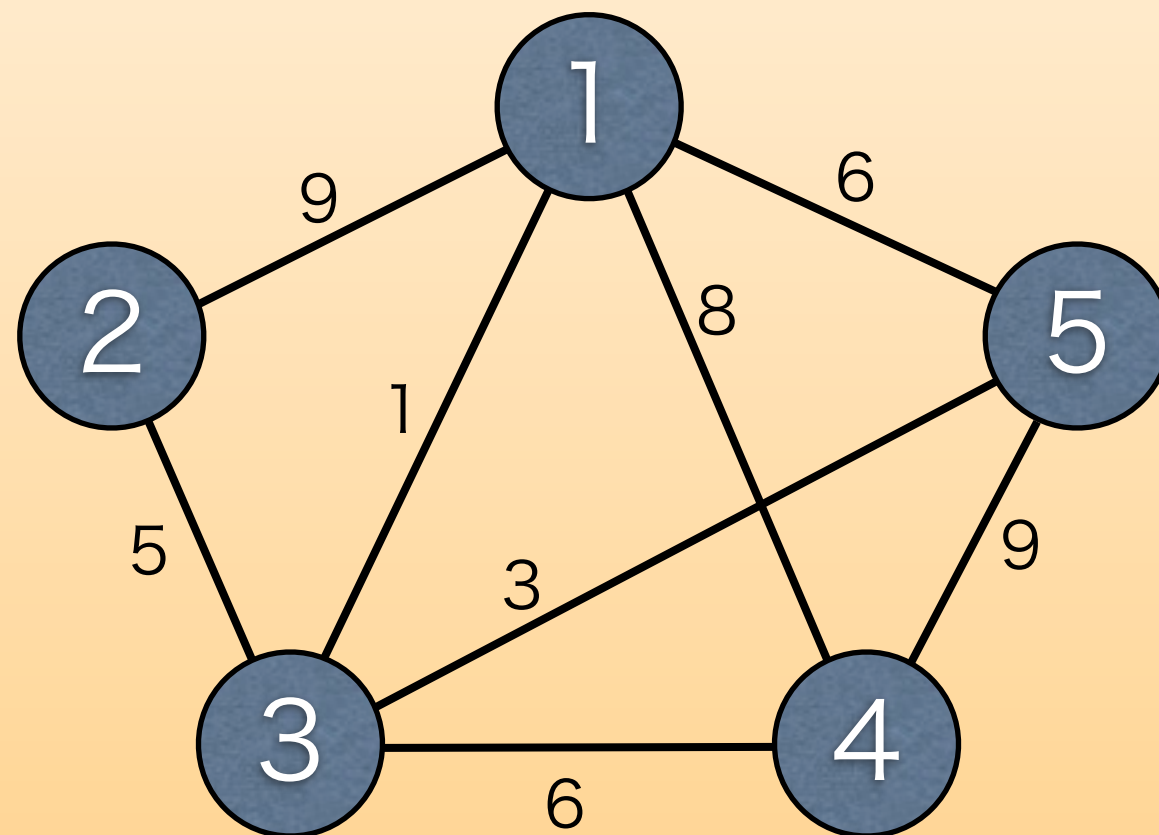
- $N \leq 16$

この問題は、**巡回セールスマン問題(TSP)**

巡回セールスマン問題を $O(2^N * N^2)$ で解けば、subtask1を正解し、10点を得られる。

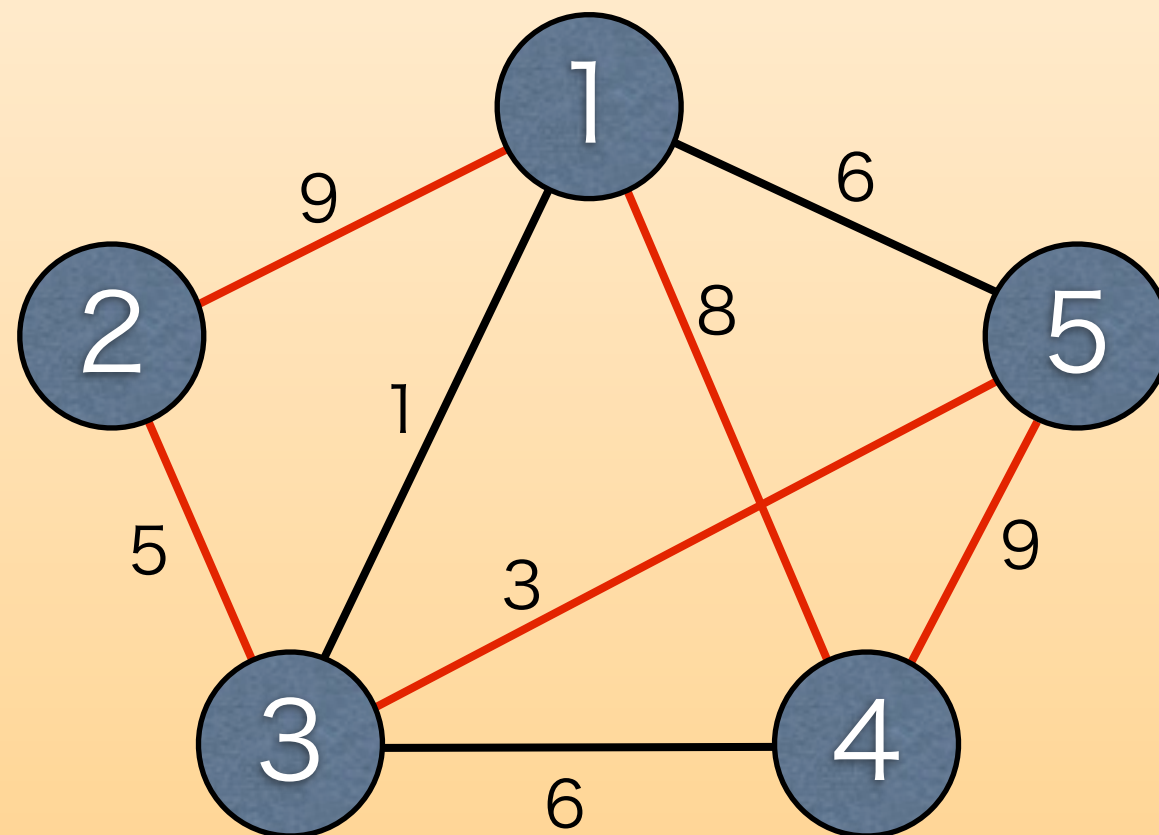
巡回セールスマン問題とは

重み付きグラフ（辺にコストがついたグラフ）が与えられた時に、最小コストで全ての頂点を1回ずつ巡る経路を求める問題。



巡回セールスマン問題とは

重み付きグラフ（辺にコストがついたグラフ）が与えられた時に、最小コストで全ての頂点を1回ずつ巡る経路を求める問題。



巡回セールスマン問題の解き方

DP[既に訪れた頂点の集合][今いる頂点]

状態数： $2^N * N$

遷移数：1つの状態あたり $O(N)$

計算量： $O(2^N * N^2)$

集合は2進数で表現すると良い。

グラフの作り方

巡回セールスマン問題として解くためには、

「駅 i のスタンプ台から駅 j のスタンプまで
行くための最短時間」

を適切に求めて完全グラフを作っておけば良い。

Subtask 2 (75点)

- ・ いい感じのDPがしたい。

とりあえず駅ごとに独立に考えることが出来ないかを考えてみる。

Subtask 2 (75点)

各駅のスタンプ台を通るための方法は、

- (A) 上り線から来て上り線に戻る : $V_i + U_i$
- (B) 下り線から来て下り線に戻る : $D_i + E_i$
- (C) 上り線から来て下り線に戻る : $V_i + E_i$
- (D) 下り線から来て上り線に戻る : $D_i + U_i$

の4通り。

Subtask 2 (75点)

図にしてみると、



という感じ。

Subtask 2 (75点)

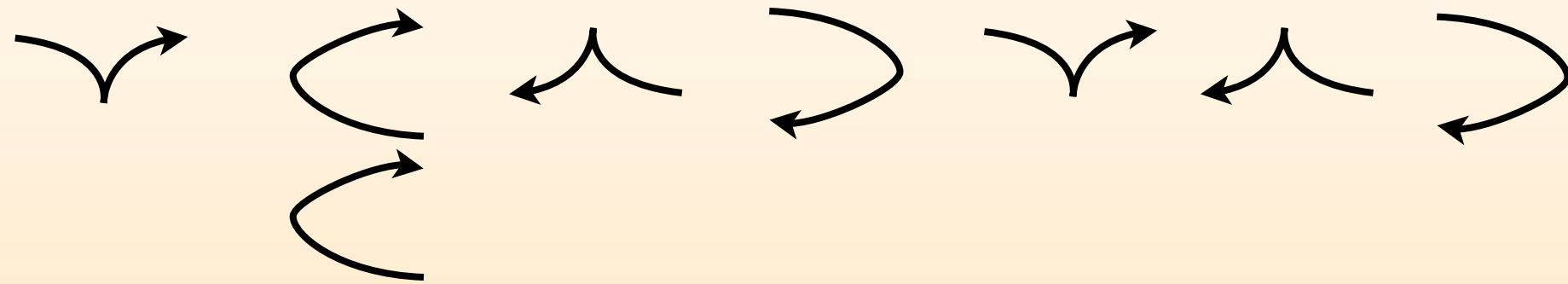
要するに、各場所に

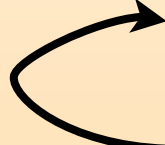
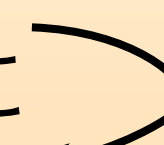
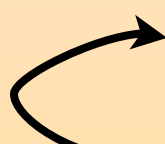


を 1 個以上ずつ並べる。

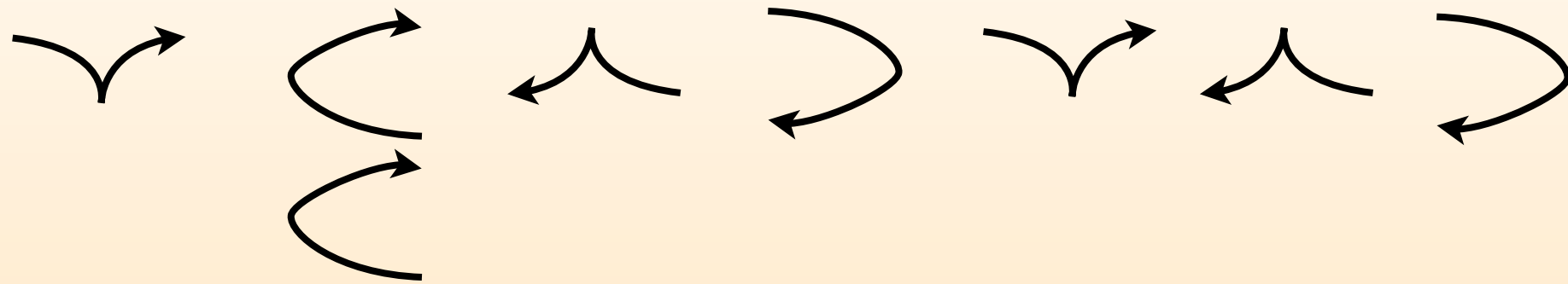
この並べ方が満たすべき性質を考察する。

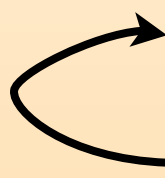
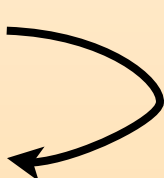
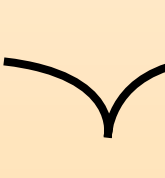
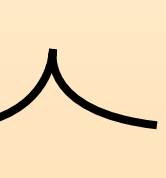
列が満たすべき性質

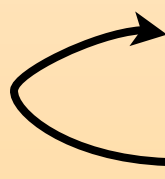
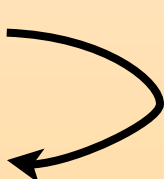


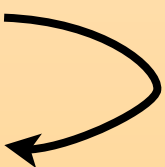
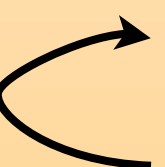
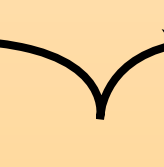
- 各場所において  の個数が常に  より多い
-  の合計が  の合計と等しい
-  は  が  より多いところだけに置ける

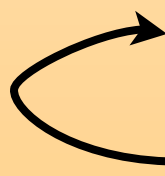
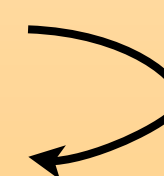
意味のある列の性質



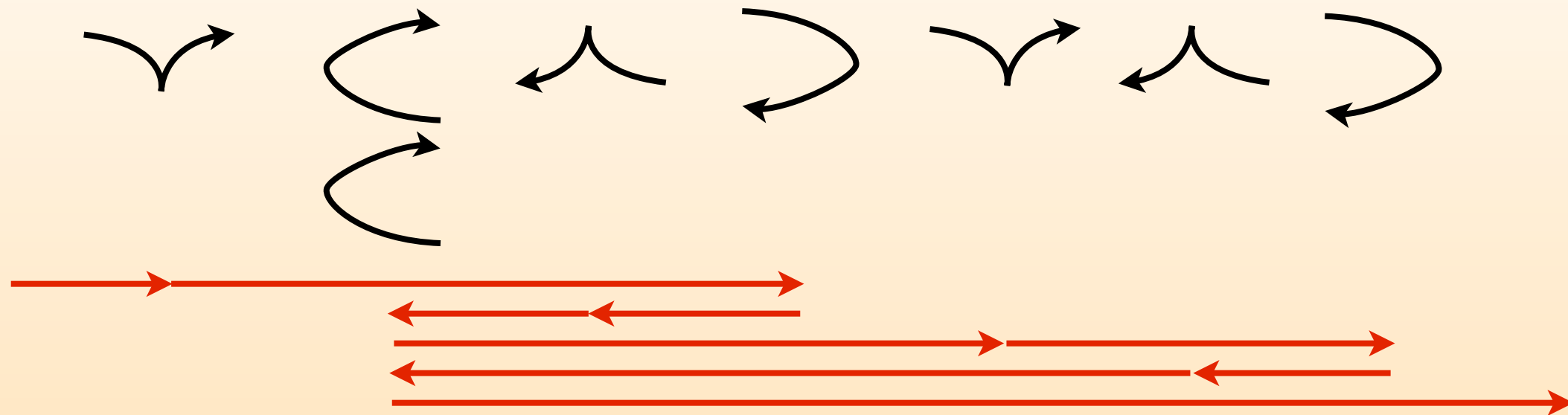
•  や  があるところには  や  は不要

•  と  を同じところに置くのは無駄

理由:  +  よりも  の方が真に小さいから

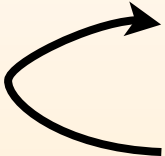
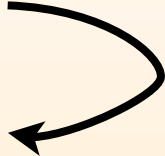
•  や  の個数は高々N個くらいで十分

電車での移動は？

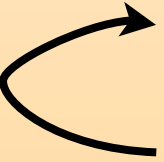


- $T \times (\text{各駅間を囲んでる } \text{↻} \text{ の数 } \times 2 + 1)$

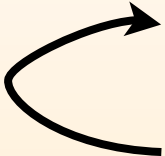
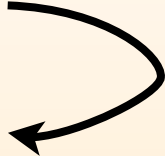
DPの状態と遷移

DP[今注目してる駅][ の個数 -  の個数]

DP[i][j]からの遷移：

-  だけ使う
 -  だけ使う ($j \geq 1$ のときのみ)
 -  だけをいくつか使う
 -  だけをいくつか使う
- $\left. \begin{array}{l} \text{ } \end{array} \right\} O(1)$
- $\left. \begin{array}{l} \text{ } \end{array} \right\} O(N)$

DPの計算量

DP[今注目してる駅][ の個数 -  の個数]

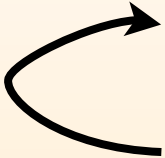
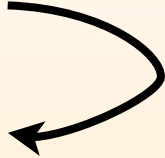
状態数 : N^2

遷移数 : $O(N)$

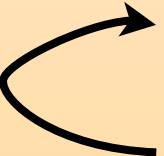
計算量 : $O(N^3)$

ここまででSubtask1とSubtask2に正解し、合計85点を得ることが出来る。

Subtask 3 (15点)

DP[今注目してる駅][の個数 - の個数]

DP[i][j]からの遷移：

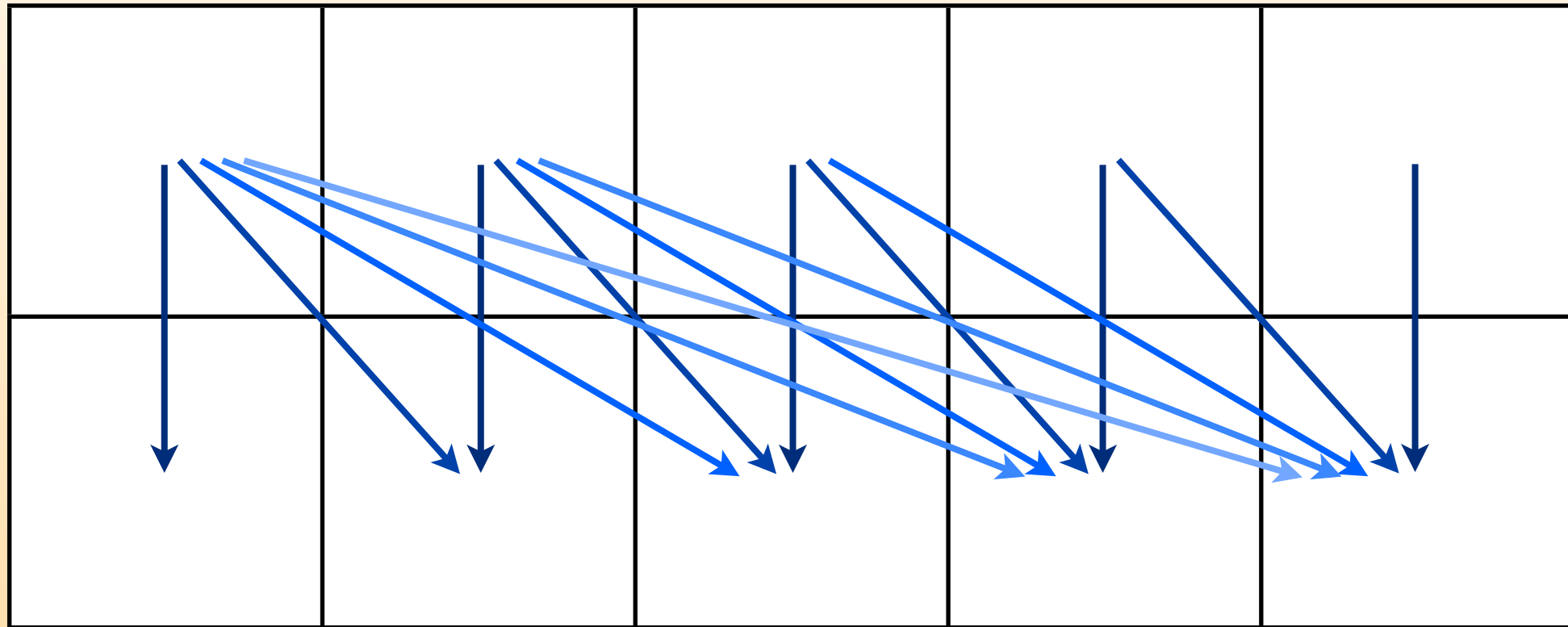
- だけ使う
 - だけ使う ($j \geq 1$ のときのみ)
 - だけをいくつか使う
 - だけをいくつか使う
-  $O(1)$ $O(N)$

ここを改善する

← だけをいくつか使う

← の個数 - → の個数

駅の番号

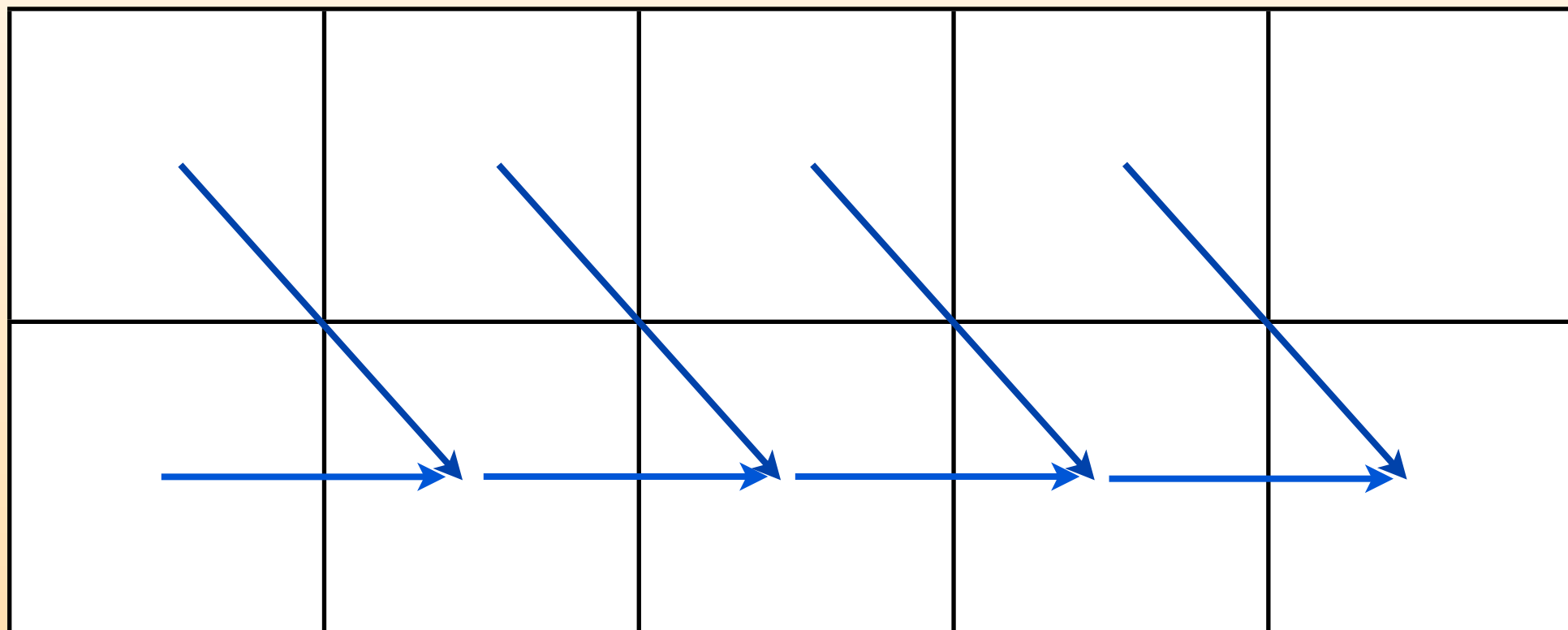


これを

← だけをいくつか使う

← の個数 - → の個数

駅の番号



こうじゃ

(よくある改善)

← だけをいくつか使う

式で書くと、

$$DP[i+1][j] = \min(DP[i][j-k] + \text{hoge} \mid k = 0 \sim j-1)$$



$$DP[i+1][j] = \min(DP[i][j-1] + \text{hoge}, DP[i+1][j-1] + \text{piyo})$$

みたいな雰囲気

→ も逆方向に同様なことをすれば良い。

改善したDPの計算量

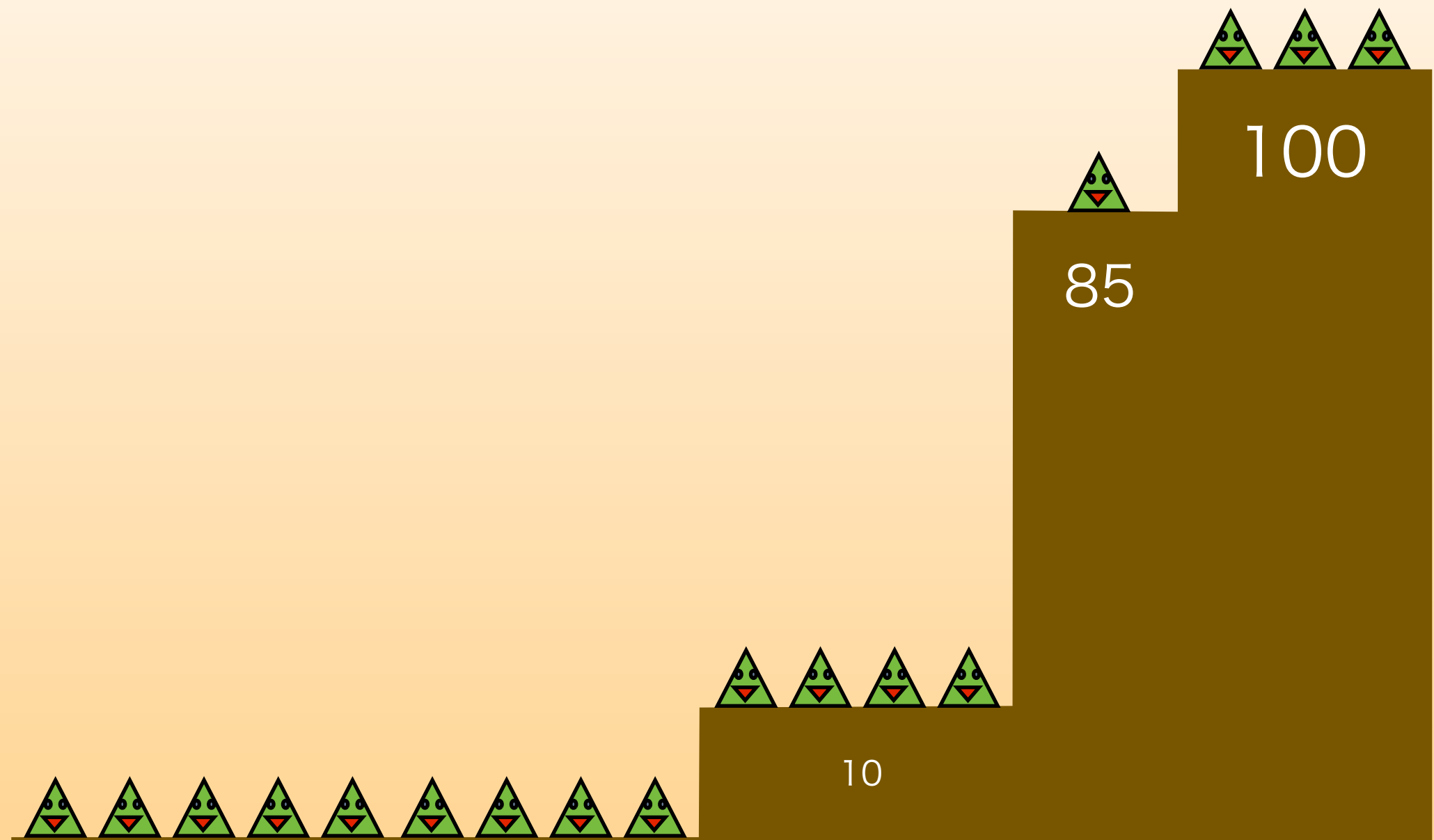
状態数： N^2

遷移数： $O(1)$

計算量： $O(N^2)$

これで満点が取れる。

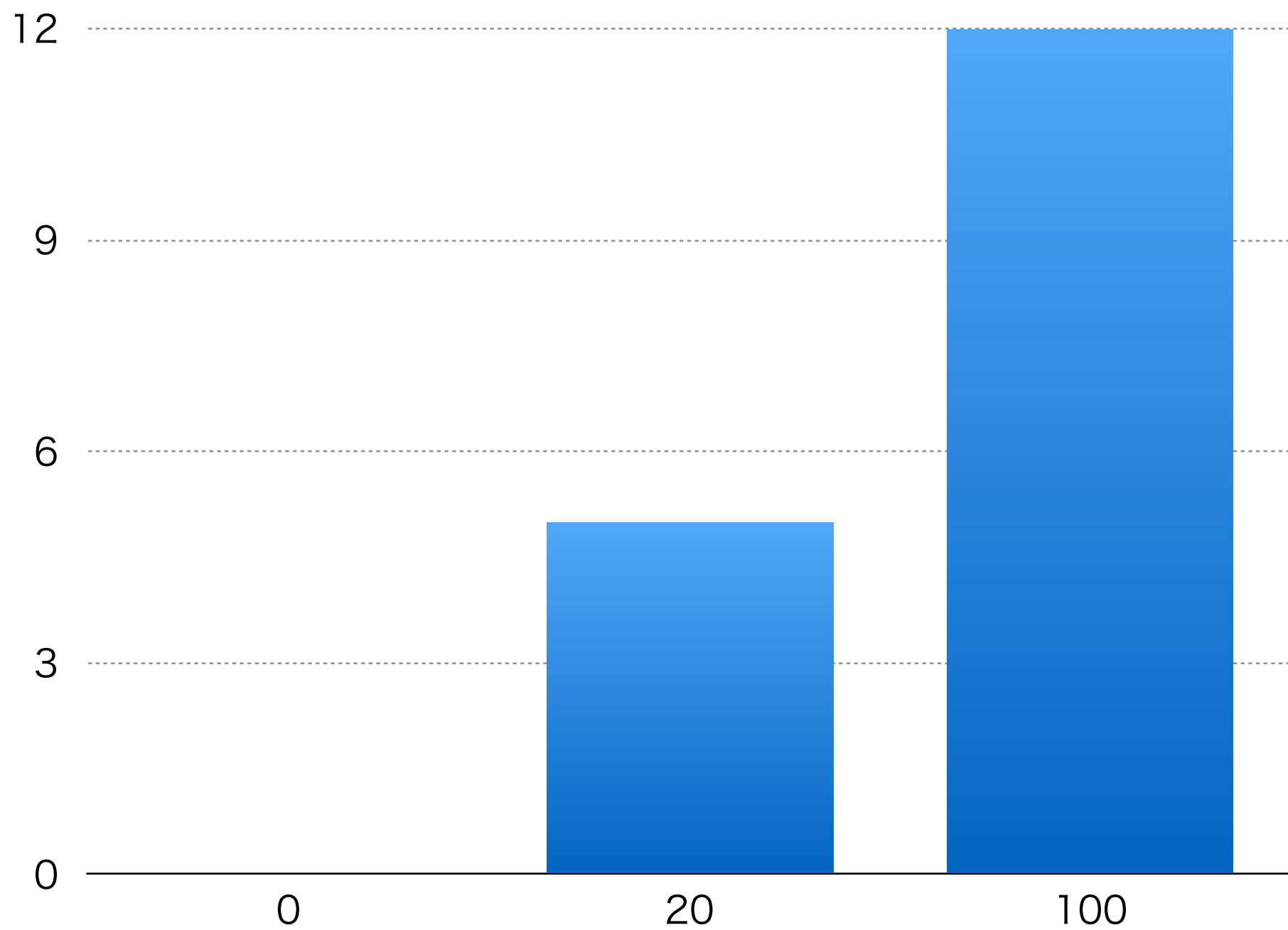
統計



JOIOJI

平野湧一郎 (nai)

得点分布



問題概要

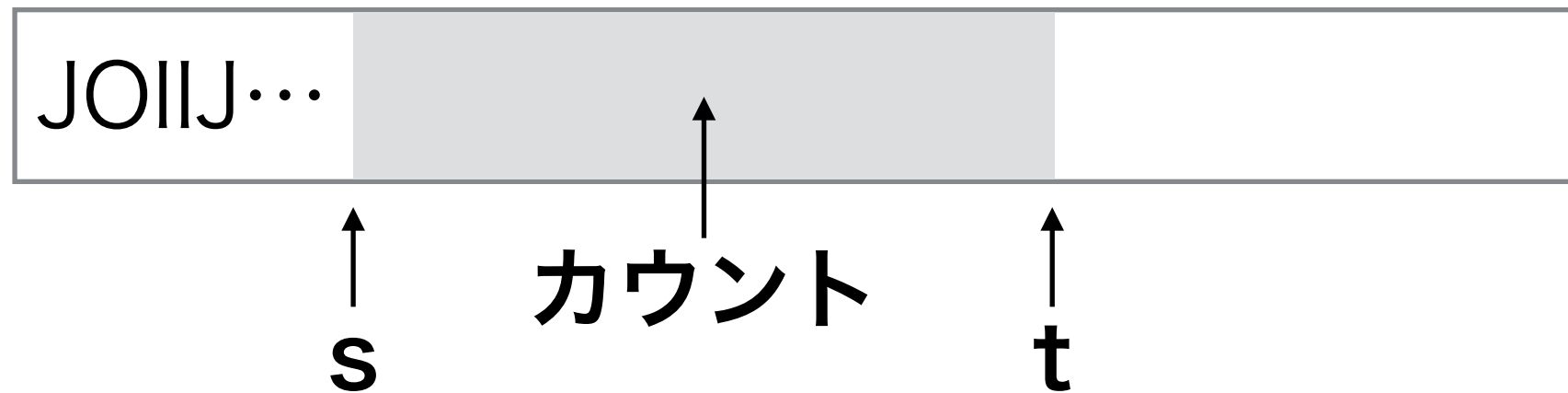
- ・ JOIOJIさんは自分の名前が大好き
- ・ 与えられた文字列の部分文字列で、J, O, I がそれぞれ同じ数だけ現れるものの長さの最大値を求めよ

JOIIJOJOI

解法1 (5点)

- ・ 総当りで全部調べる.
- ・ $O(N^3)$

- ・ 開始地点： $O(N)$
- ・ 終了地点： $O(N)$
- ・ 文字数カウント： $O(N)$



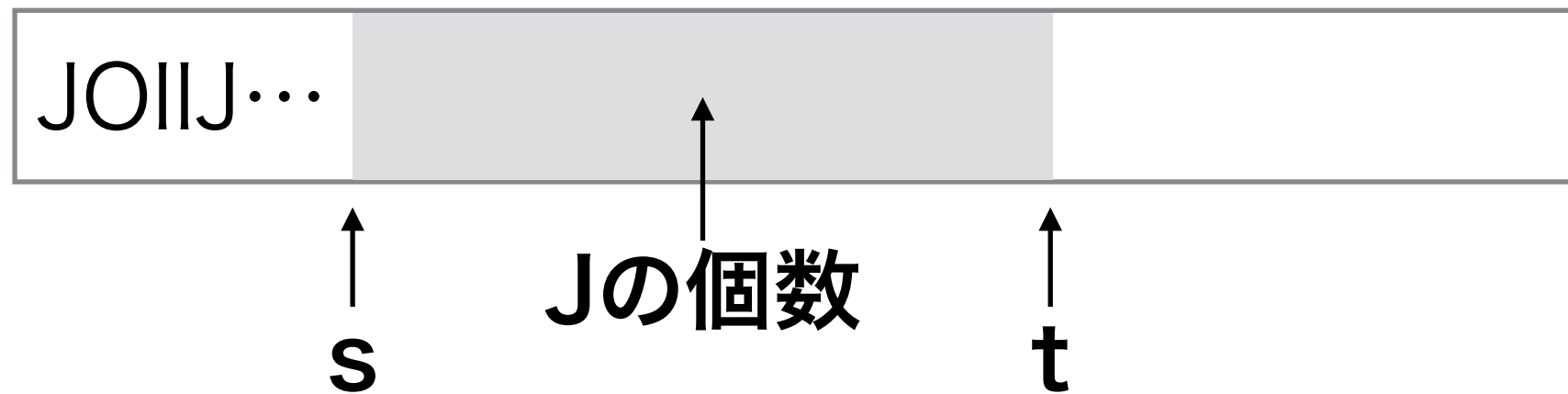
なんとかして $O(N)$ を
ひとつ減らせないか？

解法2 (20点)

- ・ 「文字数カウント」を爆速で行う
- ・ **累積和**を使う
- ・ $O(N^2)$

累積和とは

- たとえば「sからtまでの 'J' の個数」を知りたい



- ・ 「1からtまでの 'J' の個数」 - 「1から(s-1)までの 'J' の個数」 で求まる
- ・ あらかじめ 「1からxまでの 'J' の個数」 を全てのxについて求めておけば, 引き算1回で計算できる

JOIJJ...

=

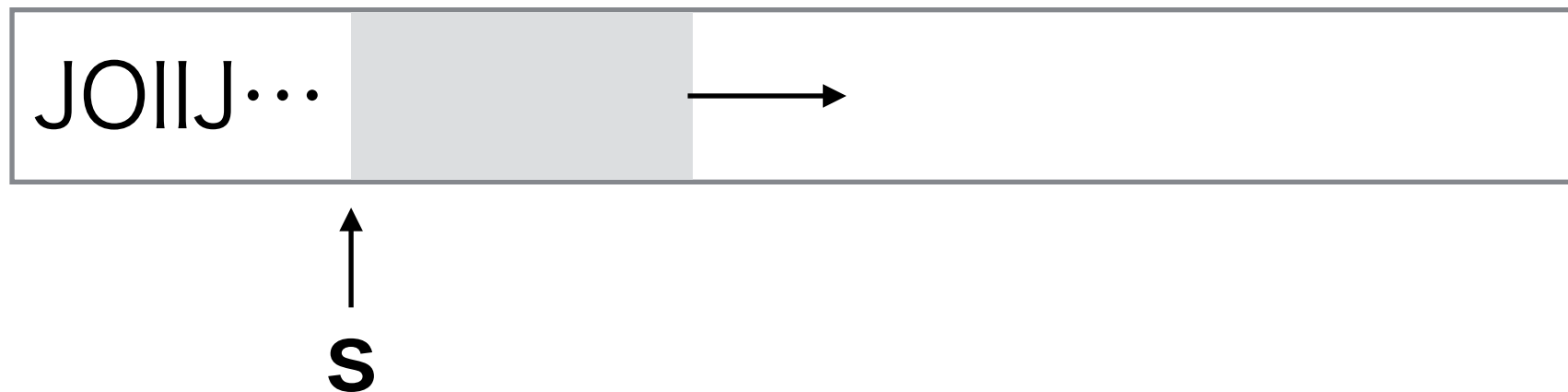
JOIJJ...

-

JOIJJ...

解法2' (別解)

- ・ 「終了地点」を固定せず考える
- ・ 開始地点を固定し，文字を数えながら終了地点を右にずらしていく
- ・ $O(N^2)$



解法2の考察

- ・ 解法2(累積和)では, 「開始地点」と「終了地点」をペアで考えていた
- ・ それぞれを独立で考えられないか？

条件の考察

- ・ 「1からxまでの 'J' の個数」を $J[x]$ と書くことにする (0, 1 も同様)
- ・ 「sからtまでの 'J' の個数」 = $J[t] - J[s-1]$

- ・ 「sからtまでの 'J' の個数」 = 「sからtまでの 'O' の個数」 となるための条件は,

- ・ $J[t] - J[s-1] = O[t] - O[s-1]$

- ・ 移項してみる

- ・ $J[\textcolor{red}{t}] - O[\textcolor{red}{t}] = J[\textcolor{blue}{s-1}] - O[\textcolor{blue}{s-1}]$

- ・ 独立して扱えそうになってきた！

条件の言い換え

- ・ $JO[x] := J[x] - O[x]$
- ・ $JI[x] := J[x] - I[x]$ とする
- ・ 「sからtまでの 'J', 'O', 'I' の個数がそれぞれ等しい」ための必要十分条件は,
 - ・ $JO[t] = JO[s-1]$ かつ
 - ・ $JI[t] = JI[s-1]$

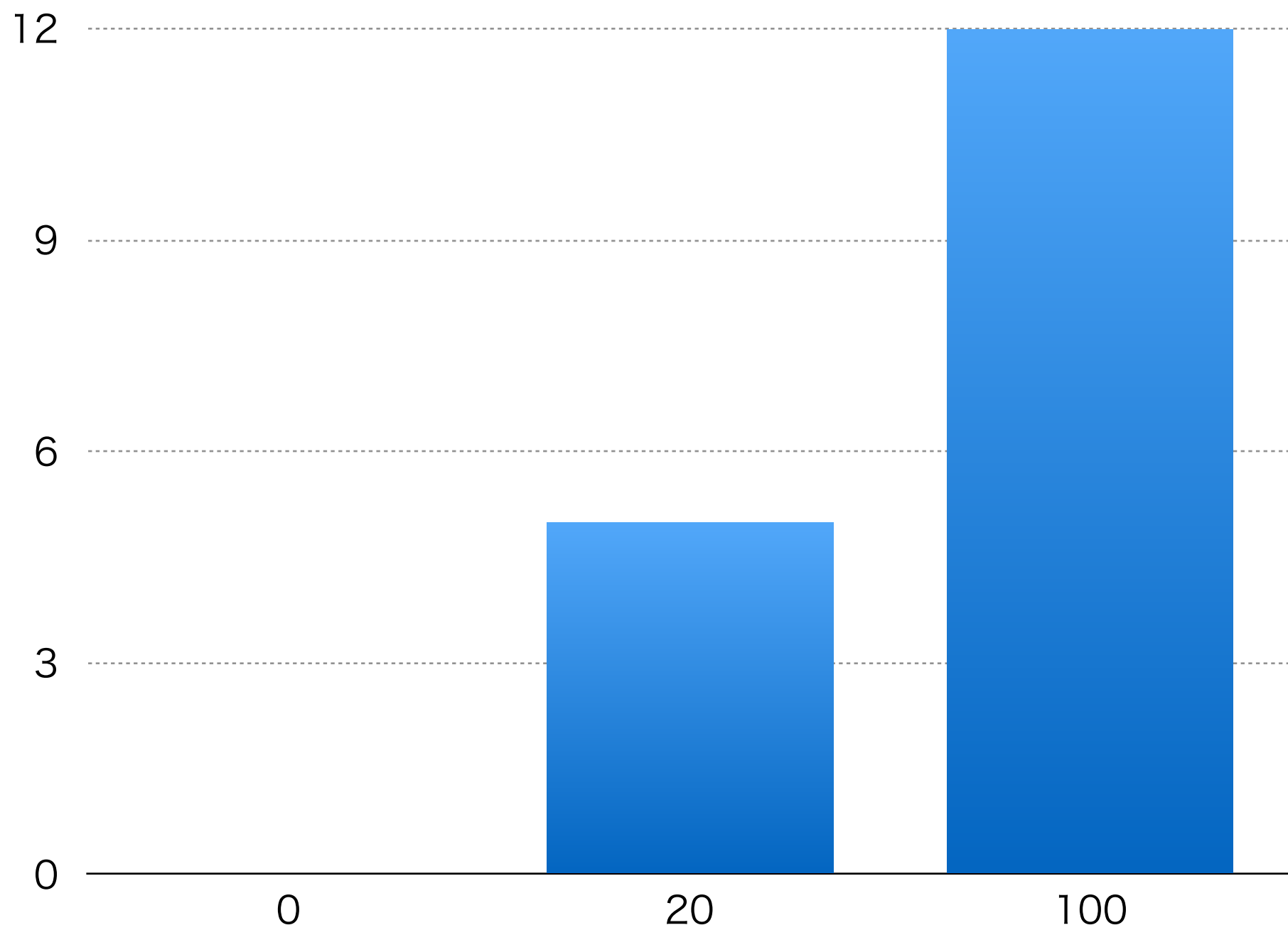
解法3 (想定解)

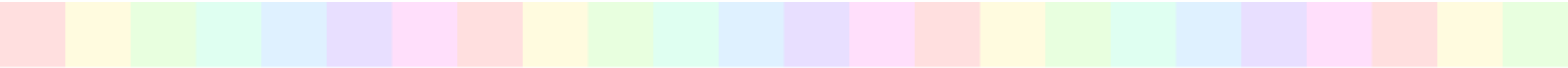
- ・ 各 x について, $JO[x]$, $Jl[x]$ を求めておく
- ・ ソートするなどして, $JO[x] = JO[y]$ かつ $Jl[x] = Jl[y]$ となるような x, y をまとめる
- ・ ↑ のような x, y ($x < y$) で, $y - x$ の最大値が答え
- ・ $O(N \log N)$

まとめ

- ・ 「sからtまでの○○の個数」→累積和！
- ・ 二次元累積和も頻出
- ・ 条件を変形し，2つ以上の変数を独立して考えるようにする

得点分布 (再掲)

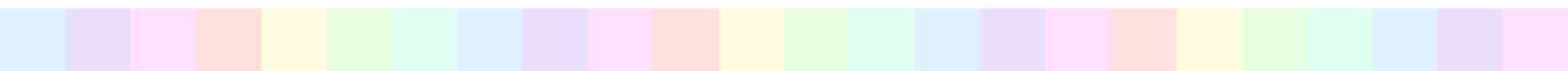




かかし (Scarecrows)

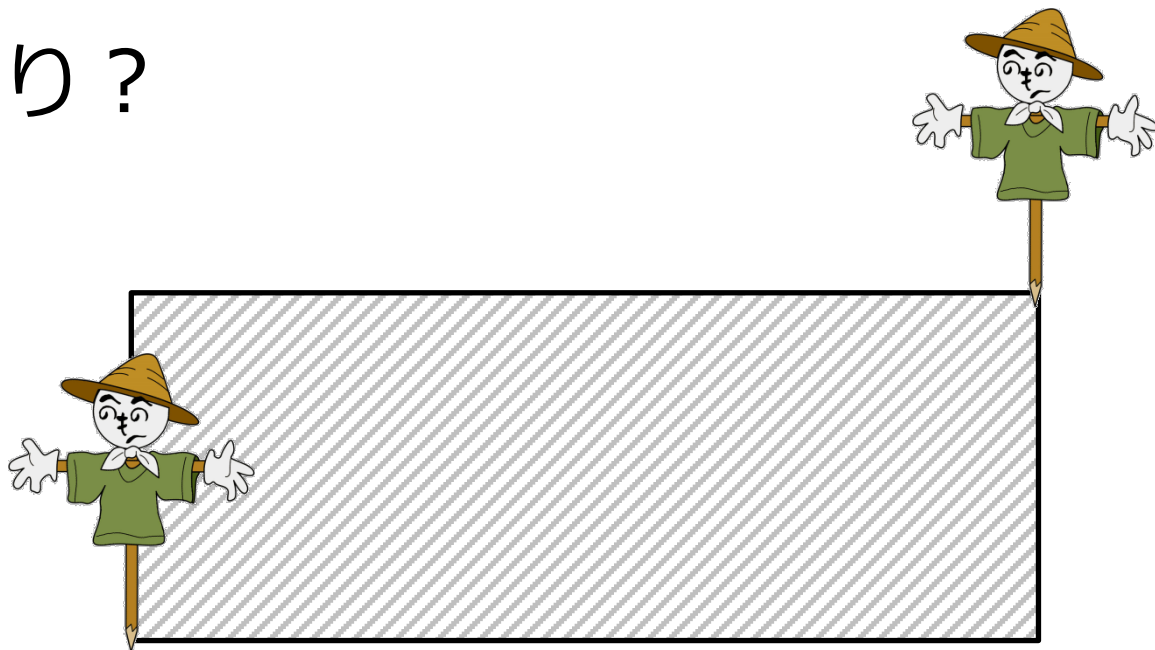
JOI 春合宿 2014 Day 3

解説： 保坂 和宏



問題概要

- N 個の点 ($N \leq 200\,000$)
- 左下と右上の点を選んで長方形を作り、内部に他の点が入らないようにする
- 何通り？

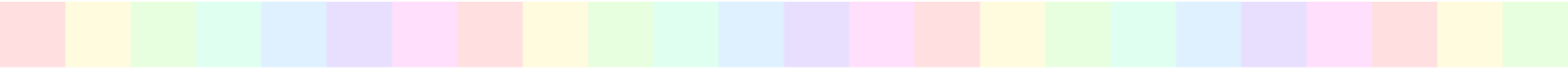


小課題 1 (5 点)

- $N \leq 400$
- 左下と右上の点を選んで長方形を作り, 内部に他の点が入らないようにする
 - 左下を決める : $O(N)$
 - 右上を決める : $O(N)$
 - 他の点が入っているか : $O(N)$
- $O(N^3)$ 時間

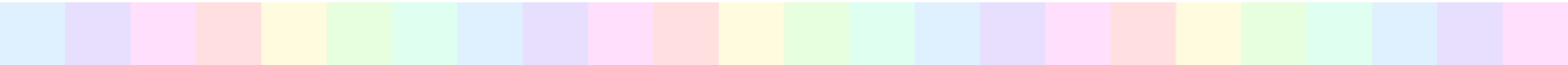
小課題 2 (10 点)

- $N \leq 5\,000$
- 左下と右上の点を選んで長方形を作り、内部に他の点が入らないようにする
 - 左下を決める : $O(N)$
 - 右上を決める : $O(N)$
 - 他の点が内部に入っているか : $O(N)$
 - これを高速にできないか？

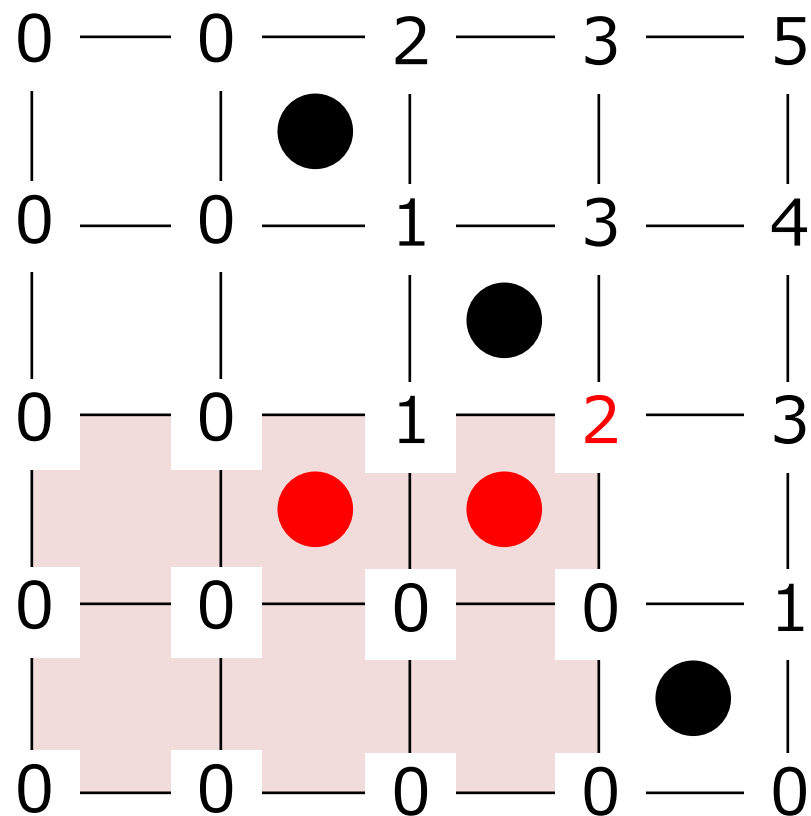


小課題 2 (10 点)

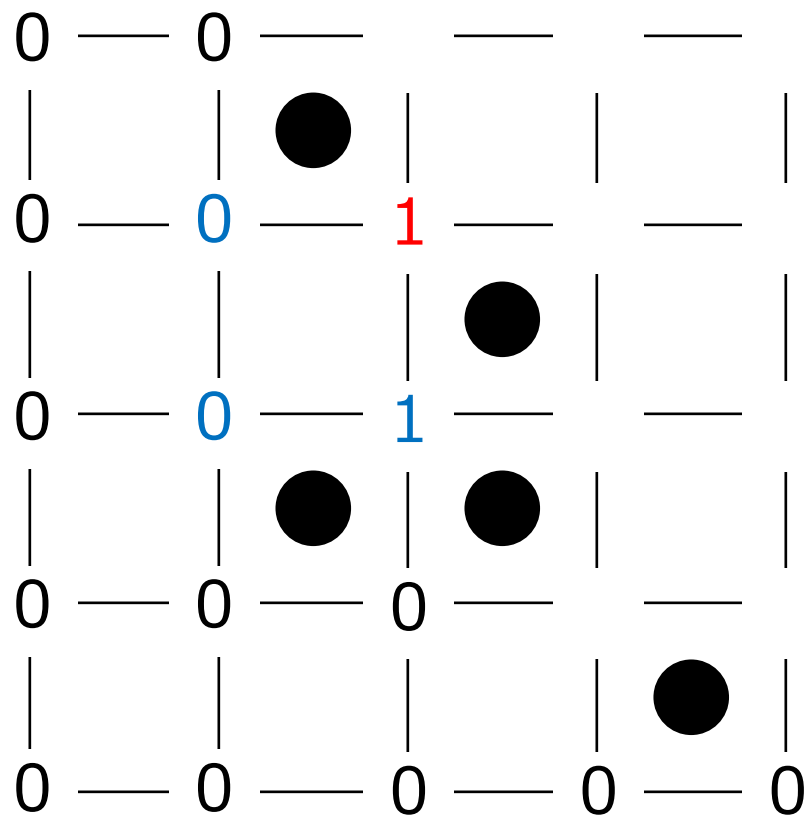
- 長方形を指定したとき, 内部の点の個数を高速に数えたい



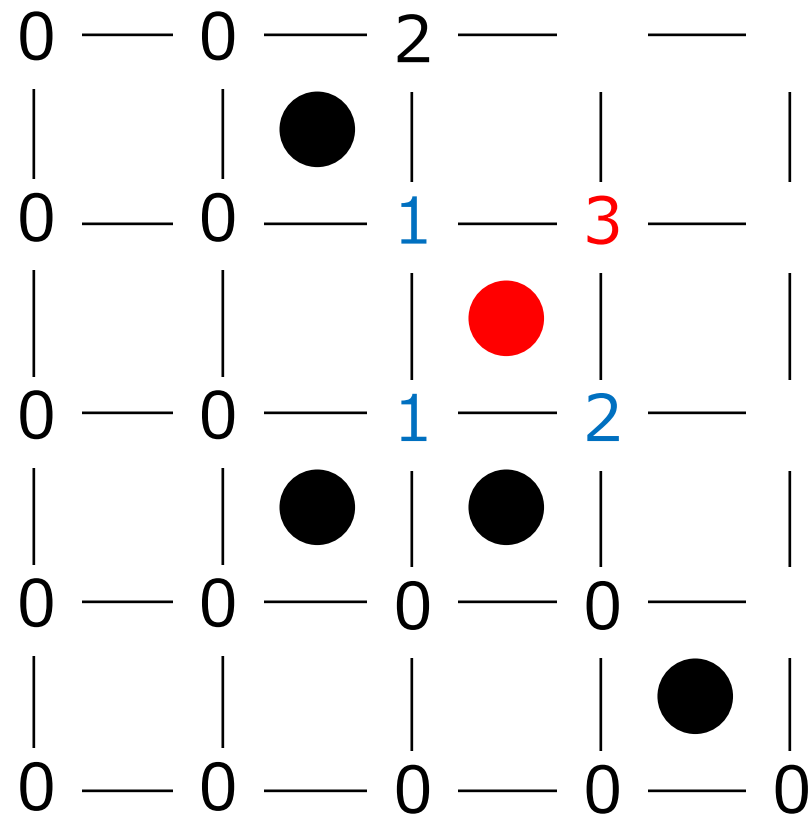
累積和



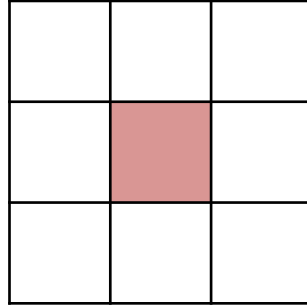
累積和



累積和



累積和



$$= \begin{array}{|c|c|c|} \hline & & \\ \hline \text{red} & \text{red} & \\ \hline \text{red} & \text{red} & \\ \hline \end{array} - \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \text{red} & \text{red} & \\ \hline \end{array} - \begin{array}{|c|c|c|} \hline & & \\ \hline \text{red} & & \\ \hline \text{red} & & \\ \hline \end{array} + \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \text{red} & & \\ \hline \end{array}$$

小課題 2 (10 点)

- 座標圧縮 ($O(N \log N)$)
- 累積和を計算 ($O(N^2)$)
- 左下と右上を決めて ($O(N^2)$) 内部の点の個数を数える ($O(1)$)
- $O(N^2)$ 時間

小課題 2 (10 点)

- 別解
 - 後述の満点解法をナイーブにするなどで $O(N^2)$ 時間解法がいくつか考えられる

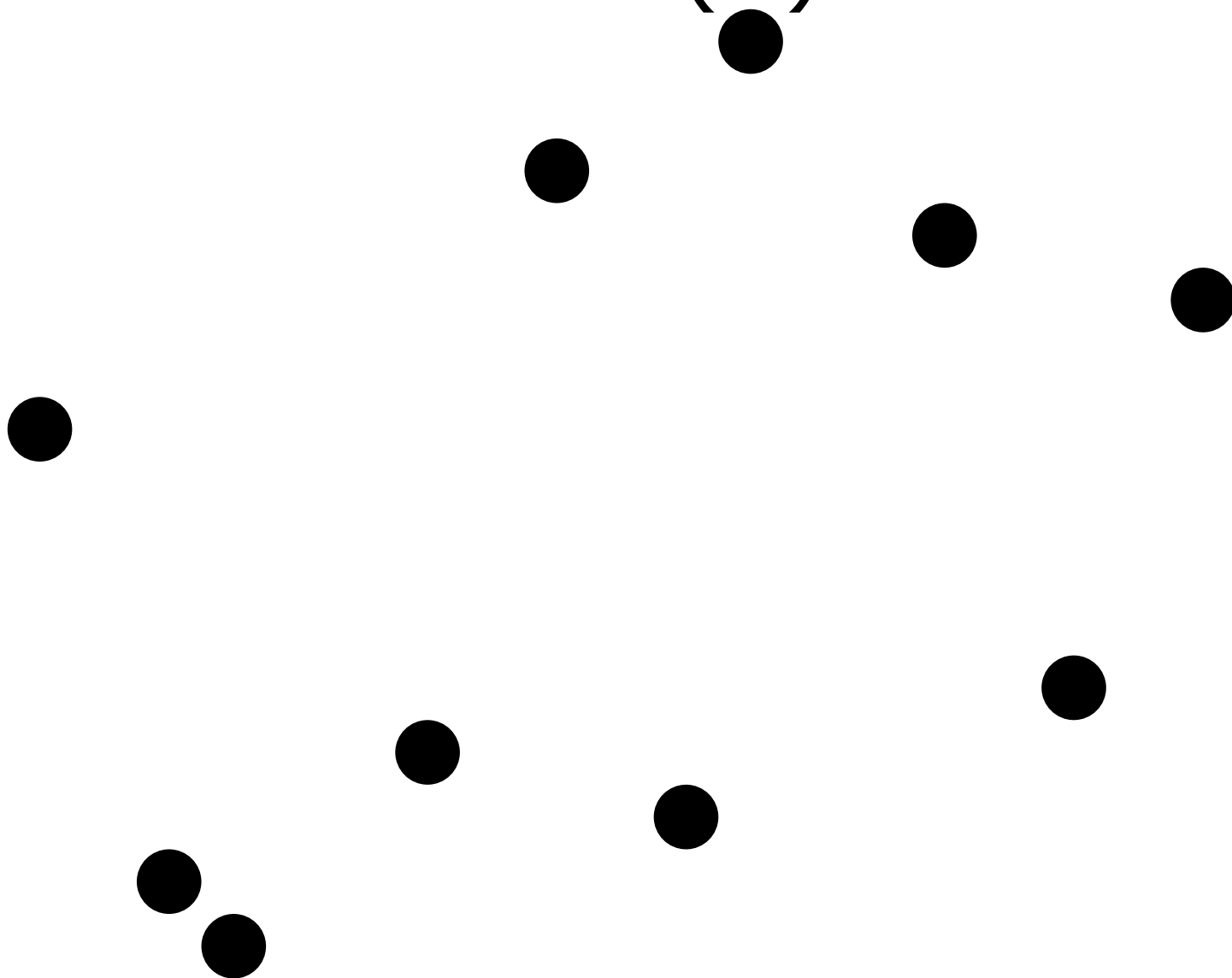
小課題 3 (85 点)

- $N \leq 200\,000$
- 左下と右上の組をすべて試せない
 - 左下 (or 右上) の点のみ決めて, 長方形が条件を満たすような右上 (or 左下) の点の個数を数える, ができるとよい

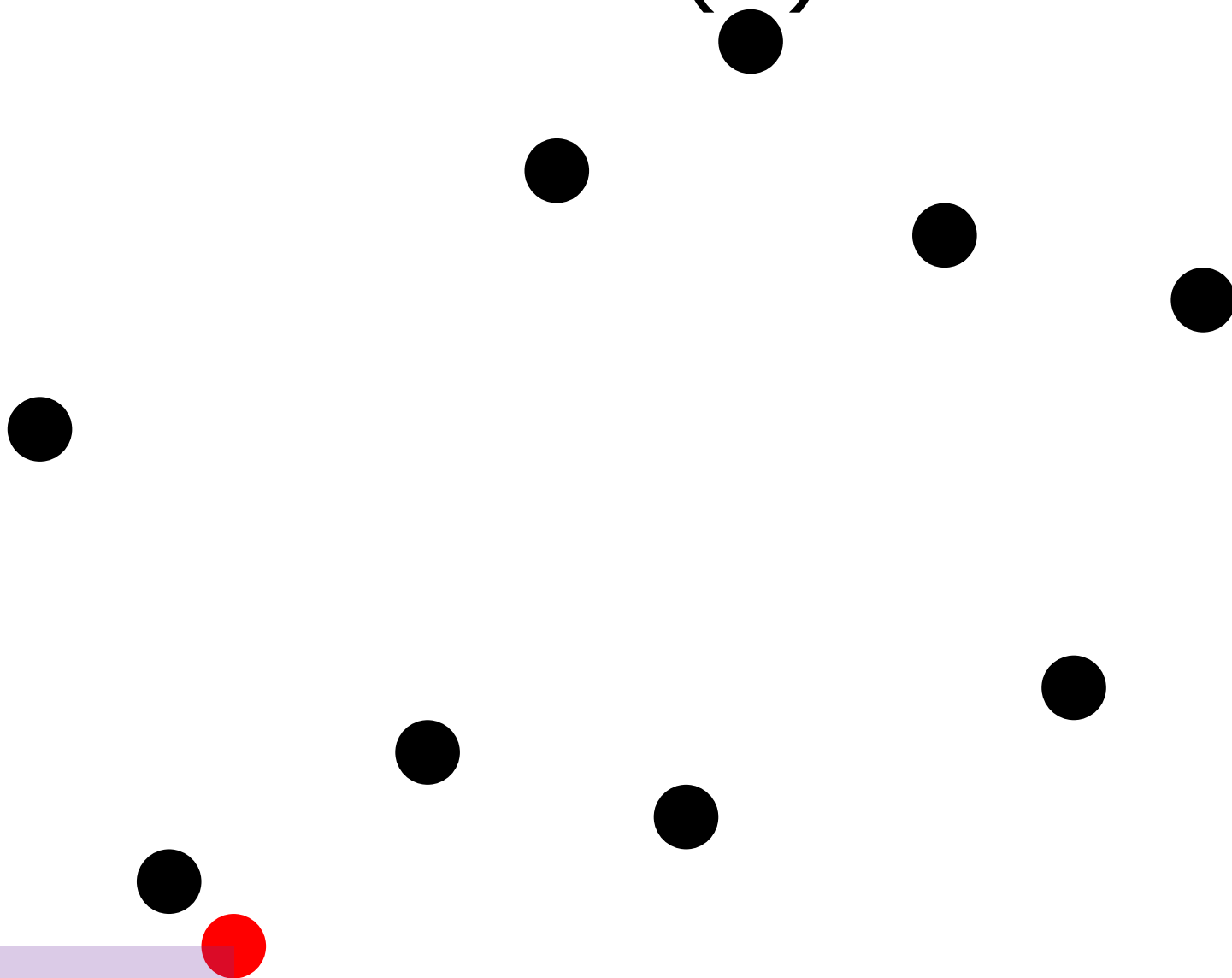
小課題 3 (85 点)

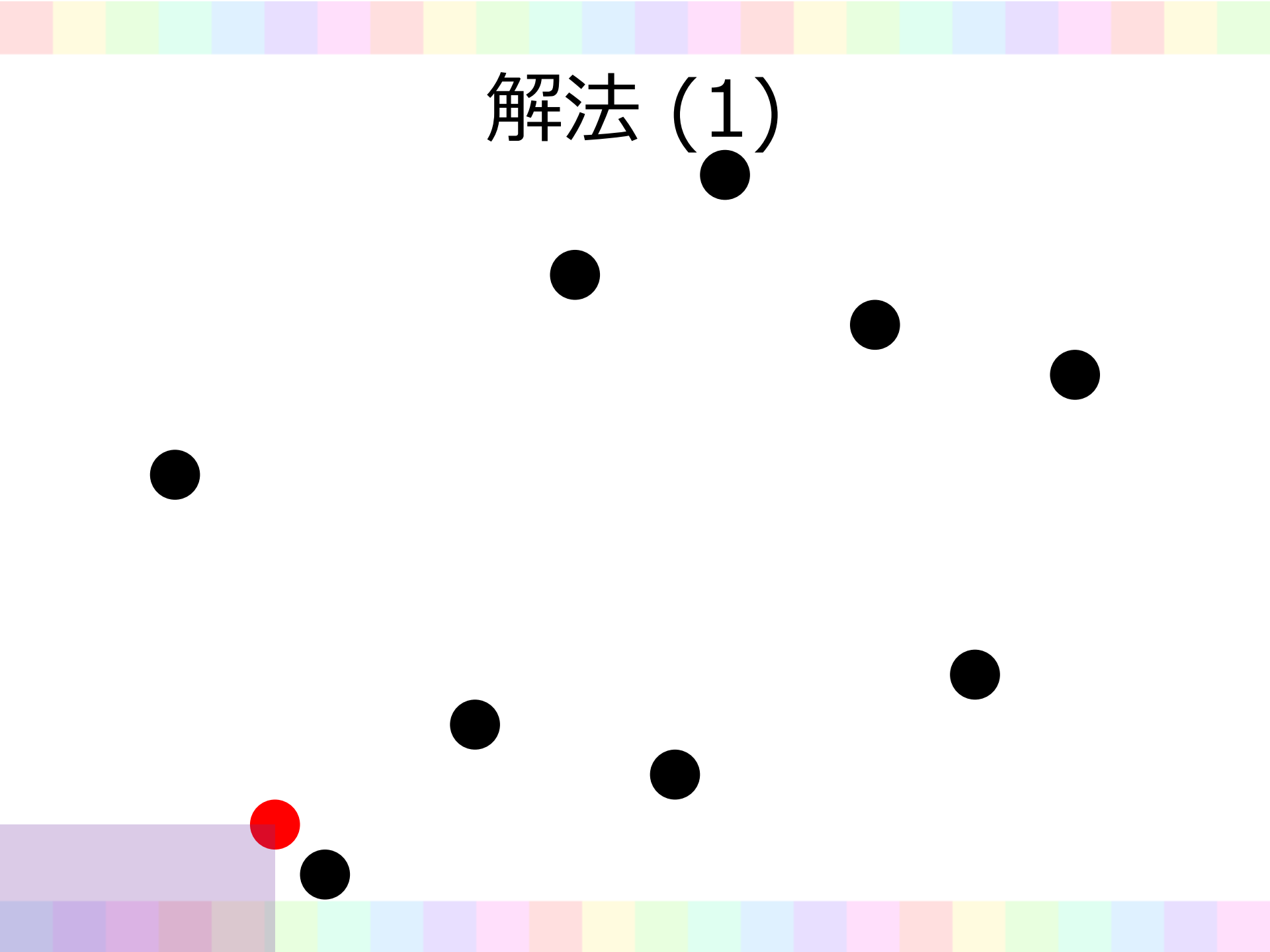
- すごいデータ構造……？
 - はい！ → 解法 (1) へ
 - いいえ → 解法 (2)(3) へ

解法 (1)

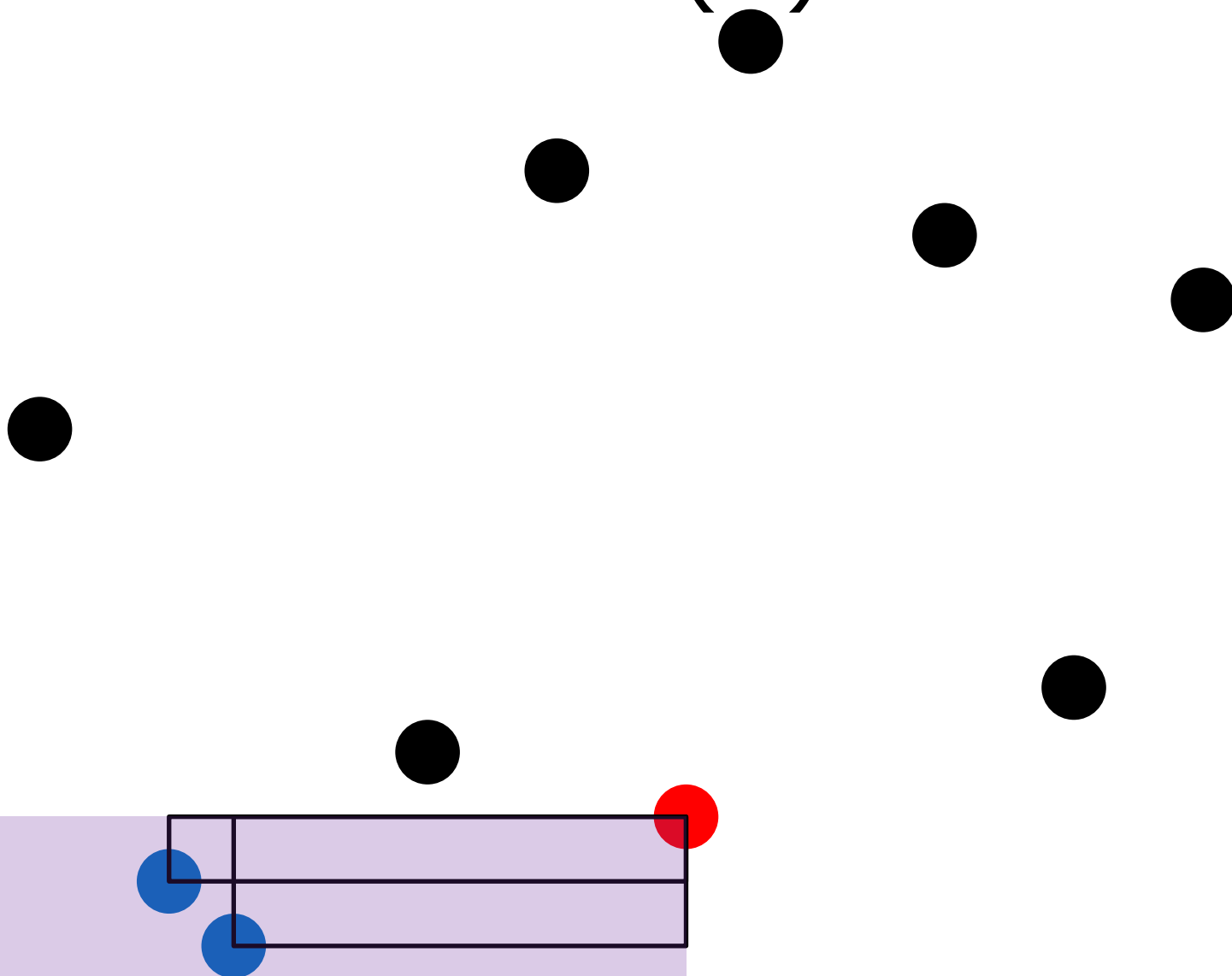


解法 (1)

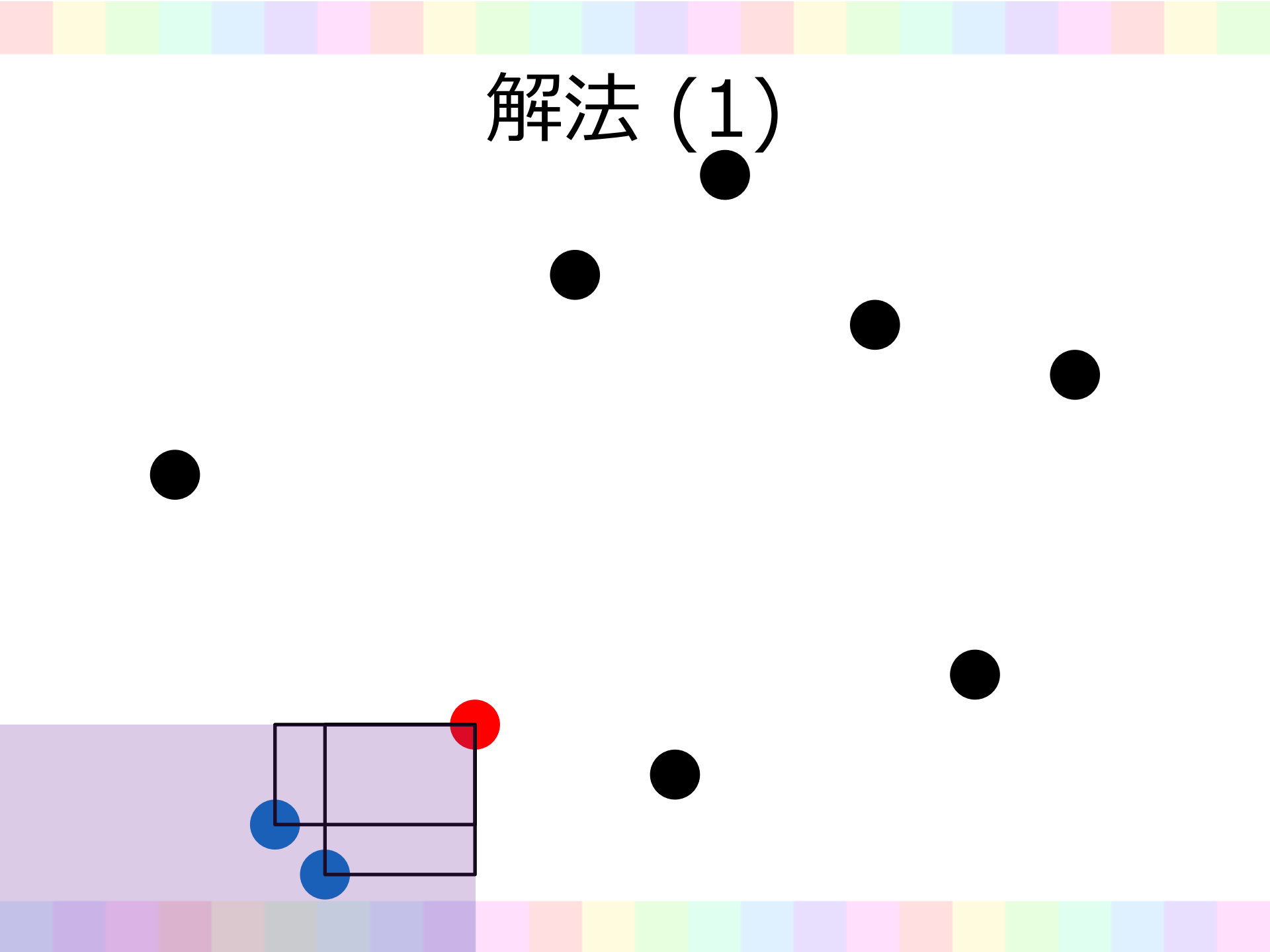




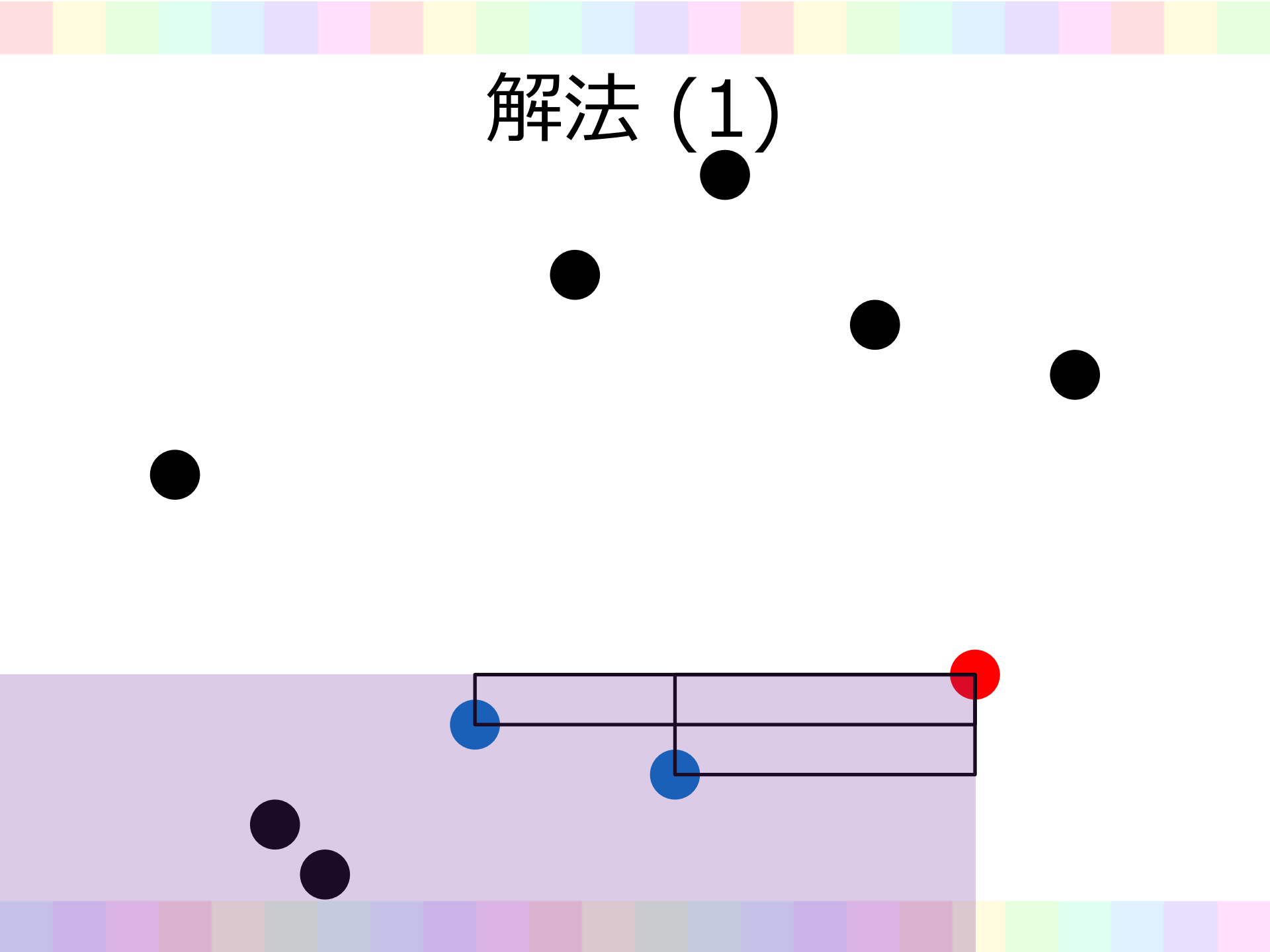
解法 (1)



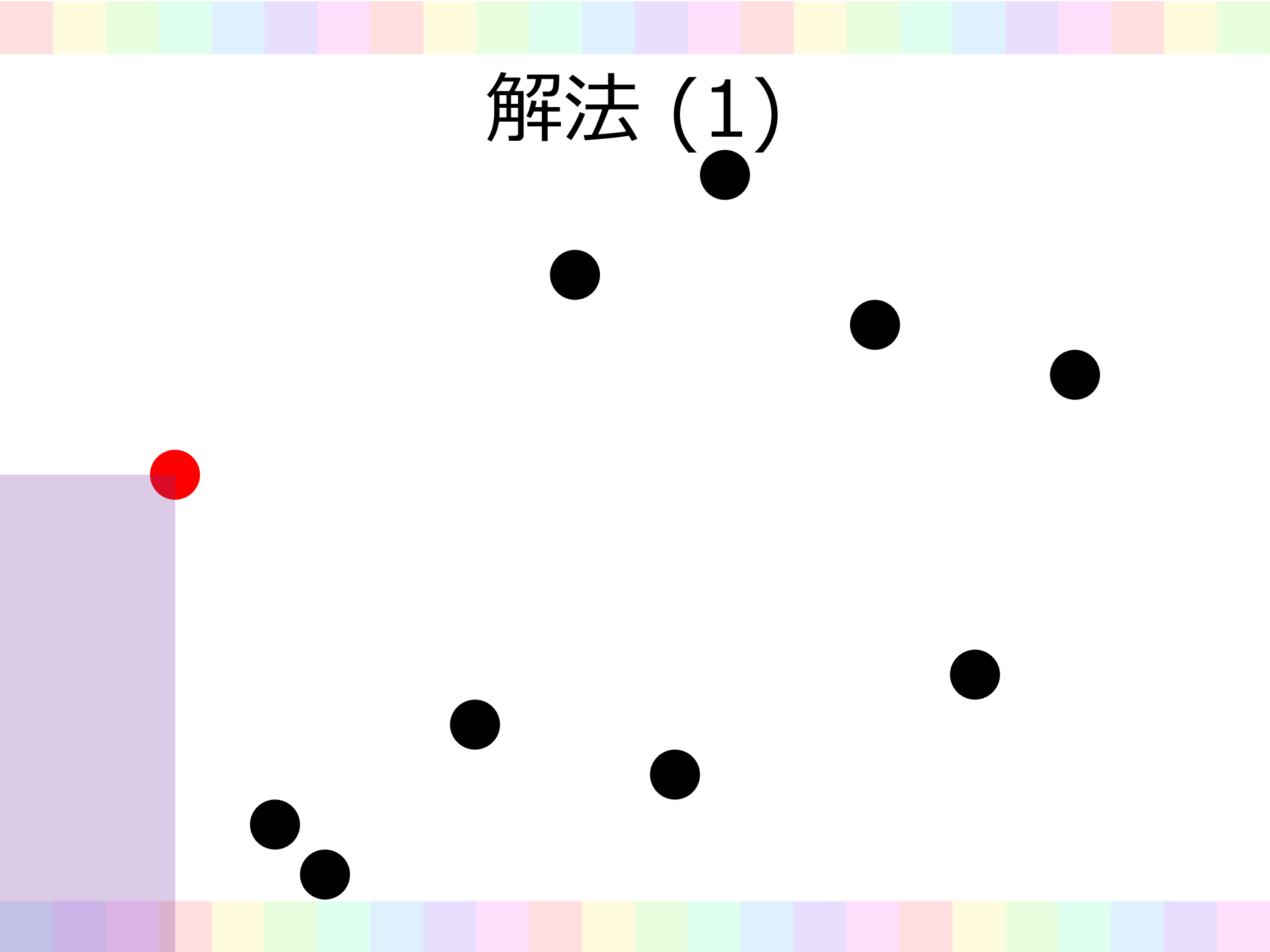
解法 (1)



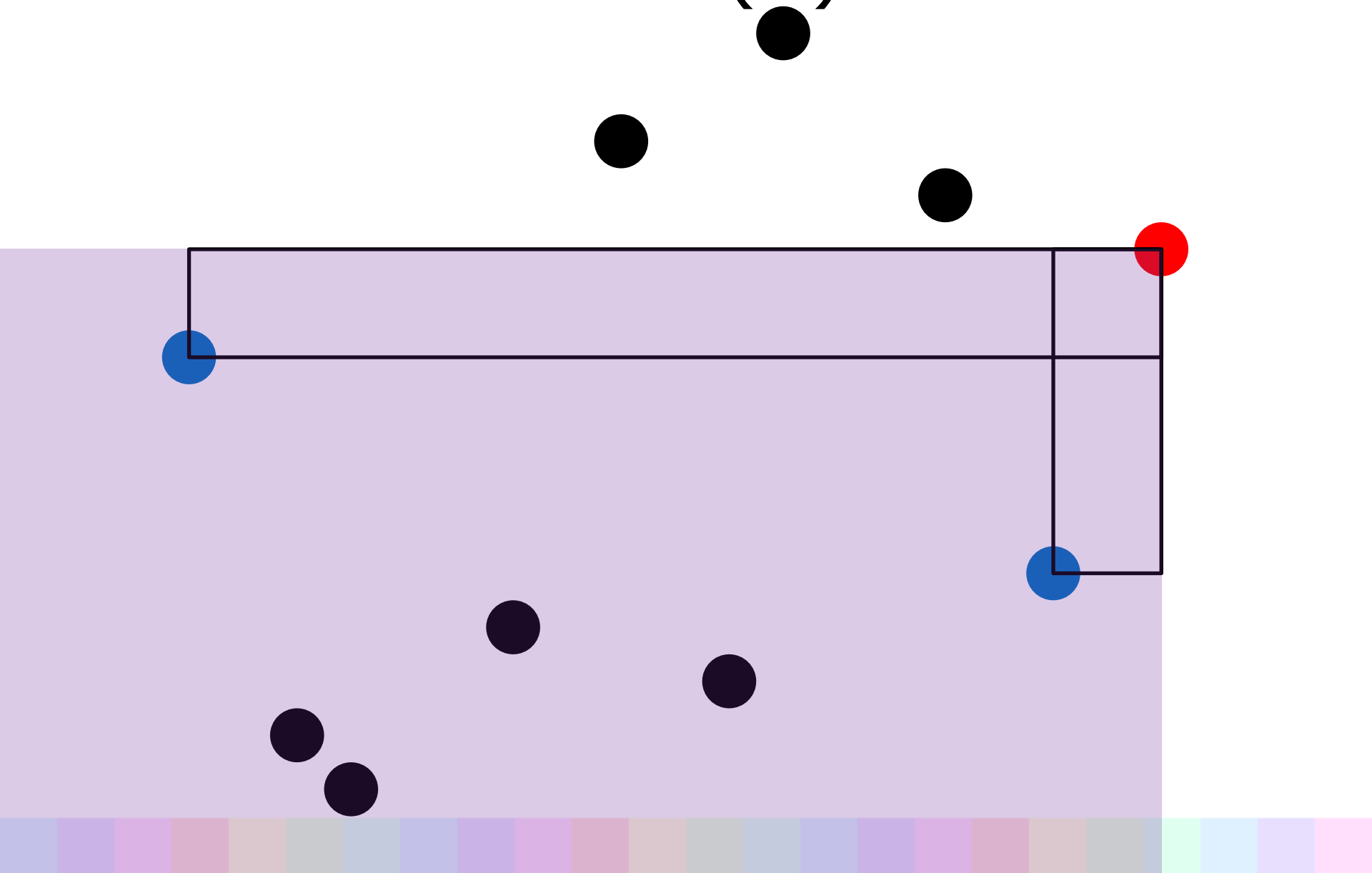
解法 (1)



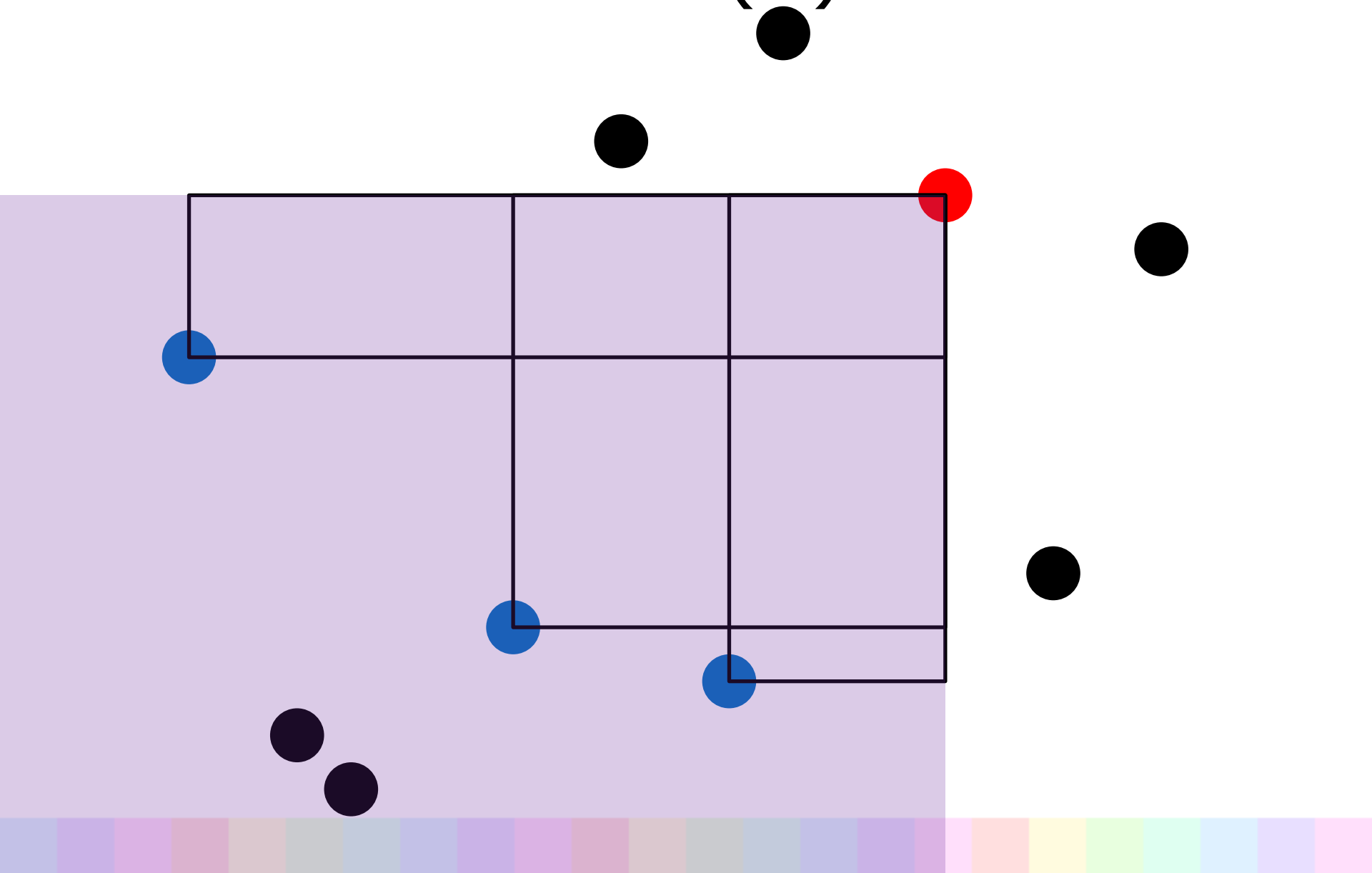
解法 (1)



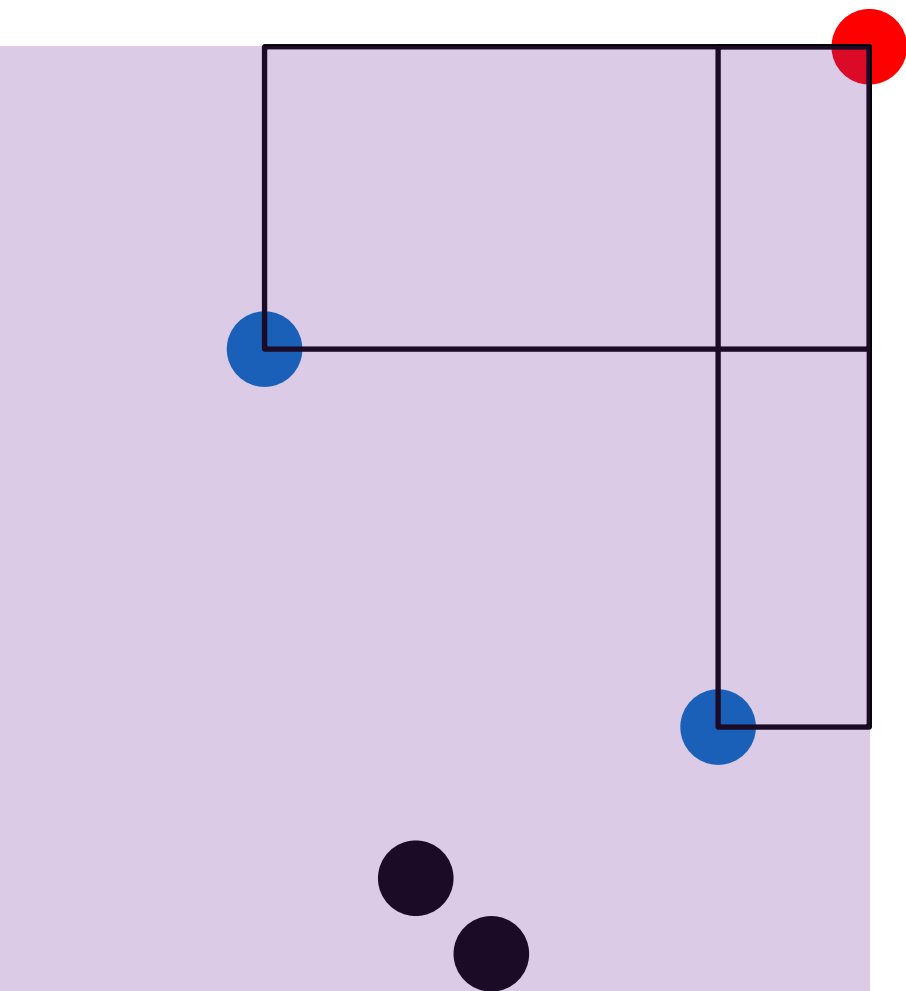
解法 (1)



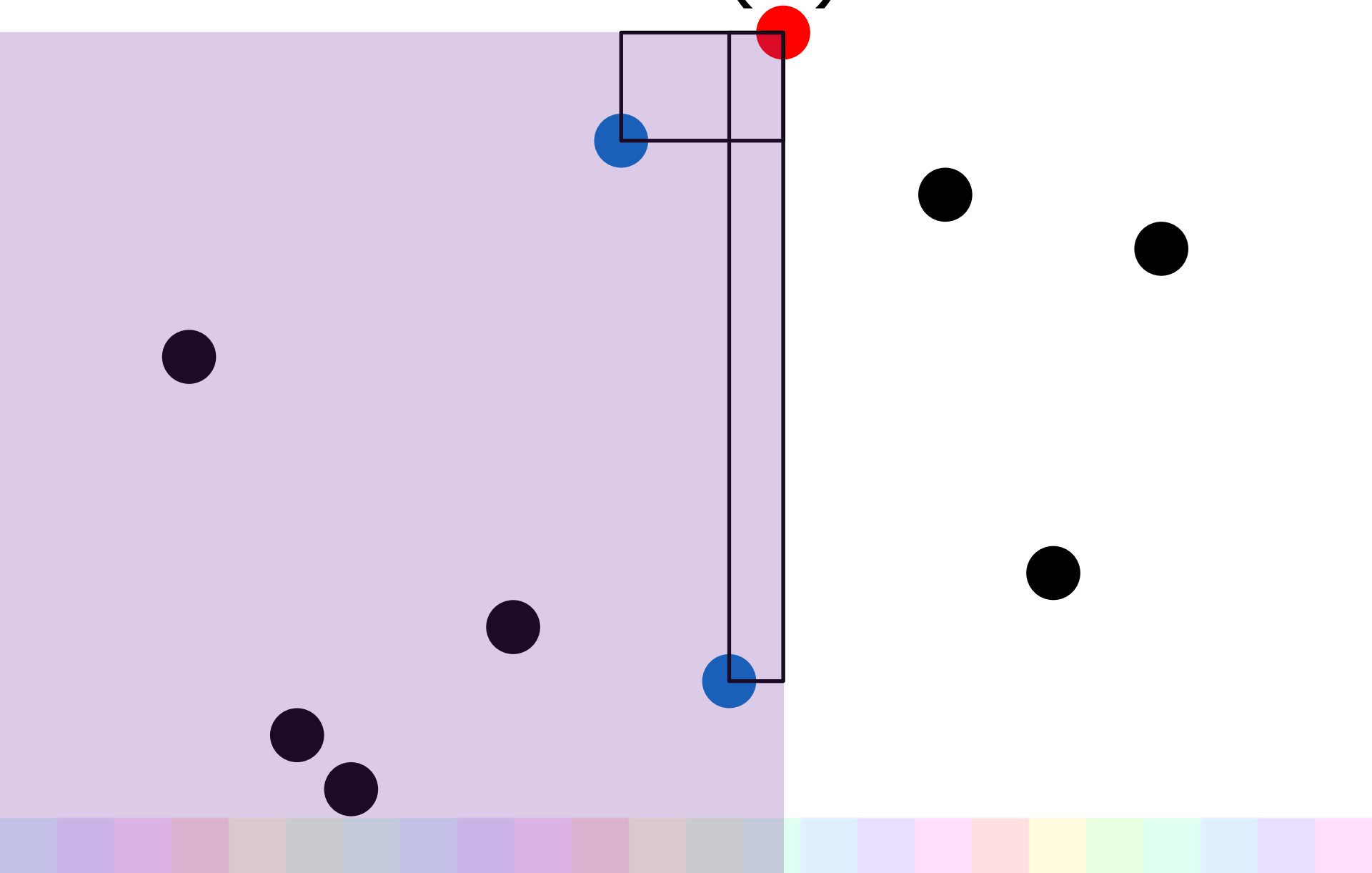
解法 (1)



解法 (1)



解法 (1)

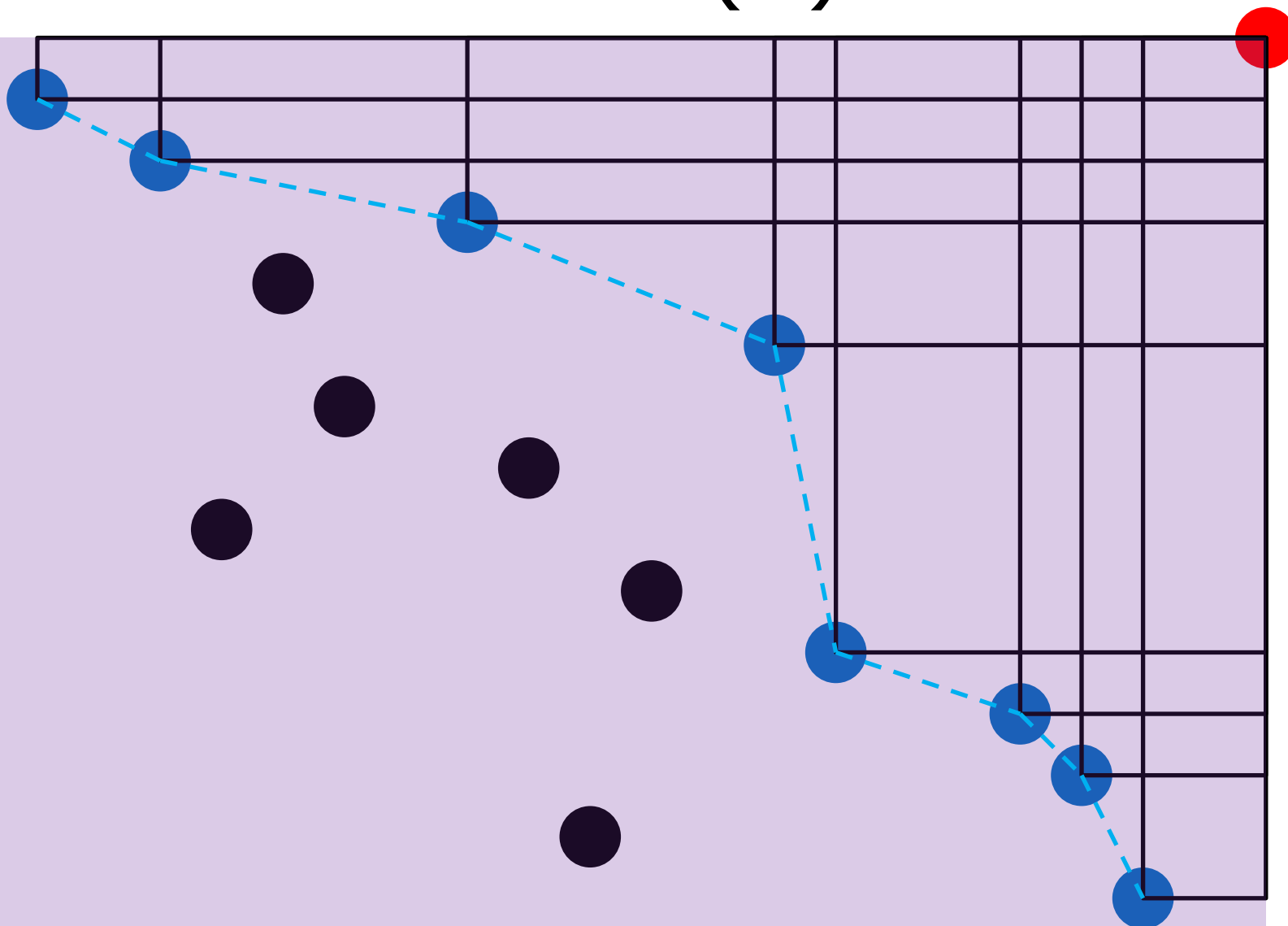


解法 (1)

- ある点から左下の領域について, 「**右上のほうに並んでいる点たち**」を高速に数えたい
 - 正確には, その領域内では右上に他の点がないような点たち

解法 (1)

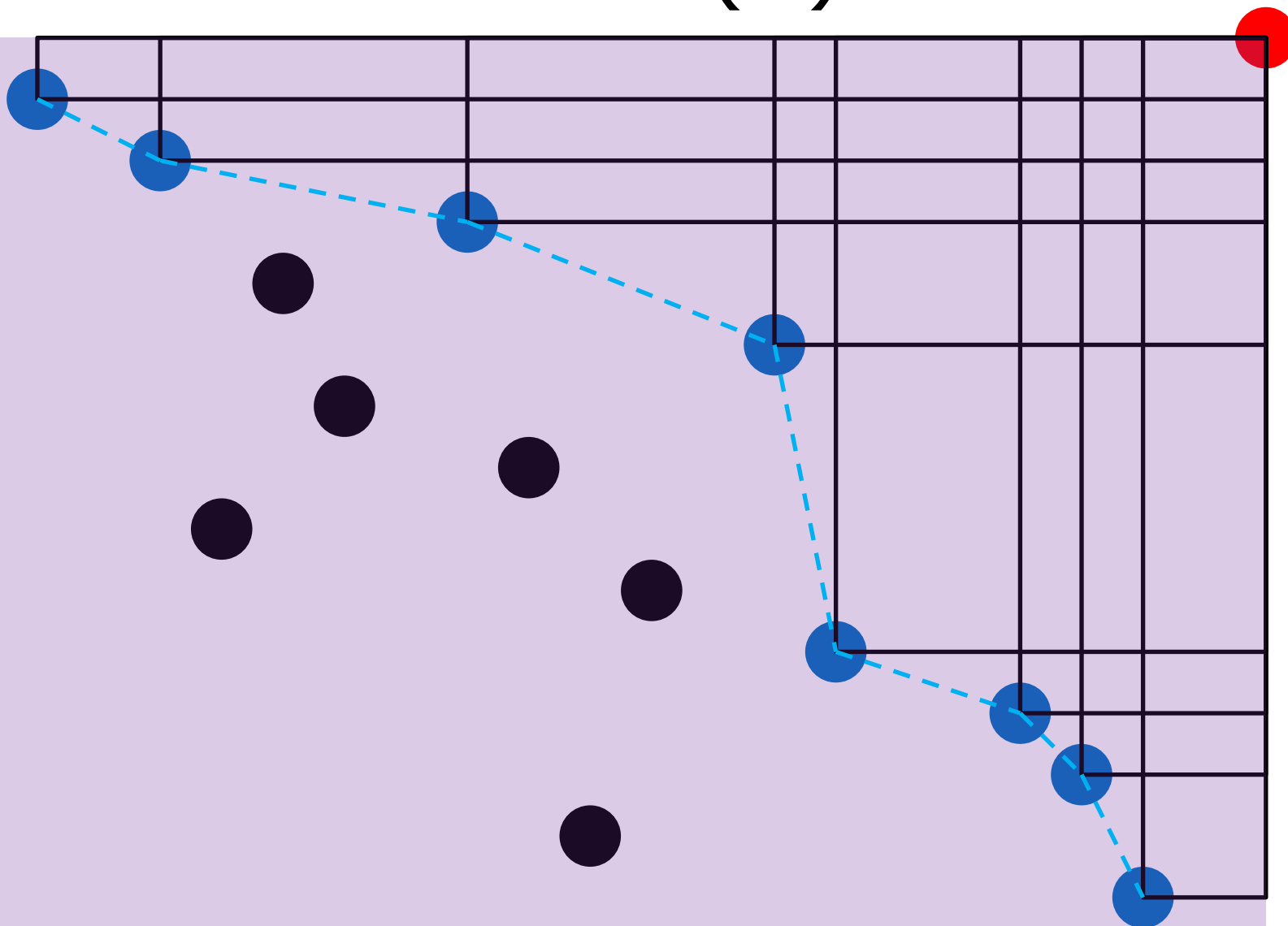
The diagram shows a 10x10 grid with a red dot at the top-right corner (9,9). A blue path starts at (0,9) and ends at (9,0). The path consists of blue dots at (0,9), (1,8), (3,7), (5,5), (6,4), (7,3), (8,2), and (9,0). A dashed blue line connects these dots. There are several black dots representing obstacles at (2,7), (2,6), (3,5), (4,4), (5,3), (6,2), (7,1), and (8,0). The grid is divided into a light purple area on the left and a light blue area on the right by a vertical line at x=5.



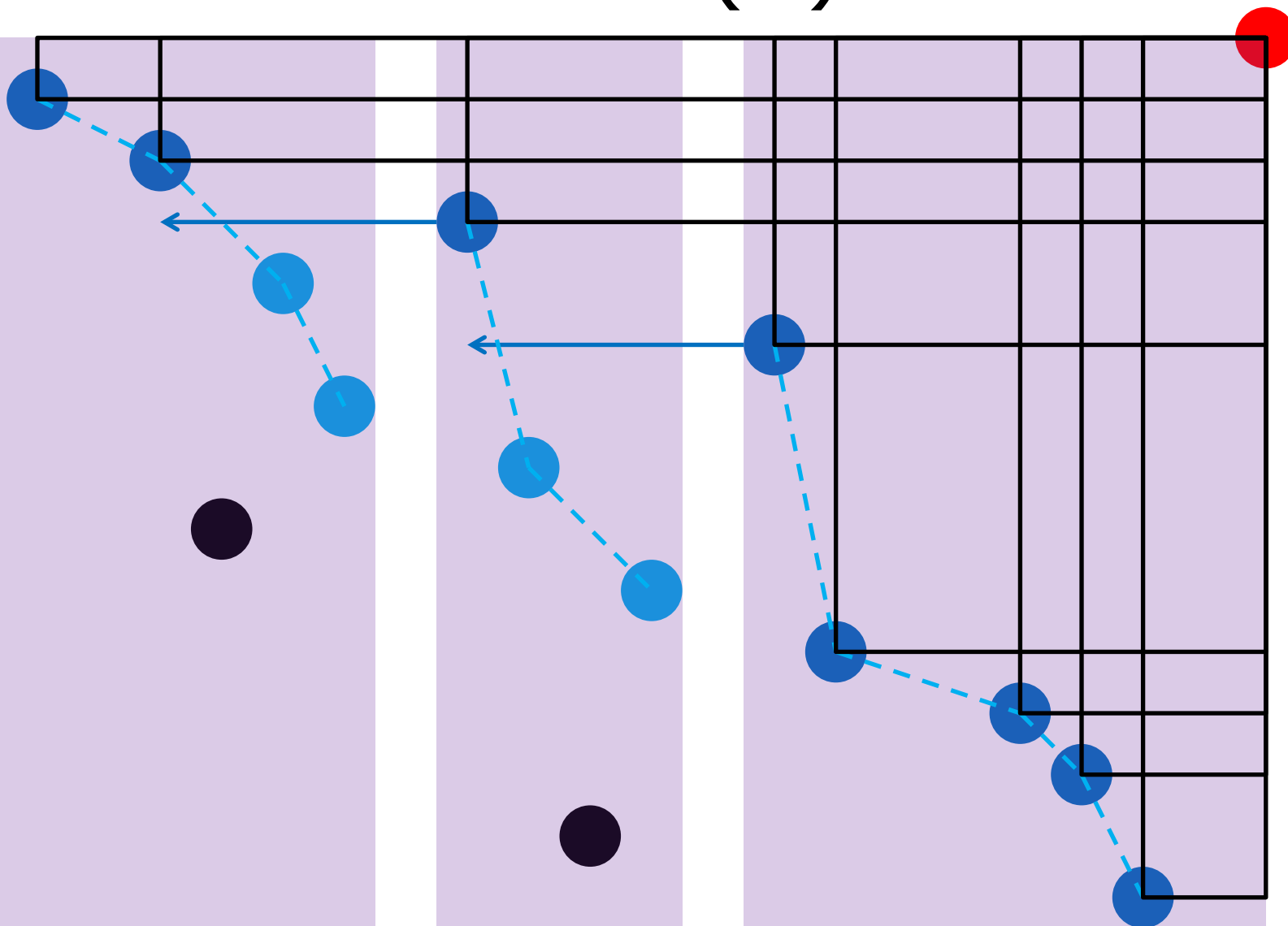
解法 (1)

- ある点から左下の領域について, 「**右上のほうに並んでいる点たち**」を高速に数えたい
 - 正確には, その領域内では右上に他の点がないような点たち
- y 座標が小さい点から順番に追加していくことにする
 - y 座標の範囲を気にしなくてよくなる

解法 (1)



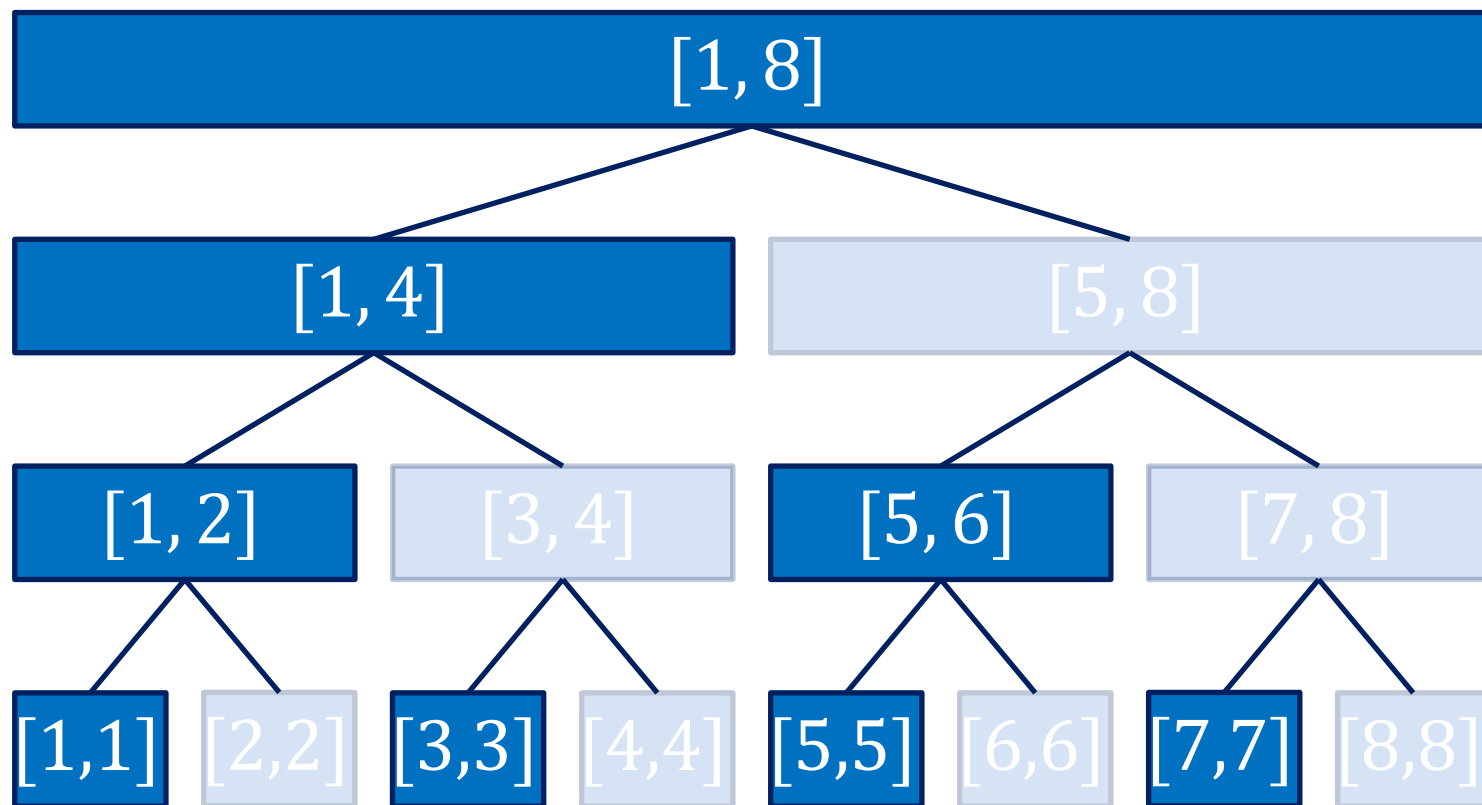
解法 (1)



解法 (1)

- x 座標の範囲をいくつかの区間に分割する
- 各区間で「右上のほうに並んでいる点たち」のどこから見ればいいのか
 - 二分探索でわかる
- どう分割するか？

解法 (1)

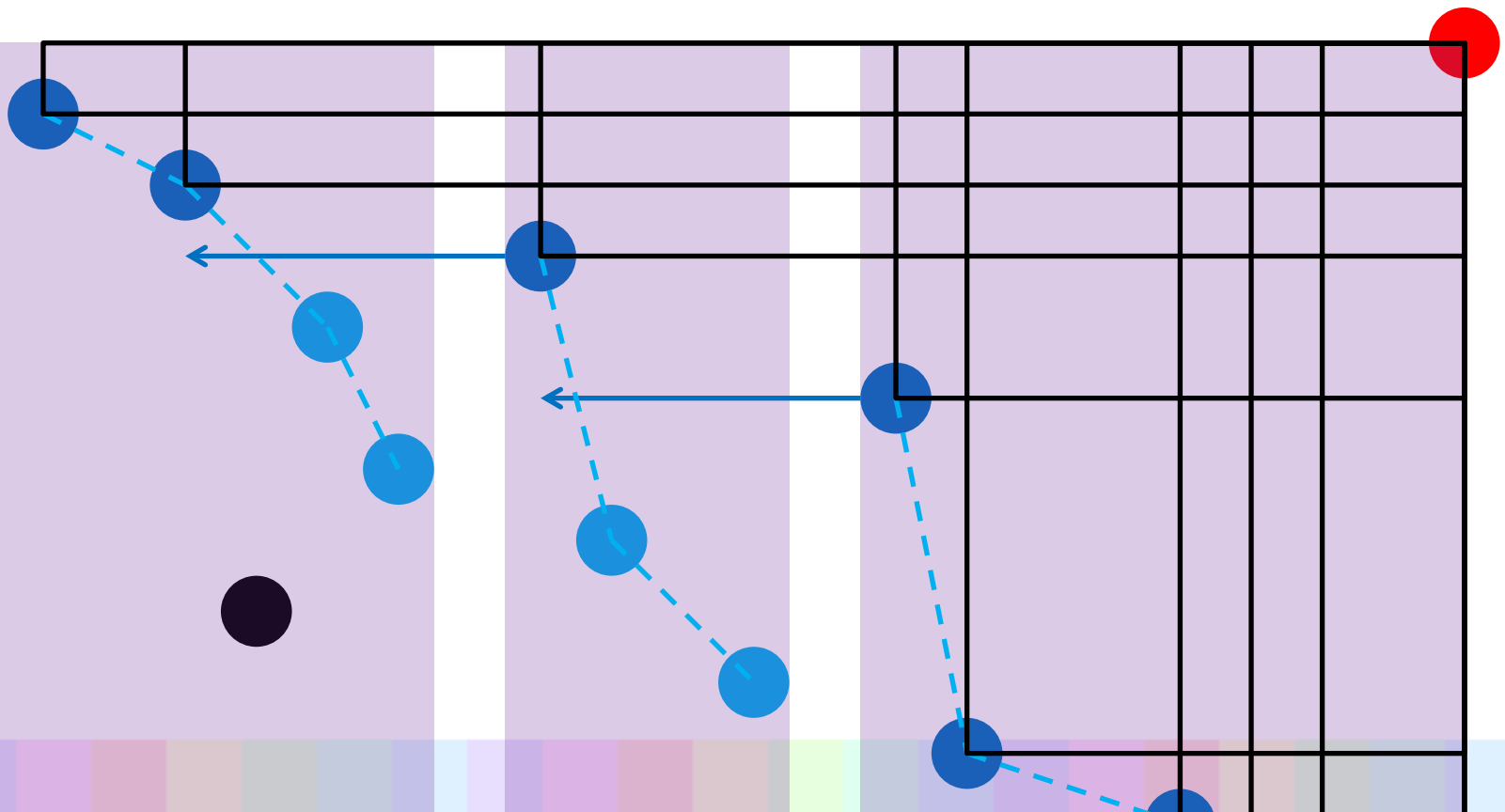


解法 (1)

- x 座標を座標圧縮して 1 から N に
- Segment Tree あるいは Binary Indexed Tree を考え、各区間に対して、「**右上のほうに並んでいる点たち**」を管理する
 - 点たちを座標でソートした列としてもつ
 - $O(N \log N)$ メモリ

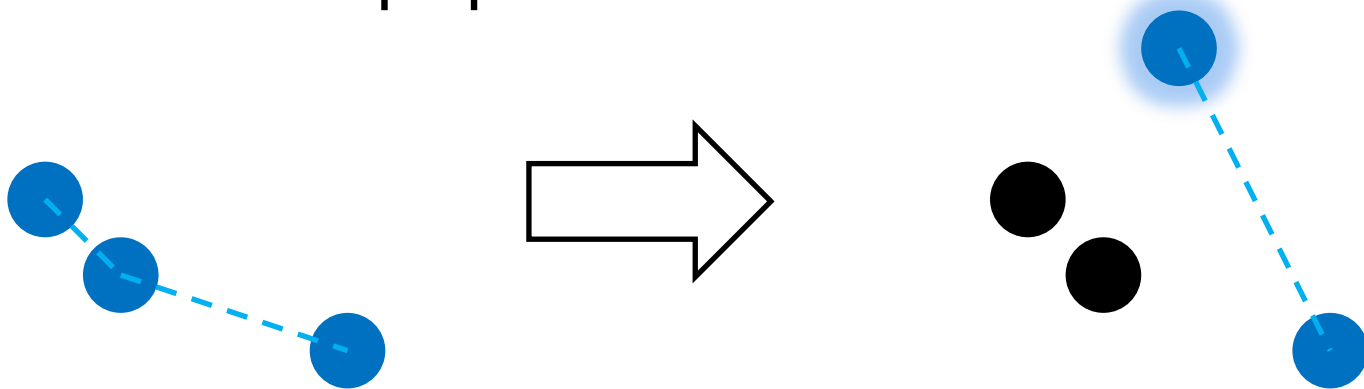
解法 (1)

- 数える
 - $O(\log N)$ 個に分割された区間を右から辿る



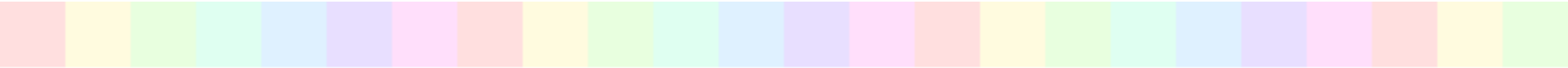
解法 (1)

- 点を追加する
 - 全体でみると, どの点も高々 1 回追加・削除されるだけ
 - y 座標が小さい順に追加するので, 配列から必要なだけ pop してから追加



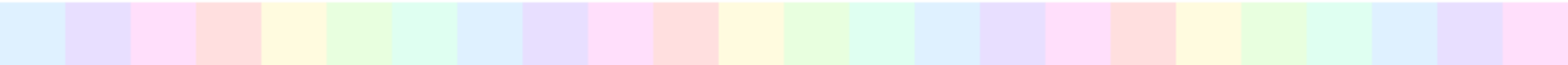
解法 (1)

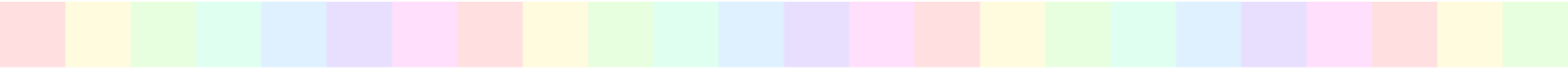
- 必要な操作
 - 末尾から削除・末尾に追加
 - 二分探索
 - 個数を数える
- 全体で $O(N(\log N)^2)$ 時間
 - 実装：ちょっと大変



小課題 3 (85 点)

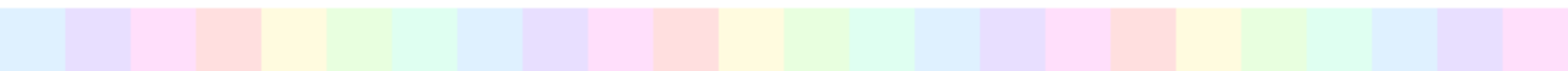
- すごいデータ構造なしでいったい??



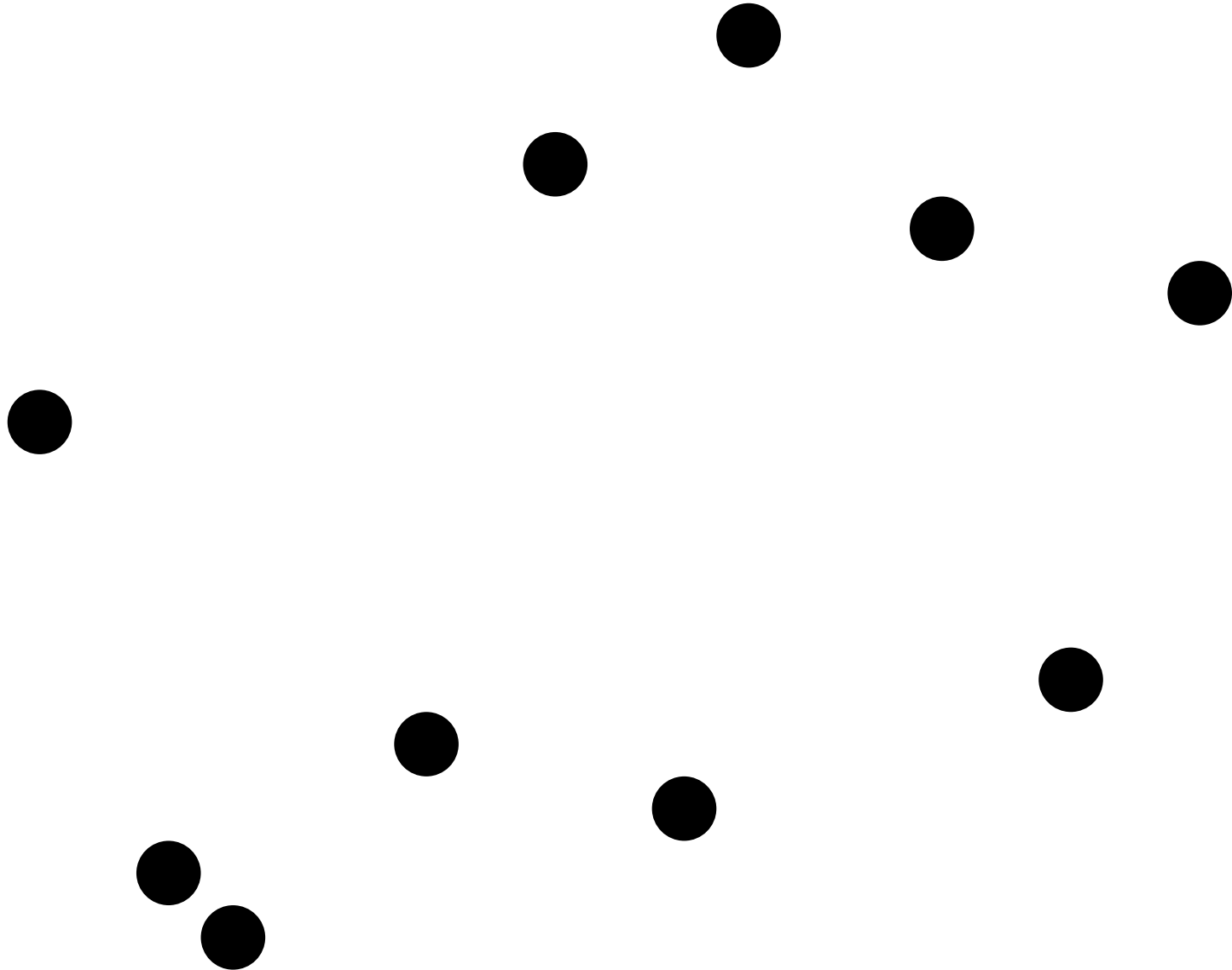


分割統治法

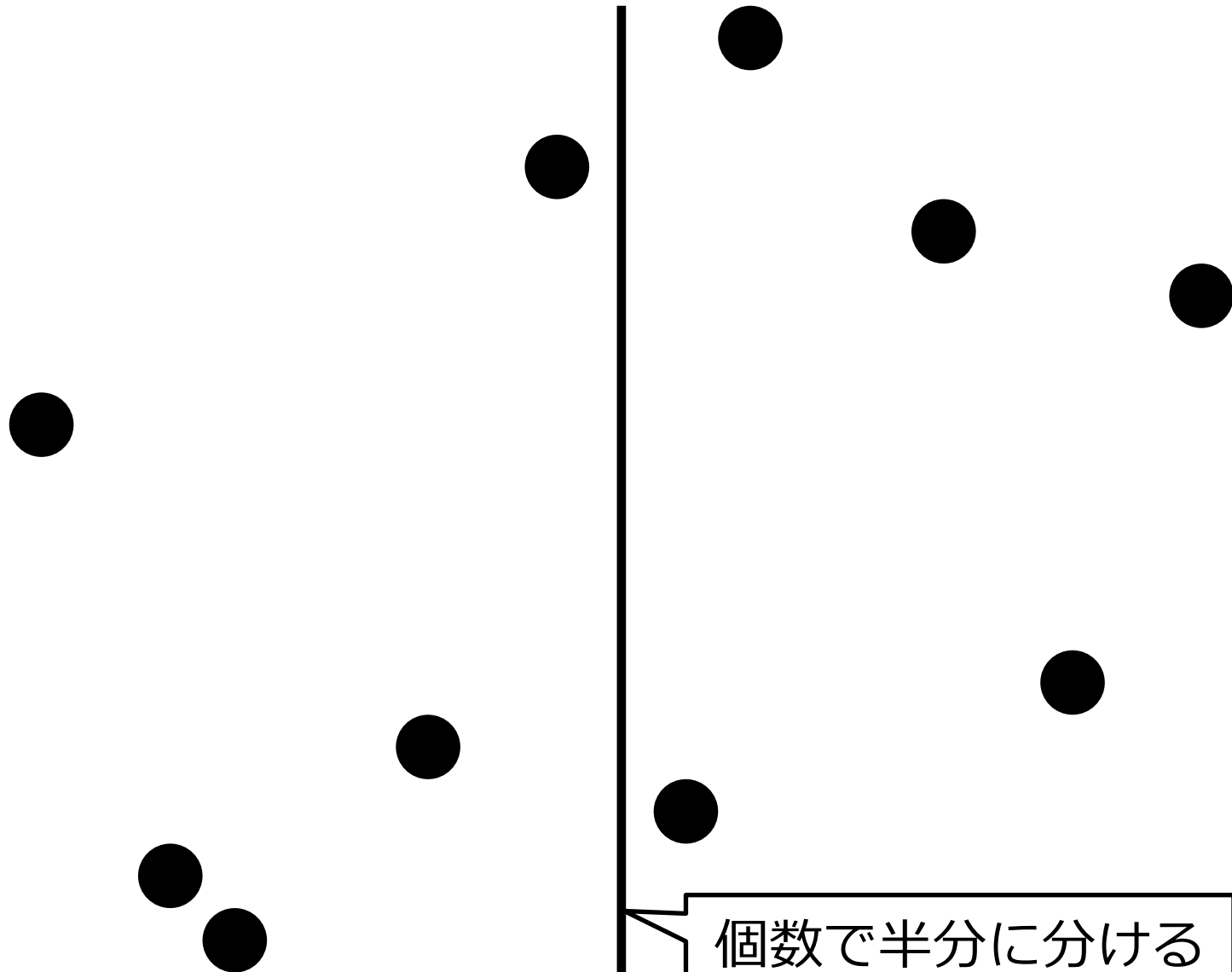
–Divide and Conquer–



分割統治法



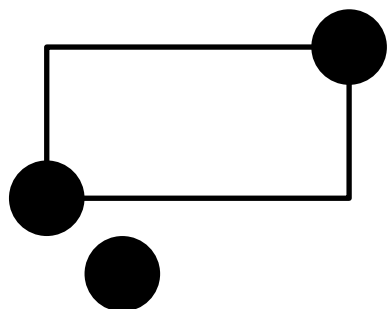
分割統治法



個数で半分に分ける

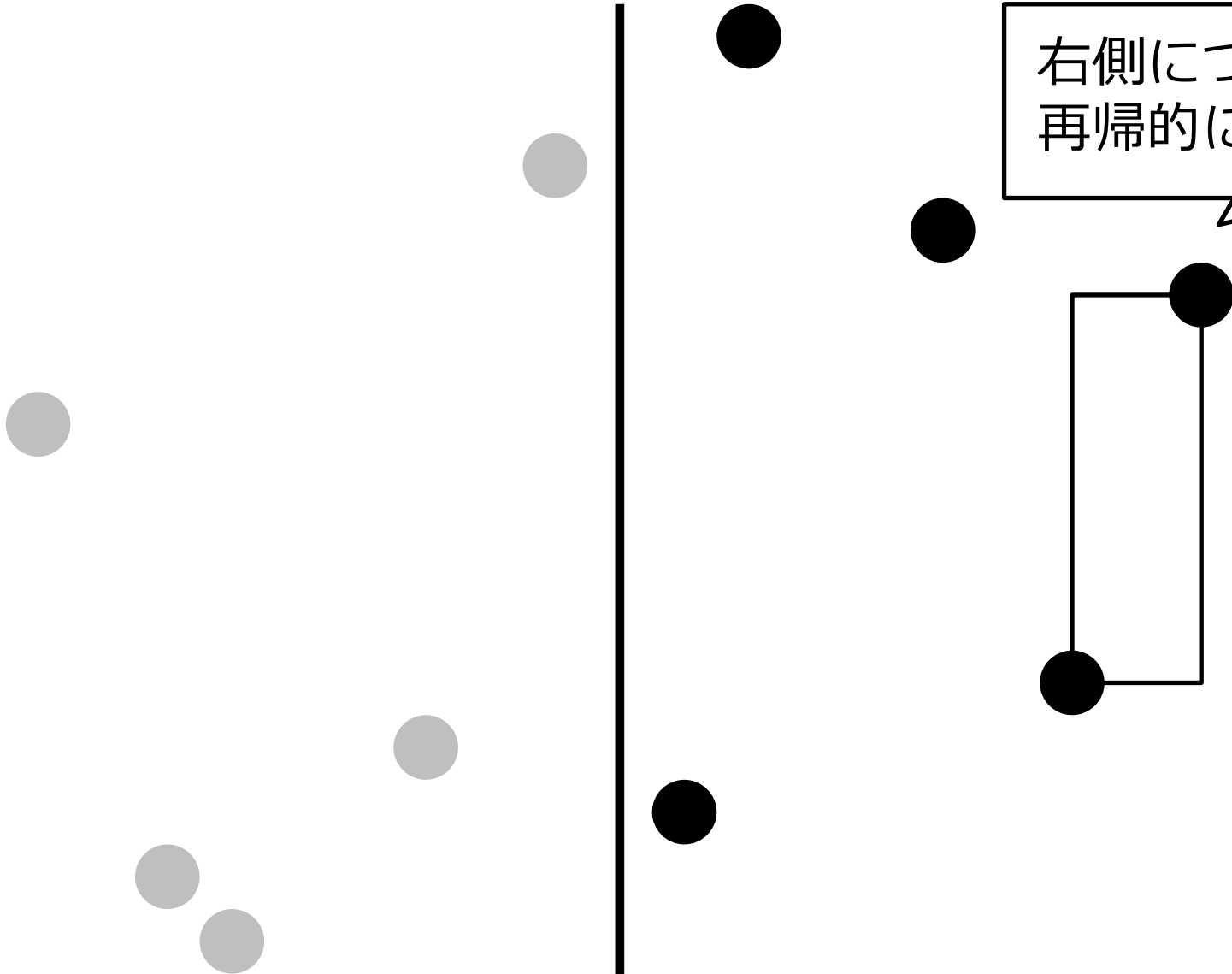
分割統治法

左側について
再帰的に解く

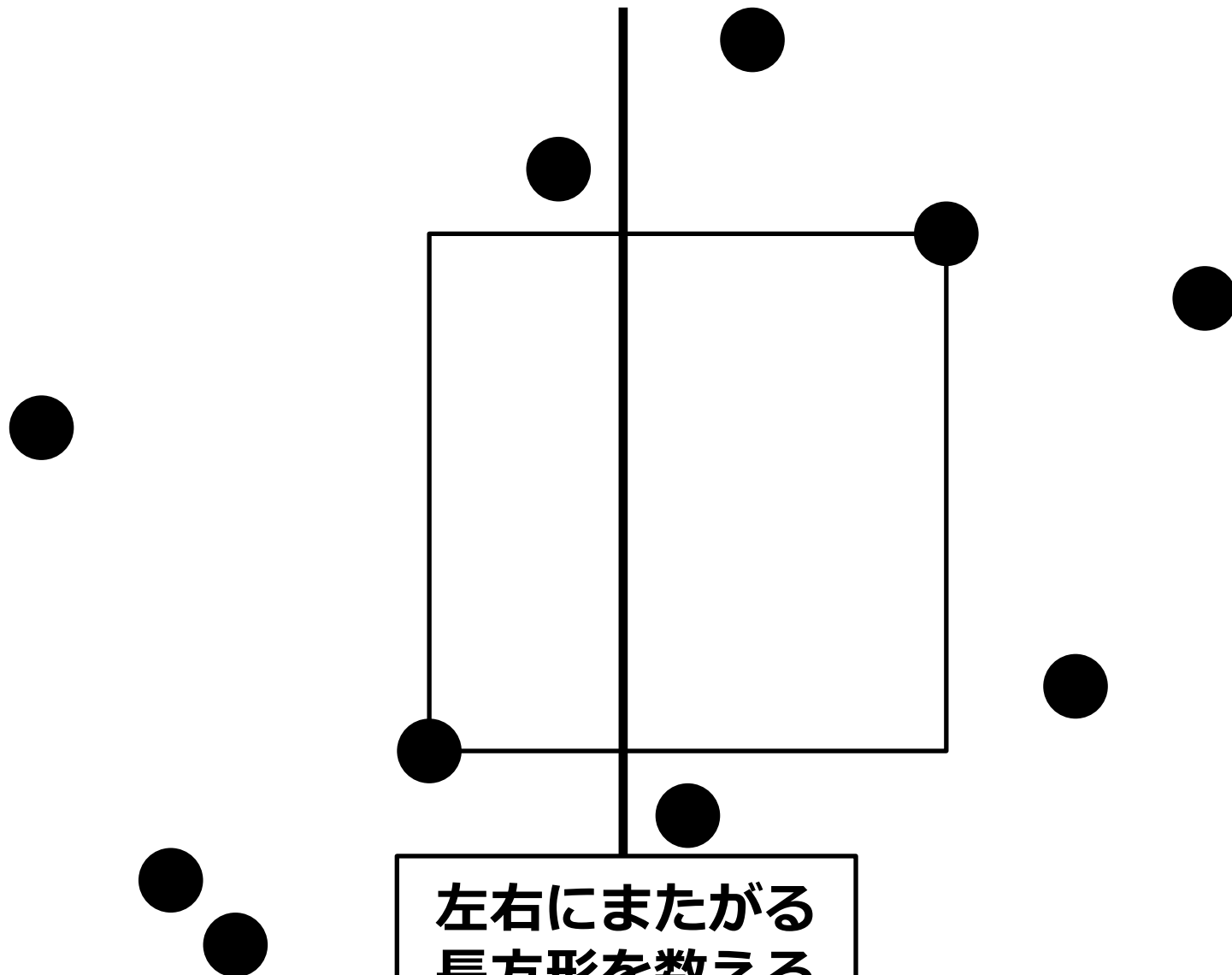


分割統治法

右側について
再帰的に解く



分割統治法



分割統治法

- 左右にまたがる長方形をどれくらい速く数えるか？
 - これが $O(N(\log N)^k)$ 時間 ($k \geq 0$) でできるなら, 全体では $O(N(\log N)^{k+1})$ 時間
 - サイズ N のとき $T(N)$ ステップかかるとし, 左右にまたがるのを $c N (\log N)^k$ ステップでできるとすると, $T(2^m) = 2 T(2^{m-1}) + c 2^m m^k$, これを解いて $\frac{T(2^m)}{2^m} = c(1^k + 2^k + \dots + m^k) = O(m^{k+1})$

分割統治法

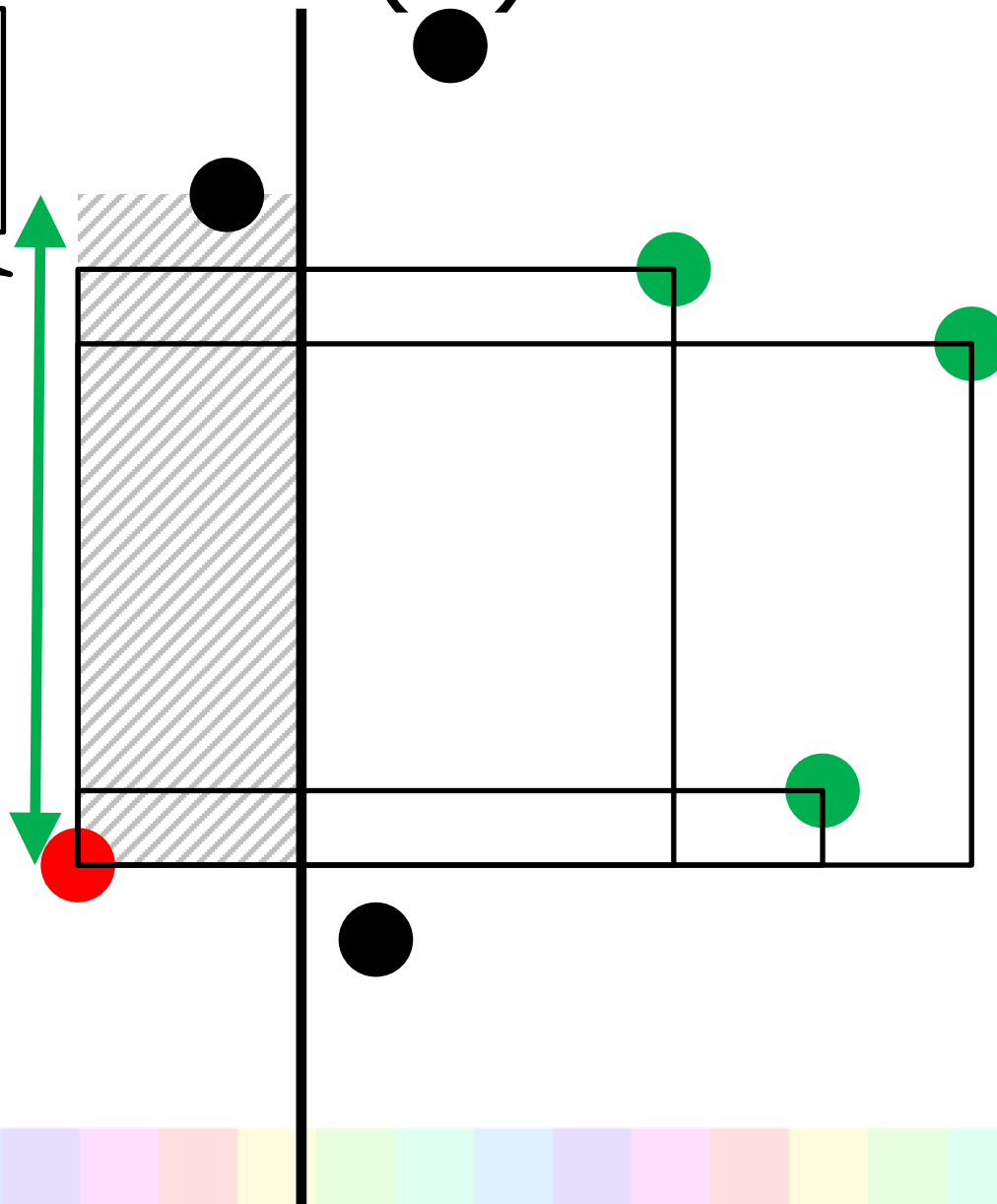
- 左右にまたがる長方形をどうやって数えるか？
- やっぱりデータ構造……？
 - はい！ → 解法 (2) へ
 - いいえ → 解法 (3) へ

解法 (2)

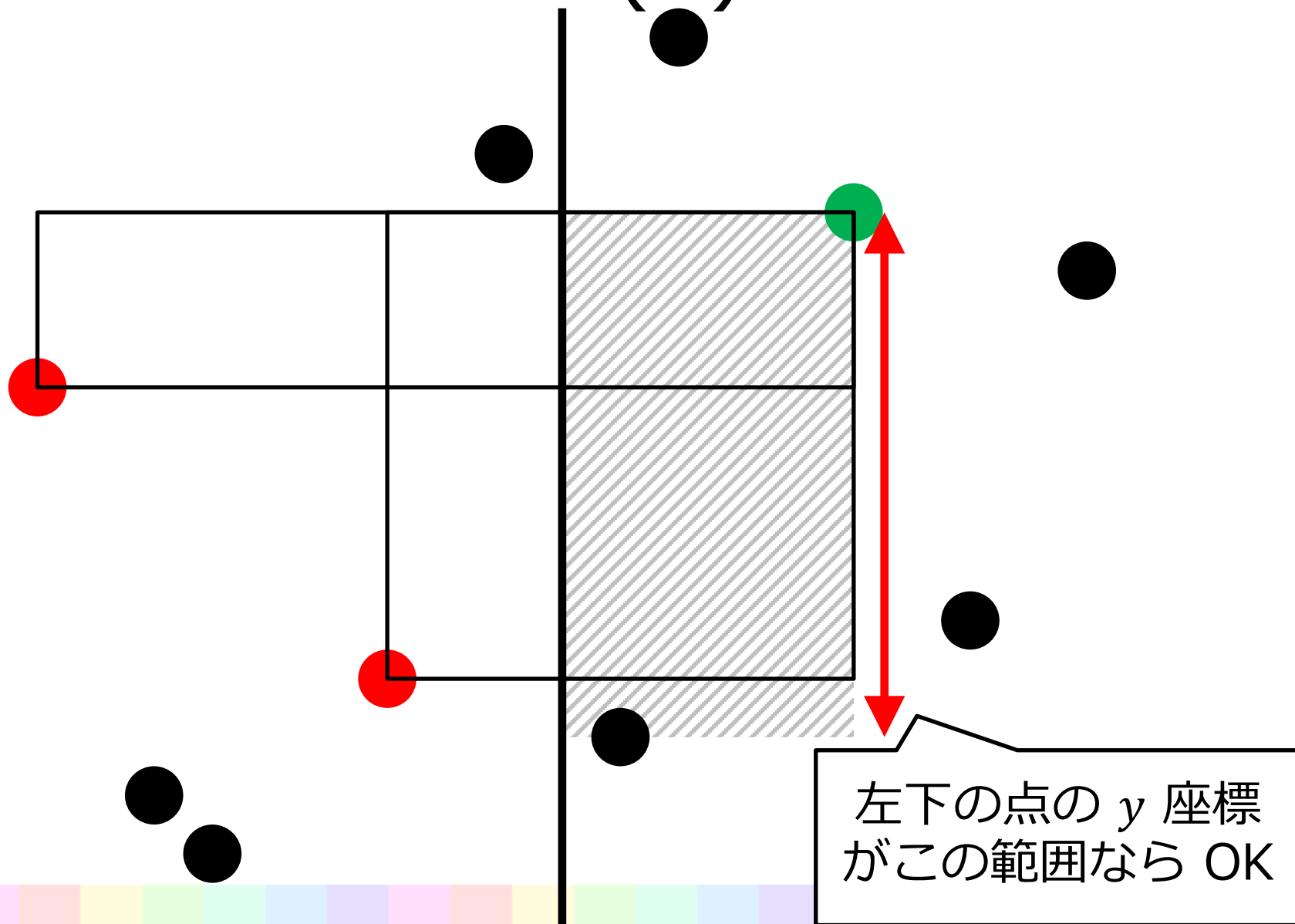
- 「内部に他の点がない」
 - 中央線より左側に他の点がない
 - 中央線より右側に他の点がない

解法 (2)

右上の点の y 座標
がこの範囲なら OK



解法 (2)



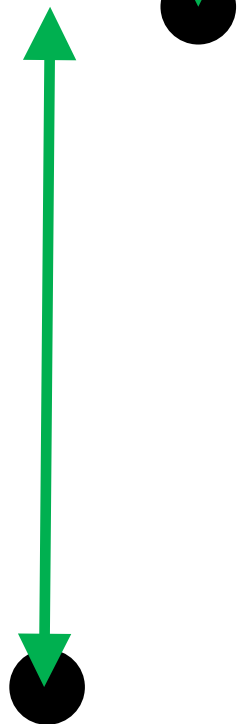
解法 (2)

中央線に近い方から
範囲を求める

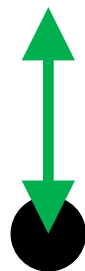
解法 (2)



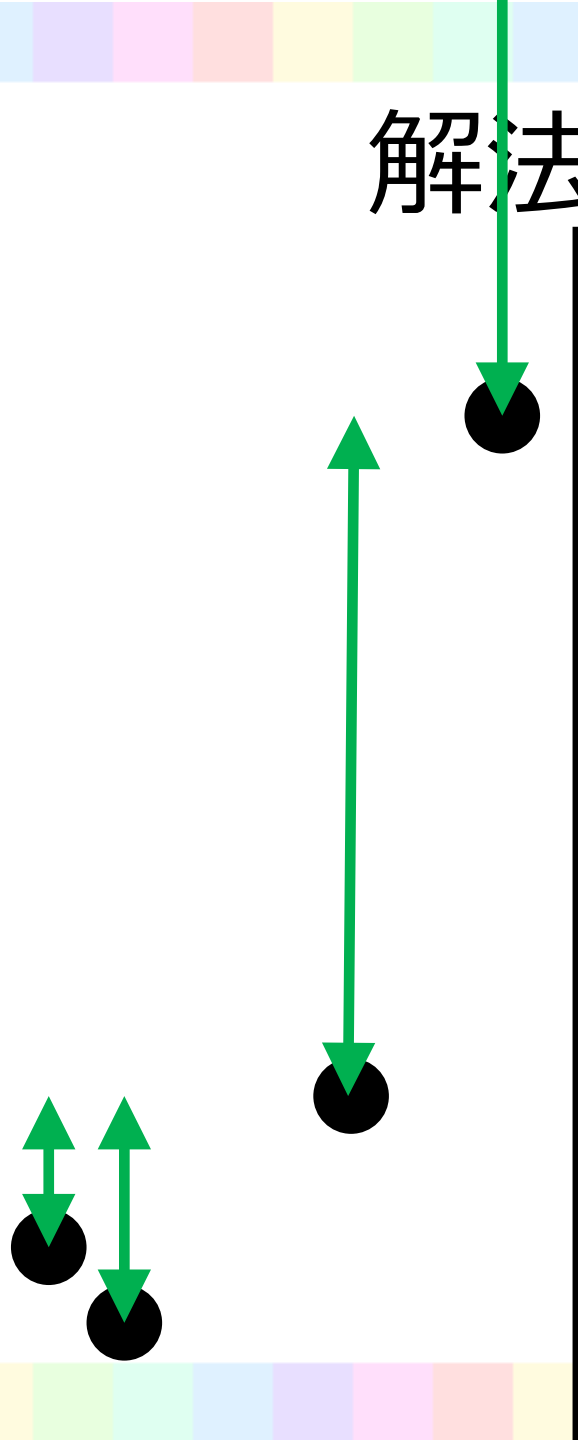
解法 (2)



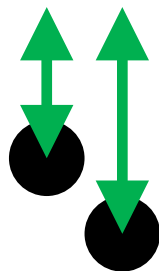
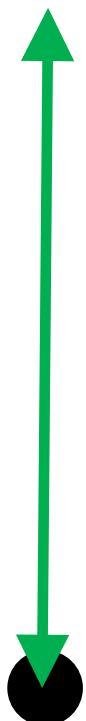
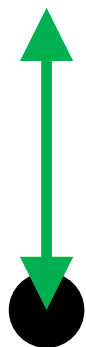
解法 (2)



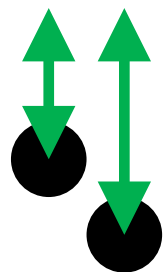
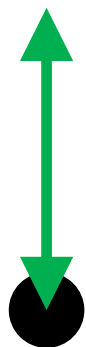
解法 (2)



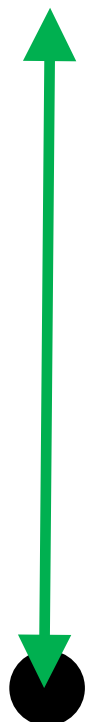
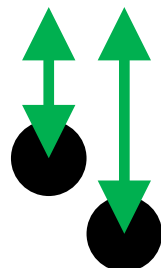
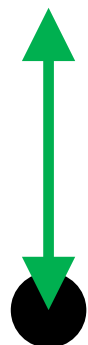
解法 (2)



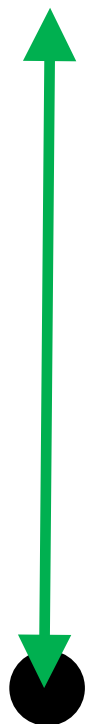
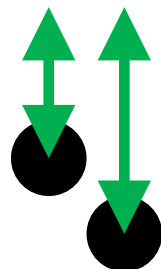
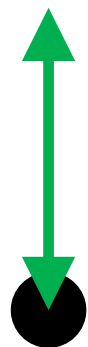
解法 (2)



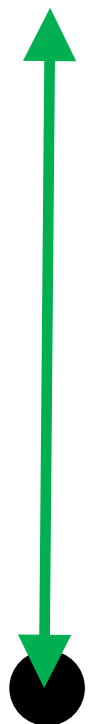
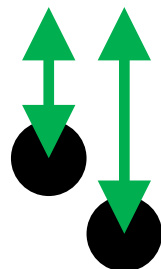
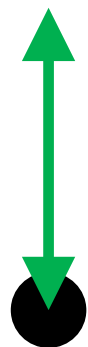
解法 (2)



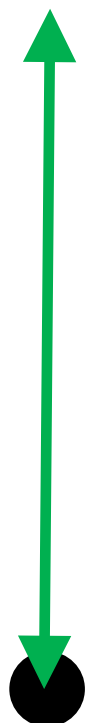
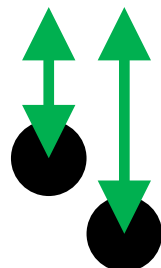
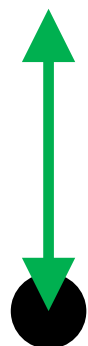
解法 (2)



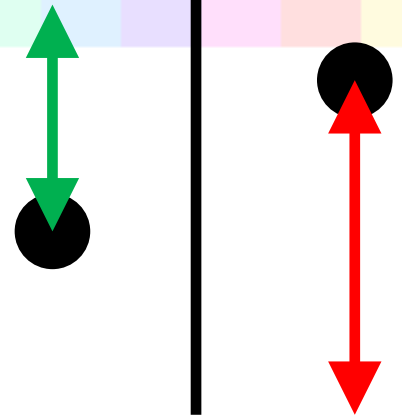
解法 (2)



解法 (2)



解法 (2)

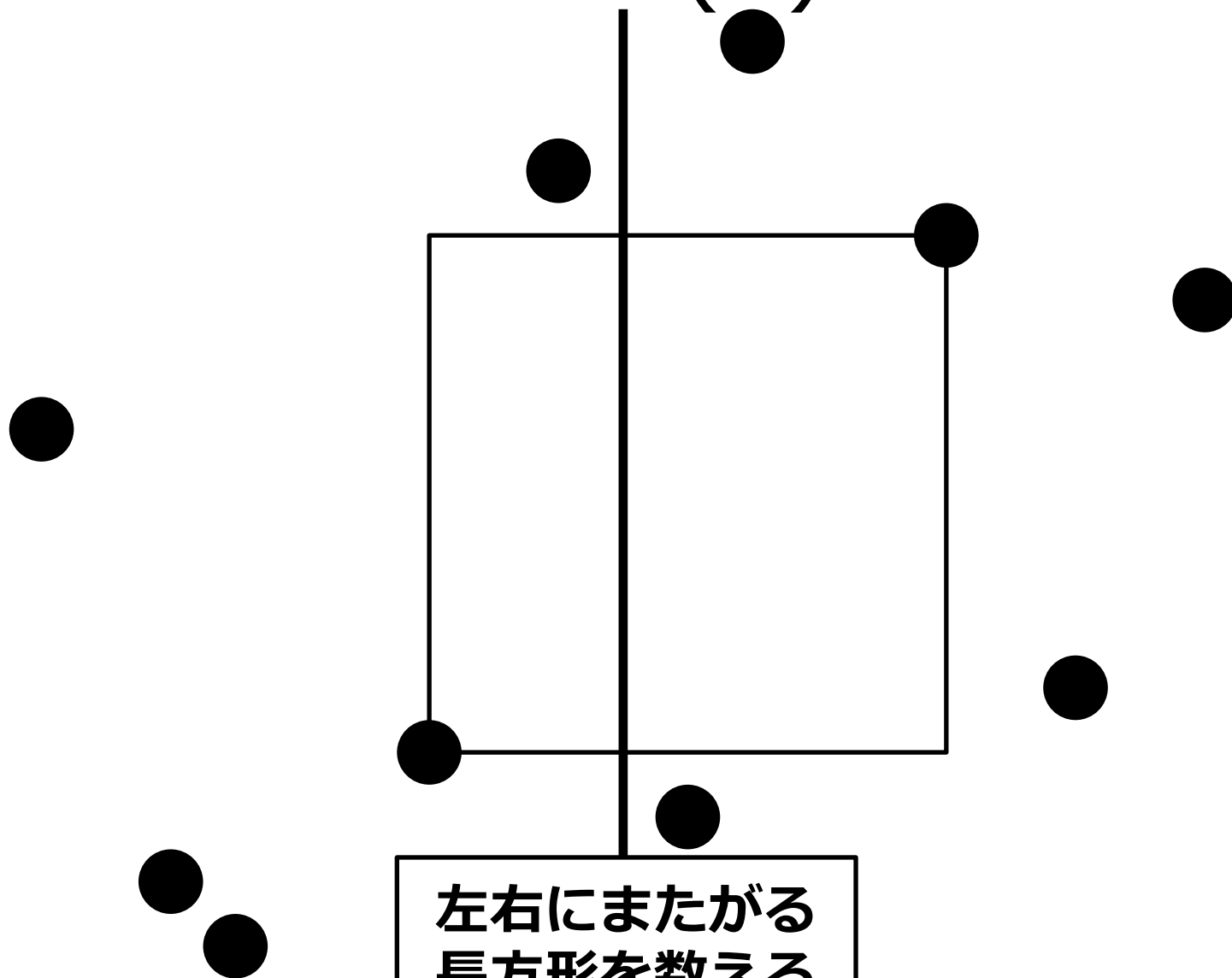


- こういう区間の組を数える：
- 左側 $[a_i, b_i]$, 右側 $[c_j, d_j]$ とする
 - OK な条件は $c_j \leq a_i \leq d_j \leq b_i$
- 下端 (a_i や c_j) の値でソート
 1. 区間 $[c_j, d_j]$ が来たら, d_j を追加する
 2. 区間 $[a_i, b_i]$ が来たら, 追加された d_j のうち $a_i \leq d_j \leq b_i$ なるものを数える

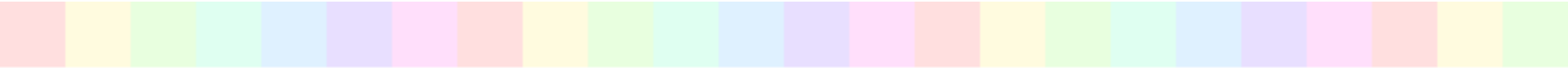
解法 (2)

- 区間を求めるのは `std::set` とその `lower_bound` などを利用
- y 座標を座標圧縮しておけば Segment Tree や BIT で数えられる
- $O(N \log N)$ 時間
- 全体で $O(N(\log N)^2)$ 時間
 - 実装：比較的かんたん

解法 (3)

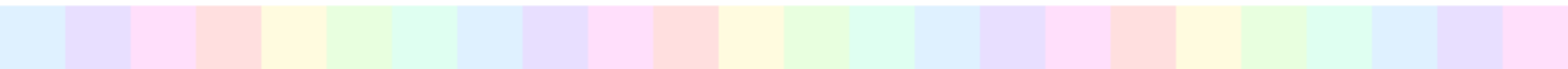


左右にまたがる
長方形を数える

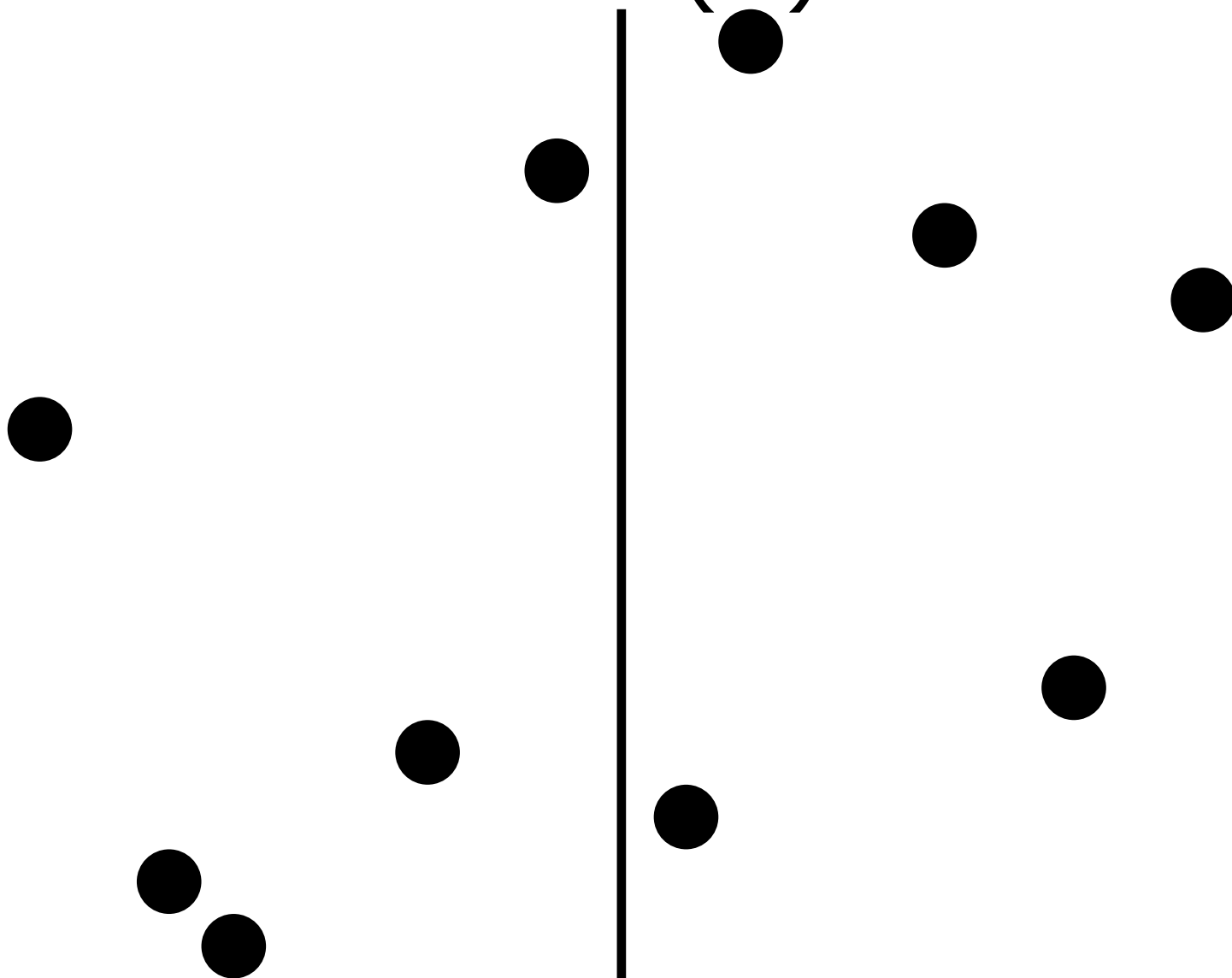


分割統治法

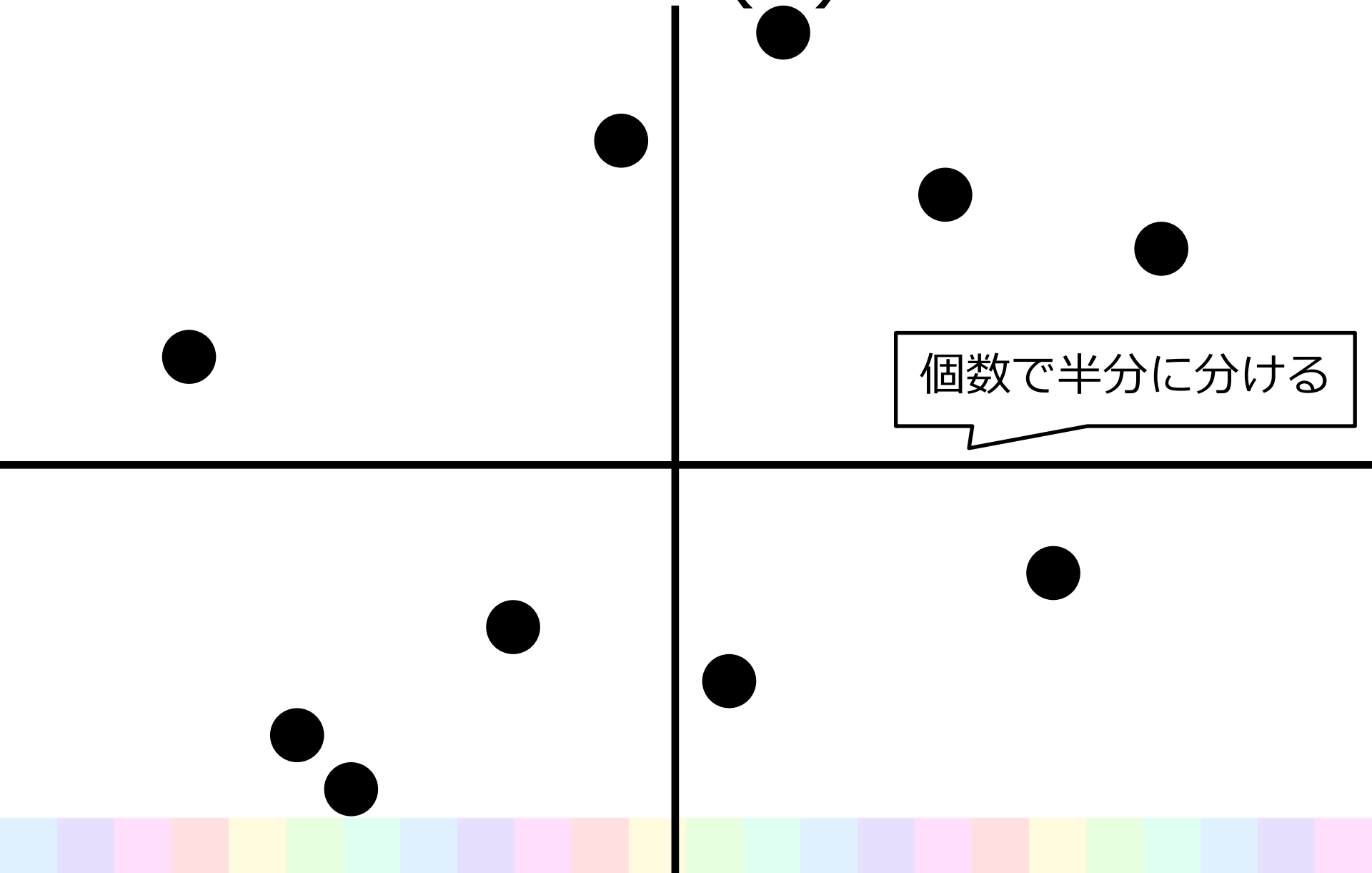
–Divide and Conquer–



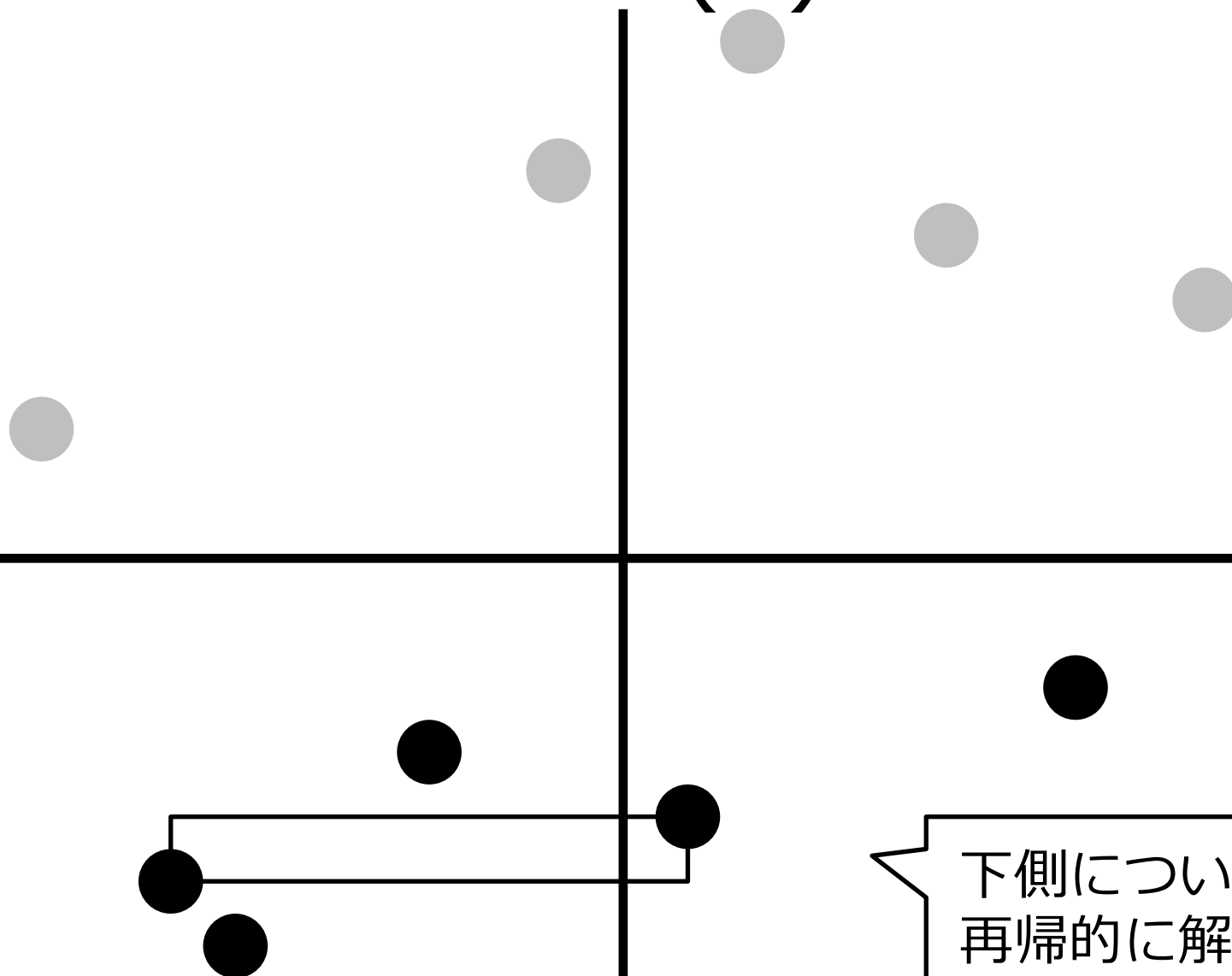
解法 (3)



解法 (3)



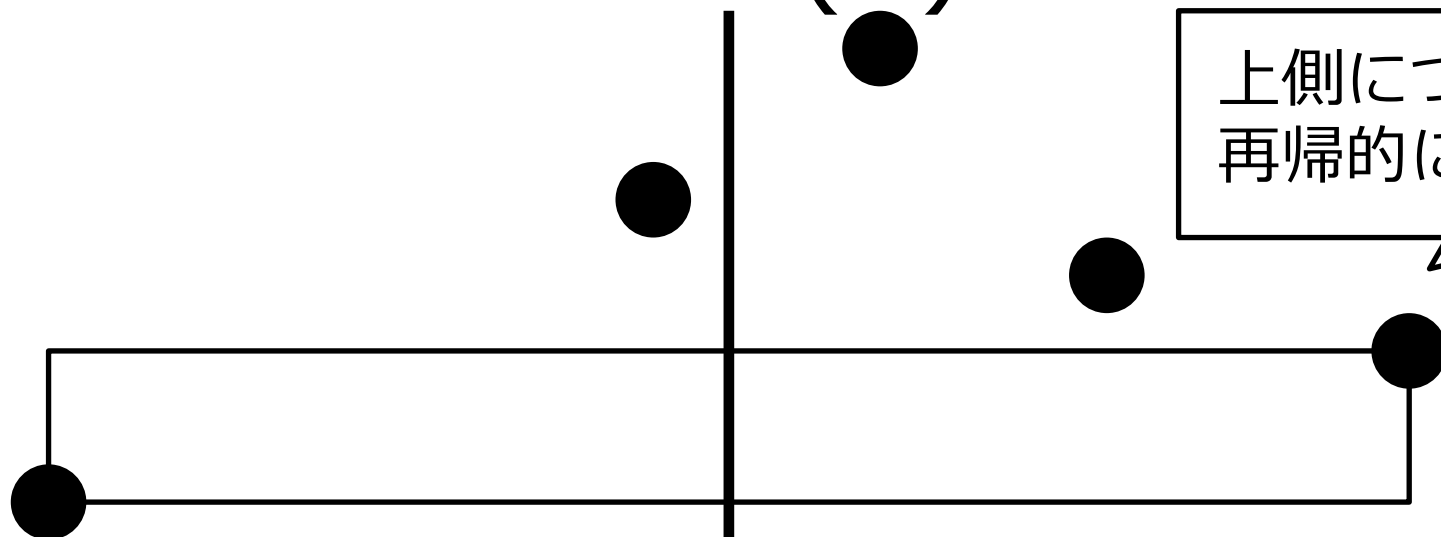
解法 (3)



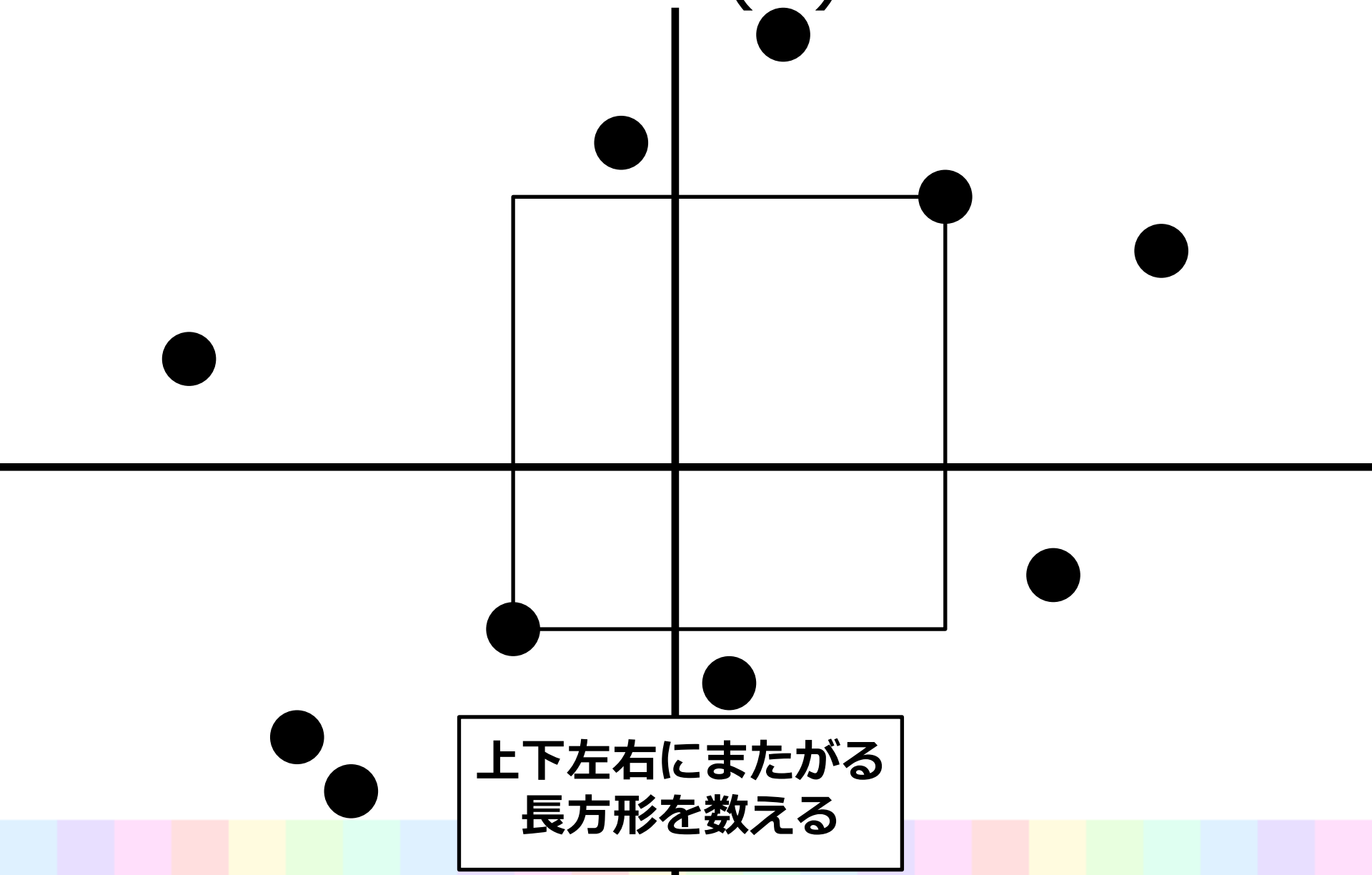
下側について
再帰的に解く

解法 (3)

上側について
再帰的に解く



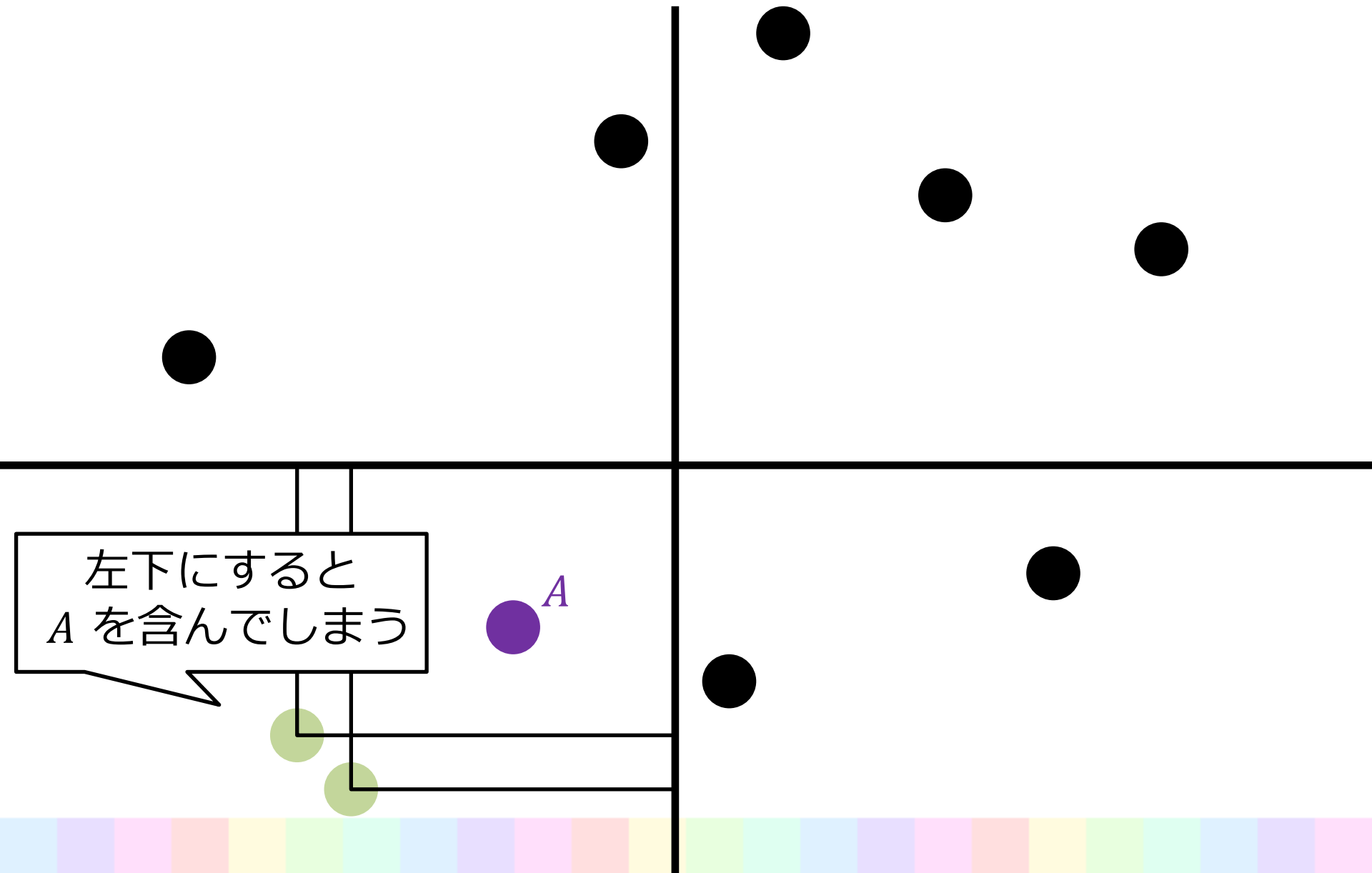
解法 (3)



解法 (3)



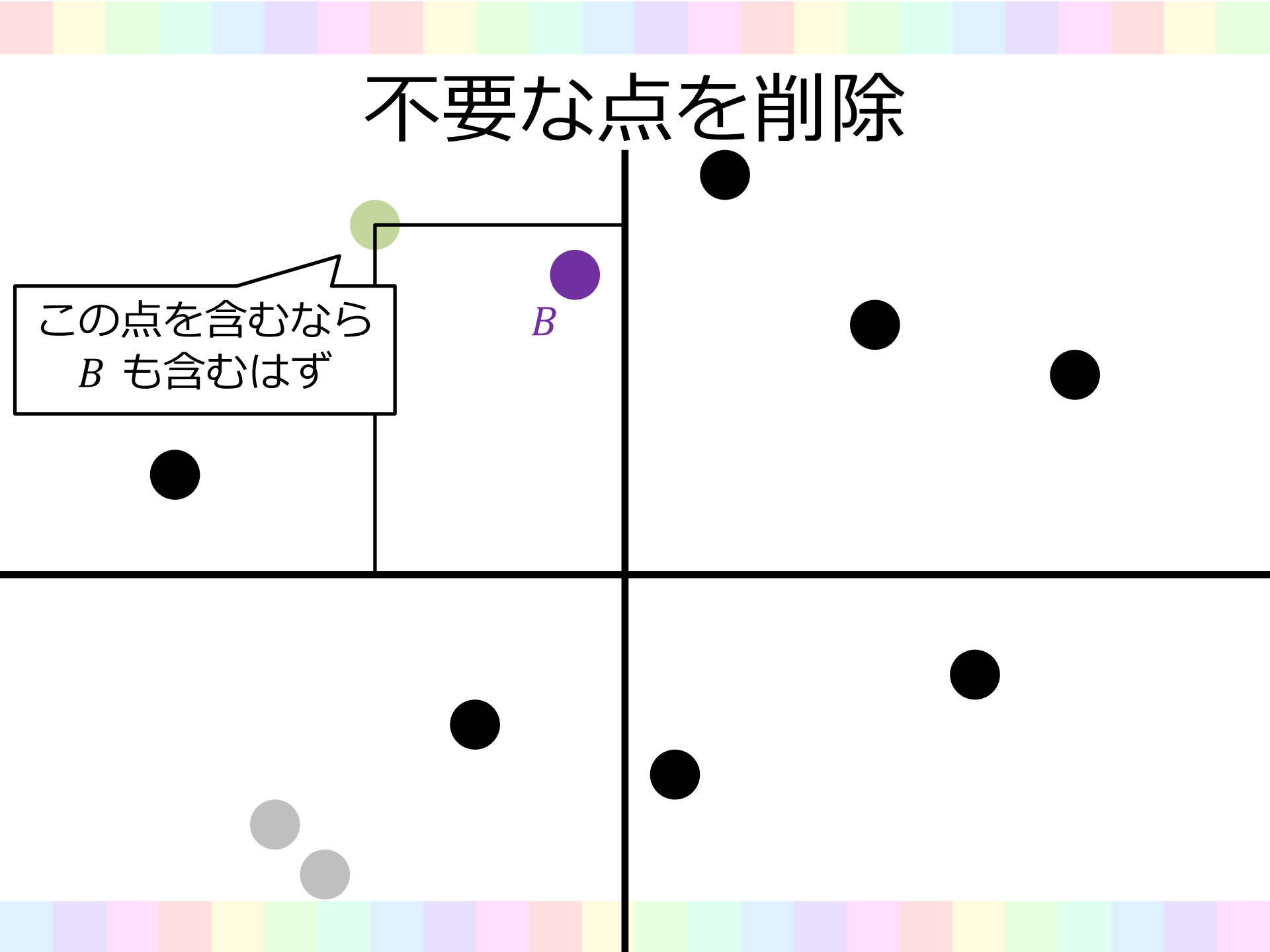
不要な点を削除



不要な点を削除

この点を含むなら
 B も含むはず

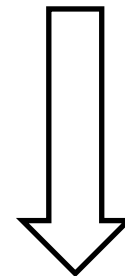
B



不要な点を削除

- 4 領域とも, 「中央のほうに並んでいる点たち」だけ残せばよい
 - 正確には, それと中央線の交点とで長方形領域を作ったとき, 他の点が含まれないような点たち
 - 残った点は, y 座標の小さい順に並べると, x 座標は小さい順 or 大きい順になる

不要な点を削除

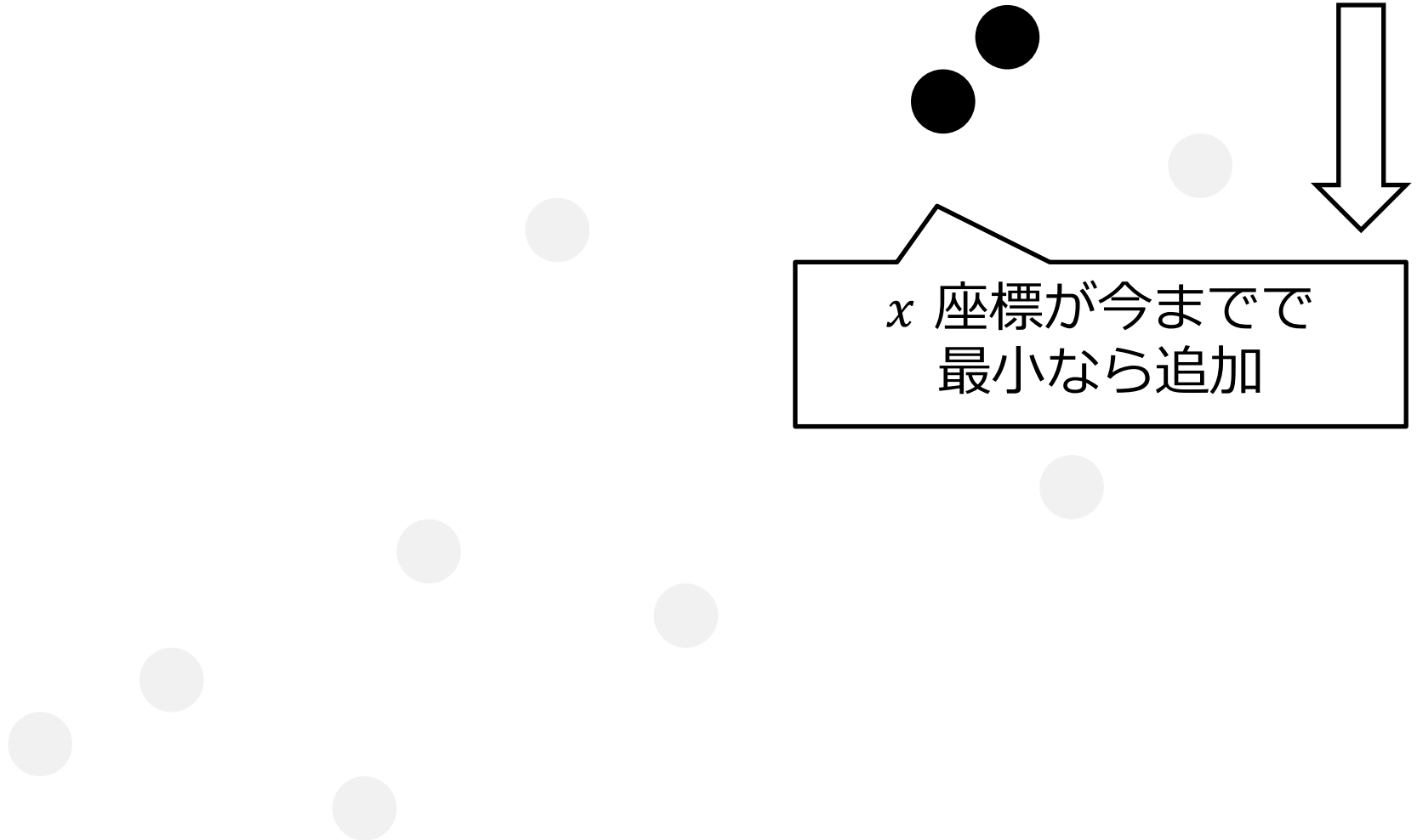


不要な点を削除

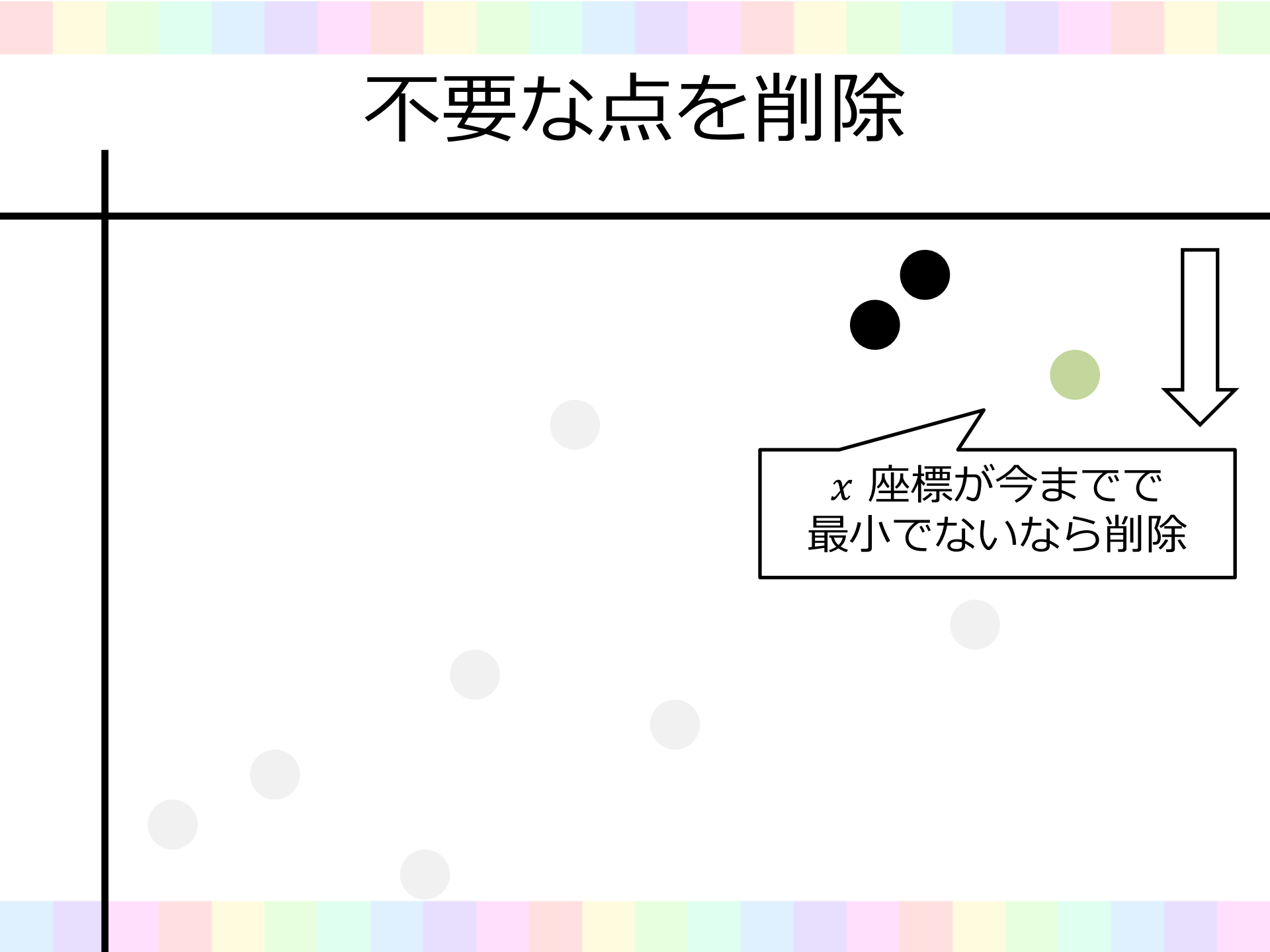


x 座標が今までで
最小なら追加

不要な点を削除

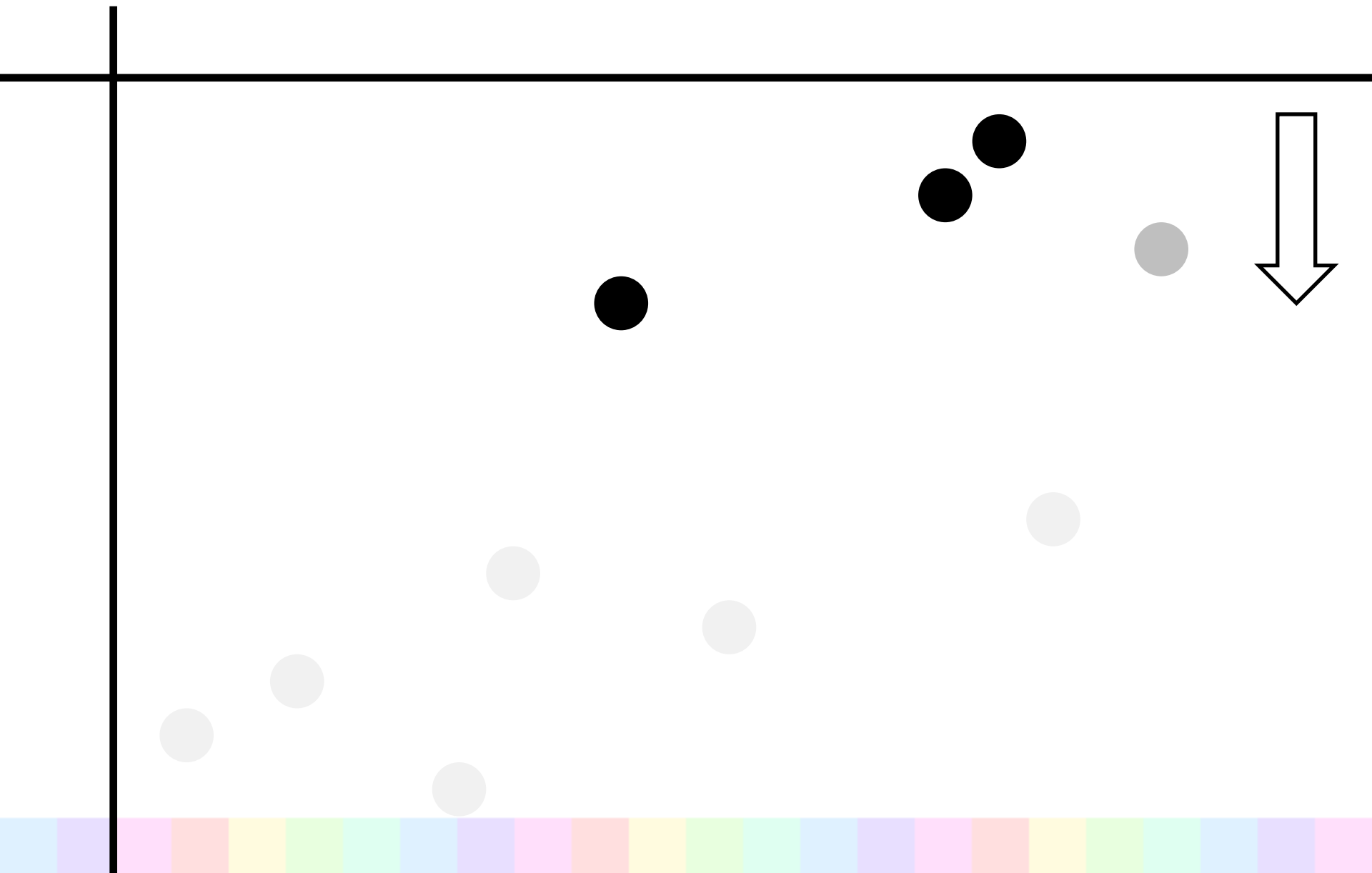


不要な点を削除

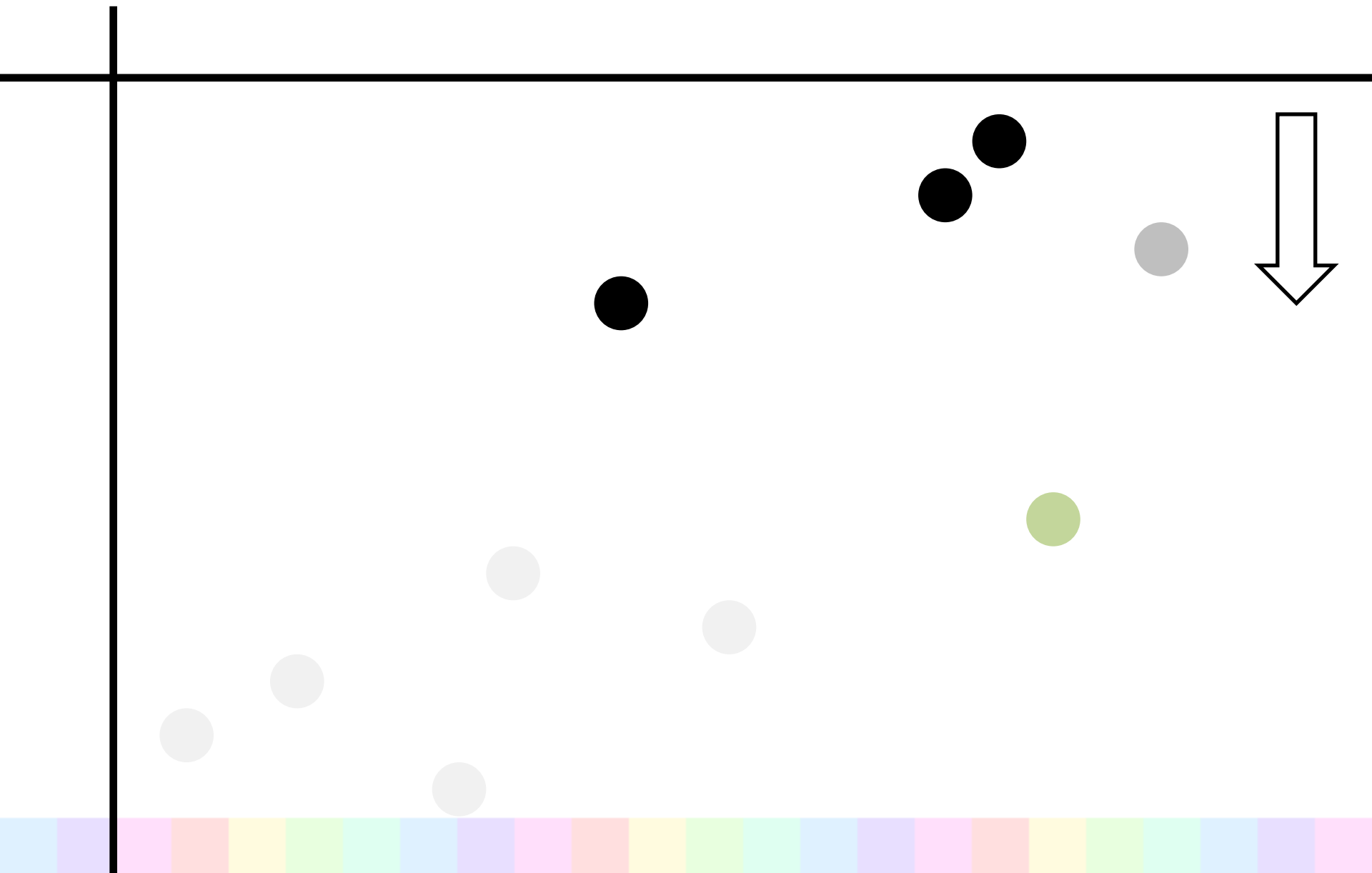


x 座標が今までで
最小でないなら削除

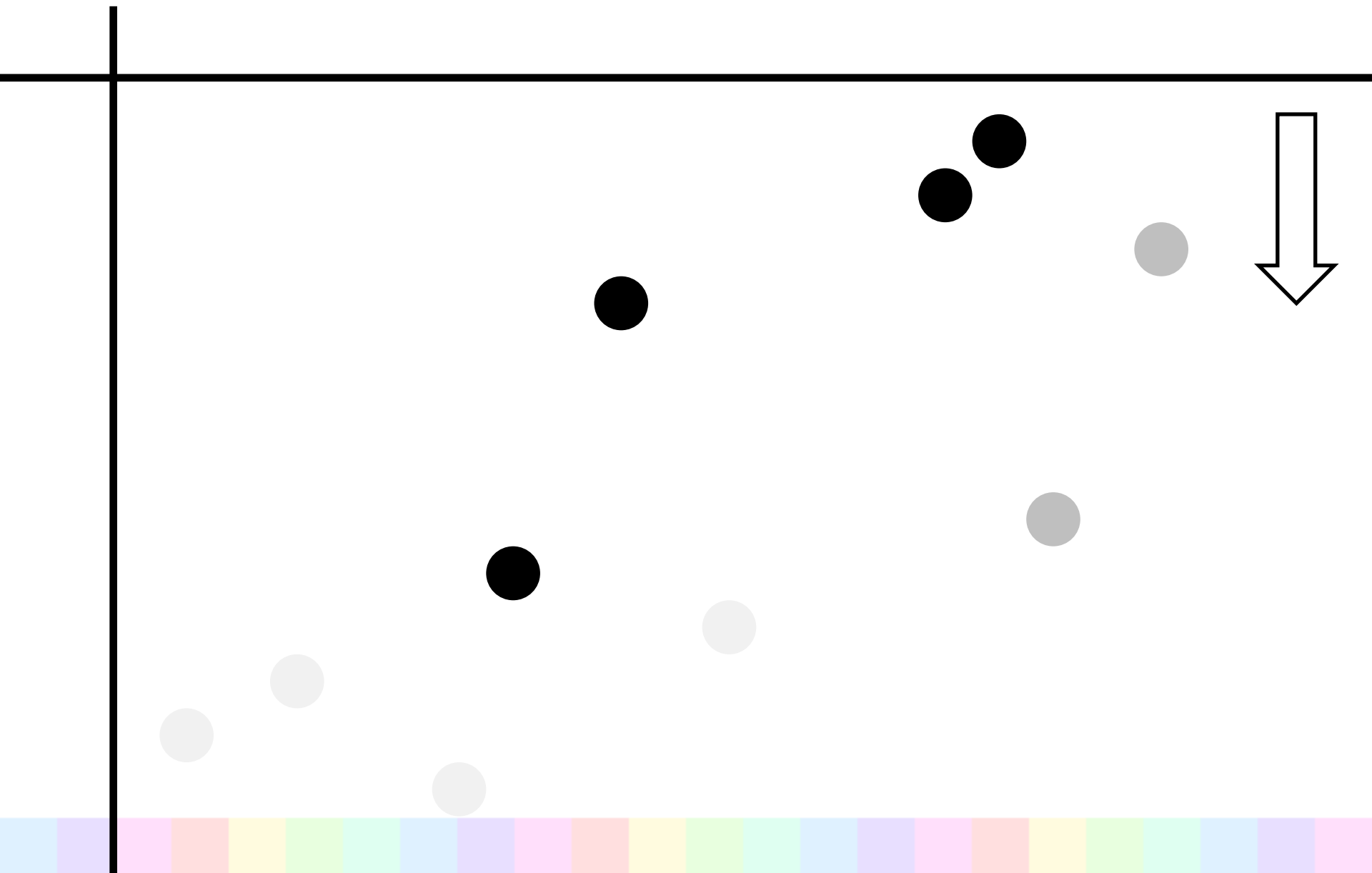
不要な点を削除



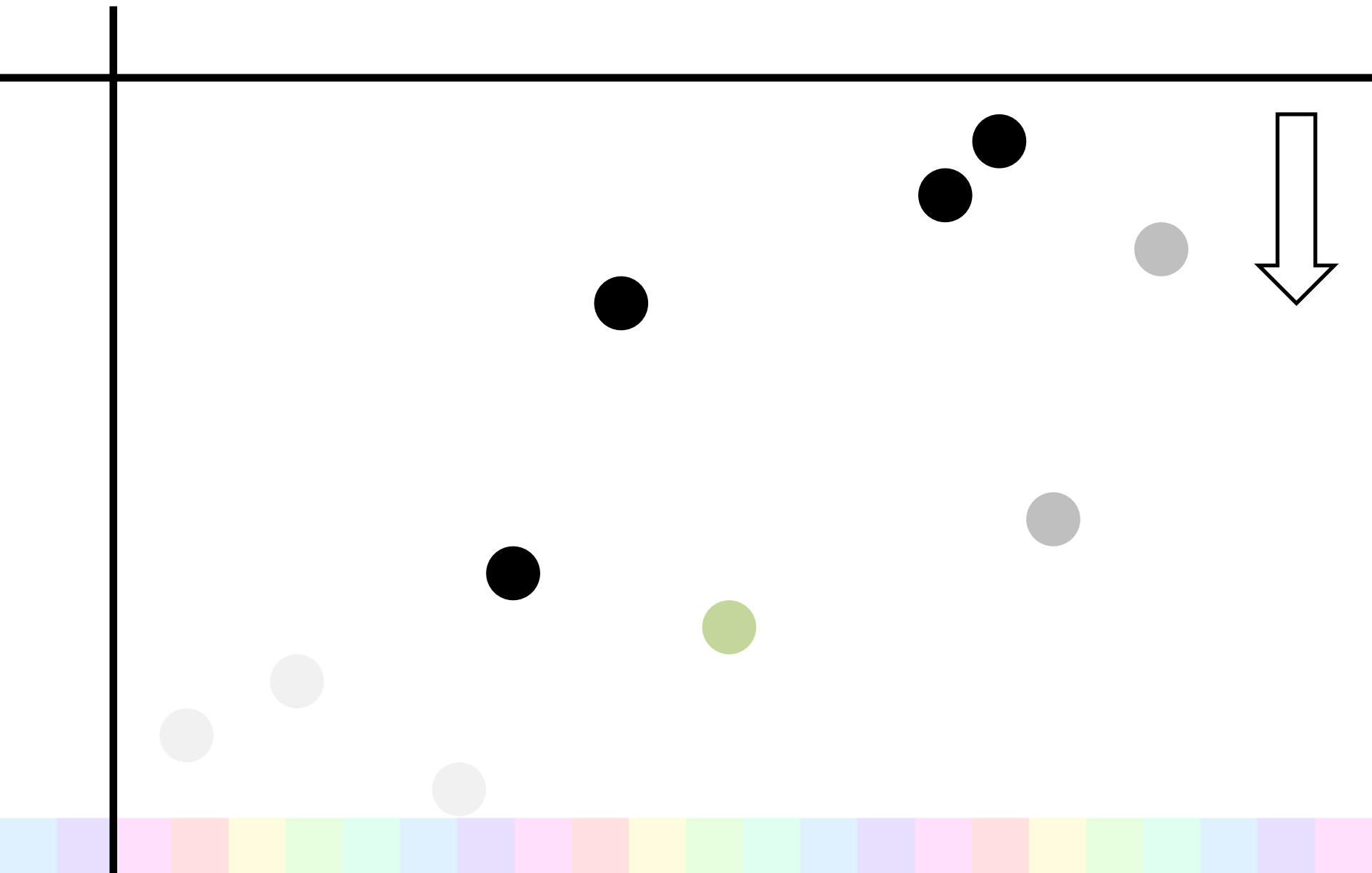
不要な点を削除



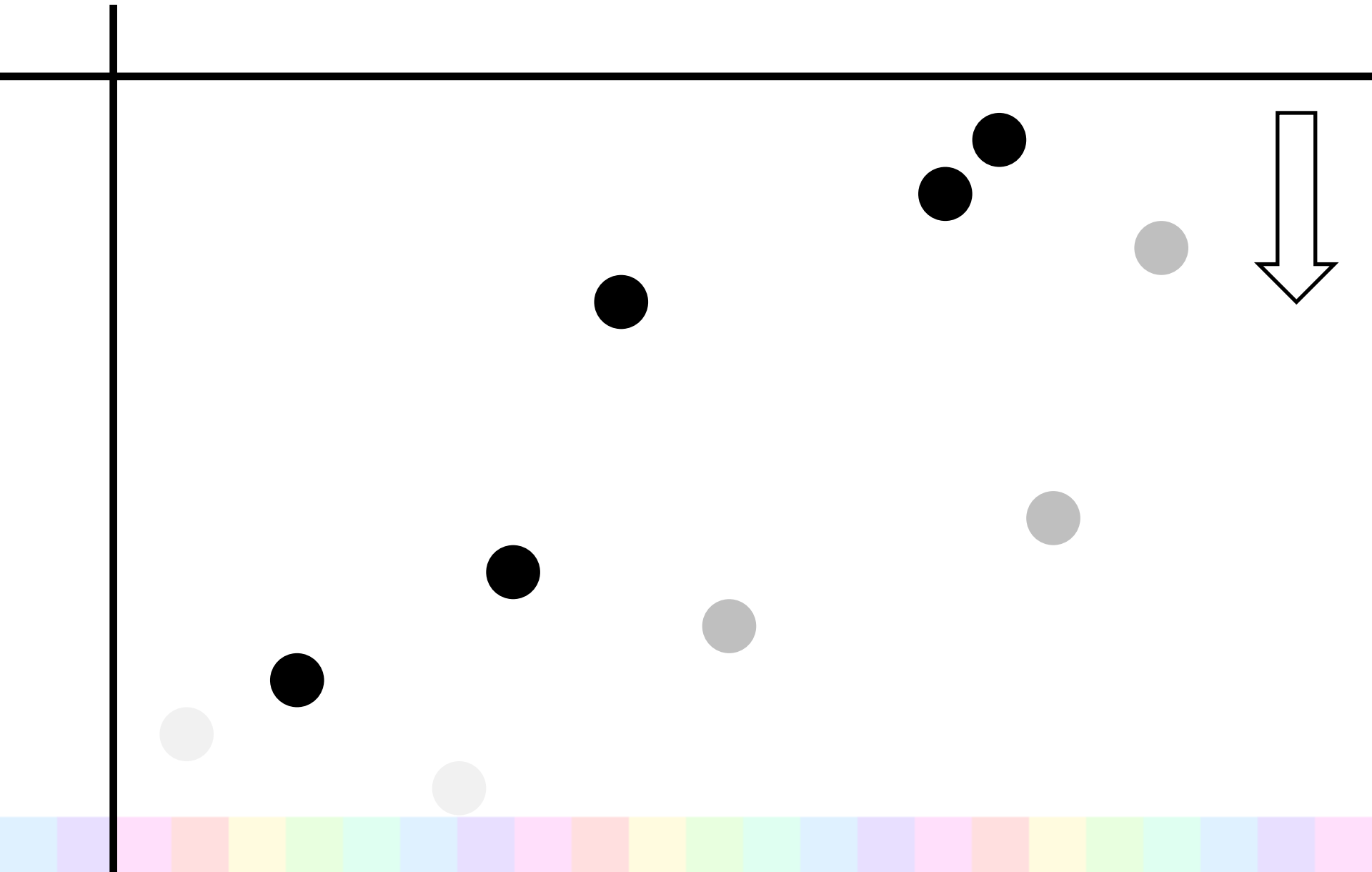
不要な点を削除



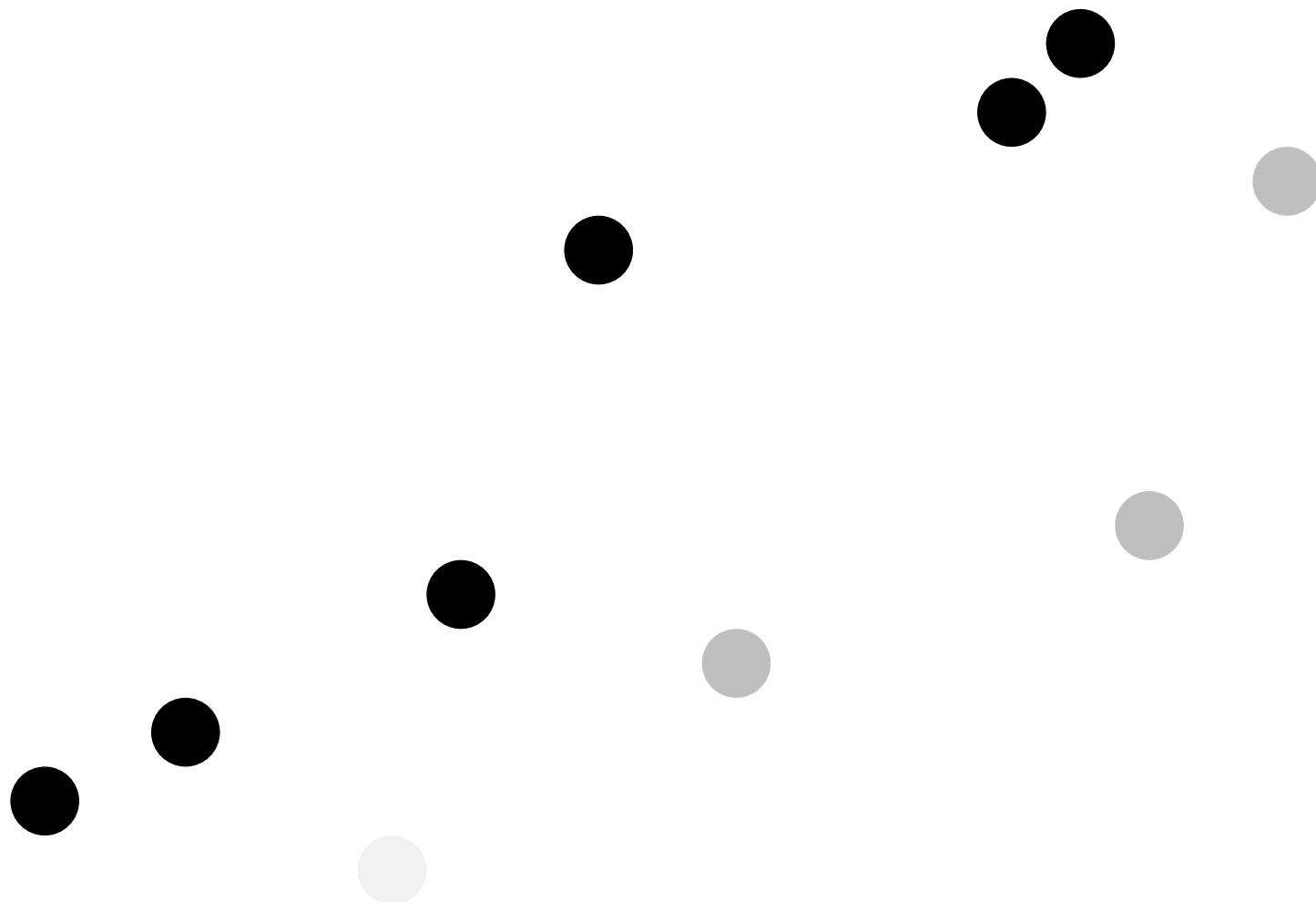
不要な点を削除



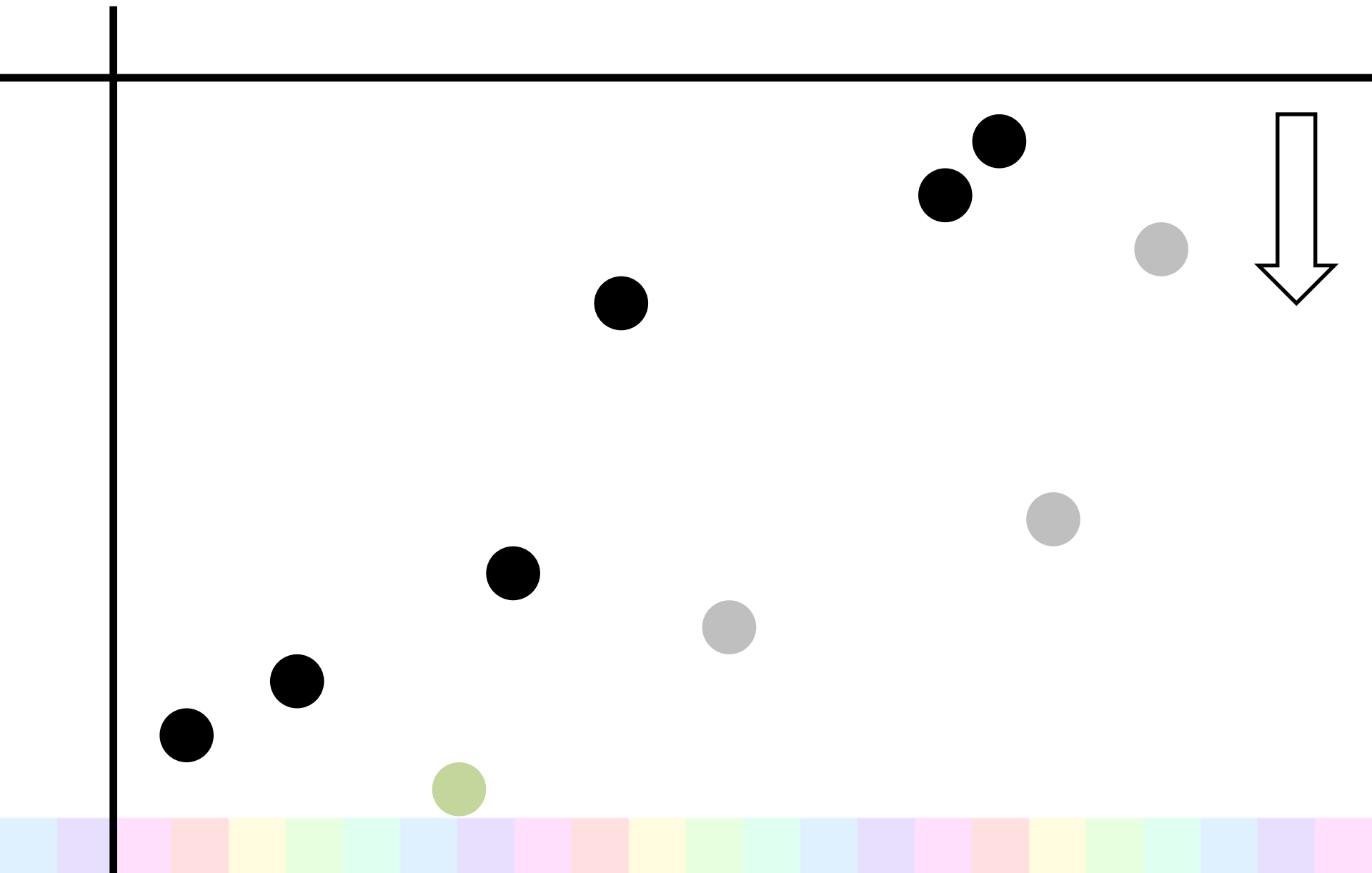
不要な点を削除



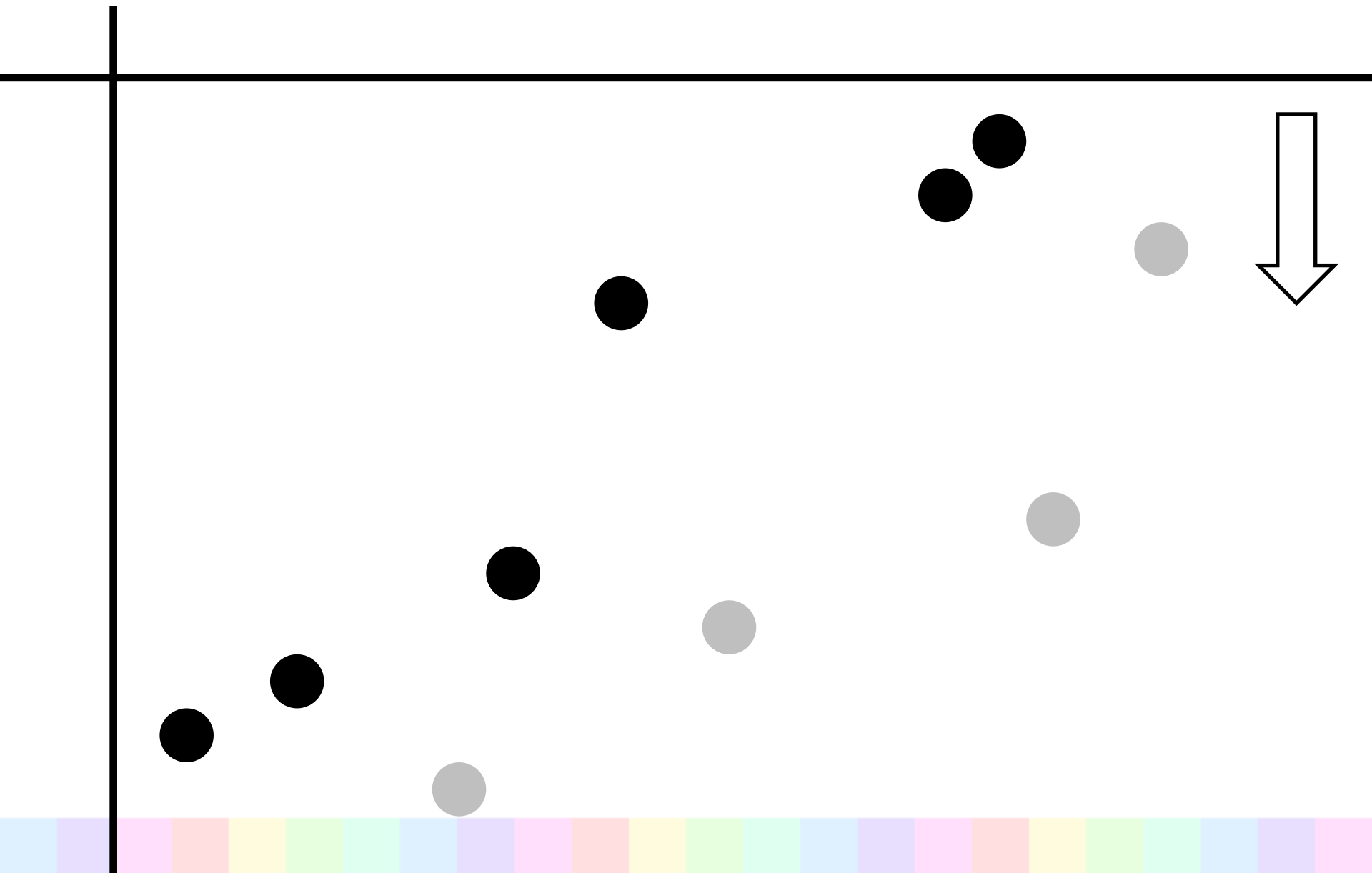
不要な点を削除



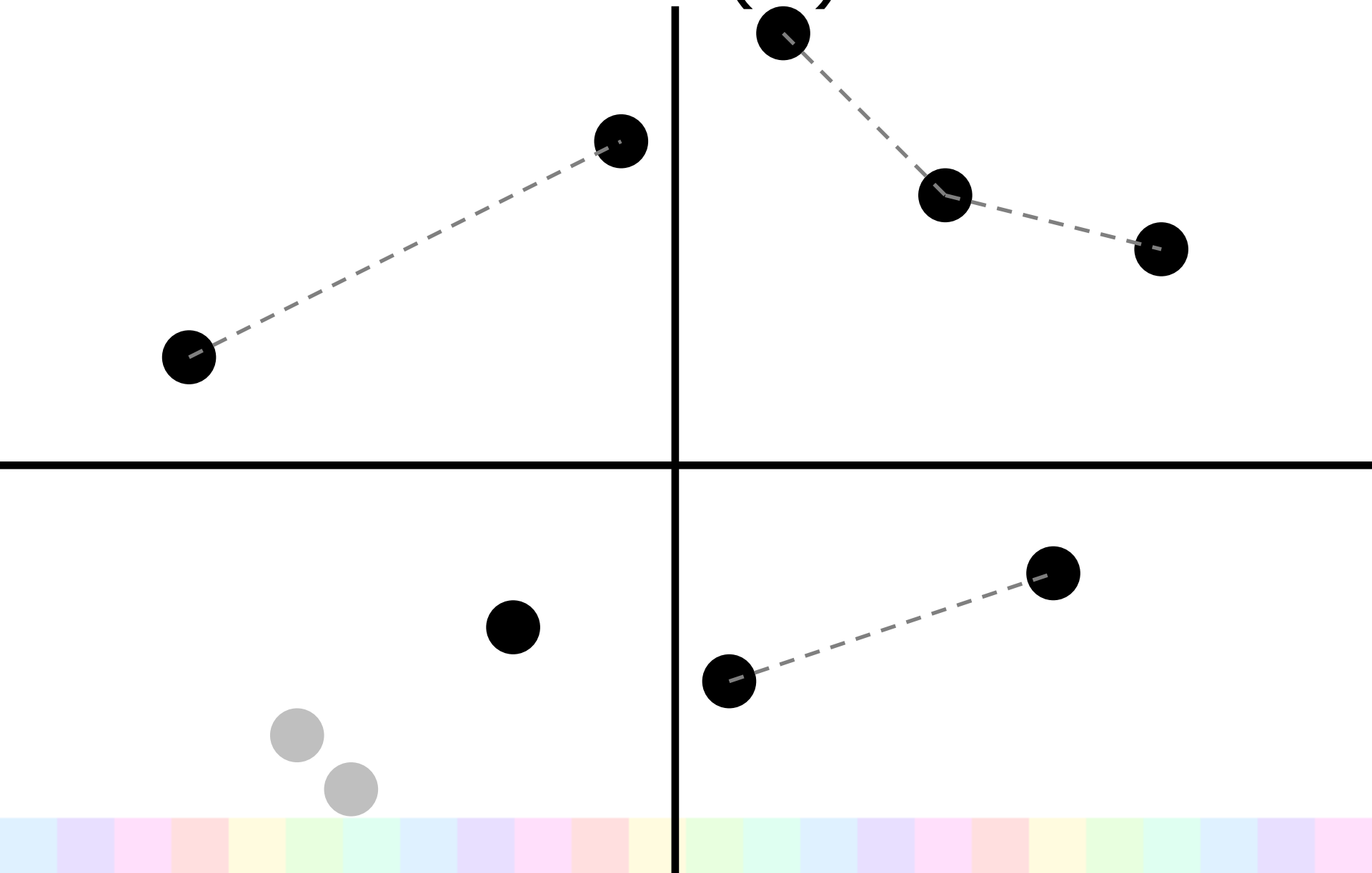
不要な点を削除



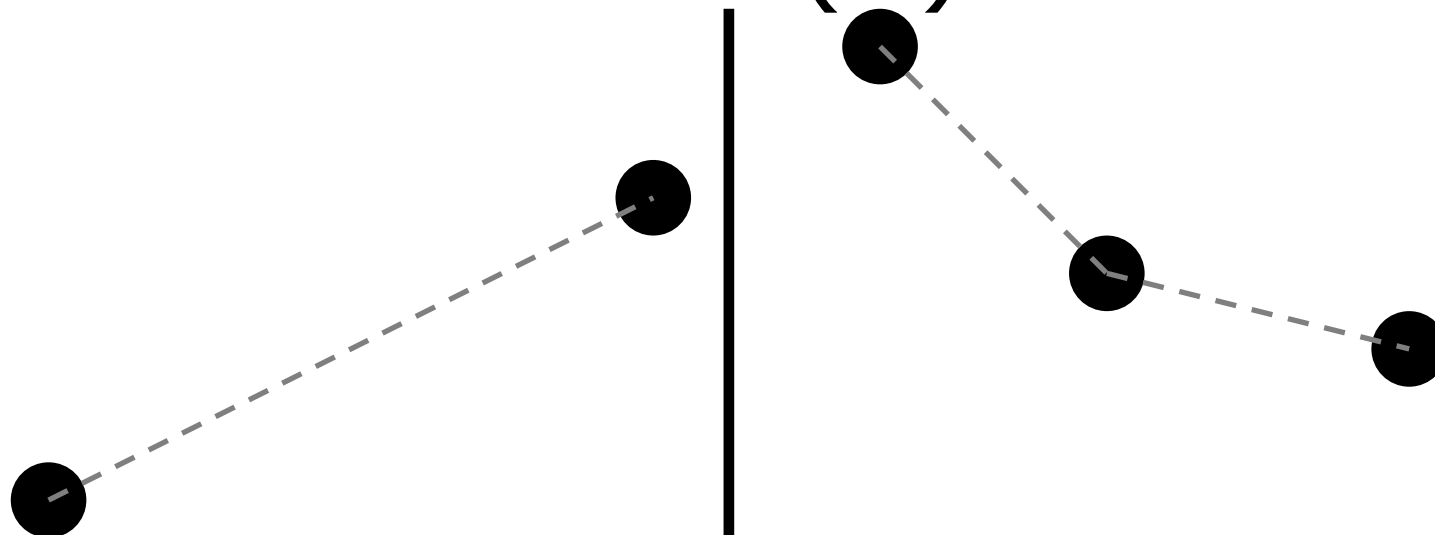
不要な点を削除



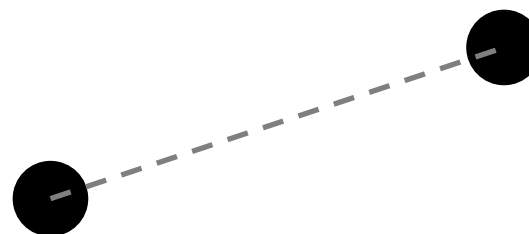
解法 (3)



解法 (3)



左下の点を
決める

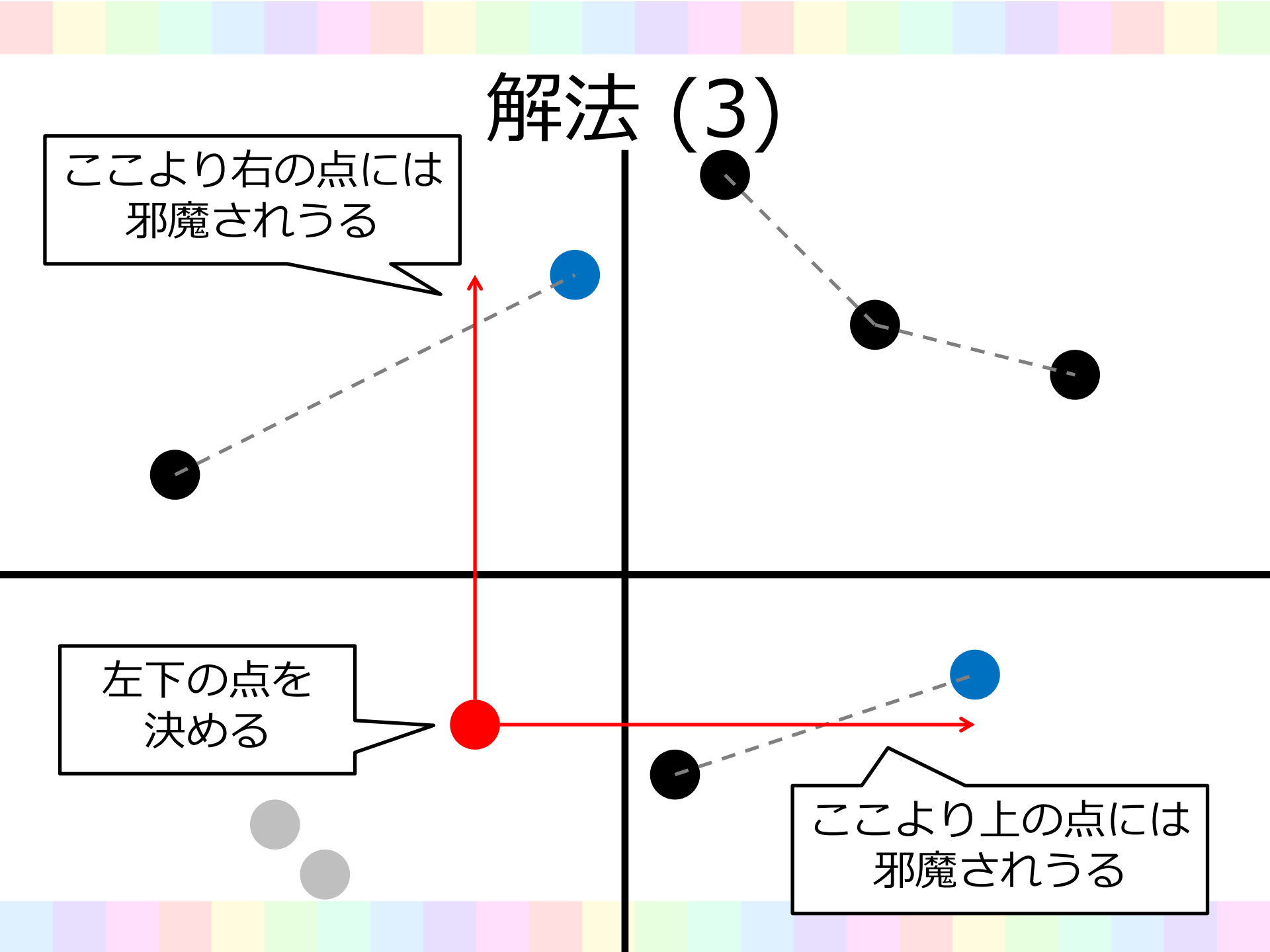


解法 (3)

ここより右の点には
邪魔されうる

左下の点を
決める

ここより上の点には
邪魔されうる



解法 (3)

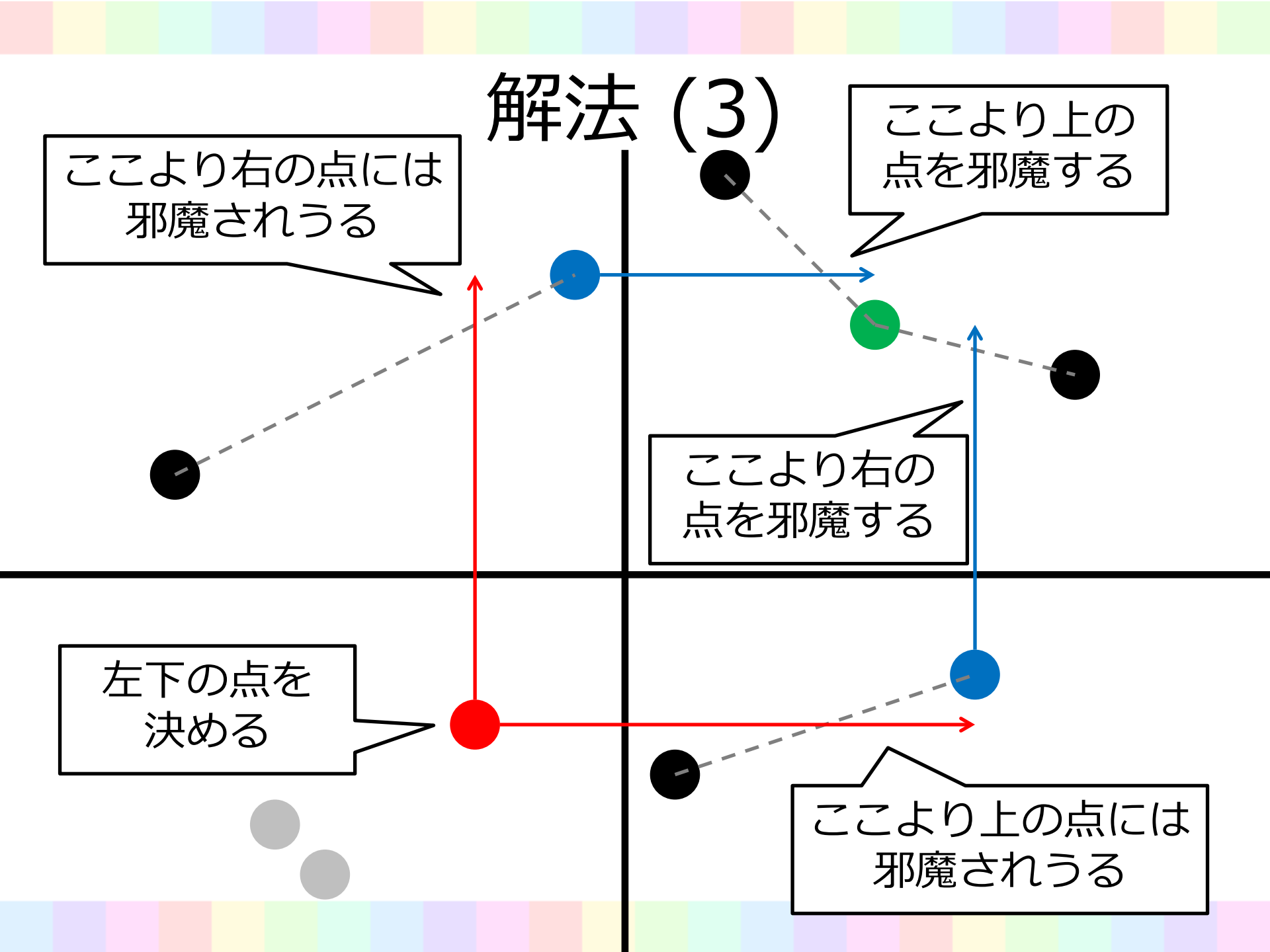
ここより右の点には
邪魔されうる

ここより上の
点を邪魔する

ここより右の
点を邪魔する

左下の点を
決める

ここより上の点には
邪魔されうる



解法 (3)

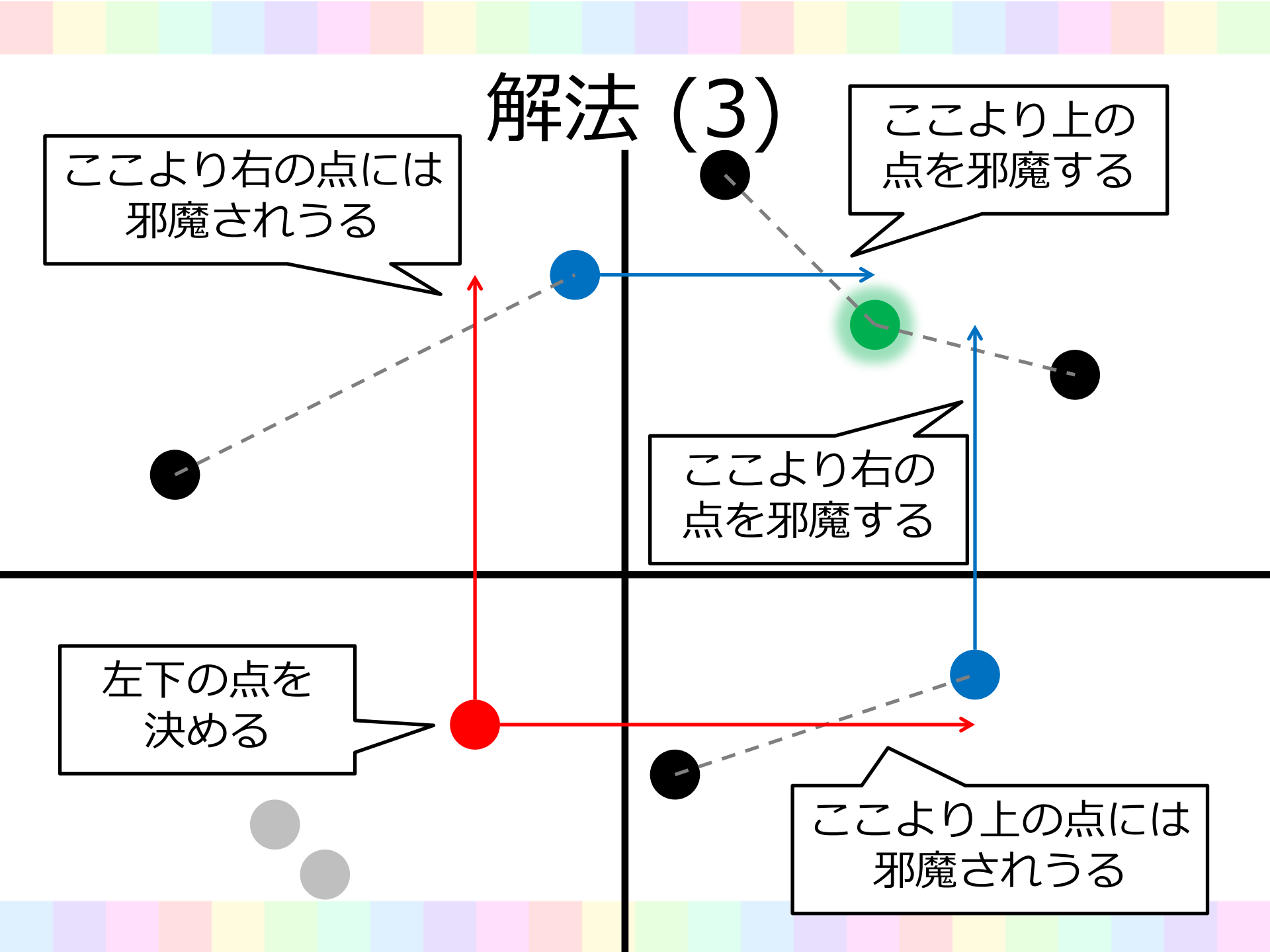
ここより右の点には
邪魔されうる

ここより上の
点を邪魔する

ここより右の
点を邪魔する

左下の点を
決める

ここより上の点には
邪魔されうる



解法 (3)

- 左下の点を決める
- 4 回二分探索する
- 右上としてとれる点の範囲がわかる
- $O(N \log N)$ 時間
- 全体で $O(N(\log N)^3)$ 時間
 - 実装：不等号や x, y で混乱しがちな単純作業

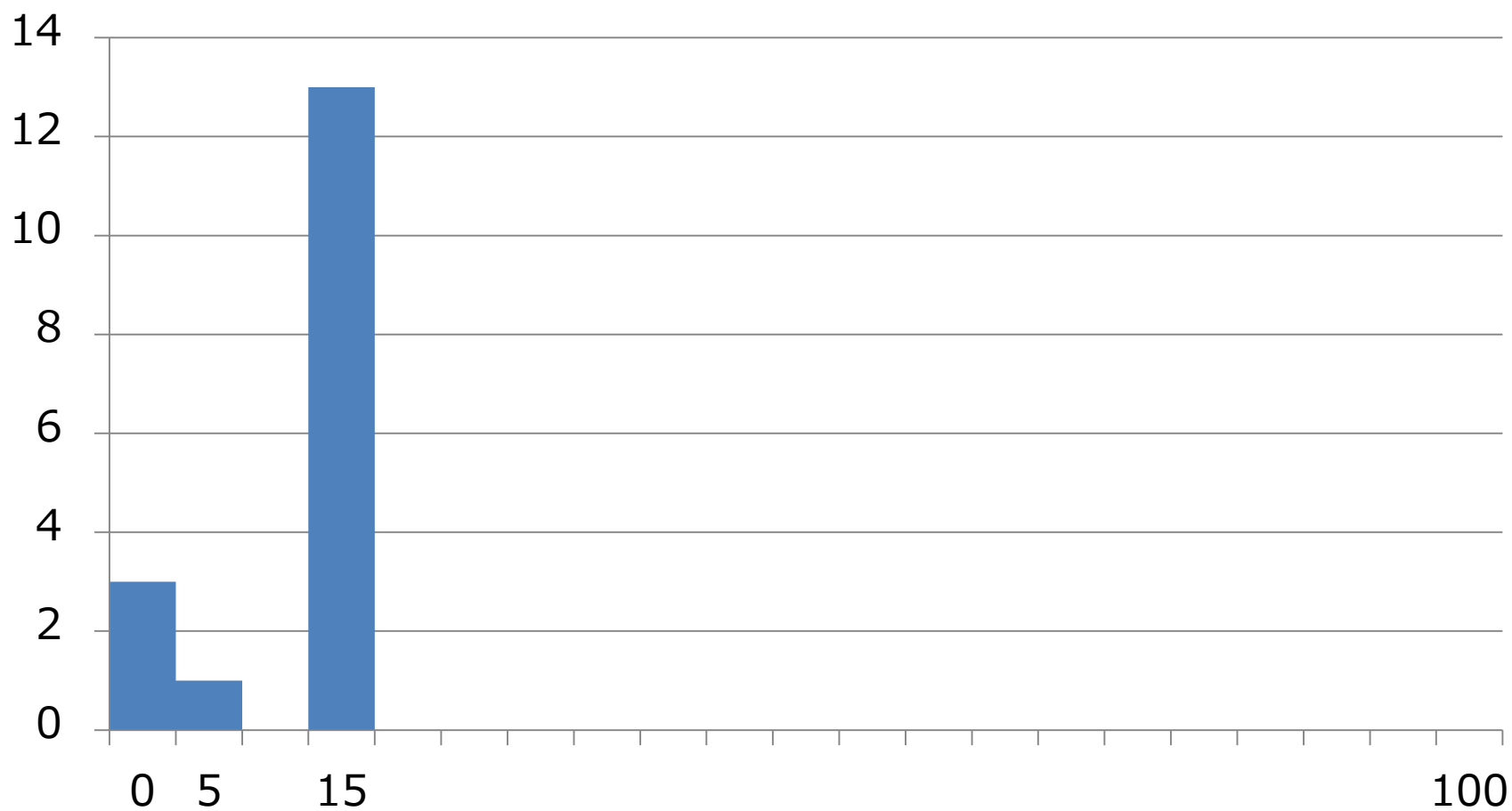
解法 (3)

- 尺取法が使える
 - 左下の点を動かしたとき, 二分探索する先が単調に変化するから
- ソートも分割統治のついでにできる
- $O(N)$ 時間
- 全体で $O(N(\log N)^2)$ 時間
 - 速度・実装ともあまり変わりません

分割統治法に関して

- 分割統治してみたら簡単な問題に変わらないか？は重要なアイデア
 - しかも直接的には気づきにくい
- 典型的な例
 - マージソート
 - 最近点对
 - 点集合から距離がもっとも短い点の対を見つける
 - 応用： Google Code Jam 2009 World Finals B
 - 応用： ACM-ICPC 2013 Asia Aizu F

得点分布



JOI2014春合宿 Day3-Voltage

解説：hogloid





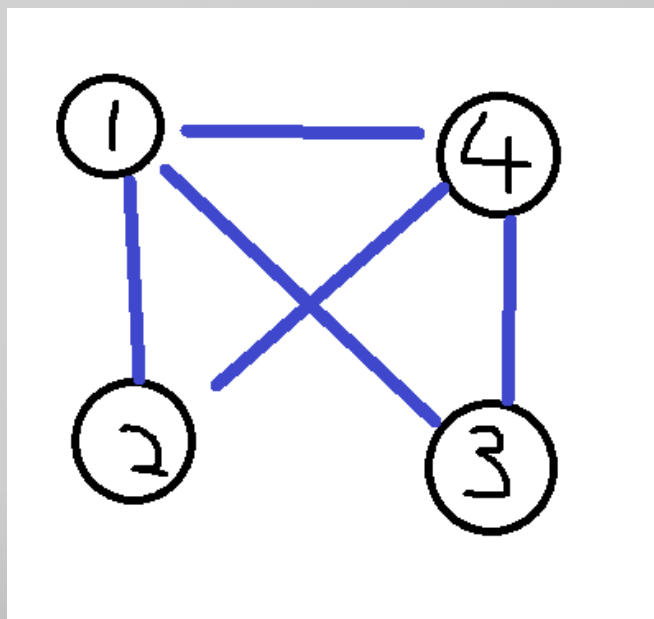
問題概要

- N 頂点、 M 辺からなるグラフがある(多重辺があったり、連結でないグラフが与えられることもある)
- 頂点に低電圧/高電圧を設定し、一つの辺のみ電流が流れず、他の辺はすべて電流が流れるようにしたい。
- 電圧を設定し、流さずにおける辺はいくつあるか数えよう。

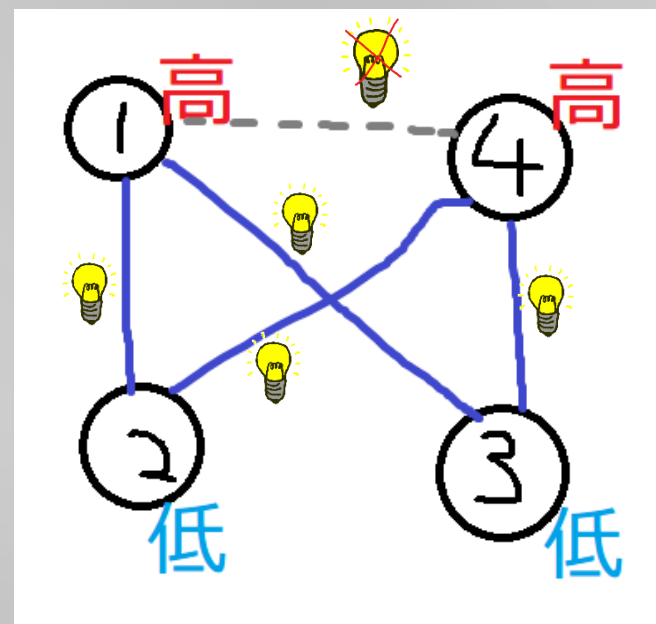


例

↓こんなグラフなら



↓灰色の点線の辺のみ
電流を止める設定がある





Subtask 1

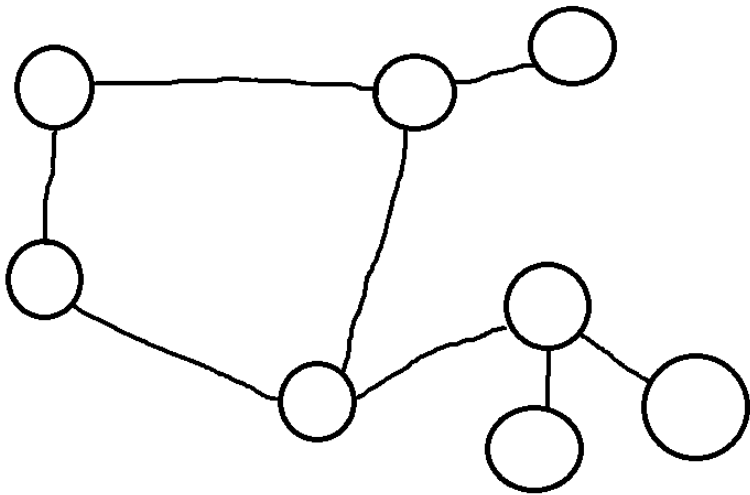
- 高電圧なら頂点に1,低電圧なら0と色を付ける。
- それぞれの辺について、その辺に電流を流さないときのことを考える。
- 流さない辺のみ、辺の両端を同じ色、他の辺は異なる色を両端に割り当てたい。
- 1と0はすべて逆にしても変わらないので、連結なグラフごとに一つの頂点の色を決めて、そこから辺をたどって色を決めていき、矛盾がないか調べればよい。
- これで $O(M(N+M))$



Subtask 2

- 連結なグラフで、 $M=N$ といえは・・・
- ↓みたいな感じの、閉路が一つ&木がくっついてるグラフになります。 ($M=N-1$ で木、それに

1本辺がついてるため)





Subtask 2 (閉路の辺の数が偶数)

- 閉路の辺の数が偶数のとき、閉路には流れない辺を作れない。
- 木の部分には、ある頂点の色を決めたら電流の流す/流さないに応じて隣の頂点の色を決められる。閉路がないので、任意に決めても条件に反する辺はない。
- これより、木の部分のどの辺も流さないようにできる。
- よって、閉路以外の辺の数が答え。



Subtask 2 (閉路の辺の数が奇数)

- 閉路の辺の数が奇数のとき、閉路には流れない辺を作らなければならない。またどこに作ってもよい。
- 木の部分は、前ページと同様に任意の流し方ができる。
- よって、閉路の辺の数が答え。



重要な考察

- 簡単のため、しばらくグラフは連結と仮定する。
- 一つ任意の全域木を取ると、全域木のすべての辺に電流を流すとき・・・
- ある頂点の色を決め、そこから全域木をたどって色をすべて決められる。
- 残りの辺で、電流が流れないものが一つのみあればOK. そうでないなら不可



重要な考察

- 全域木^①の辺(S とおく)で電流を流さないものを選ぶとき . . .
- 全域木^①以外の辺(T とおく)はすべて電流を流さなければならない。
- S の辺すべてに電流を流すときの色付けをして、 S の辺の一つに電流を流さないことにすると、その辺以下のすべてのノードの色が反転する。

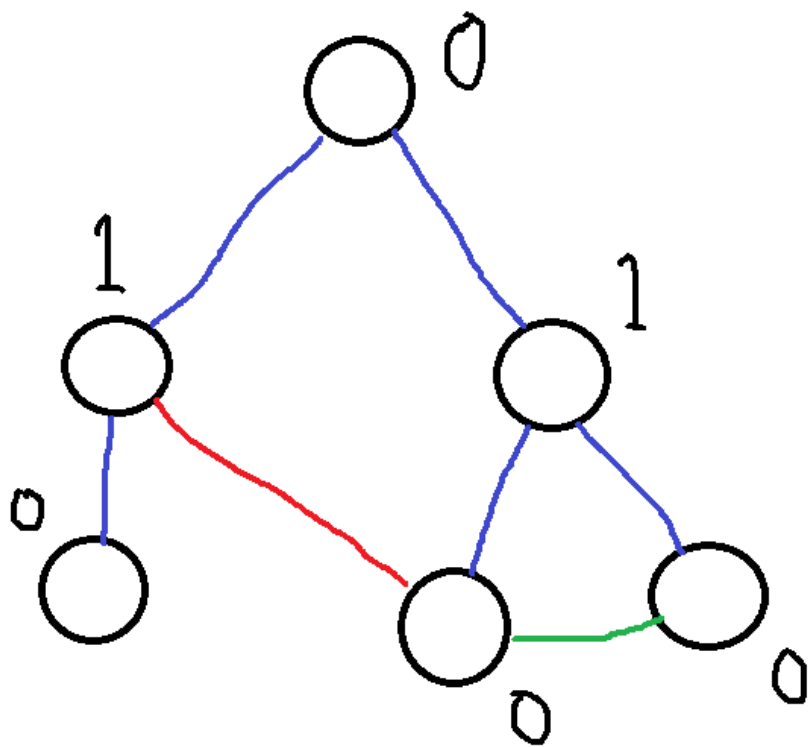


重要な考察

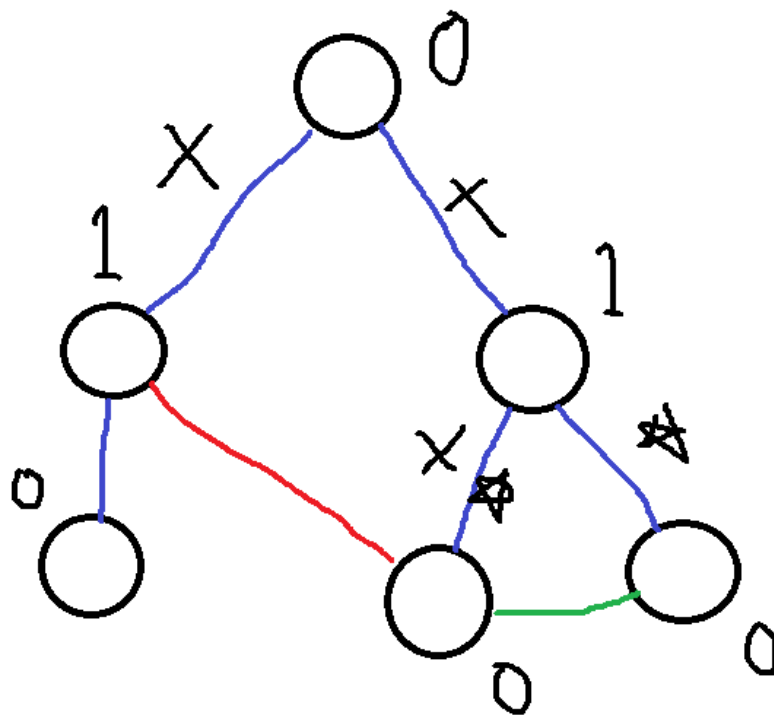
(Sから電流を流さないものを選ぶとき)

- 反転の性質から、全域木で電流を流すとしたときの色付けで、Tの辺で電流が流れるなら、その辺のつなぐ二つの頂点の全域木内パス間では流さない辺があってはいけない。
- 同様に、Tの辺で電流が流れないなら、その辺のつなぐ二つの頂点の全域木内パス間で流さない辺を作らなければならない。

例: 丸が頂点、隣の数字が s にすべて電流を流すときの色、青の辺が全域木の辺。赤の辺はもとの色付けで電流が流れ、緑の辺は流れないもの。



青い辺で流さない辺を作るとき、Tの辺の条件から、
×の辺は使ってはいけない。また、☆の辺から選ばなければならない。（ここでは1本のみ）





Subtask 3

- 全域木は様々なやり方みたいなので作れます
- Tの辺について、前の例のようにパスに☆マーク、×マークを愚直に書きまくる・・・①
- ×マークが0で、☆マークがすべて書かれている辺の数が、Sの辺の中で条件を満たす数。
- これにTで流さなくできる辺を加えると答え・・・②
- ①のパートは $O((M-N+1)N)$, ②のパートは $O(M-N+1)$
- 全域木も高速に作れるので、これで55点。



Subtask 4

- ①の書きまくる段階で、二つの頂点のLCAを取り、木の上で下から上へ累積和を取ると、愚直に書かずに☆と×の数が分かる。
- 連結でないときは、それぞれの部分グラフで全域木を作り、全域木以外の辺で流れないものが一つだけあるならその辺は流れなくすることができる。



Subtask 4

- 全域木の辺で流さないものを作るとき、満たすべき条件はやっぱり☆をすべて取り、×をひとつも取らないとき。
- これで $O(M\log N + N)$



Subtask 4

- これでも一応通ると思いますが、ここからがすごい部分(だと思われる)
- LCAを求めるのはそこそこ面倒。全域木は自由で作っていいので、うれしい性質を持つ木を作りたい。
- あれ、そういえばDFS木というのが・・・



DFS木

- 根っこの頂点から始めて、まだ訪れていない点を再帰で次々と訪れていく(=DFS) このときの訪れていく経路の辺とそれにつながるとは、連結な部分の全域木(=DFS木)
- 訪れる経路に使われない辺は後退辺と呼ぶ。
- このDFS木を以前の全域木として使うと、後退辺のつながり2点は片方が片方の子なので、LCAは深さの浅い方。 $O(M+N)$
- DFS木については以下も参照
http://hos.ac/slides/20110504_graph.pdf,
http://hos.ac/slides/20120322_joi_sokoban.pdf



諸注意

- 多重辺があり得るので、再帰で辺を戻らないようにするときには前の頂点を覚えるだけでは不十分
- ☆の数、✕の数はひとつにまとめて行える



得点情報

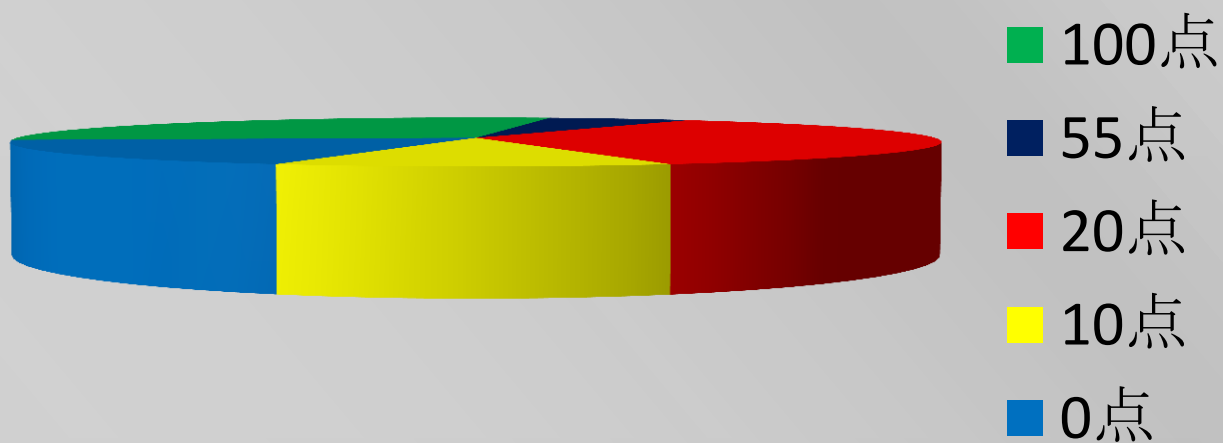
- ルーマニアかな？
- 最高20点かな？？





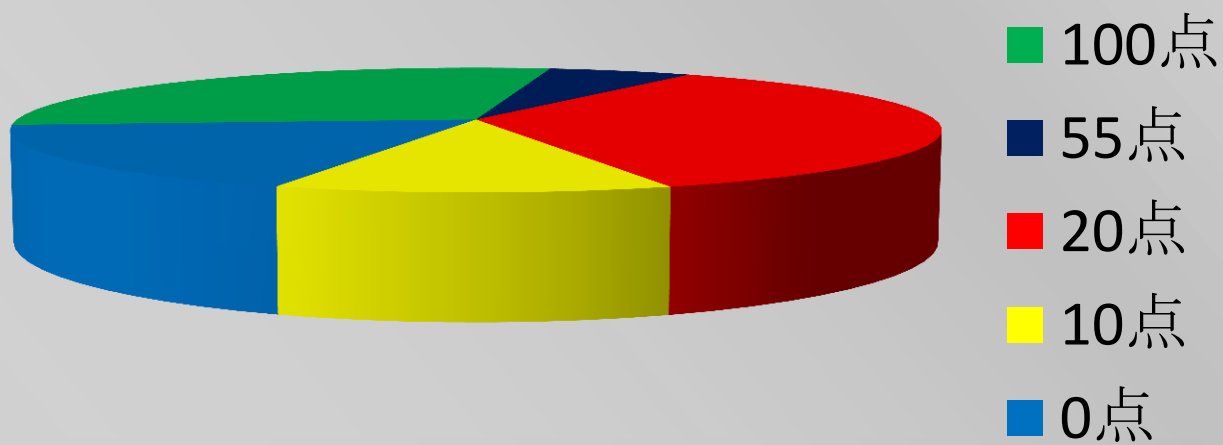
得点情報

■ ん？？？



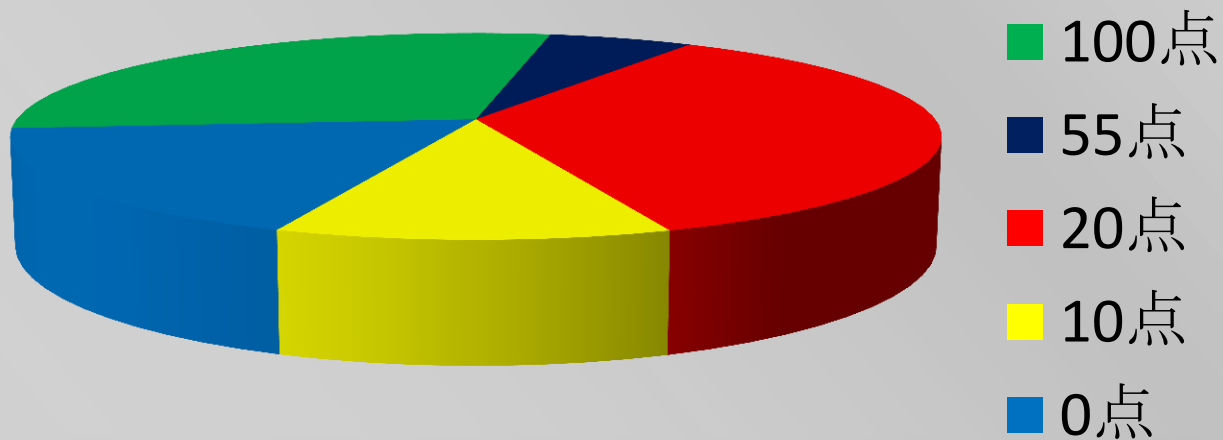


得点情報



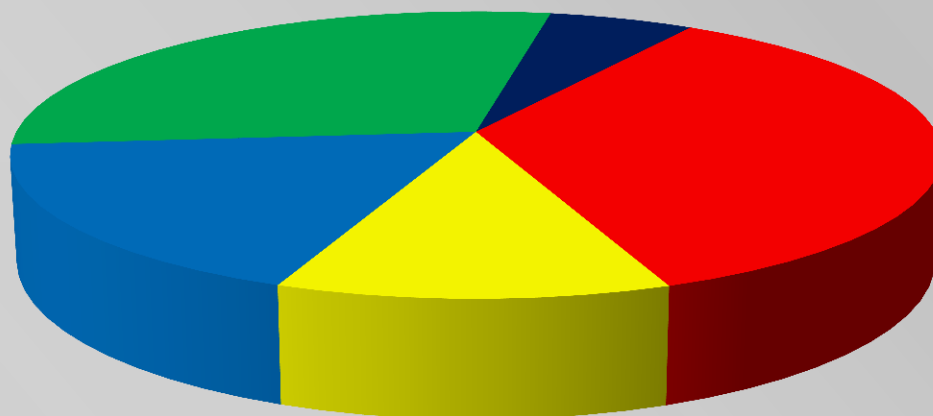


得点情報





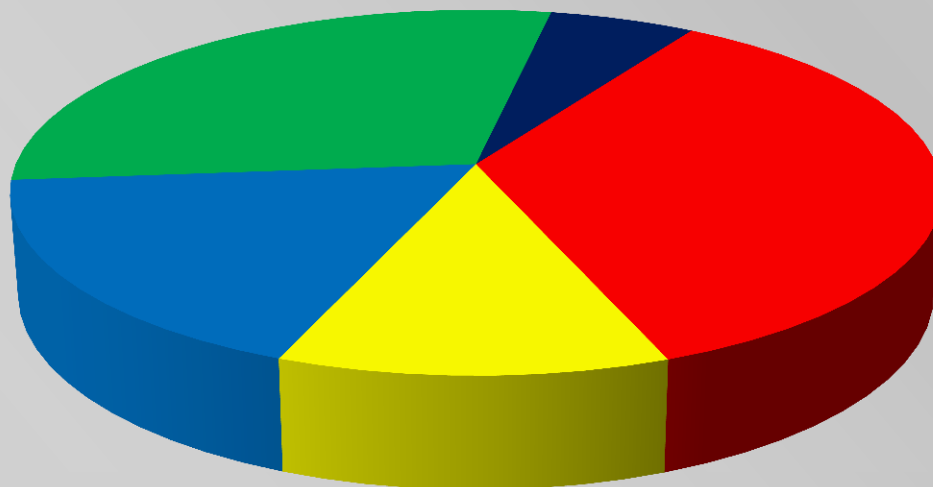
得点情報



- 100点
- 55点
- 20点
- 10点
- 0点



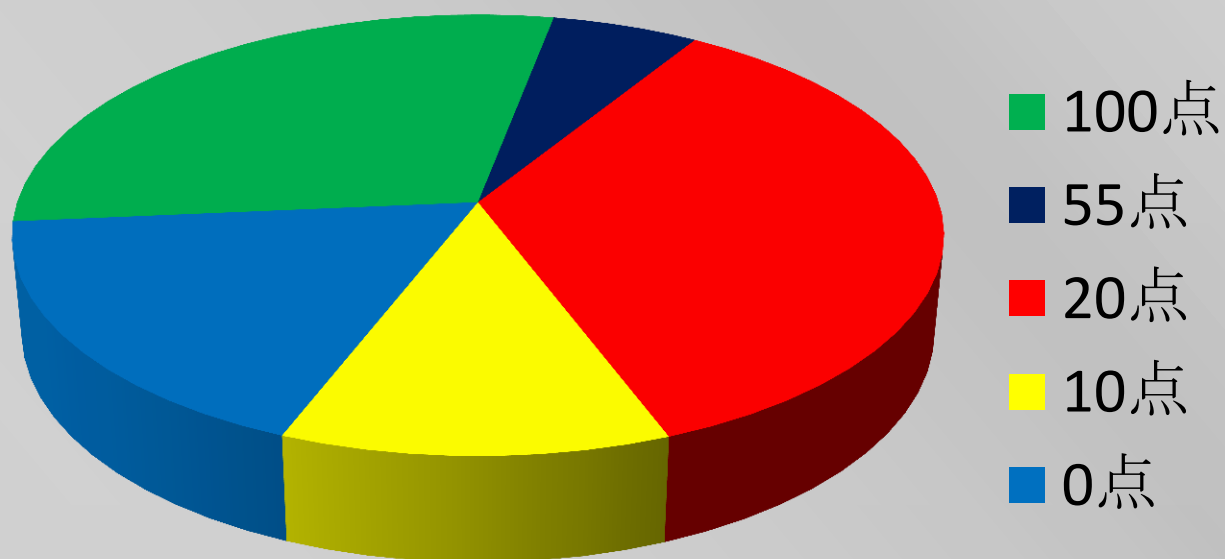
得点情報



- 100点
- 55点
- 20点
- 10点
- 0点

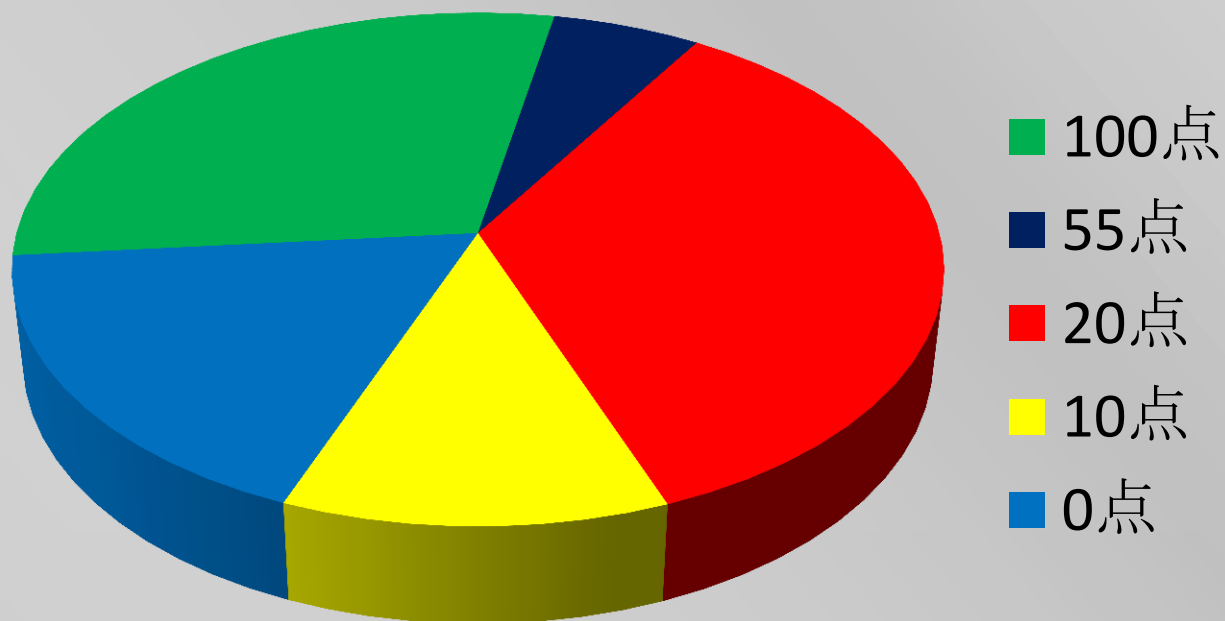


得点情報



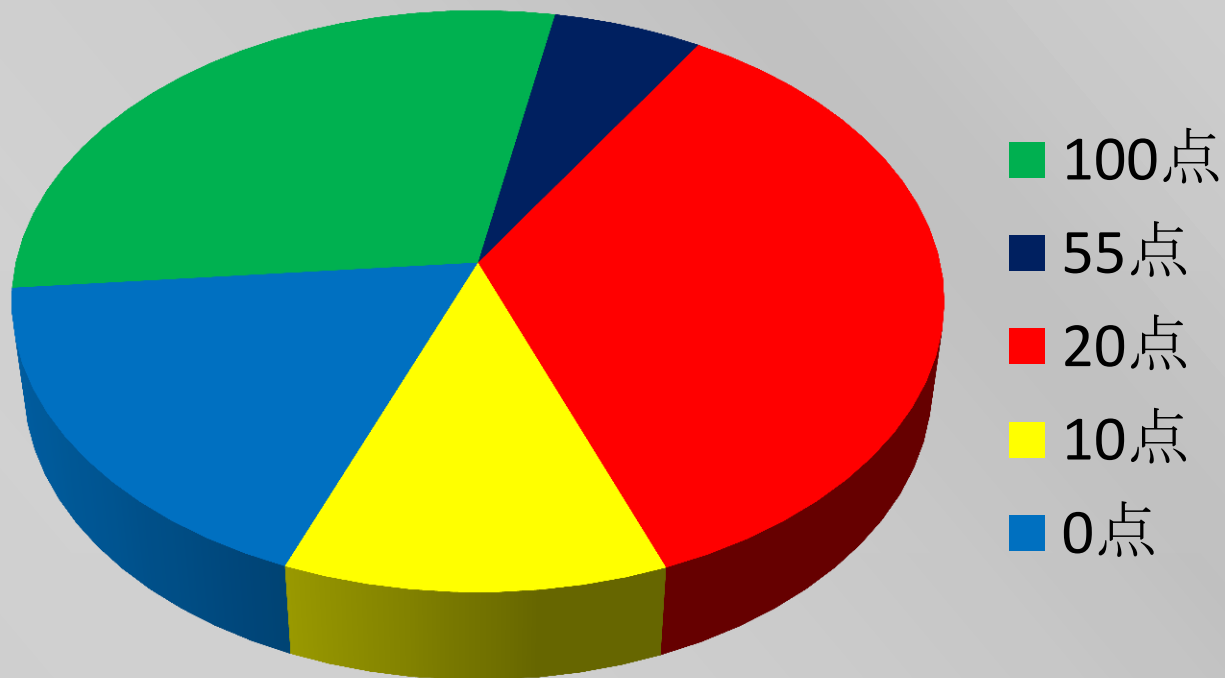


得点情報



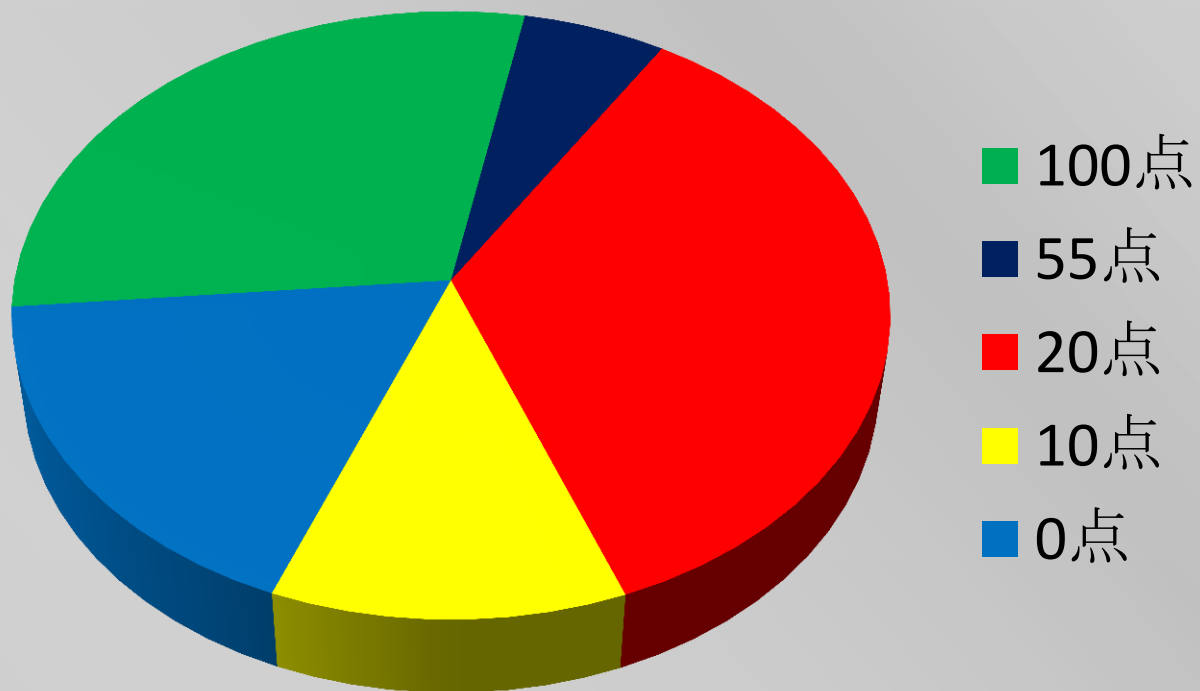


得点情報



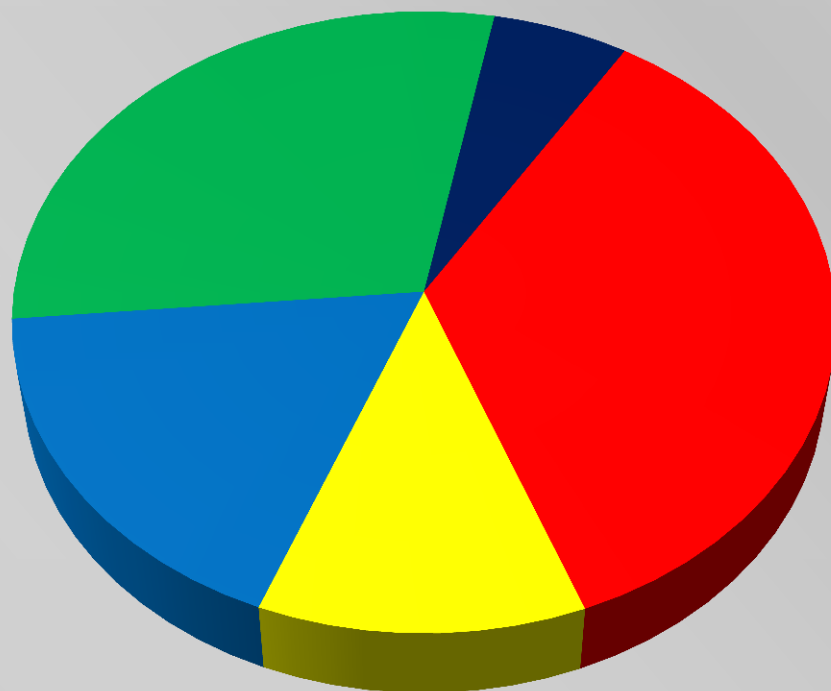


得点情報





得点情報



■ 100点

■ 55点

■ 20点

■ 10点

■ 0点

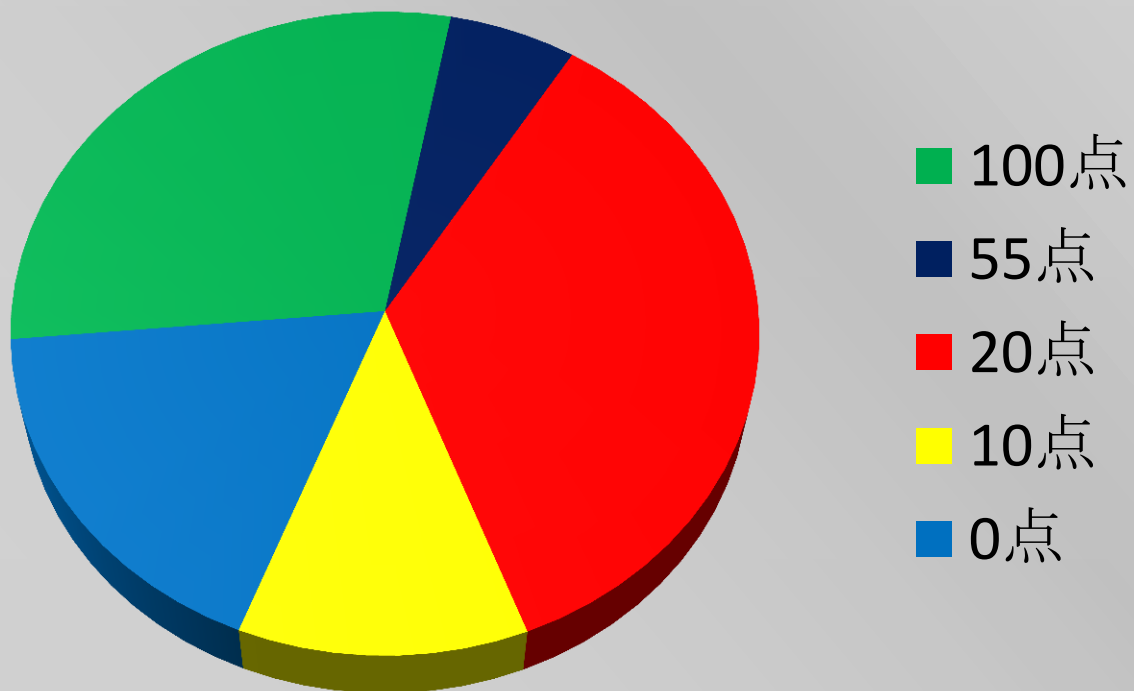


得点情報





得点情報





得点情報





得点情報



■ 100点

■ 55点

■ 20点

■ 10点

■ 0点



得点情報



■ 100点

■ 55点

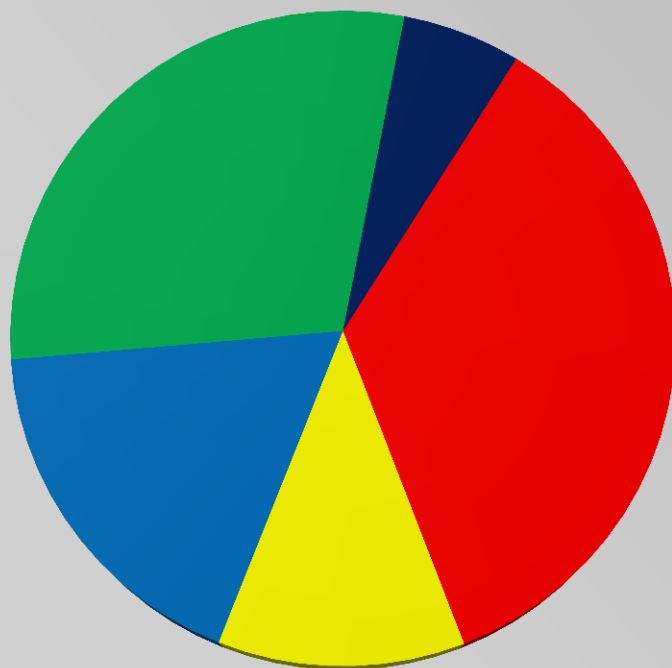
■ 20点

■ 10点

■ 0点



得点情報



■ 100点

■ 55点

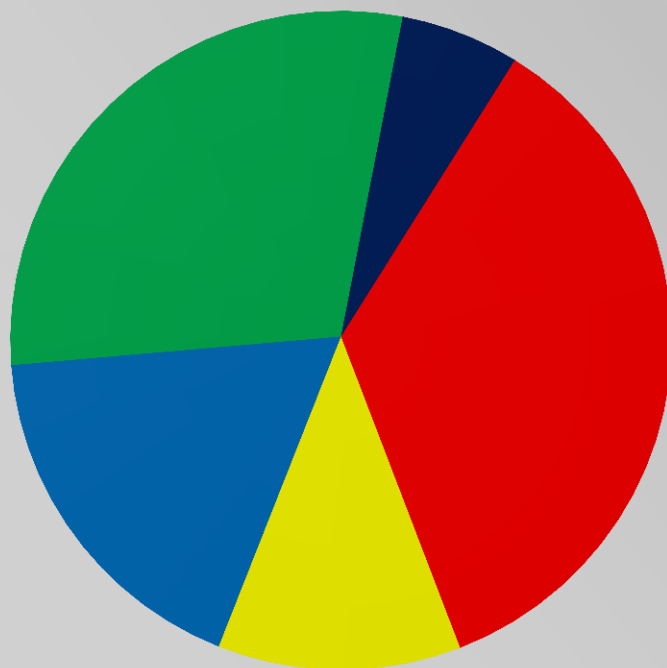
■ 20点

■ 10点

■ 0点

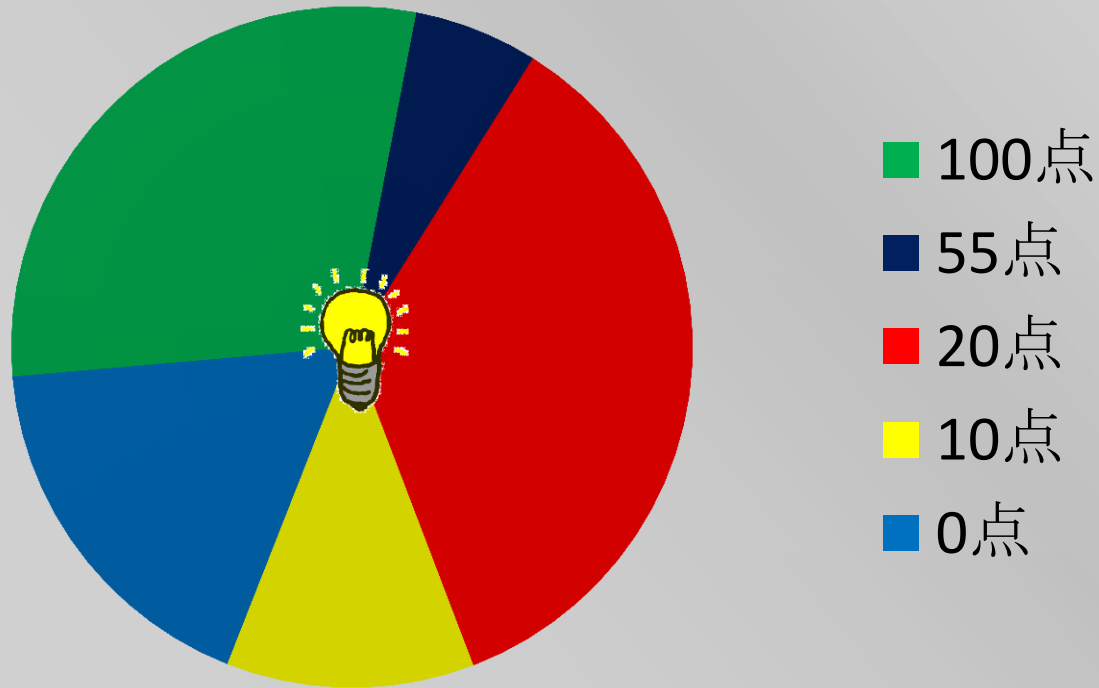


得点情報



- 100点
- 55点
- 20点
- 10点
- 0点

得点情報



100点:5人 55点:1人 20点:6人 10点:2人 0点:3人

強い(確信)

JOI 2014 春合宿

Constellation 2 解説

城下 慎也

(@phidnight)

はじめに

- JOI 春合宿は様々な過去問が存在します。

はじめに

- JOI 春合宿は様々な過去問が存在します。
- 過去問から学べることはたくさんあります。予習、復習等に是非とも役立ててください。

はじめに

- JOI 春合宿は様々な過去問が存在します。
- 過去問から学べることはたくさんあります。予習、復習等に是非とも役立ててください。
- ところで、なぜ突然このような話をしたのかというと、実は JOI には競技の問題についていくつかの JOI 都市伝説※があり、様々な憶測が飛び交っています。

※JOI 都市伝説とは、○○系の問題は難しい、○○を読むと代表になれるとかいったもので、特に科学的根拠とかはないもののことです。

参考：過去問からの統計

問題設定が星系（宇宙系）問題の一覧

- Starry Sky (2009 Day 4)
- UFO (2011 Day 3)
- Constellation (2012 Day 2)
- JOI Poster (2013 Day 1)
- Spaceships (2013 Day 4)
- Constellation 2 (2014 Day 4) ← new!!

問題概要

- 展望台から N 個の星が見える。

問題概要

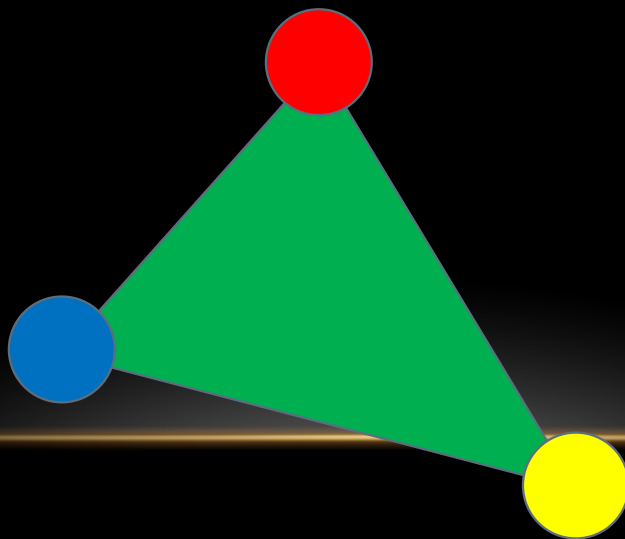
- 展望台から N 個の星が見える。
- JOI ちゃんと IOI ちゃんが JOIOI 座という星座を作ろうとしている。

問題概要

- 展望台から N 個の星が見える。
- JOI ちゃんと IOI ちゃんが JOIOI 座という星座を作ろうとしている。
- JOIOI 座は 2 つの良い三角形の組で構成される。

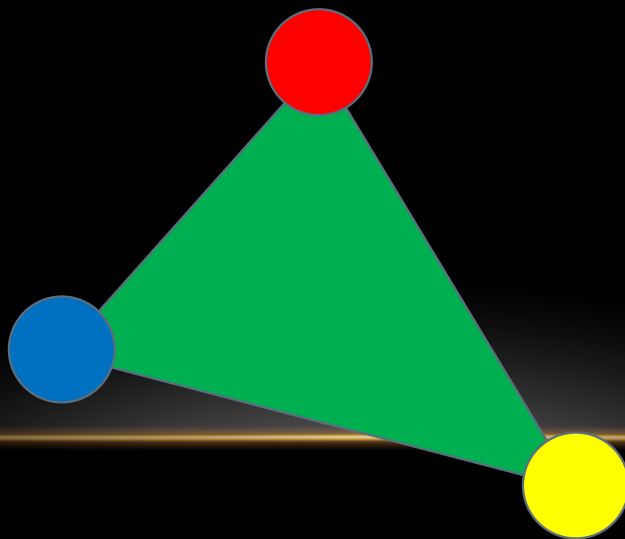
問題概要

- 展望台から N 個の星が見える。
- JOI ちゃんと IOI ちゃんが JOIOI 座という星座を作ろうとしている。
- JOIOI 座は 2 つの良い三角形の組で構成される。
- 良い三角形とは、下図のように 3 頂点が赤、青、黄の三角形である。



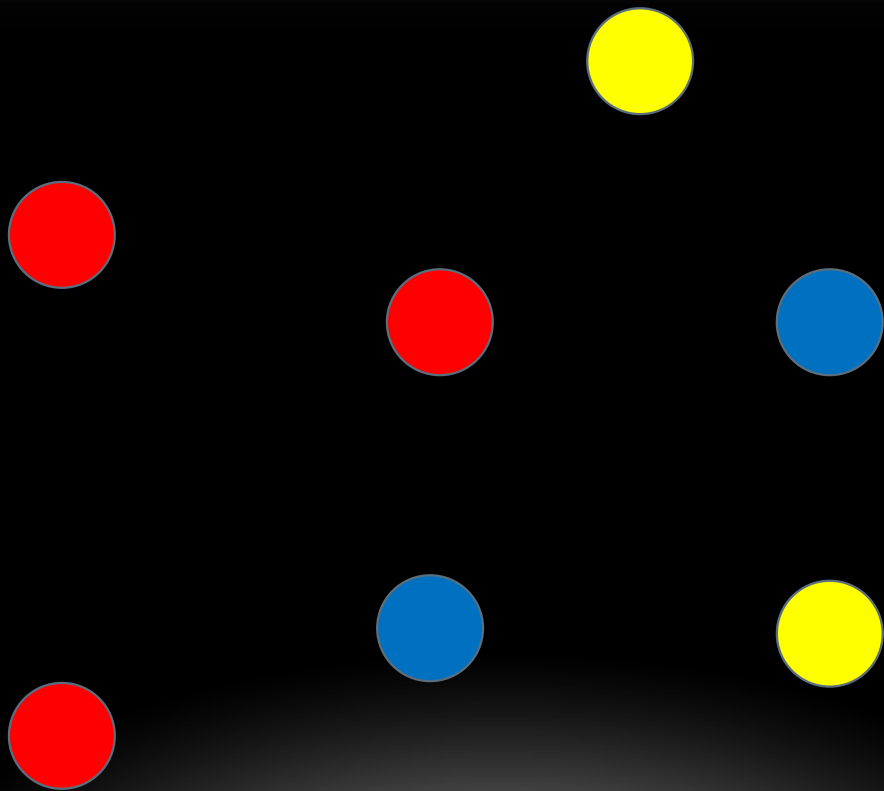
問題概要

- 展望台から N 個の星が見える。
- JOI ちゃんと IOI ちゃんが JOIOI 座という星座を作ろうとしている。
- JOIOI 座は 2 つの良い三角形の組で構成される。
- 良い三角形とは、下図のように 3 頂点が赤、青、黄の三角形である。
- JOIOI 座は全部で何個形成できるか？



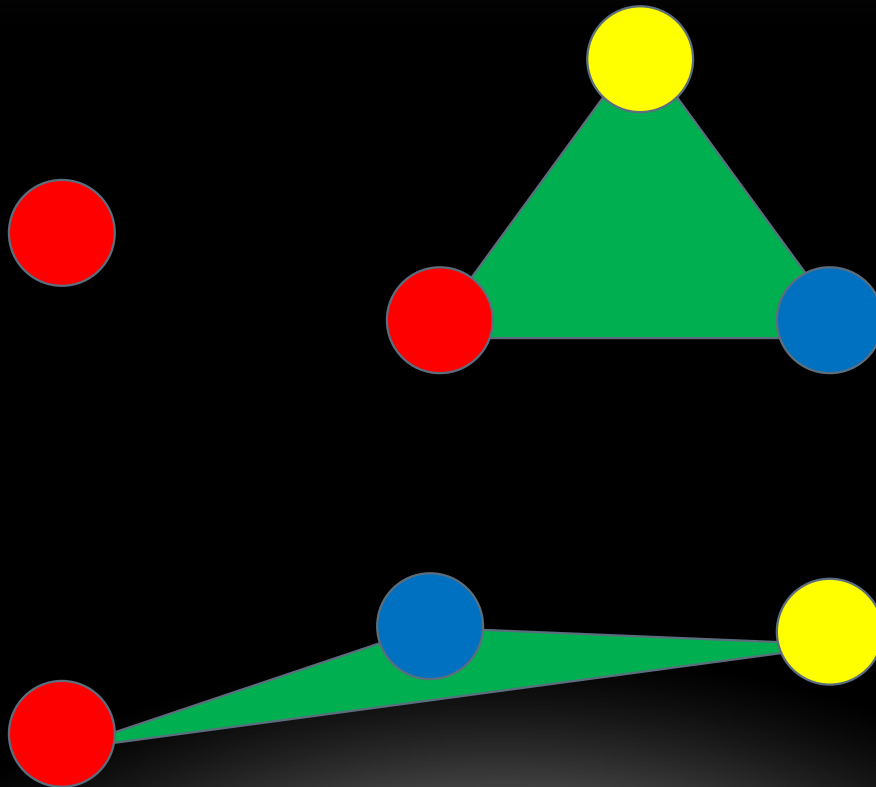
サンプル 1

- 以下の配置になっている。



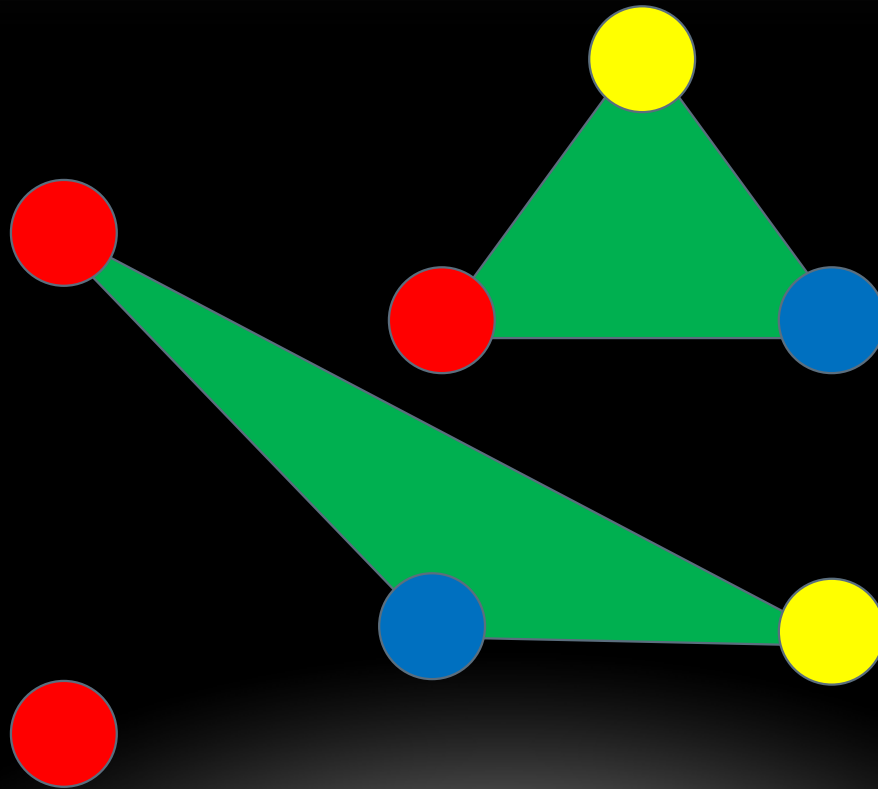
サンプル 1

- 以下の配置になっている。
- 累計 1 通り



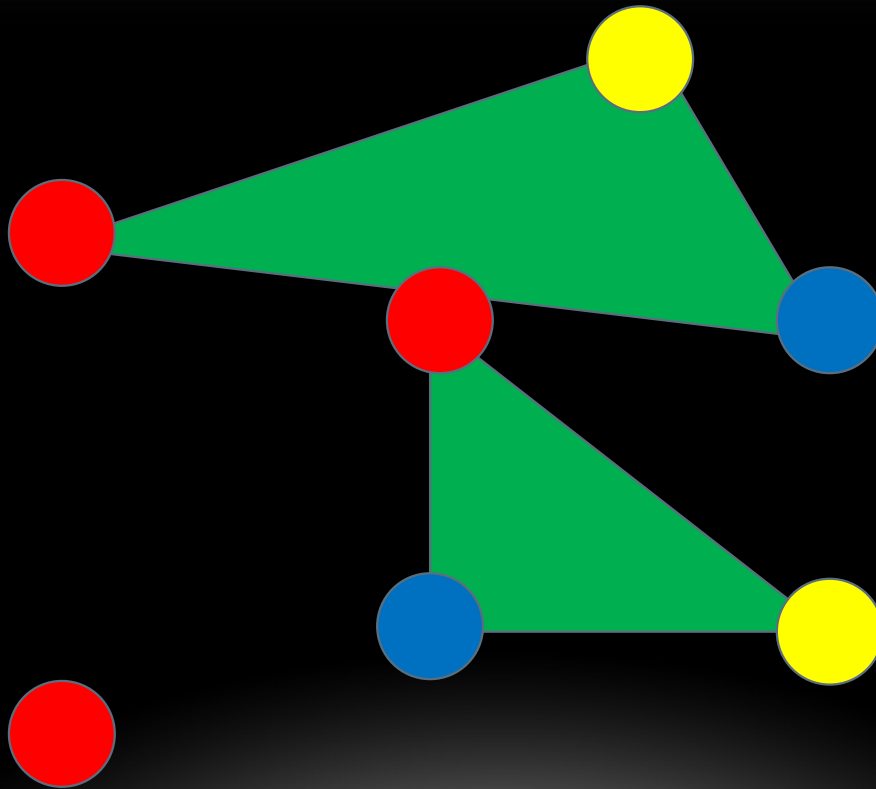
サンプル 1

- 以下の配置になっている。
- 累計 2 通り



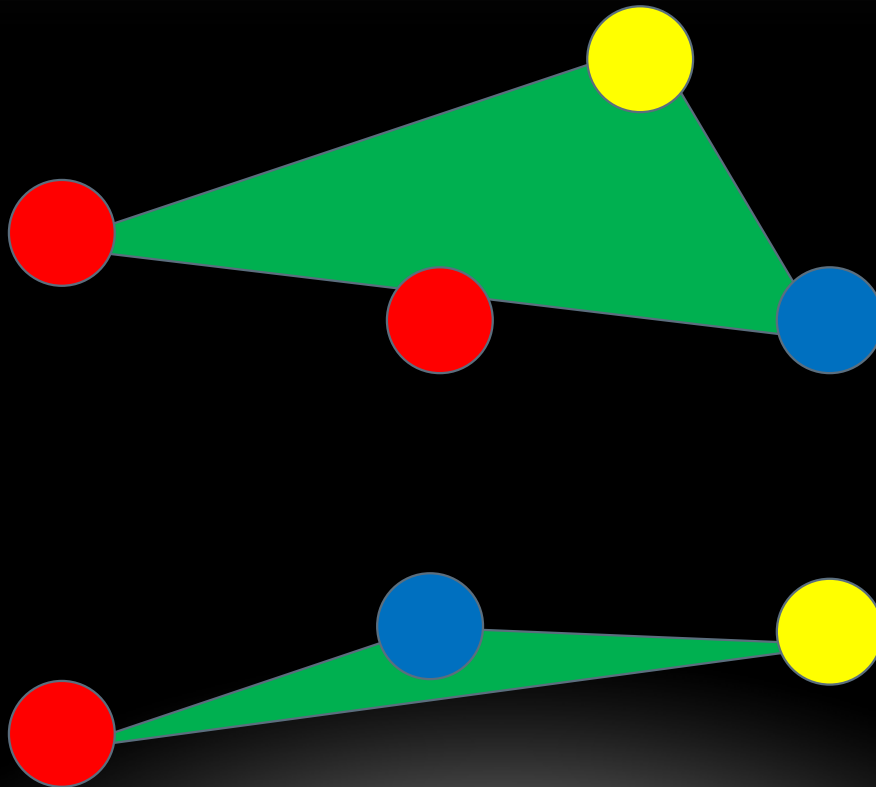
サンプル 1

- 以下の配置になっている。
- 累計 3 通り



サンプル 1

- 以下の配置になっている。
- 累計 4 通り
- 以上 4 通り



幾何について

- この問題は幾何の問題です。

幾何について

- この問題は幾何の問題です。
- 幾何の問題では、点や線分の処理方法、幾何特有のアルゴリズムなどを用いることが多いです。

幾何について

- この問題は幾何の問題です。
- 幾何の問題では、点や線分の処理方法、幾何特有のアルゴリズムなどを用いる場合が多いです。
- 浮動小数点計算とかは誤差に注意して計算しましょう。

幾何について

- この問題は幾何の問題です。
- 幾何の問題では、点や線分の処理方法、幾何特有のアルゴリズムなどを用いる場合が多いです。
- 浮動小数点計算とかは誤差に注意して計算しましょう。
- 幾何の問題は、解ける（計算量的に）でも実装方針や使用する要素などの違いで実行時間や計算結果（誤差）に影響が出ます。

幾何について

- この問題は幾何の問題です。
- 幾何の問題では、点や線分の処理方法、幾何特有のアルゴリズムなどを用いる場合が多いです。
- 浮動小数点計算とかは誤差に注意して計算しましょう。
- 幾何の問題は、解ける（計算量的に）でも実装方針や使用する要素などの違いで実行時間や計算結果（誤差）に影響が出ます。
- 浮動小数点計算は遅いこともあるので、誤差を回避するためにも整数で計算できることは整数で計算するほうが良い場合もあります。

幾何について

- この問題は幾何の問題です。
- 幾何の問題では、点や線分の処理方法、幾何特有のアルゴリズムなどを用いる場合が多いです。
- 浮動小数点計算とかは誤差に注意して計算しましょう。
- 幾何の問題は、解ける（計算量的に）でも実装方針や使用する要素などの違いで実行時間や計算結果（誤差）に影響が出ます。
- 浮動小数点計算は遅いこともあるので、誤差を回避するためにも整数で計算できることは整数で計算するほうが良い場合もあります。
- JOI では使用はできませんが、ライブラリがあると便利ことがあります。（アルゴリズムを覚えていて、必要なときに頭から引き出せると理想です）

幾何について

- この問題は幾何の問題です。
- 幾何の問題では、点や線分の処理方法、幾何特有のアルゴリズムなどを用いる場合が多いです。
- 浮動小数点計算とかは誤差に注意して計算しましょう。
- 幾何の問題は、解ける（計算量的に）でも実装方針や使用する要素などの違いで実行時間や計算結果（誤差）に影響が出ます。
- 浮動小数点計算は遅いこともあるので、誤差を回避するためにも整数で計算できることは整数で計算するほうが良い場合もあります。
- JOI では使用はできませんが、ライブラリがあると便利ことがあります。（アルゴリズムを覚えていて、必要なときに頭から引き出せると理想です）

小課題 1 (15 点)

- $N \leq 30$ なので、全通り試すことが可能
- 三角形の個数が $O(N^3)$ あるので、ペアの総数は $O(N^6)$

小課題 1 (15 点)

- $N \leq 30$ なので、全通り試すことが可能
- 三角形の個数が $O(N^3)$ あるので、ペアの総数は $O(N^6)$
- 幾何学的な計算があります。

小課題 1 (15 点)

- $N \leq 30$ なので、全通り試すことが可能
- 三角形の個数が $O(N^3)$ あるので、ペアの総数は $O(N^6)$
- 幾何学的な計算があります。
- ここでは、線分の交差判定や、三角形に点が含まれるか否かを判定する必要があります。

小課題 1 (15 点)

- $N \leq 30$ なので、全通り試すことが可能
- 三角形の個数が $O(N^3)$ あるので、ペアの総数は $O(N^6)$
- 幾何学的な計算があります。
- ここでは、線分の交差判定や、三角形に点が含まれるか否かを判定する必要があります。
- 外積や内積を用いた計算がわりと簡単です。

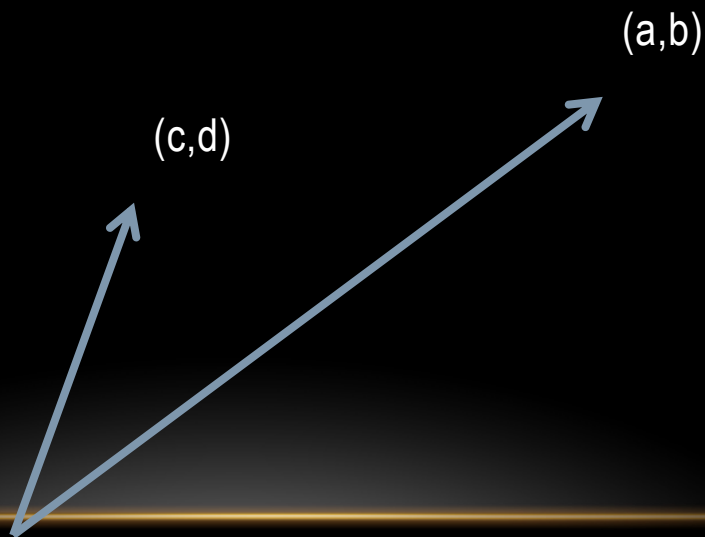
外積について

- 外積の計算は、以下のようにベクトル (a,b) と (c,d) があった際に

- $a \times d - b \times c$

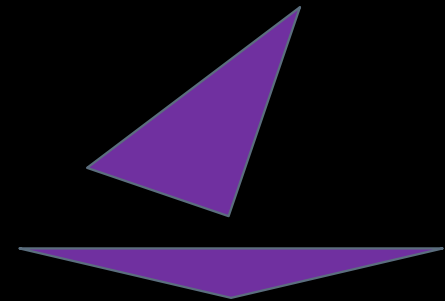
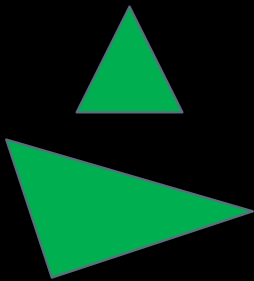
で計算できる値のこと。

- 整数で平行四辺形の面積や、点が直線のどちら側にあるかが判定できてとても便利です。



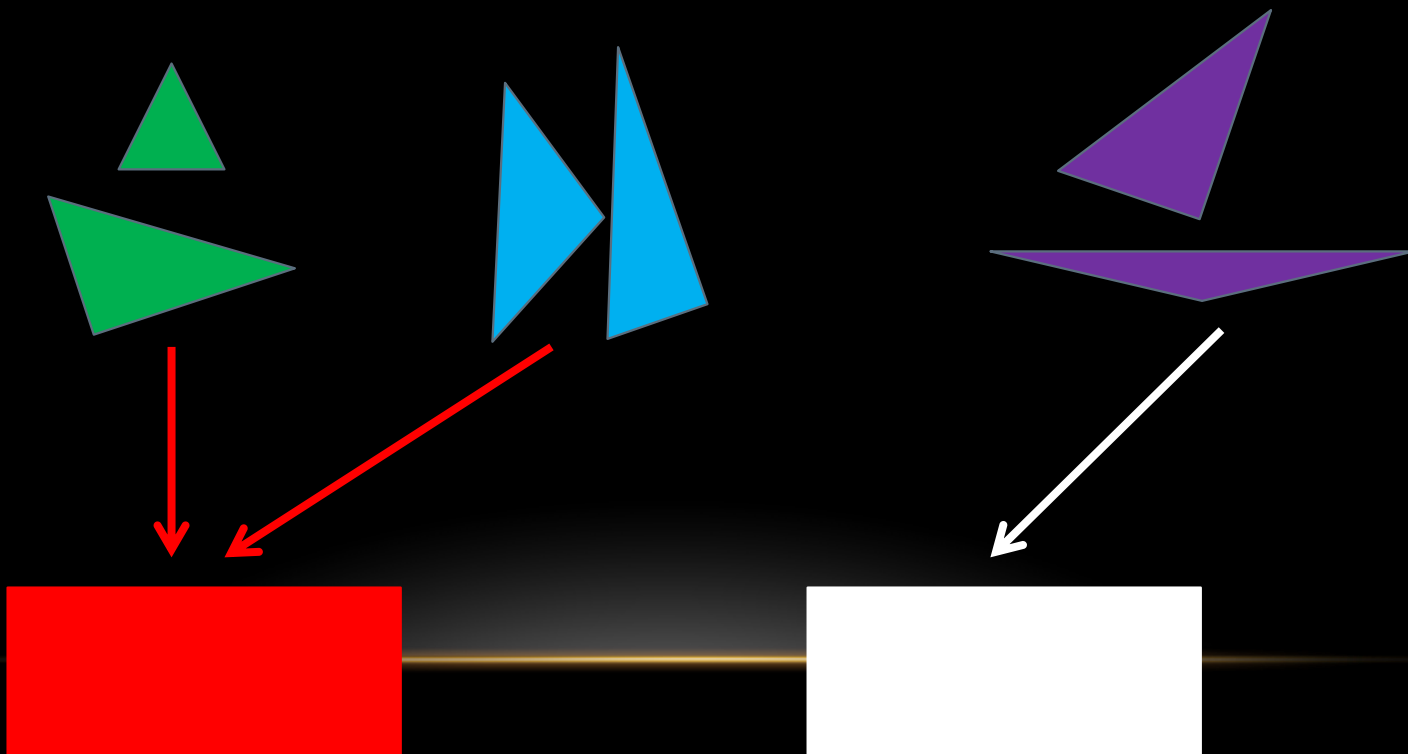
小課題 2 以降へのアプローチ

- N がでかくなると、全探索で数え上げることができなくなります。



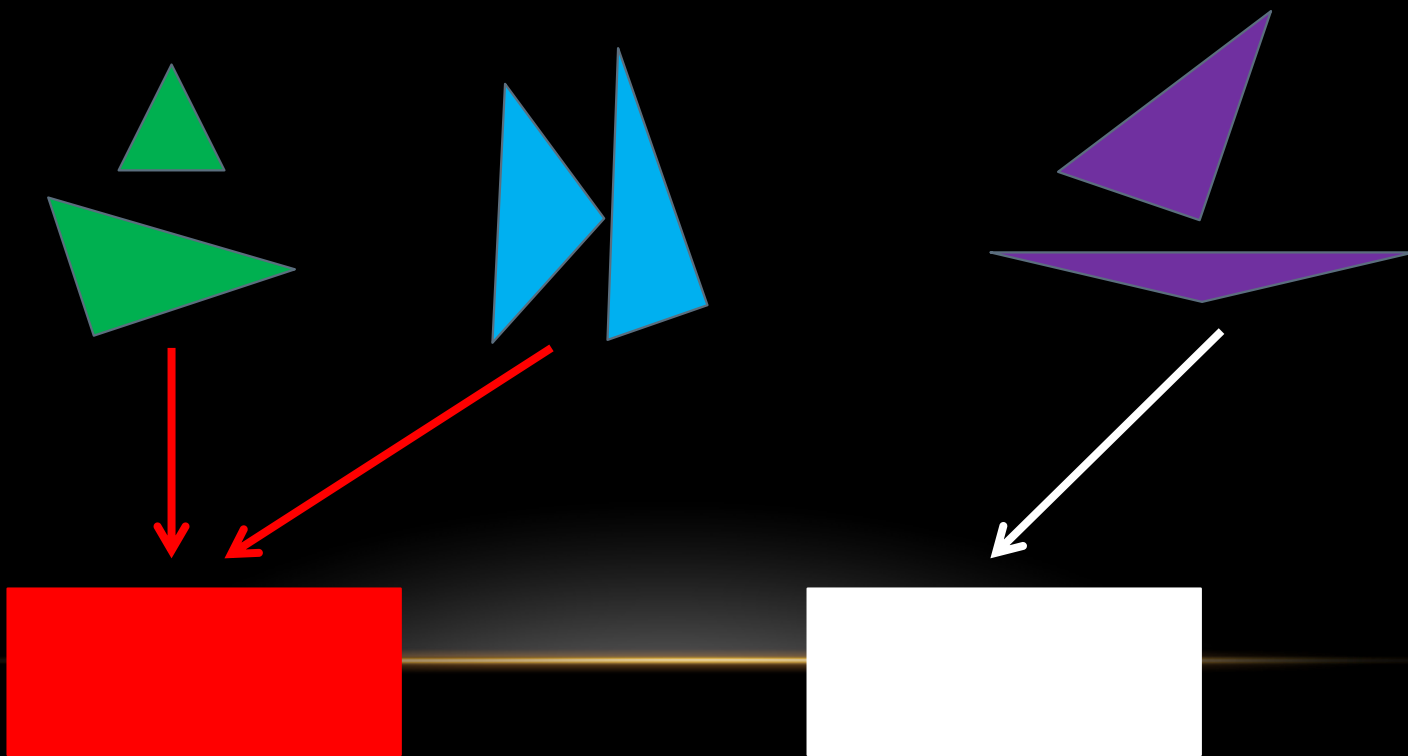
小課題 2 以降へのアプローチ

- N がでかくなると、全探索で数え上げることができなくなります。
- 複数の候補を一度に計算できる方法があれば...



小課題 2 以降へのアプローチ

- N がでかくなると、全探索で数え上げることができなくなります。
- 複数の候補を一度に計算できる方法があれば...
- 2つの三角形の性質について考えてみよう！



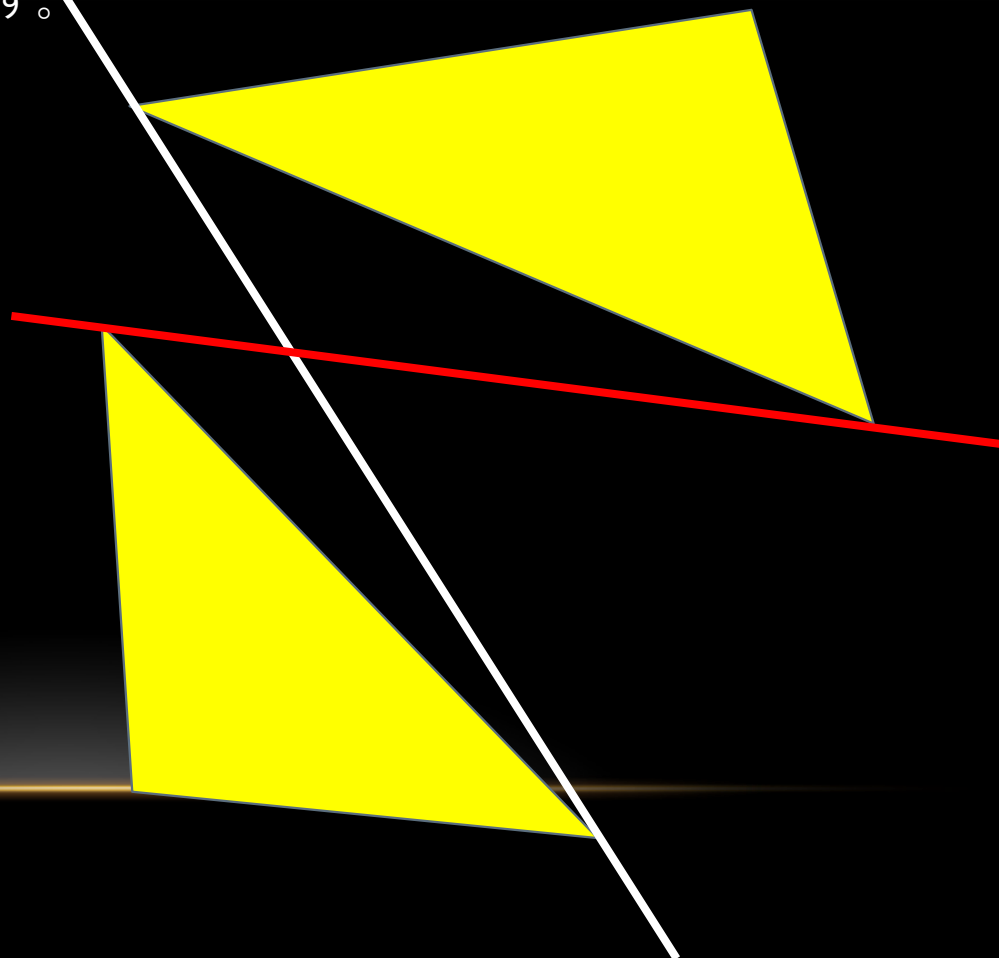
三角形の性質

- 三角形が2つあるとき、



三角形の性質

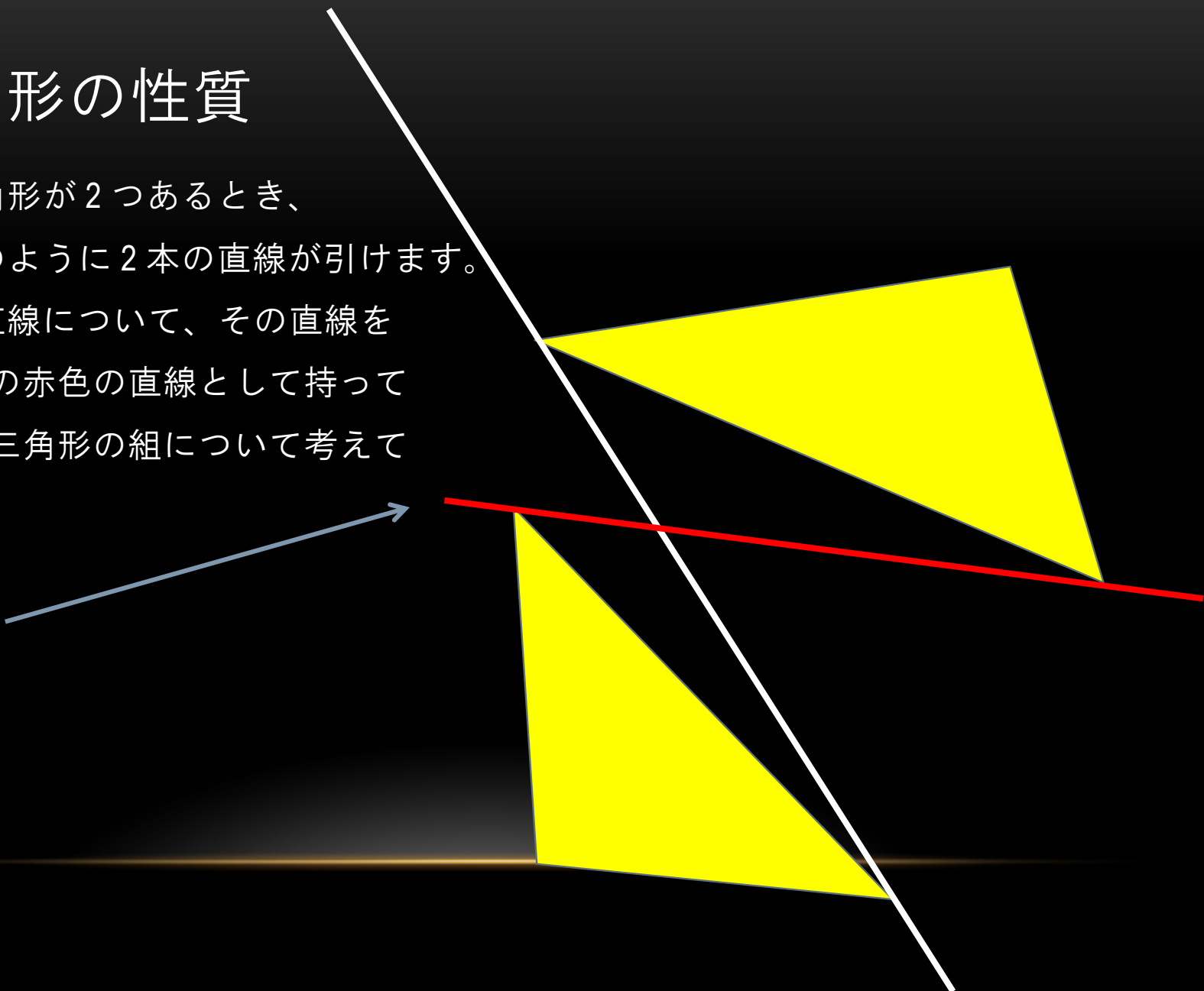
- 三角形が2つあるとき、
- 次のように2本の直線が引けます。



三角形の性質

- 三角形が2つあるとき、
- 次のように2本の直線が引けます。
- 各直線について、その直線を右図の赤色の直線として持っている三角形の組について考えてみる。

こっち



直線について

- 直線を固定してみます。



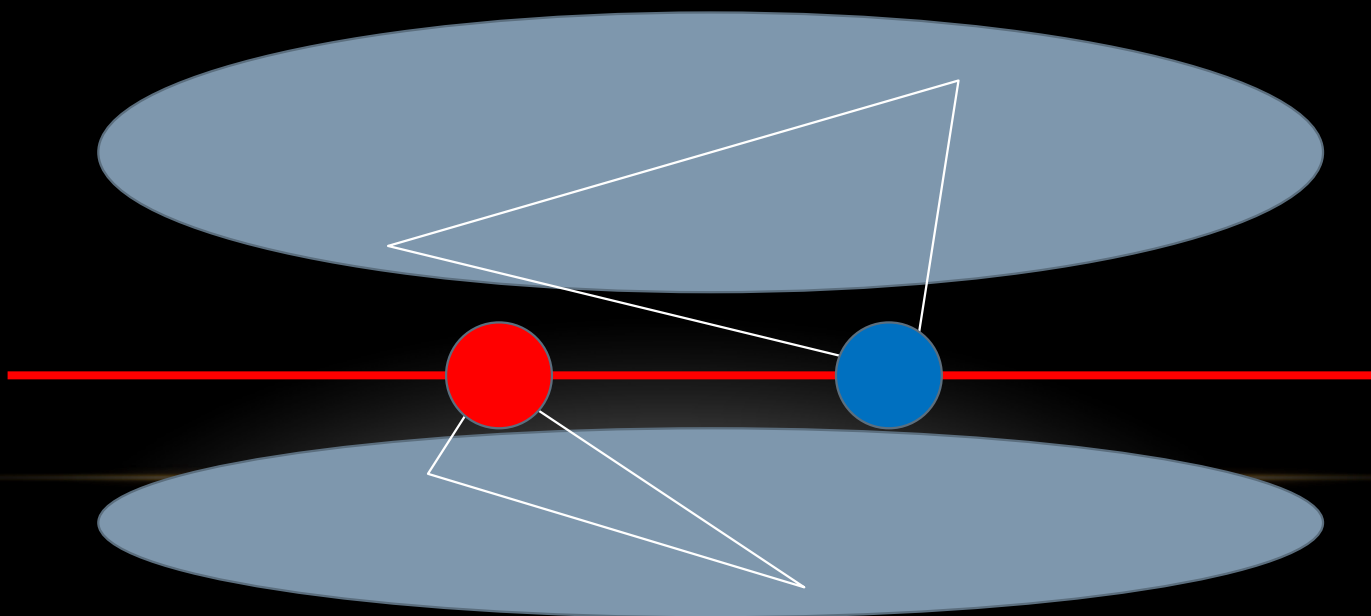
直線について

- 直線を固定してみます。
- 問題文の条件やその性質より、丁度2つの頂点が乗っています。



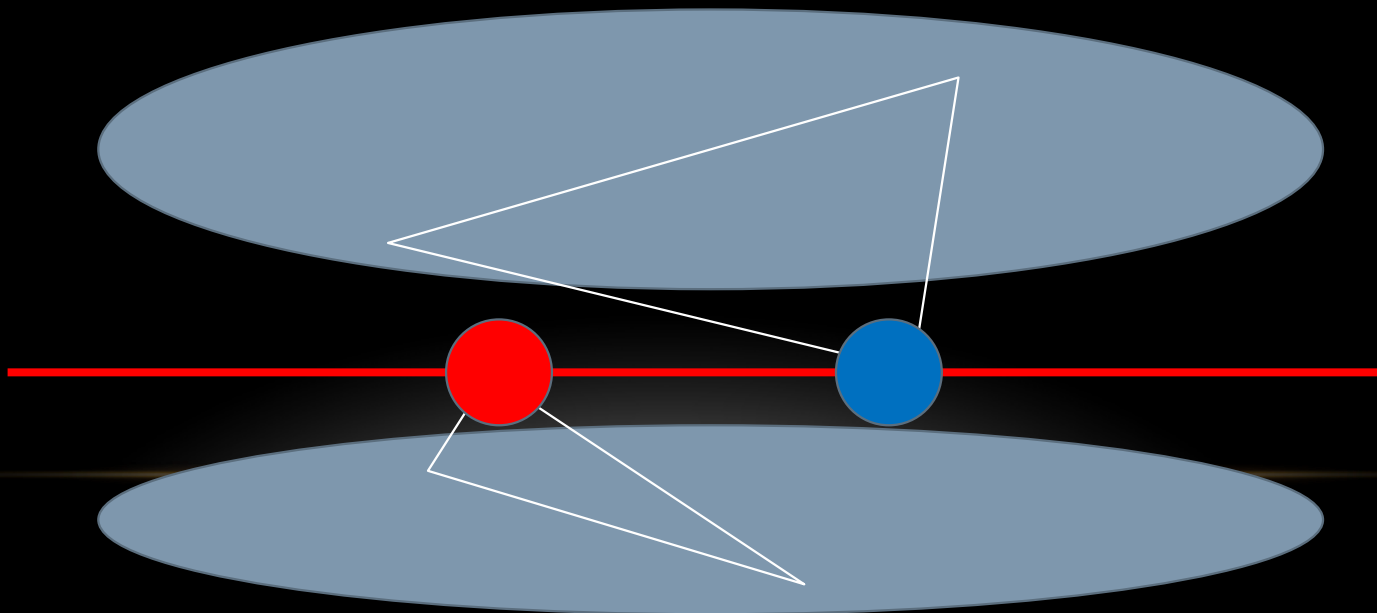
直線について

- 直線を固定してみます。
- 問題文の条件やその性質より、丁度2つの頂点が乗っています。
- 直線で分かれた2つの領域について、それぞれから2つ選んで三角形を作成すれば、その直線を先ほどの条件を満たす直線としている三角形が復元できる。



直線について

- 直線を固定してみます。
- 問題文の条件やその性質より、丁度2つの頂点が乗っています。
- 直線で分かれた2つの領域について、それぞれから2つ選んで三角形を作成すれば、その直線を先ほどの条件を満たす直線としている三角形が復元できる。
- こうすることで、三角形同士の交差を気にしないで済むようになります。



小課題 2 (累計 55 点)

- 先ほどの組み合わせについては、1つの三角形について、直線上の頂点の色以外の2つの色に関して、頂点を1つずつどのように取り出しても必ず三角形を作ることができる。

(例えば、赤色の頂点が直線上にあるなら、(青色)×(黄色)が答えになる)

小課題 2 (累計 55 点)

- 先ほどの組み合わせについては、1つの三角形について、直線上の頂点の色以外の2つの色に関して、頂点を1つずつどのように取り出しても必ず三角形を作ることができる。

(例えば、赤色の頂点が直線上にあるなら、(青色)×(黄色)が答えになる)

- 各直線について、直線上にはない頂点を1個ずつ、どちらに属するか判定し、個数をまとめておいて、さっきの計算を用いれば、 $O(N)$ で組の総数を計算することができる。

小課題 2 (累計 55 点)

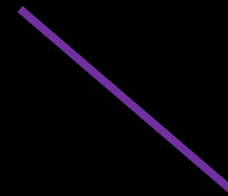
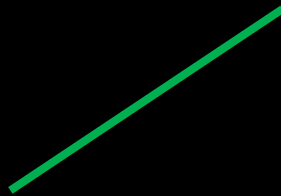
- 先ほどの組み合わせについては、1つの三角形について、直線上の頂点の色以外の2つの色に関して、頂点を1つずつどのように取り出しても必ず三角形を作ることができる。

(例えば、赤色の頂点が直線上にあるなら、(青色)×(黄色)が答えになる)

- 各直線について、直線上にはない頂点を1個ずつ、どちらに属するか判定し、個数をまとめておいて、さっきの計算を用いれば、 $O(N)$ で組の総数を計算することができる。
- 直線の個数は $O(N^2)$ なので、全体で $O(N^3)$ で計算できる。

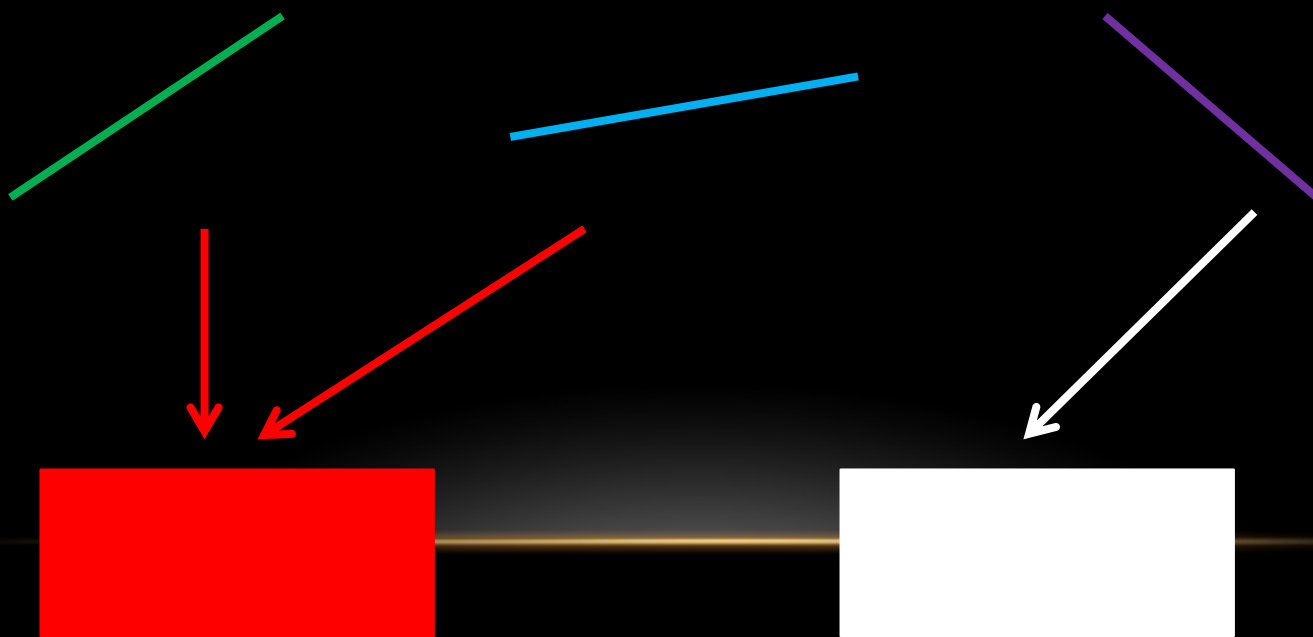
小課題 3 へのアプローチ

- N がでかくなると、 $O(N^3)$ では間に合わなくなります。



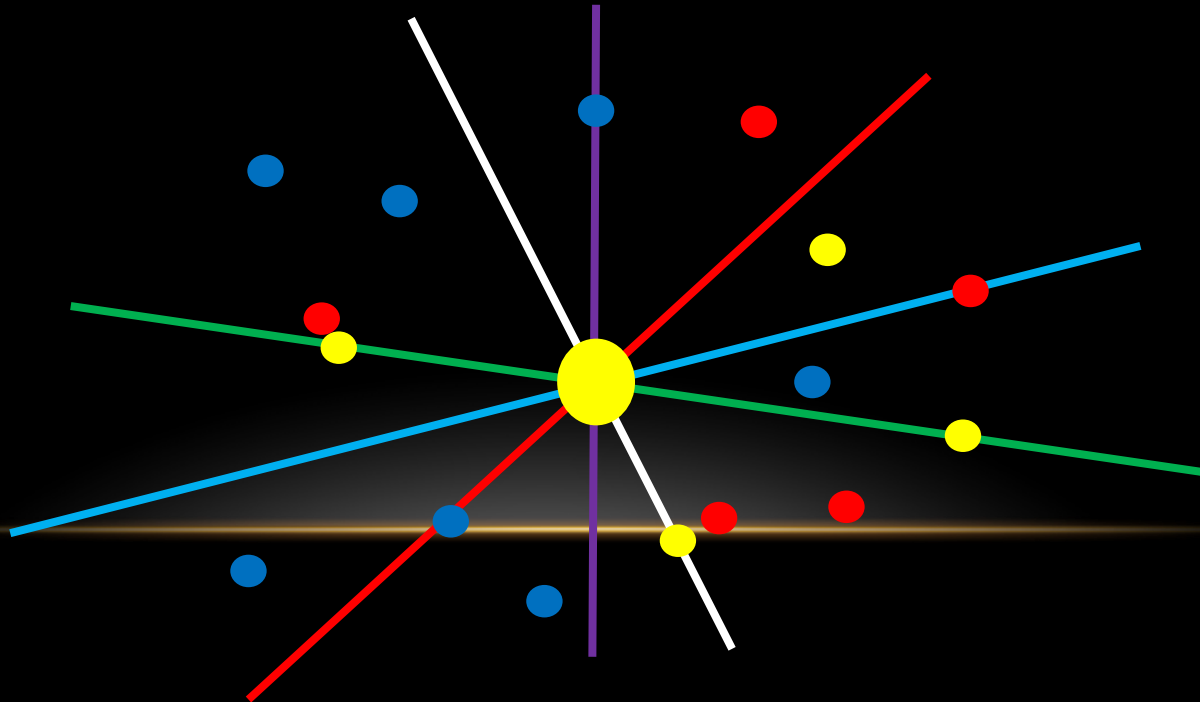
小課題 3 へのアプローチ

- N がでかくなると、 $O(N^3)$ では間に合わなくなります。
- 先ほどの計算について、直線の個数が $O(N^2)$ のため、頂点の分類判定をどうにか高速化したい。
- いくつかの直線を分類してまとめて数え上げできないだろうか？



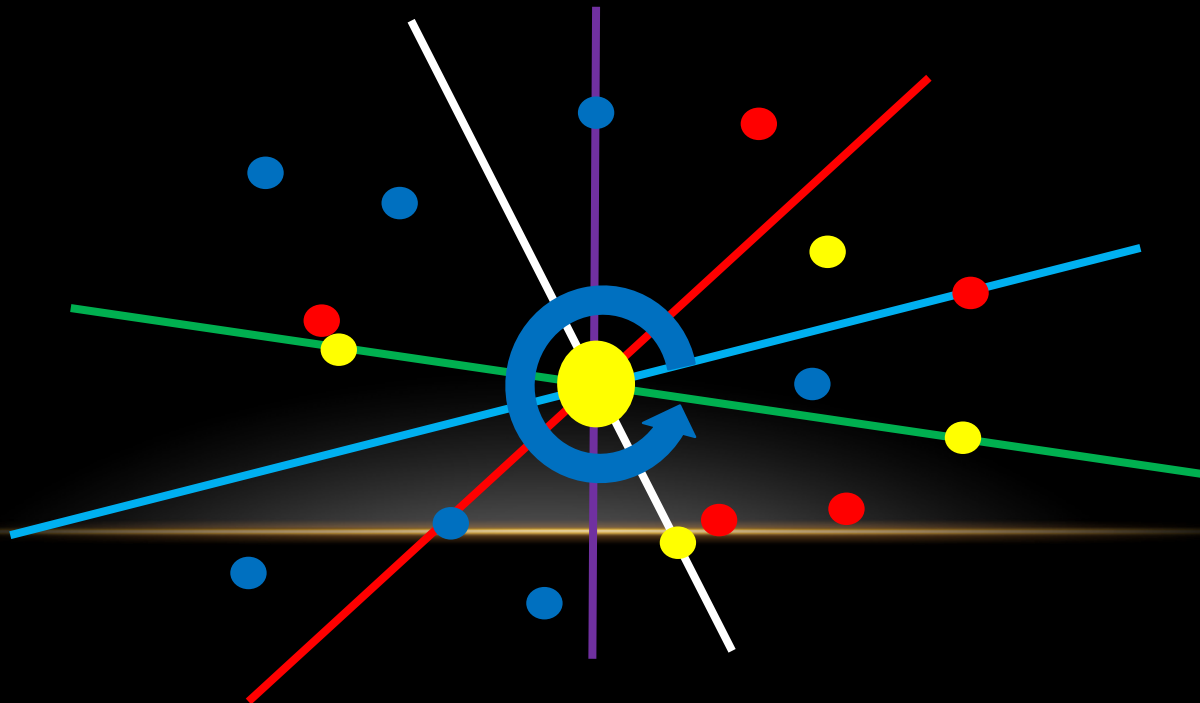
満点解法

- ある頂点 x を通る直線についてまとめてみる。



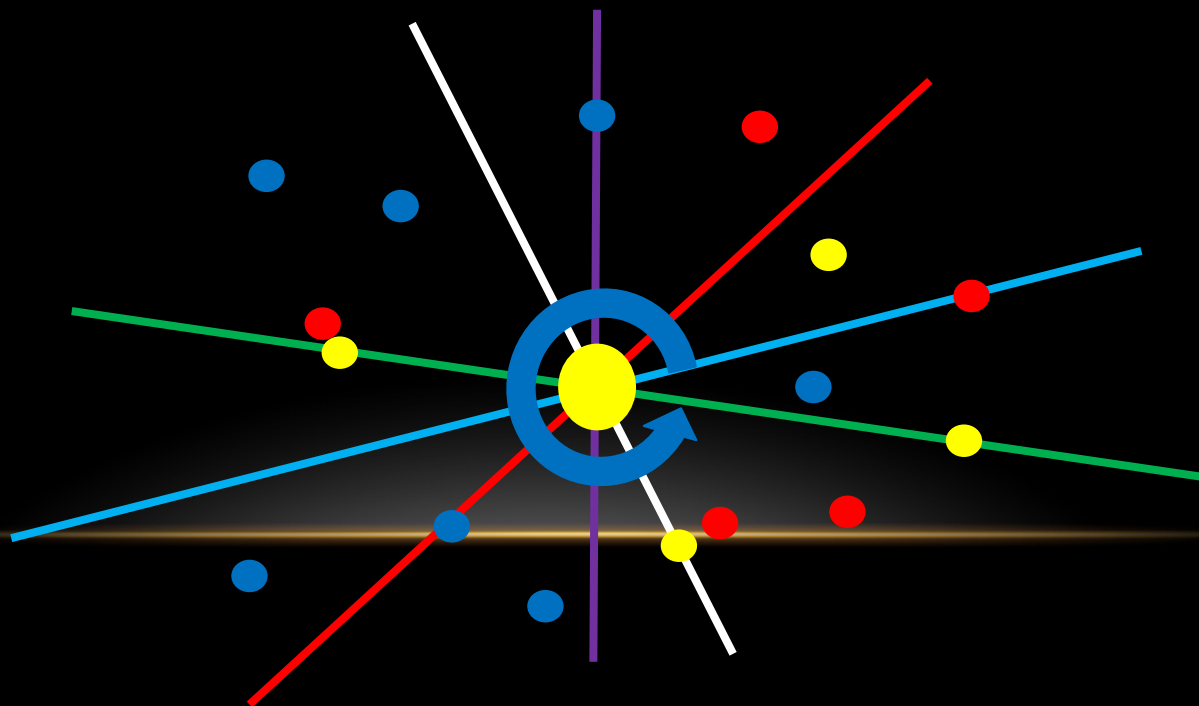
満点解法

- ある頂点 x を通る直線についてまとめてみる。
- 下図矢印のようにグルリと軸を回転させれば、外部の頂点が高々定数回出入りするだけで、まとめた直線すべてについて計算することができる。



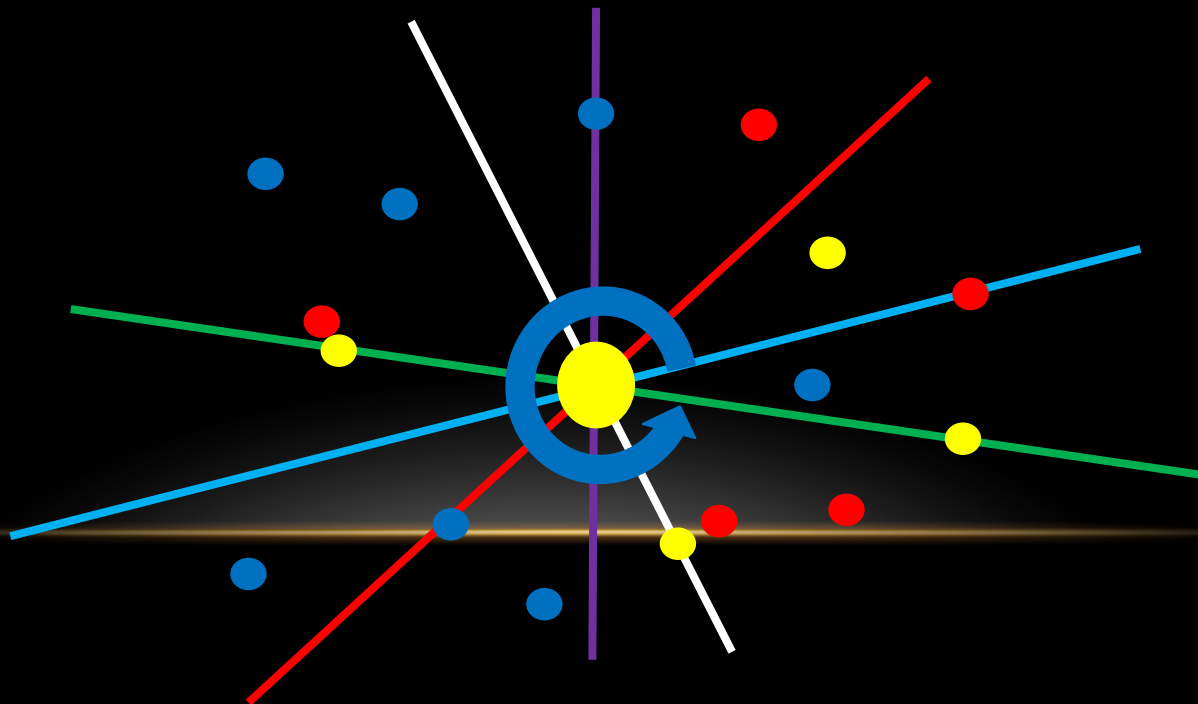
満点解法

- ある頂点 x を通る直線についてまとめてみる。
- 下図矢印のようにグルリと軸を回転させれば、外部の頂点が高々定数回出入りするだけで、まとめた直線すべてについて計算することができる。
- ソート分を含めて $O(N \log N)$ でまとめた直線すべてについて計算できる。



満点解法

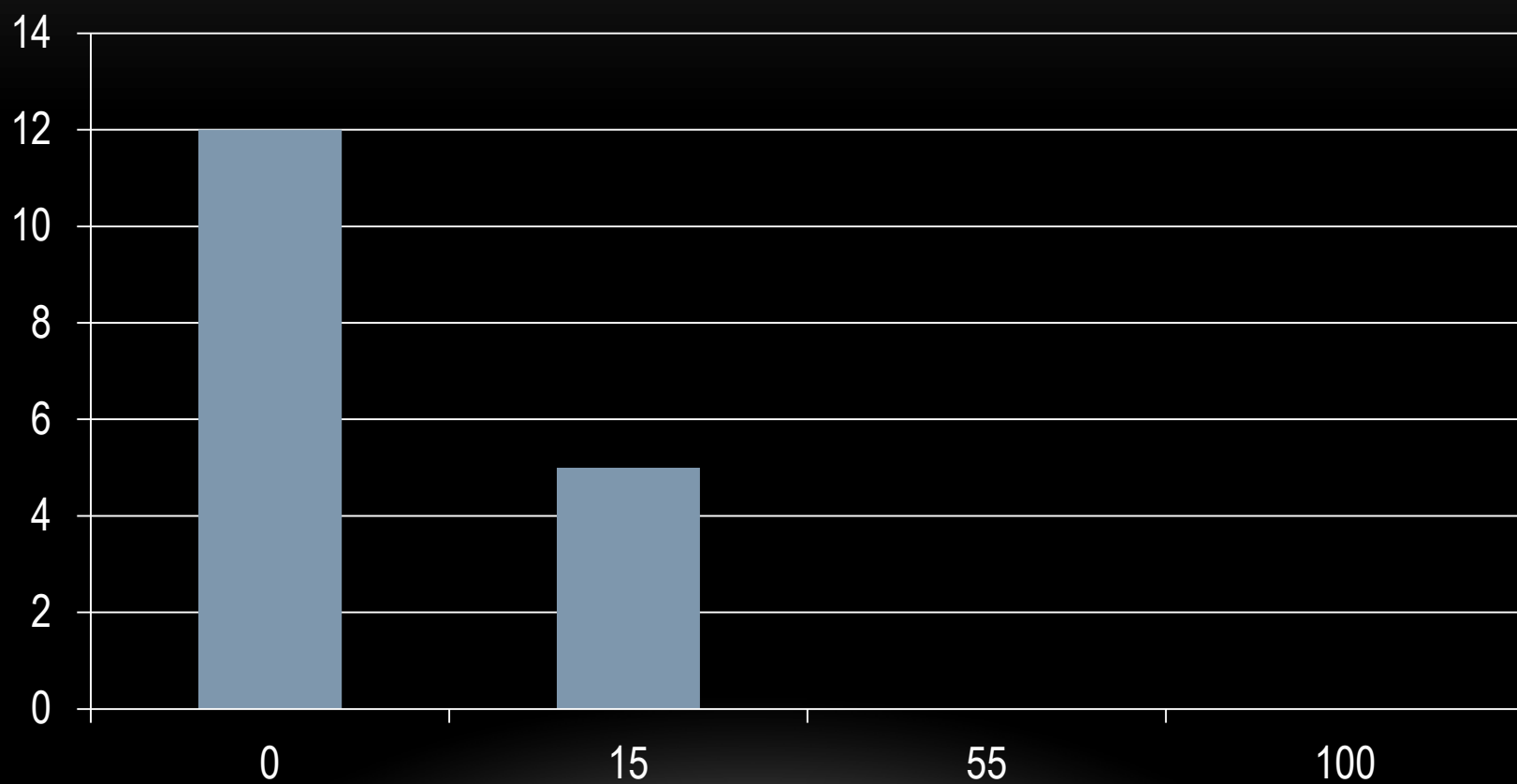
- ある頂点 X を通る直線についてまとめてみる。
- 下図矢印のようにグルリと軸を回転させれば、外部の頂点が高々定数回出入りするだけで、まとめた直線すべてについて計算することができる。
- ソート分を含めて $O(N \log N)$ でまとめた直線すべてについて計算できる。
- X は $O(N)$ 通りあるので、各々計算して全体で $O(N^2 \log N)$ で計算できる。



まとめ

- 数え上げの問題は、数え上げる対象の性質に基づいて分類してまとめることで、効率よく計算できる場合があります。
- 幾何の問題では、平面走査などのテクニックがよく問われます。過去問にもあるので、忘れていた、知らなかった方は復習しておいてください。
- 今回のセットのような、癖の強い問題に関しては、コンテストの時間中にどの問題にどれだけ時間をかけるか、計画を立てておくとう安全です。
- 数え上げの問題を解く際に、解法によっては同じ組み合わせを複数回数える解法となることがあります。この場合、全部が同数ずつ足し合わされているなら、定数で除算することにより解を求めることができます。
- ただし、複数解足す場合は、答えが 64 bit に収まっても、途中経過が 64 bit に収まらない場合があるのでその点に注意してください。
- **過去問は重要。**

得点分布





2013 JOI春合宿 Day4

漢字しりとり (Kanji Shiritori) 解説

2014/03/24

山下 洋史

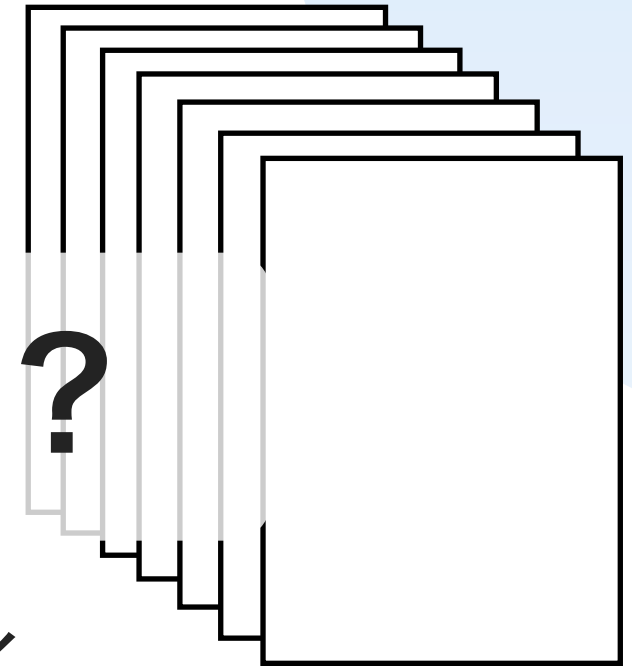
@utatakiyoshi

はじめに

- ・ 問題文が 7 ページもありますが、

ちゃんと読みましたか？

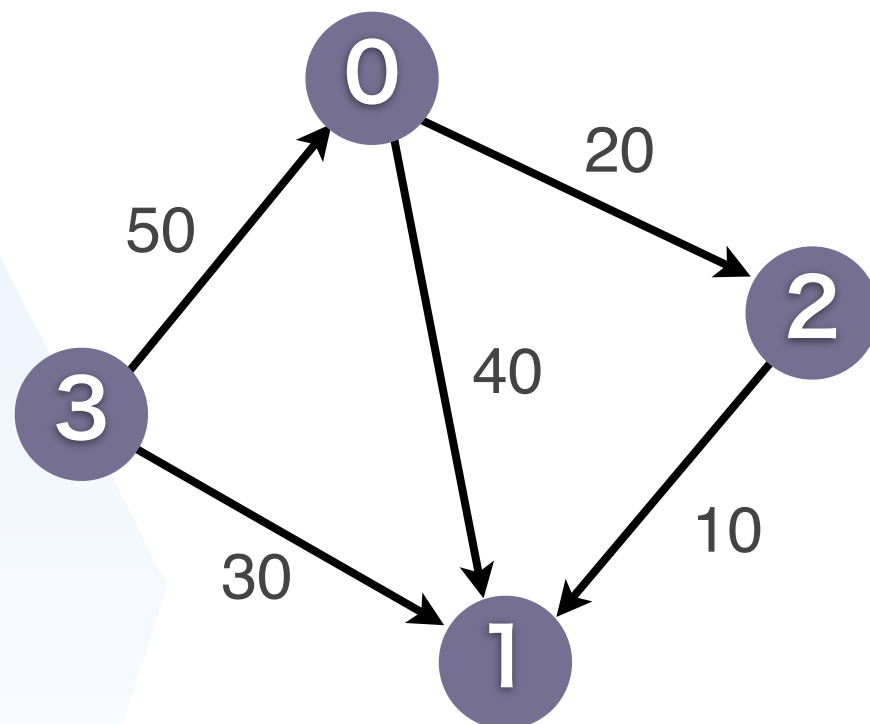
- ・ 最近の IOI は Reactive な出題が多く、
問題文が長くなる傾向にあります
- ・ 時間はたっぷりあるので、焦らずに読んでください



概要

Anna

の視点



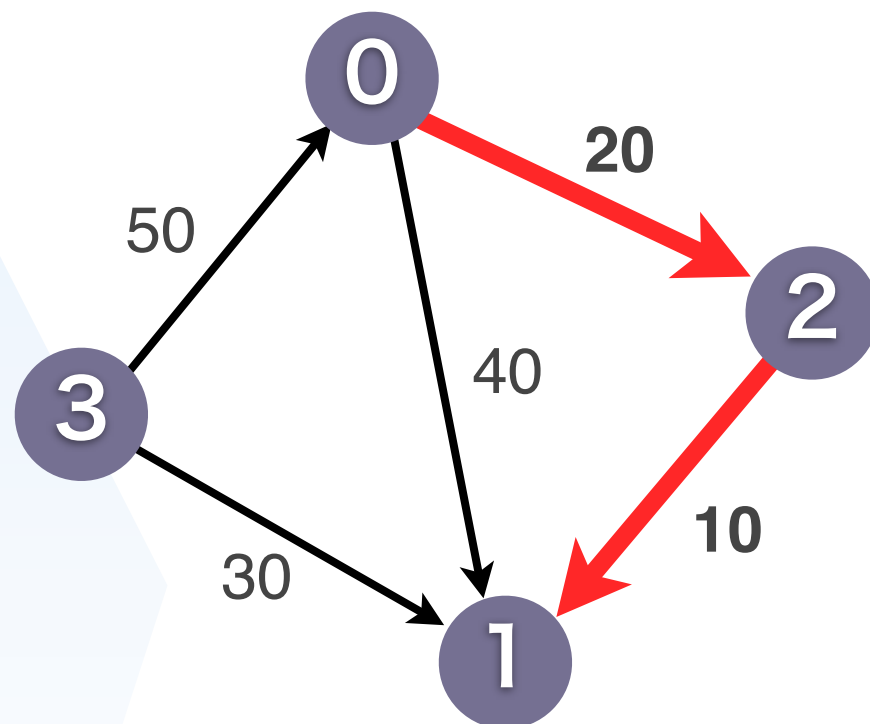
N頂点 M辺 の有向グラフ

0 → 1 の最短経路は？
 というクエリが Q 個飛んでくる

概要

Anna

の視点



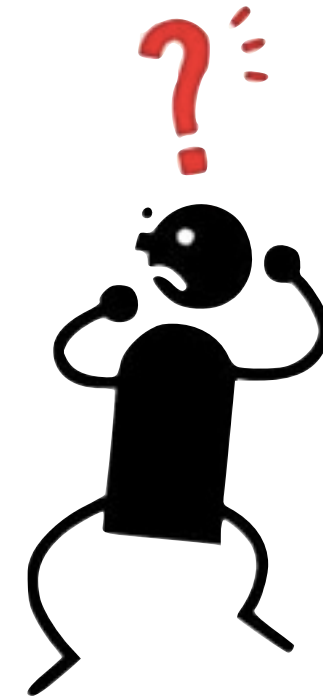
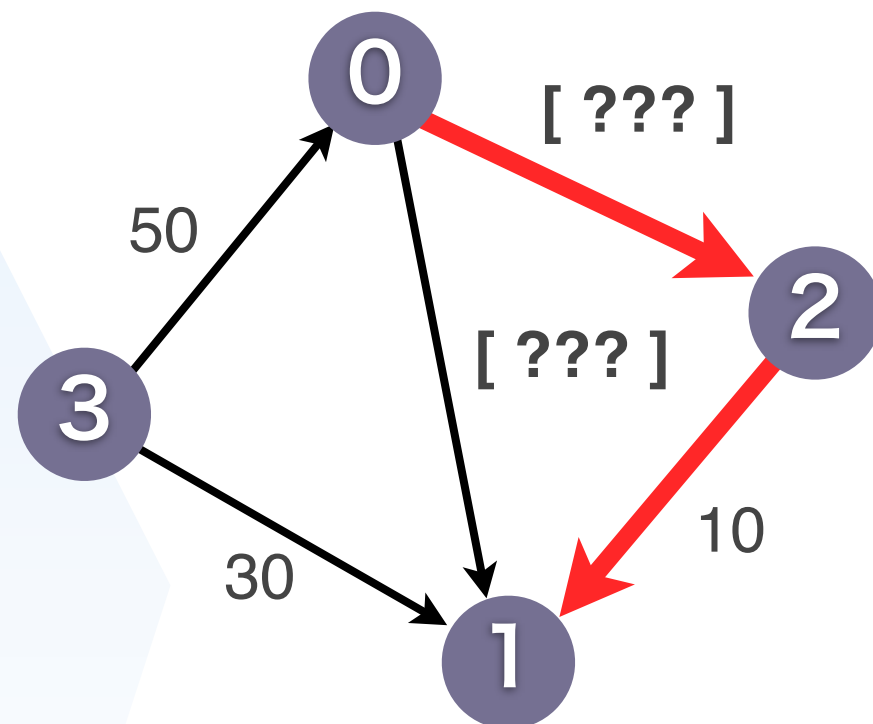
N頂点 M辺 の有向グラフ

0 → 1 の最短経路は？
 というクエリが Q 個飛んでくる

概要

Bruno

の視点



N頂点 M辺 の有向グラフ

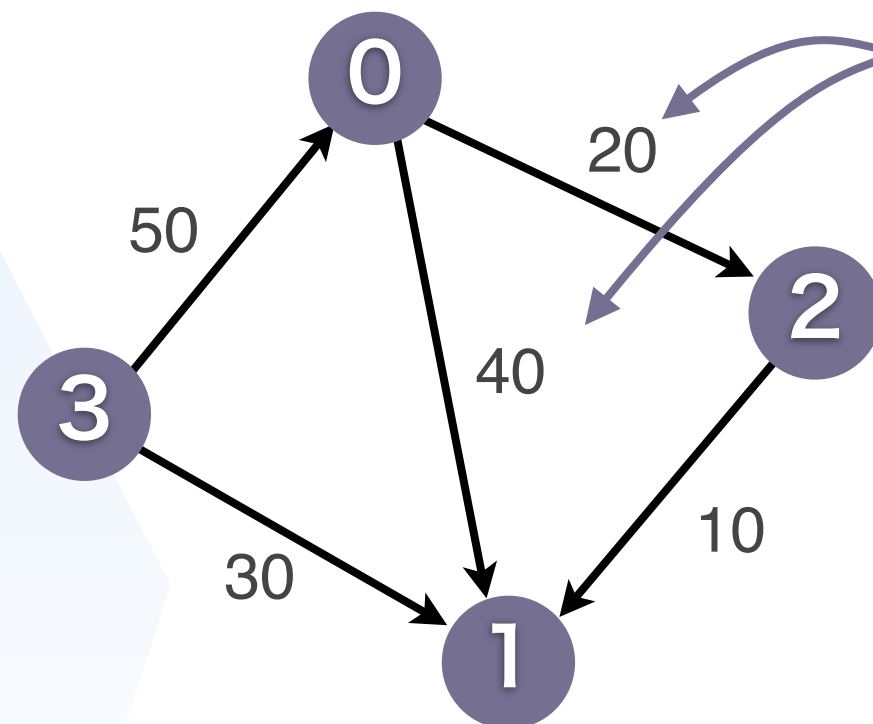
0 → 1 の最短経路は？

コストが[???]の辺が K 本 というクエリが Q 個飛んでくる

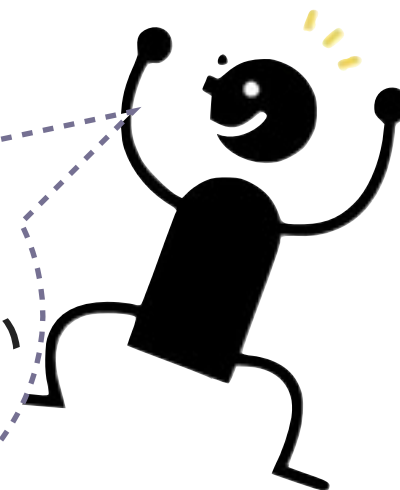
概要

Anna

の視点



Bruno は この辺の
長さを知らないらしい



N頂点 M辺 の有向グラフ

0 → 1 の最短経路は？
というクエリが Q 個飛んでくる

概要

Anna

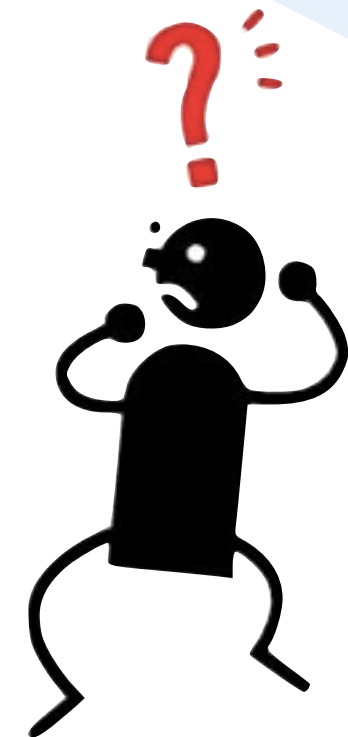


L個

010010010...011



Bruno



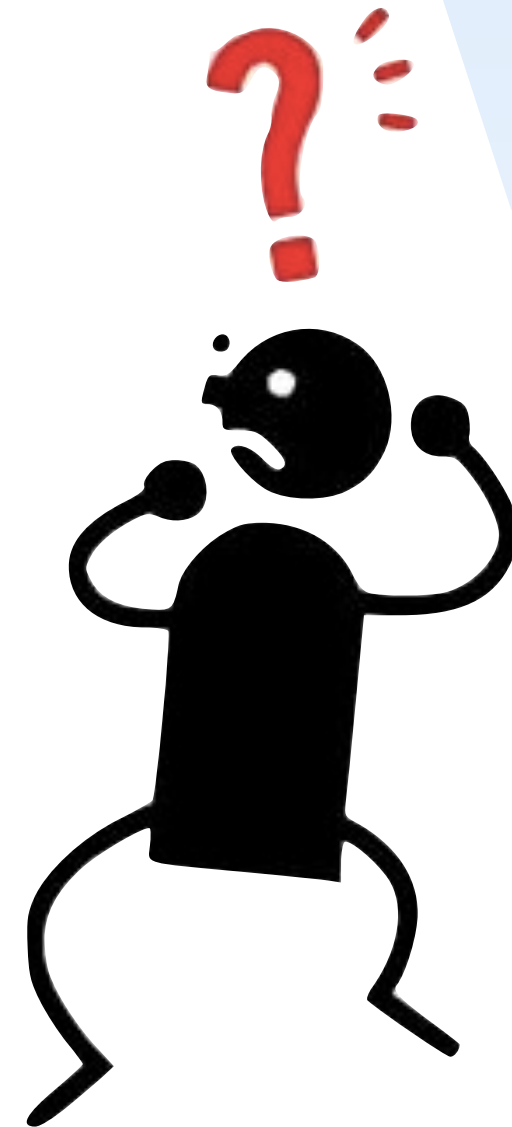
小課題

- 小課題 1 [10点] ($L \leq 1000$ いろいろ小さい)
- 小課題 2 [22点] ($L \leq 180$)
- 小課題 3 [8点] ($L \leq 160$)
- 小課題 4 [40点] ($L \leq 90$)
- 小課題 5 [20点] ($L \leq 64$)

制約

すべての入力データは以下の条件を満たす.

- $2 \leq N \leq 300$.
- $1 \leq M \leq N \times (N-1)$.
- $0 \leq A_i < N$ ($0 \leq i < M$).
- $0 \leq B_i < N$ ($0 \leq i < M$).
- $A_i \neq B_i$ ($0 \leq i < M$).
- $(A_i, B_i) \neq (A_j, B_j)$ ($0 \leq i < j < M$).
- $1 \leq C_i \leq 10^{16}$ ($< 2^{54}$) ($0 \leq i < M$).
- $1 \leq Q \leq 60$.
- $0 \leq S_j < N$ ($0 \leq j < Q$).
- $0 \leq T_j < N$ ($0 \leq j < Q$).
- $S_j \neq T_j$ ($0 \leq j < Q$).
- $(S_i, T_i) \neq (S_j, T_j)$ ($0 \leq i < j < Q$).
- 漢字 S_j から始まり漢字 T_j で終わる漢字しりとりが存在する ($0 \leq j < Q$).
- $1 \leq K \leq 5$.
- $0 \leq U_k < M$ ($0 \leq k < K$).
- $U_i \neq U_j$ ($0 \leq i < j < K$).
- Bruno が忘れてしまった単語の最初の文字は共通である. すなわち $A_{U_0} = A_{U_1} = \dots = A_{U_{K-1}}$.



重要な制約

- $N \leq 300$ ← そこそこ
- $Q \leq 60$ ← そこそこ
- $C_i \leq 10^{16} (< 2^{54})$ ← かなりでかい
- $K \leq 5$ ← かなり小さい
- $A[U_0] = A[U_1] = \dots A[U_{K-1}]$ ← 考察しよう

小課題 1 (10点)

- $Q \leq 10$
- 答えのサイズ = $W \leq 10$
- $L \leq 1000$

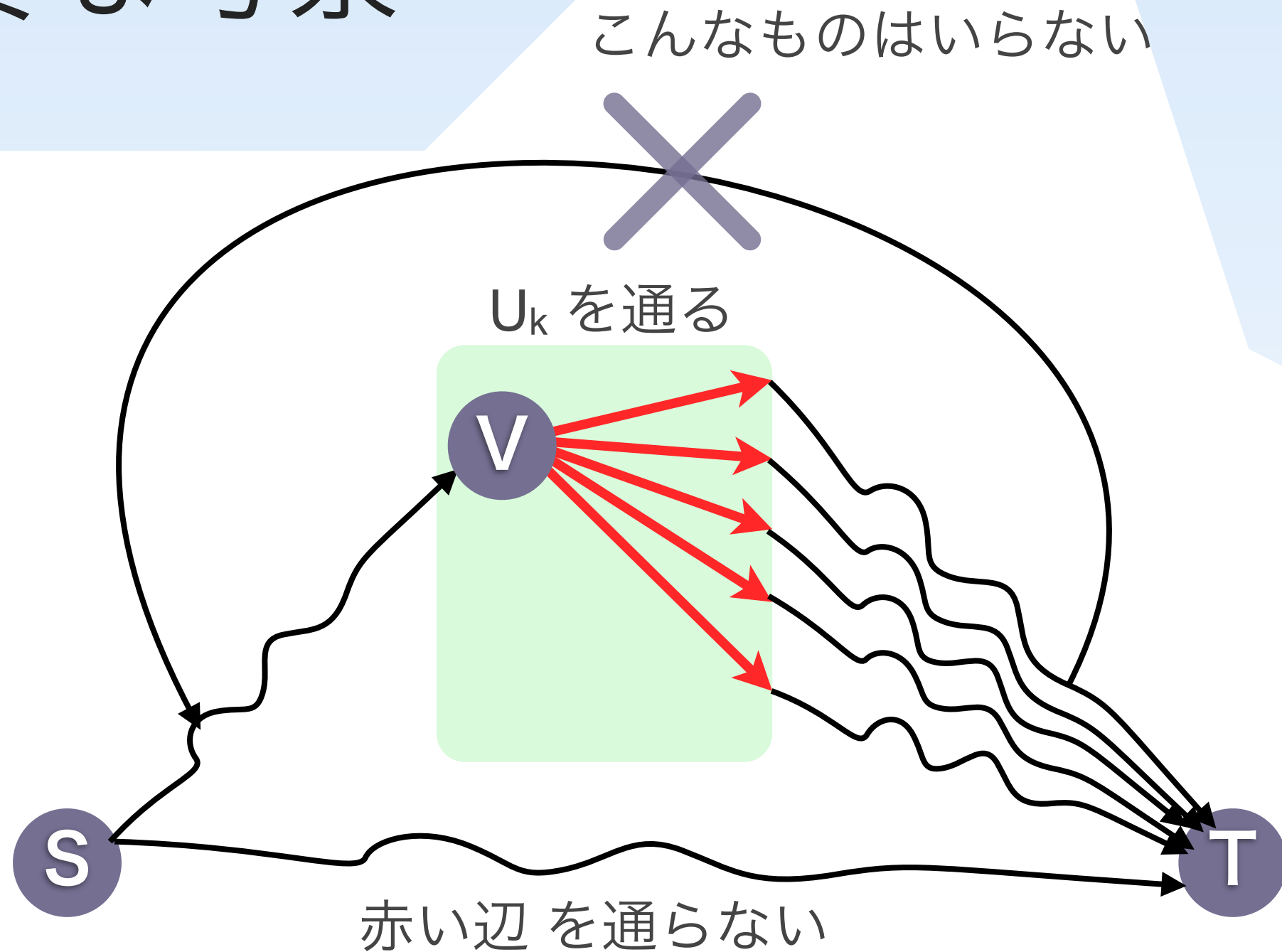
→ Anna で最短路して, **答えを全部**送る

($QW\log(N) = 10 \times 10 \times \log(300) \rightarrow$ **900 bit**)

小課題 1' (10点)

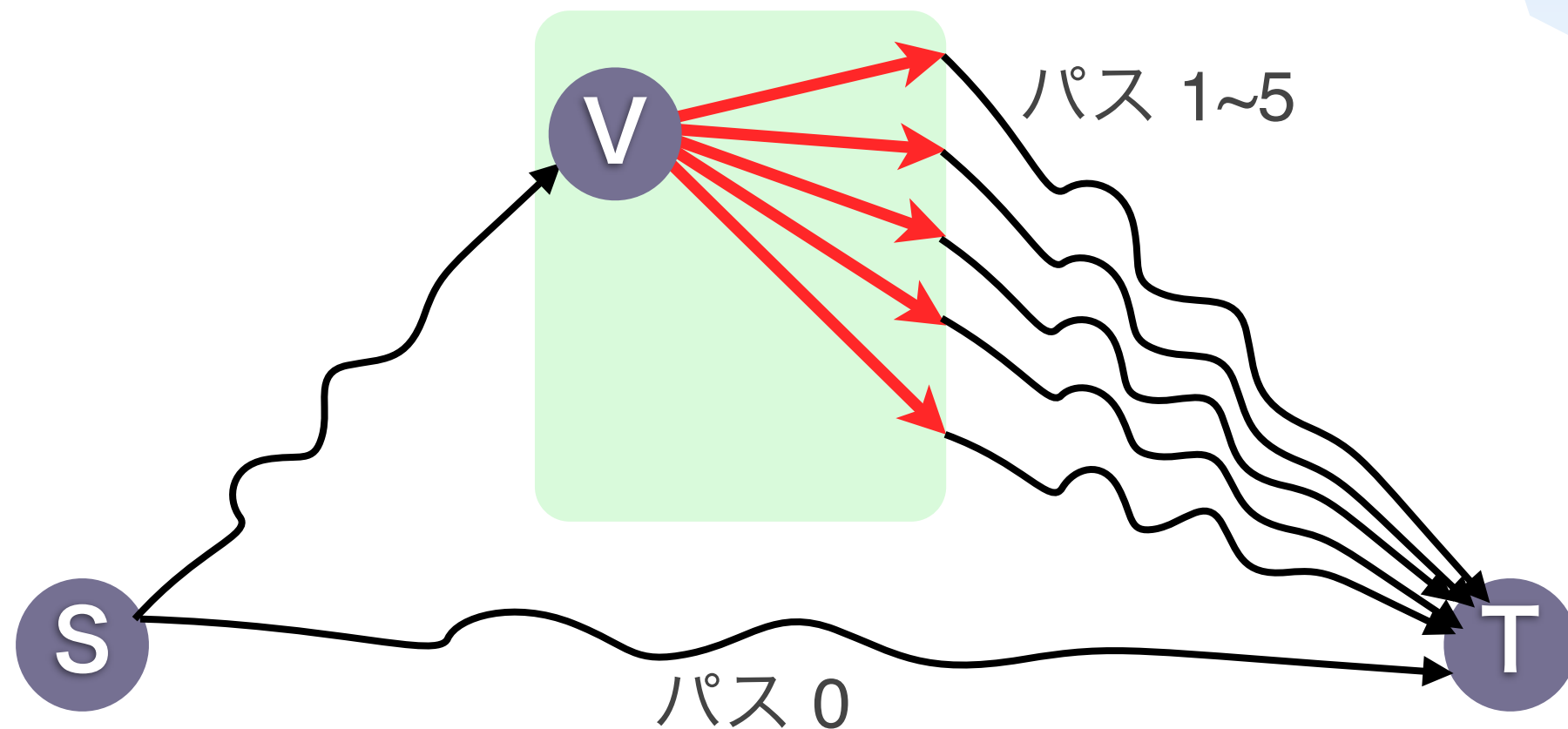
- $Q \leq 10$
 - 答えのサイズ = $W \leq 10$
 - $L \leq 1000$
- Anna が **U_k のコストをそのまま**送る
- ($\log(C)K = \log(2^{54}) \times 5 =$ **270 bit**)

重要な考察



最短経路はこの 6 通りしかない

重要な考察



とりあえず番号付け

小課題 2 (22点)

- ・ この **6** ($=K+1$) **通りのどれを通るか**を送る
- ・ 1クエリあたり 3 bit
- ・ $Q \times 3 = 60 \times 3 = \mathbf{180 \text{ bit}}$

数字を0/1列で送る 一般的なテク

- $0 \leq a < 5, 0 \leq b < 5, 0 \leq c < 10$ を送りたい
- そのまま送ると
a に 3 bit , b に 3 bit , c に 4 bit → 合計 10 bit
- (a,b,c) は $5 \times 5 \times 10 = 250$ 通り
- 250 通りのうちどれか？を送ることにする
→ 8 bit (2 bit 短縮)

小課題 3 (8点)

- **3つずつまとめて送る**
- x, y, z を送るとき, $36x + 6y + z$ を送る
- $6 \times 6 \times 6 = 216$ 通り $\rightarrow$ 8 bit
- $(Q / 3) \times 8 = (60 / 3) \times 8 = \mathbf{160 \text{ bit}}$

小課題 4 (40点)

Compare(j, a, b):

問題 j でパス a とパス b はどっちが短い？

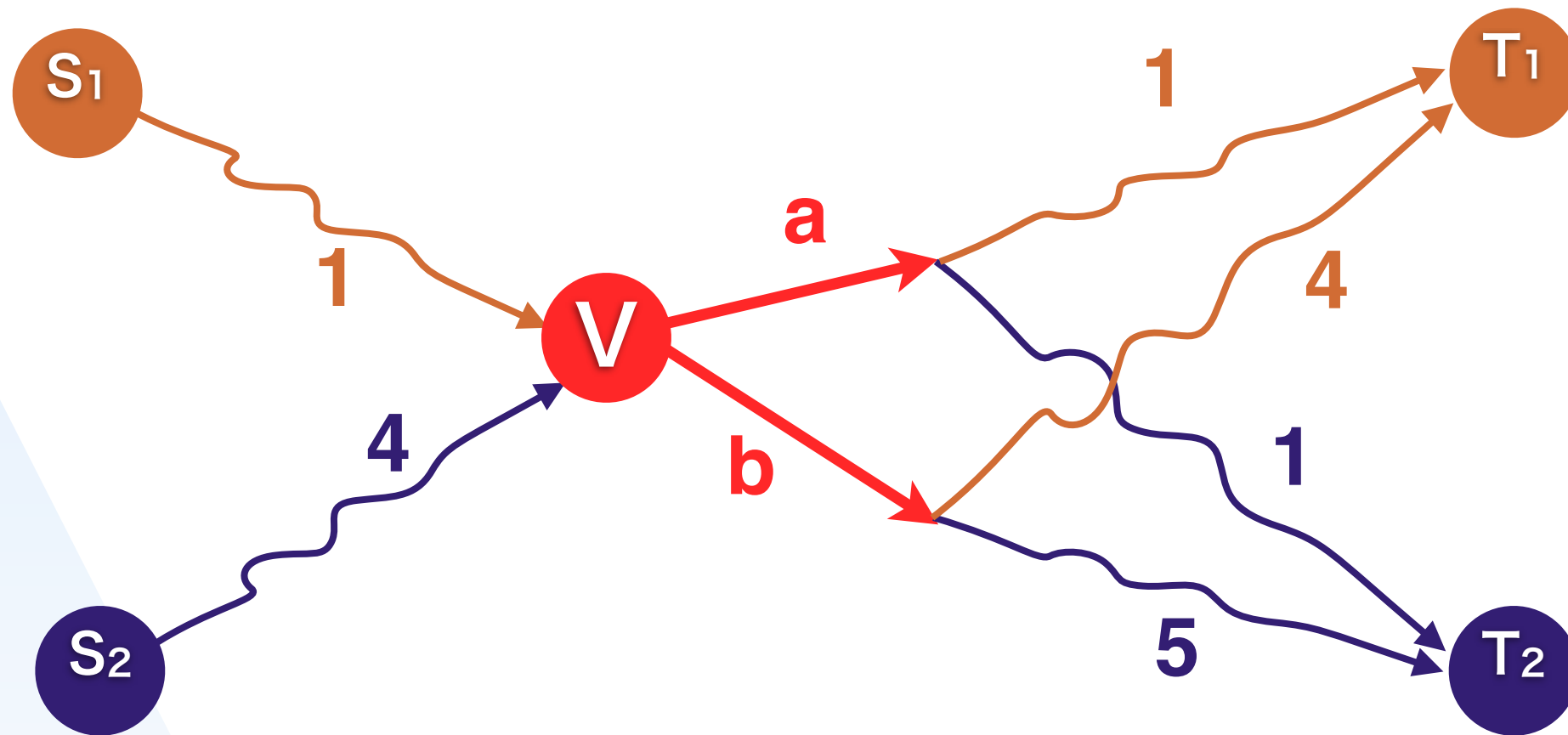
これが $0 \leq j < Q, 0 \leq a \leq 5, 0 \leq b \leq 5$

について全部分かれれば解ける

小課題 4 (40点)

Bruno

の視点



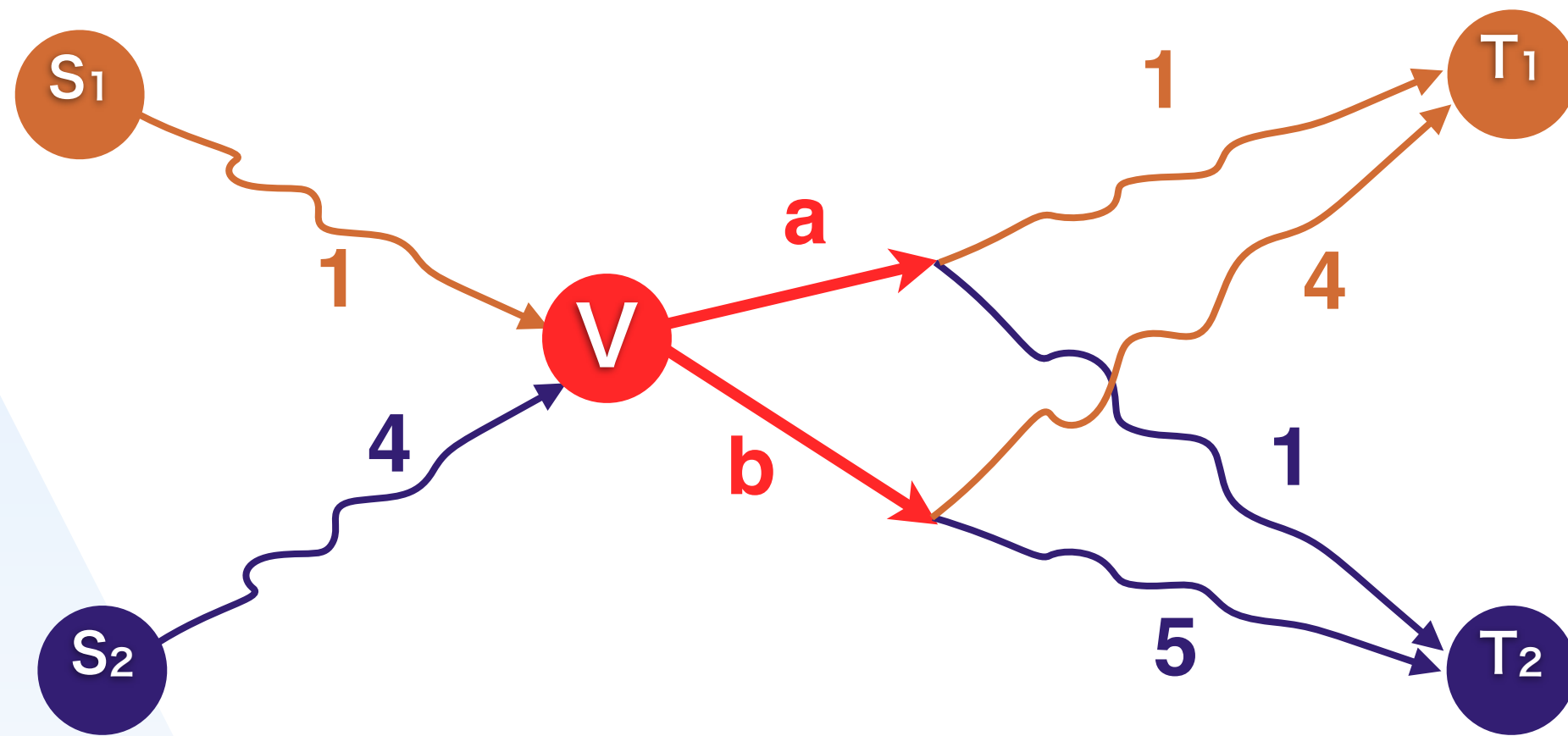
$a+1 < b+4$ なら 問題 1 は上ルートが勝ち

$a+1 < b+5$ なら 問題 2 は上ルートが勝ち

小課題 4 (40点)

Bruno

の視点



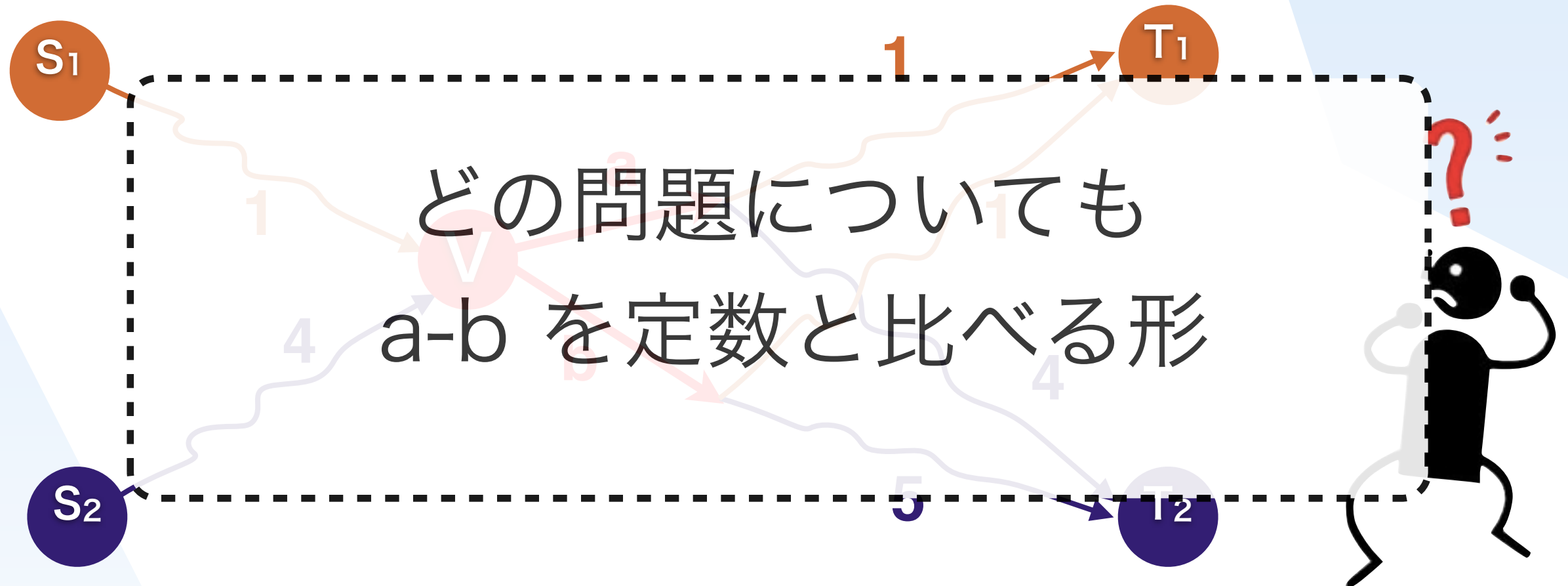
$a-b < -1+4$ なら 問題 1 は上ルートが勝ち

$a-b < -1+5$ なら 問題 2 は上ルートが勝ち

小課題 4 (40点)

Bruno

の視点



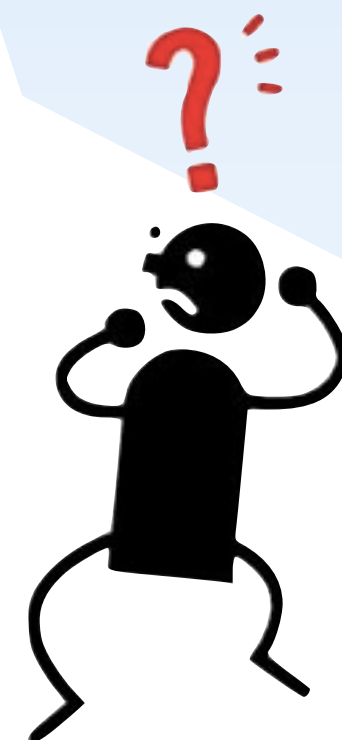
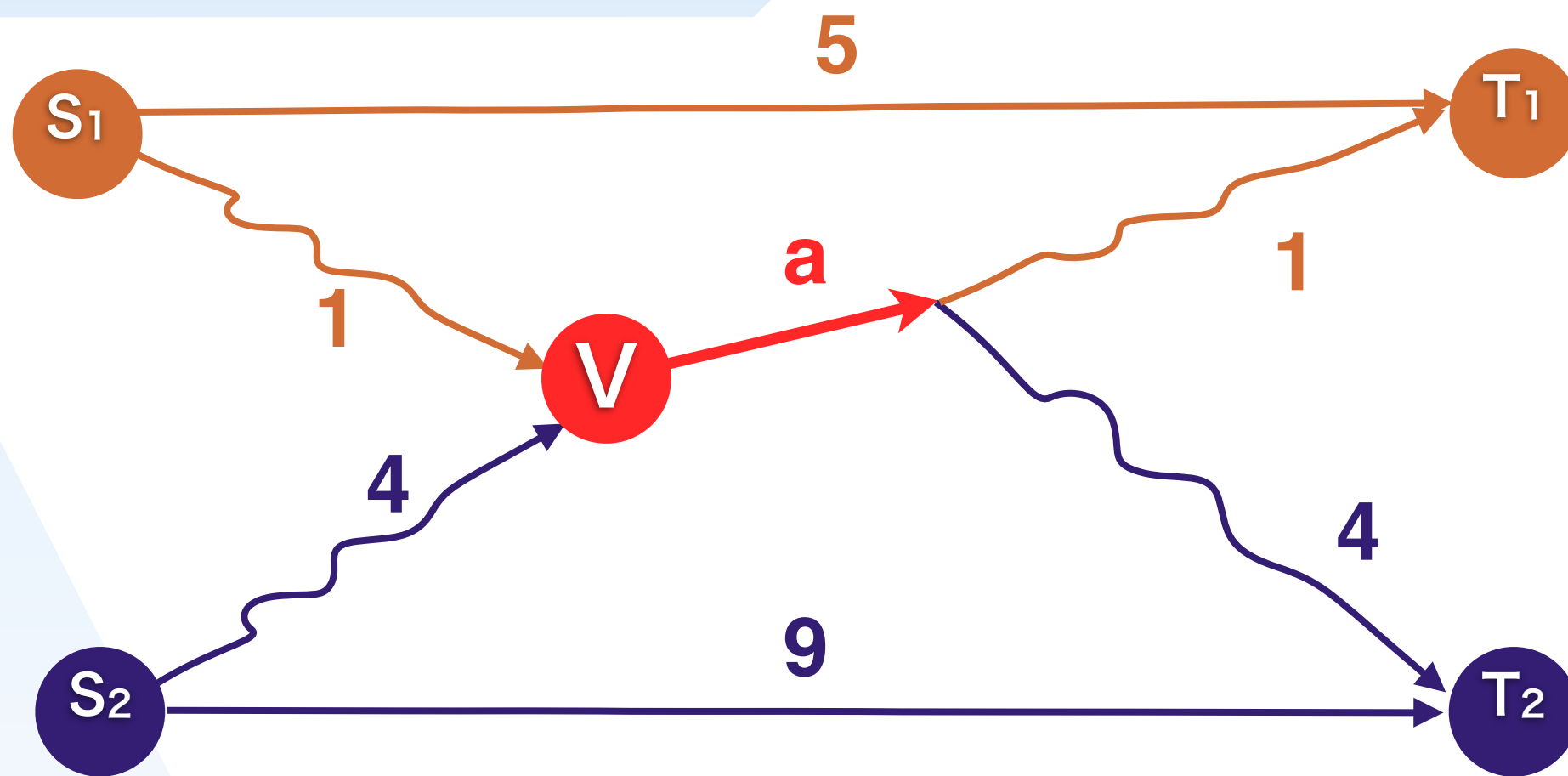
$a-b < -1+4$ なら 問題 1 は上ルートが勝ち

$a-b < -1+5$ なら 問題 2 は上ルートが勝ち

小課題 4 (40点)

Bruno

の視点



$a-0 < -2+5$ なら 問題 1 は中ルートが勝ち

$a-0 < -8+9$ なら 問題 2 は中ルートが勝ち

小課題 4 (40点)

$a-b < -3$ なら 問題 1 は上ルートが勝ち

$a-b < -1$ なら 問題 2 は上ルートが勝ち

$a-b < 4$ なら 問題 3 は上ルートが勝ち

$a-b < -1$ なら 問題 4 は上ルートが勝ち

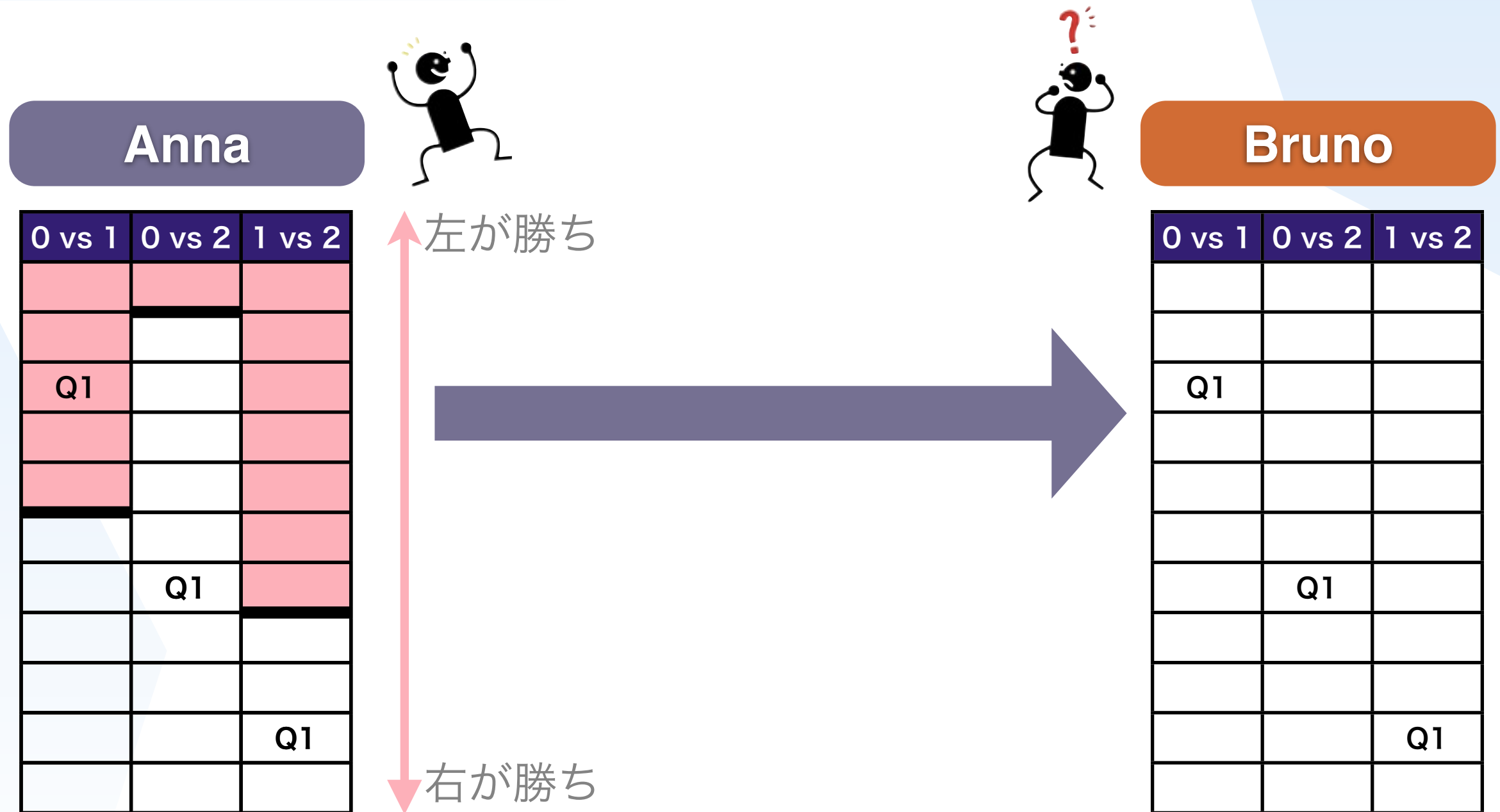
$a-b < -5$ なら 問題 5 は上ルートが勝ち

$a-b < 9$ なら 問題 6 は上ルートが勝ち

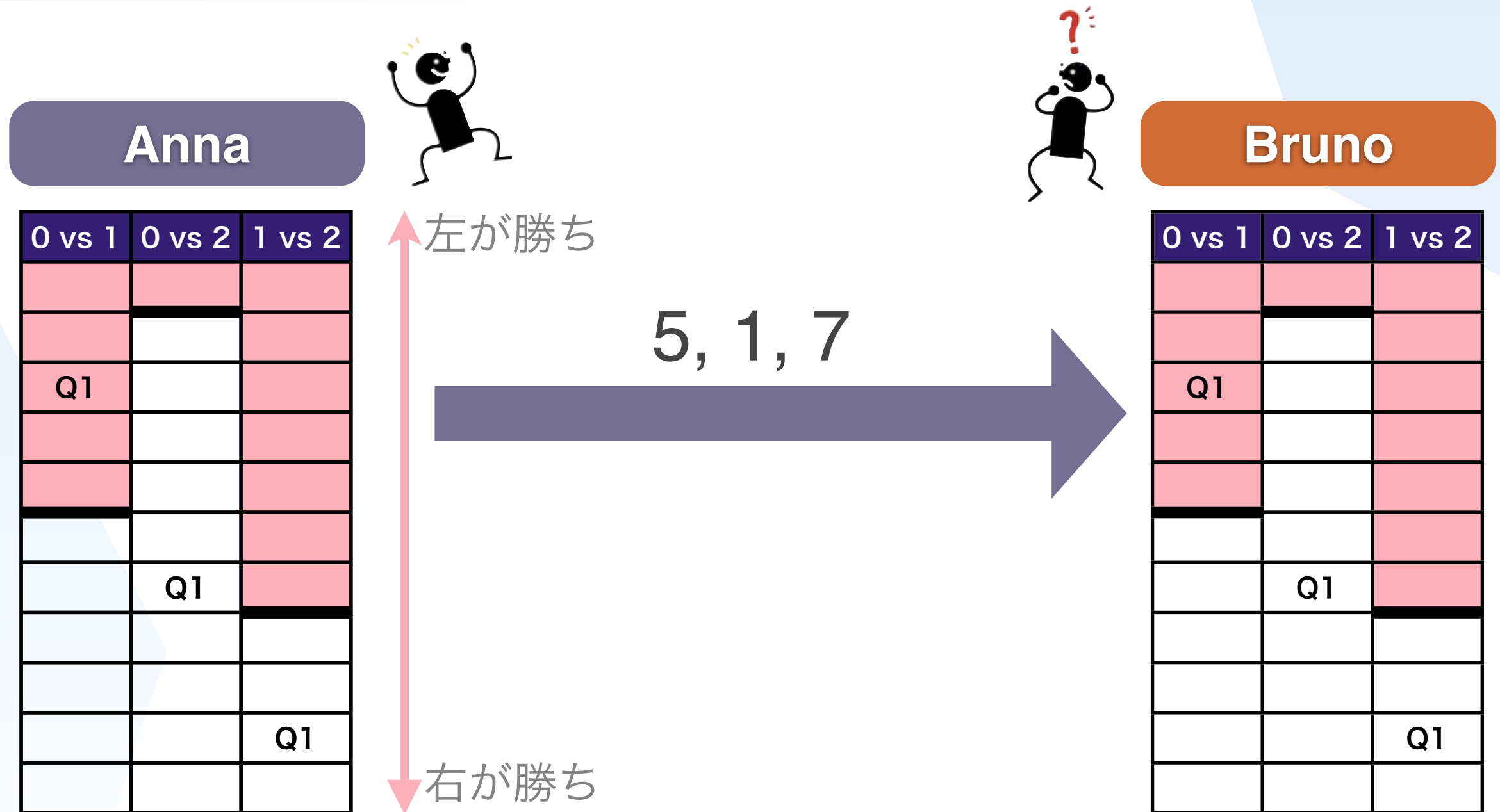
小課題 4 (40点)

- ・ パス a と パス b の比較で パス a が勝つのはいくつか？
を送る.
- ・ 「パス a 勝ち」の個数が分かれば Bruno で復元できる
- ・ 送るのは $\log(Q+1) \rightarrow 6 \text{ bit}$
- ・ a, b の組み合わせが $(K+1) \times K / 2 = 15$ 通り
- ・ $6 \times 15 = \mathbf{90 \text{ bit}}$

小課題 4 の解のイメージ



小課題 4 の解のイメージ



小課題 4 の解のイメージ

パス 0 < パス 1
パス 0 > パス 2
パス 1 > パス 2
だからパス 2 が最短やな！

Anna



0 vs 1	0 vs 2	1 vs 2
Q1		
	Q1	
		Q1

左が勝ち

5, 1, 7



Bruno



0 vs 1	0 vs 2	1 vs 2
Q1		
	Q1	
		Q1

右が勝ち

小課題 4 (40点)

- ・ パス a と パス b の比較で パス a が勝つのはいくつか？
を送る.
- ・ 「パス a 勝ち」の個数が分かれば Bruno で復元できる
- ・ 送るのは $\log(Q+1) \rightarrow 6 \text{ bit}$
- ・ a, b の組み合わせが $(K+1) \times K / 2 = 15$ 通り
- ・ $6 \times 15 = \mathbf{90 \text{ bit}}$

小課題 4 の解のイメージ

パス 0 < パス 1
 パス 0 > パス 2
~~パス 1 > パス 2~~
 だからパス 2 が最短やな！

Anna



左が勝ち

Bruno



0 vs 1	0 vs 2	1 vs 2

0 vs 1	0 vs 2	1 vs 2

5, 1, 7

Compare(j, a, b) を全部送るのはムダ

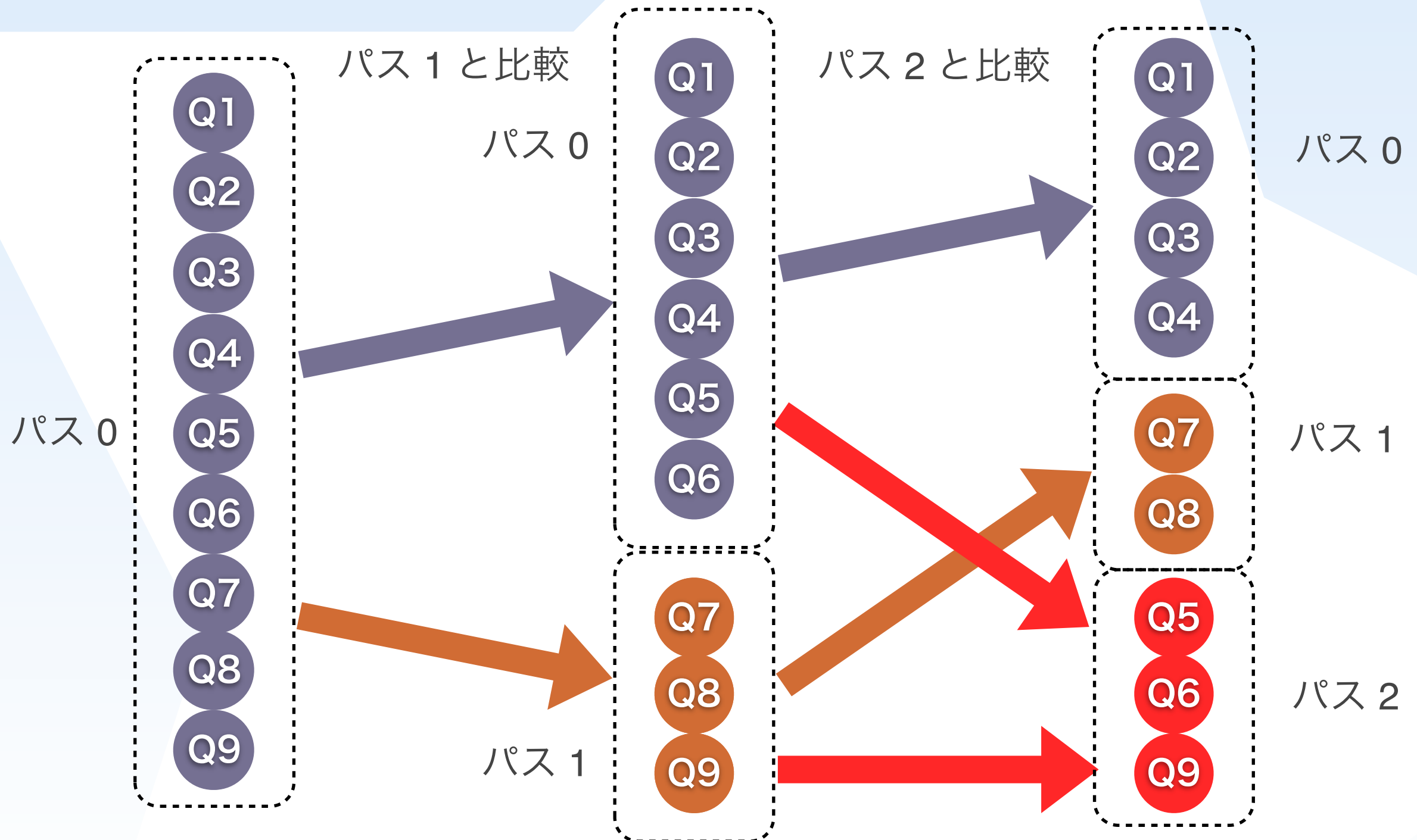
Q1		
	Q1	
		Q1

Q1		
	Q1	
		Q1

右が勝ち

小課題 5 (20点)

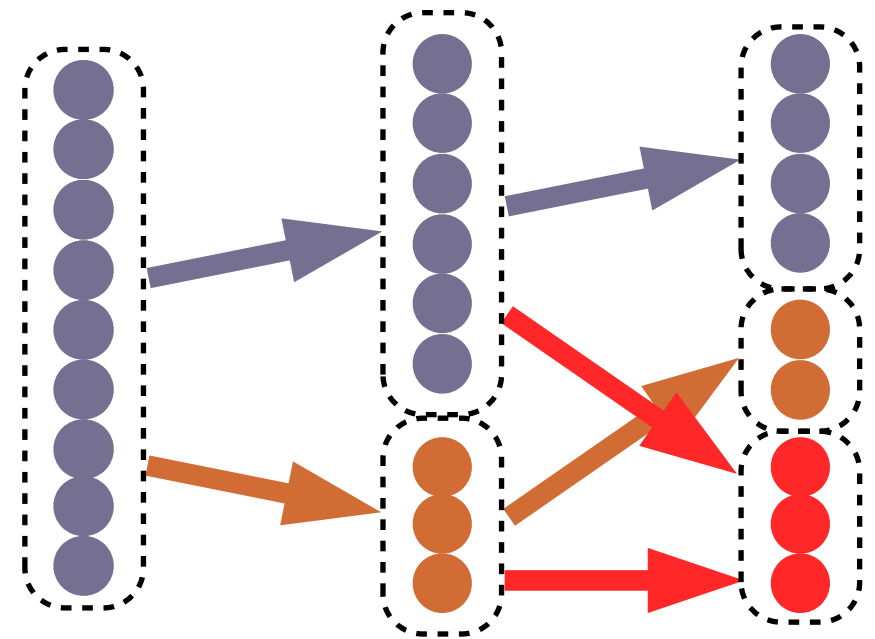
最短パスをパス i と比較



小課題 5 (20点)

使う Compare は

- Compare(?, 0, 1) : 9個
- Compare(?, 0, 2) : 6個
- Compare(?, 1, 2) : 3個



小課題 5 (20点)

- Compare(?, a, b) を使う個数を X_{ab} とする
 - 左が勝つのは何個かを送る. ($0 \sim X_{ab}$ の範囲)
- (まとめて送るテクでの圧縮もする)

$\log(X_{01}+1) + \log(X_{02}+1) + \log(X_{12}+1) + \dots + \log(X_{45}+1)$ [bit]

- $(60) \rightarrow (30,30) \rightarrow (20,20,20) \rightarrow (15,15,15,15) \rightarrow (12,12,12,12,12)$

となるように分割された時が最悪

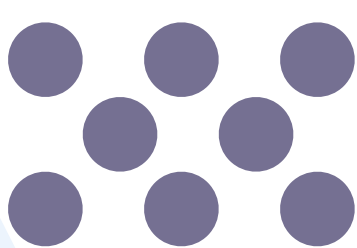
$\rightarrow \log(61) + \log(31) \times 2 + \log(21) \times 3 + \log(16) \times 4 + \log(12) \times 5$

$\rightarrow$ **64 bit**

小課題 5 の解のイメージ

Anna



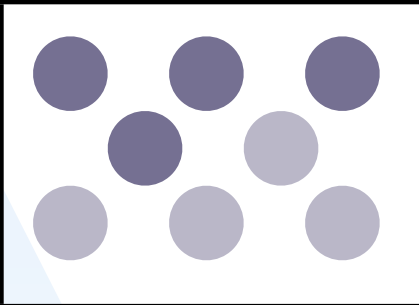
	



小課題 5 の解のイメージ

Anna



4



小課題 5 の解のイメージ

Anna



● ● ● ●	
● ● ● ●	

4



小課題 5 の解のイメージ

Anna



4, 2, 1



小課題 5 の解のイメージ

Anna



● ●	
● ● ●	
● ● ●	

4, 2, 1



小課題 5 の解のイメージ

Anna



● ●	
● ● ●	
● ● ●	

4, 2, 1, 1, 2, 0



小課題 5 の解のイメージ

Anna



●	● ● ●
●	
● ● ●	

4, 2, 1, 1, 2, 0, ...



小課題 5 の解のイメージ



Bruno

4, 2, 1, 1, 2, 0, ...



小課題 5 の解のイメージ



Bruno

4, 2, 1, 1, 2, 0, ...



小課題 5 の解のイメージ



Bruno

2, 1, 1, 2, 0, ...



小課題 5 の解のイメージ



Bruno

2, 1, 1, 2, 0, ...



小課題 5 の解のイメージ



Bruno

1, 2, 0, ...



● ●	
● ● ●	
● ● ●	

小課題 5 の解のイメージ



Bruno

1, 2, 0, ...



小課題 5 の解のイメージ



Bruno



●	● ● ●
●	
● ● ●	

小課題 5 (20点)

- $\text{Compare}(?, a, b)$ を呼ぶ回数を C_{ab} とする
- それぞれ小課題 4 と同じように送る
(まとめて送るテクでの圧縮もする)
- $\log(C_{01}+1) + \log(C_{02}+1) + \log(C_{12}+1) + \dots + \log(C_{45}+1)$ [bit]
- $(60) \rightarrow (30, 30) \rightarrow (20, 20, 20) \rightarrow (15, \dots, 15) \rightarrow (12, \dots, 12)$

となるように分割された時が最悪

→ $\log(61) + \log(31) \times 2 + \log(21) \times 3 + \log(16) \times 4 + \log(12) \times 5$

→ **64 bit** (おめでとうございます)

諸注意

- ・ long long (引数の型見てネ)
- ・ パス i が無い ([???)の辺を消すと非連結になる)

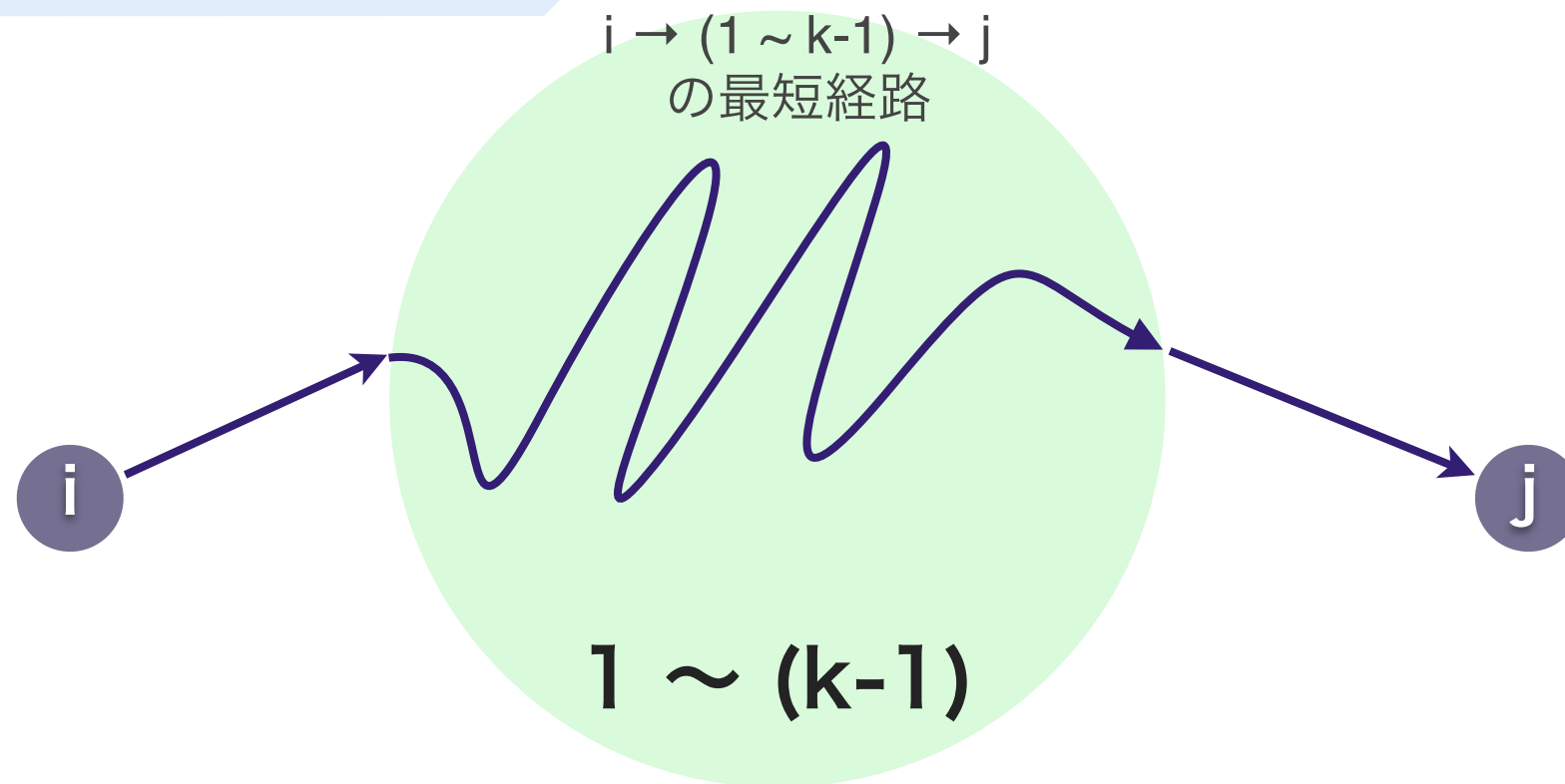
- ・ グラフが密なので Dijkstra なら

priority queue を使わない $O(V^2)$ の方で

(今回はどっちでもたぶん OK)

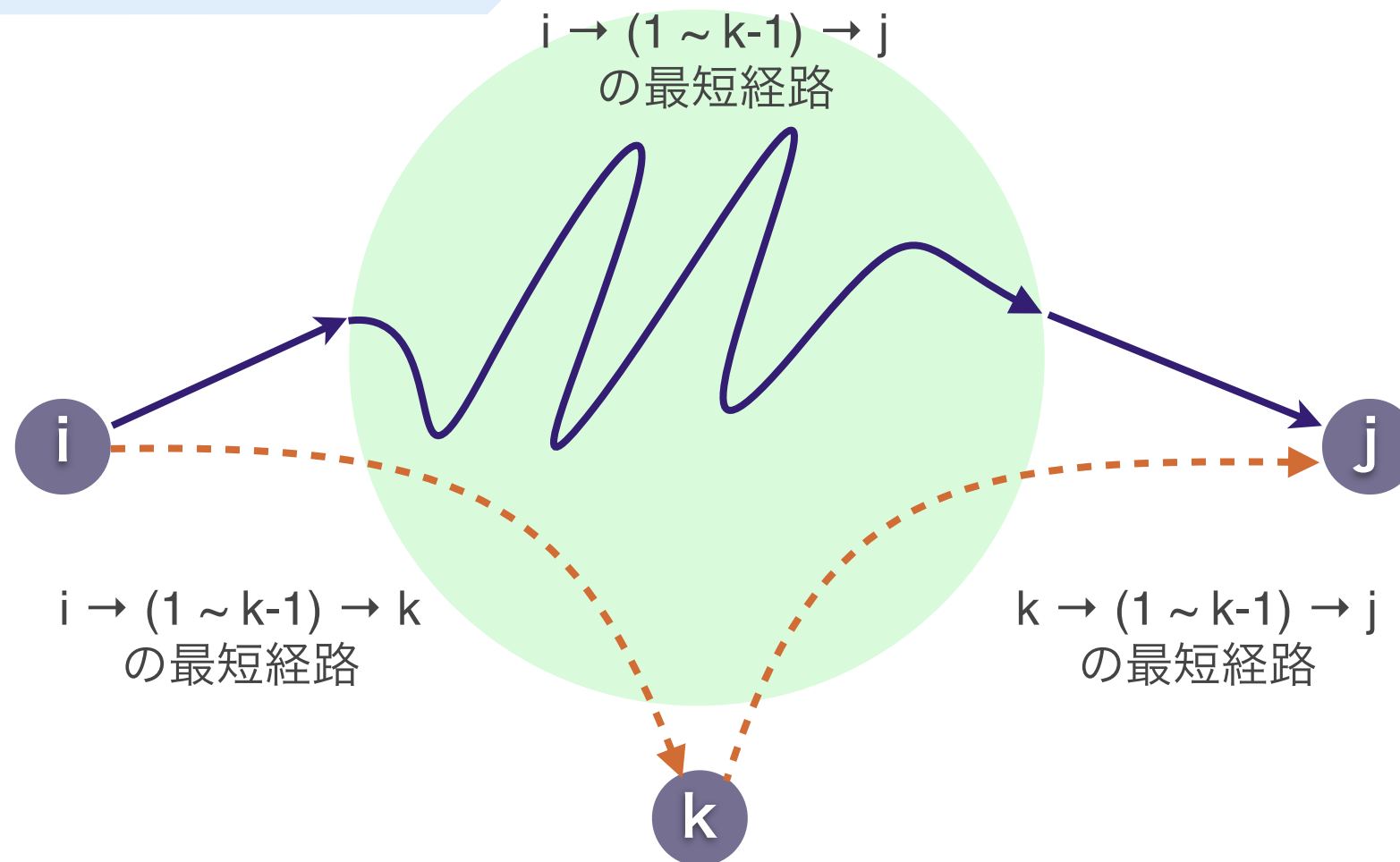
- ・ 不正はいけない

Warshall-Floyd 法



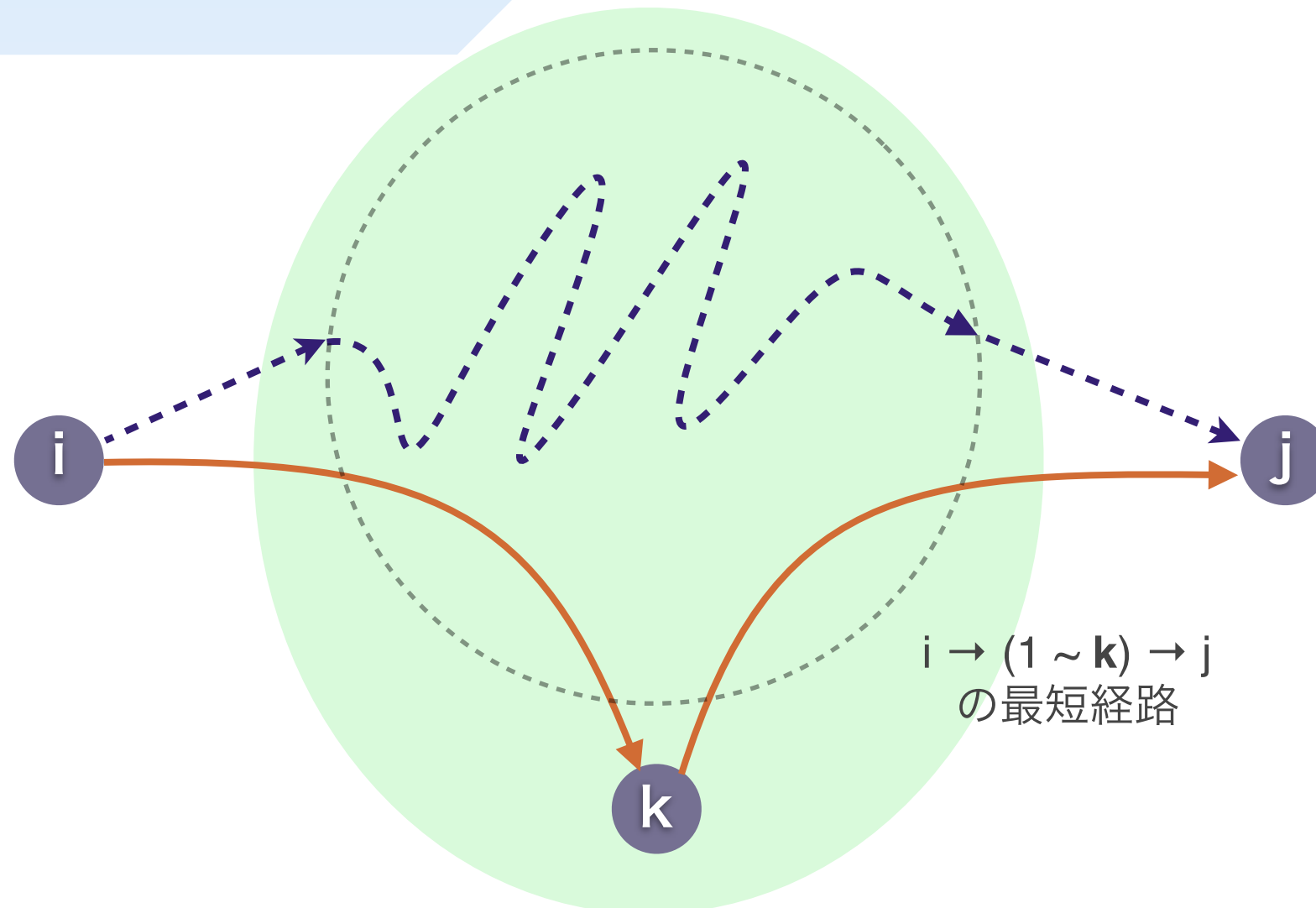
全頂点对間の最短距離を求めます
 $1 \sim (k-1)$ だけを通って行く最短路がわかっている

Warshall-Floyd 法



$(i \rightarrow k) + (k \rightarrow j)$ が $(i \rightarrow j)$ より短かったら

Warshall-Floyd 法



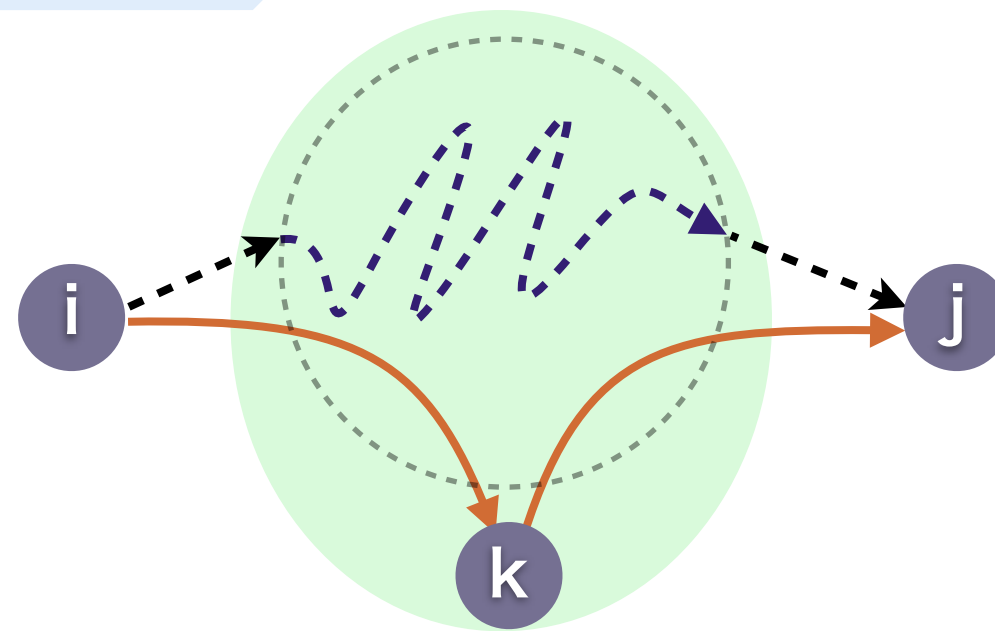
$(i \rightarrow k) + (k \rightarrow j)$ が $(i \rightarrow j)$ より短かったら
 $i \rightarrow j$ を更新



Warshall-Floyd 法

```
for (k = 1 ... N) {  
    for (i = 1 ... N) {  
        for (j = 1 ... N) {  
            i → j を更新する  
        }  
    }  
}
```


経路復元

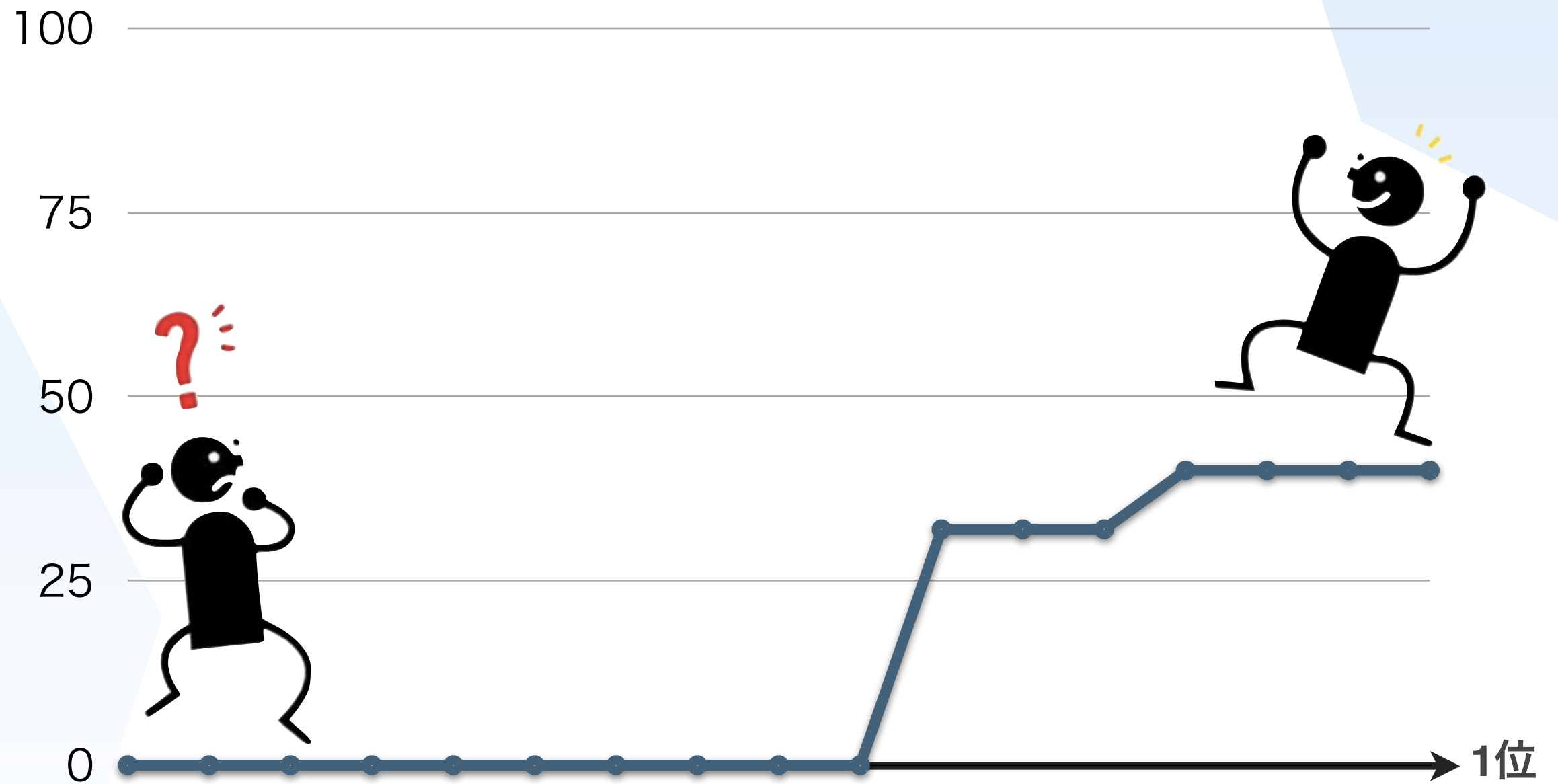


$\text{next}(i \rightarrow j) : i \rightarrow j$ の経路で初めにたどる辺

更新 : $\text{next}(i \rightarrow j) \leftarrow \text{next}(i \rightarrow k)$

辿る : $i \leftarrow \text{next}(i \rightarrow j)$ の終点

点数





JOI春合宿 Straps 解説

三谷庸
2014/3/23

問題概要

- N 個のストラップがある
- ストラップの下にストラップを付けられる
- ストラップにうれしさが定まっている
- うれしさの合計を最大化

小課題1

- $N \leq 15$
- $2^{15} = 32768$
- 全探索

小課題1

- 「ストラップがいくつか与えられたとき、それらを全部使った取り付け方が存在するか」

小課題1

- 「ストラップがいくつか与えられたとき、それらを全部使った取り付け方が存在するか」
- 「取り付け端子の個数」 $\geq$ 「ストラップの個数」であればよい
- なぜか？
- 必要性は明らか

小課題1

- 取り付け方
- 端子の数が0のストラップをいきなりつけるとまずい
- 端子の数0のストラップを最後に取り付けるようにすればよい

小課題1

- 計算量は $O(2^N N)$

考察1

- 端子の数が0のストラップを最初につけると損
- 端子の数が0でないストラップを取り付けることで、それ以外に取り付けられるストラップが減ることはない

考察1

- ストラップを、端子の数が0のものが最後になるように並べ替えてから処理する
- これにかかる時間は $O(N)$ または $O(N \log N)$

小課題2

- 全ての i に対して、 $B_i \geq 0$
- 取り付けてうれしさが減ることはない
- 貪欲法

小課題2

- 最初に端子の数が0でないストラップをすべて取り付ける
- 端子の数は減らないので、つけられなくなることはない

小課題2

- 最初に端子の数が0でないストラップをすべて取り付ける
- 端子の数は減らないので、つけられなくなることはない
- 次に、端子の数が0であるストラップをうれしさが大きい順に取り付ける。

小課題2

- 計算量は $O(N)$

考察2

- 「残っている端子がいくつあるか」だけが重要
- 取り付けた端子の集合が同じならば、取り付け方によらず残った端子の数は同じ

考察2

- 「残っている端子がいくつあるか」だけが重要
- 取り付けた端子の集合が同じならば、取り付け方によらず残った端子の数は同じ

○ 動的計画法

小課題3

- $N \leq 2000$
- それぞれのストラップは15個以下しか下にストラップを付けられない

小課題3

- DPの状態として、「どのストラップまで考えたか」と「今使える端子はいくつあるか」を持っておく
- 端子数0のストラップを最後に処理すれば、処理の途中で端子の数が負になることはないとしてよい

小課題3

- $dp[i][j]$ = 「ストラップ $0, 1, \dots, i-1$ について取り付けるかどうかを決め、 j 個の端子が残っている状態にするときのうれしさの最大値」
とする

小課題3

- i 番目のストラップの端子の数を x , うれしさを y とすると、遷移は

$$dp[i+1][j] = \max(dp[i+1][j], dp[i][j])$$

$$dp[i+1][j-1+x] =$$

$$\max(dp[i+1][j-1+x], dp[i][j] + y)$$

となる

小課題3

- 端子の個数は $N \cdot A$ 以下 (A は $a[i]$ の最大値)
- よって、計算量は $O(N^2 A)$

小課題3

- MLEに注意
- `int dp[2001][2000*15]`は256MBに対してぎりぎりになる

小課題3

- MLEに注意
- `int dp[2001][2000*15]`は256MBに対してぎりぎりになる
- メモリの使い回し
- `dp[i][]`の計算には`dp[i-1][]`しか必要ない
- `dp[0][]`と`dp[1][]`を交互に使う
- 実際に確保する配列は`dp[2][2000*15]`でよい

満点解法

- 小課題3のDPとほぼ同じ

満点解法

- 小課題3のDPとほぼ同じ
- 端子が N 個あれば、それ以上端子を使うことはない
- 端子が N 個以上あるときは N 個とみなす

満点解法

- 計算量は $O(N^2)$
- 100点

得点分布

